

Programación Web

Programación IV



Ing. Alejandro Di Battista

TUP26

Programación Web

Programación IV

Ing. Alejandro Di Battista

2026

Contenido

Parte I. Entender el lenguaje y sus valores

- Capítulo 1. Del problema al programa
- Capítulo 2. Variables, alcance y expresiones
- Capítulo 3. El tipo `boolean`
- Capítulo 4. Los tipos `number` y `bigint`
- Capítulo 5. El tipo `string` y Unicode
- Capítulo 6. `undefined`, `null` y la ausencia de datos
- Capítulo 7. Conversión y coerción de tipos
- Capítulo 8. El tipo `symbol` y los protocolos del lenguaje

Parte II. Modelar colecciones y entidades

- Capítulo 9. Arrays y matrices
- Capítulo 10. Objetos, propiedades y referencias
- Capítulo 11. `Map`, `Set` y colecciones especializadas

Parte III. Organizar el comportamiento

- Capítulo 12. Estructuras de control
- Capítulo 13. Errores y excepciones
- Capítulo 14. Funciones
- Capítulo 15. Programación funcional y pipelines de datos
- Capítulo 16. Recursividad y árboles binarios

Parte IV. Aplicar e integrar

- Capítulo 17. Programación asíncrona con `async` y `await`
- Capítulo 18. Expresiones regulares
- Capítulo 19. Gestión de archivos con Node.js

Parte V. Construir para la Web

- Capítulo 20. Desarrollo web: del documento a una aplicación del clima
- Capítulo 21. Arrow.js: del contador al inventario
- Capítulo 22. Cómo consumir una API REST

Apéndices

- Apéndice A. Byte Pair Encoding: construir un tokenizador
- Apéndice B. Evaluar expresiones mediante pilas
- Apéndice C. Fundamentos de computación digital
- Apéndice D. Una agenda de terminal con React e Ink
- Apéndice E. Referencia rápida de expresiones regulares
- Apéndice F. Glosario

Fuentes

Parte I. Entender el lenguaje y sus valores

Capítulo 1. Del problema al programa

Idea central

JavaScript es un lenguaje; para convertirlo en una solución necesita un motor que lo ejecute y un entorno que le permita interactuar con el mundo. Distinguir esas tres capas evita buena parte de la confusión inicial y permite decidir si un programa debe vivir en el navegador, en Node.js o en ambos.

JavaScript no es “el lenguaje de una etiqueta `<script>`” ni un sinónimo de navegador. Es una especificación con múltiples implementaciones, anfitriones y herramientas. Comprender ese ecosistema permite interpretar correctamente mensajes de error, documentación y decisiones de arquitectura.

1.1 Una tarea concreta antes que un lenguaje

Imaginemos una inscripción a una materia. Recibimos el nombre y la edad de una persona, validamos los datos y producimos una respuesta. El problema existe antes del código:

```
entrada → reglas → salida
```

JavaScript permite expresar esas reglas:

```
function evaluarInscripcion({ nombre, edad }) {  
  if (!nombre.trim()) return { aceptada: false, motivo: "Falta el nombre" };  
  if (edad < 16) return { aceptada: false, motivo: "Edad insuficiente" };  
  
  return { aceptada: true, mensaje: `Bienvenido, ${nombre}` };  
}
```

Este pequeño programa ya contiene las piezas que recorreremos en el libro: valores, una estructura de datos, una función, condiciones y un resultado explícito.

1.2 Por qué apareció JavaScript

La Web inicial publicaba documentos enlazados. Cada interacción importante requería pedir una página nueva al servidor. Los navegadores necesitaban un lenguaje pequeño que pudiera reaccionar cerca del usuario: validar un formulario, cambiar contenido y responder a eventos sin recargar todo.

JavaScript fue creado en 1995 dentro de Netscape. Su primer prototipo se desarrolló con enorme rapidez y el lenguaje cambió de nombre durante su lanzamiento. Su nombre final aprovechó la popularidad de Java, pero JavaScript y Java son lenguajes distintos:

- tienen sistemas de tipos y modelos de objetos diferentes;
- se ejecutan mediante plataformas diferentes;
- compartir parte de la sintaxis superficial no vuelve equivalentes sus programas.

La estandarización llegó mediante Ecma International. El estándar se llama **ECMAScript** y JavaScript es su implementación más conocida.

Una línea de tiempo mínima

- **1995:** nace el lenguaje en el navegador.
- **1997:** se publica la primera edición de ECMAScript.
- **1999:** ECMAScript 3 consolida características que dominaron la Web durante años.
- **2009:** ECMAScript 5 formaliza modo estricto, JSON y métodos modernos de arrays, entre otras mejoras.
- **2015:** ECMAScript 2015 introduce módulos, clases, `let`, `const`, promesas, iteradores, generadores, flechas y una gran actualización del lenguaje.
- **Etapas posteriores:** el estándar adopta entregas regulares e incrementales en lugar de esperar muchos años entre grandes versiones.

El lenguaje actual conserva compatibilidad con una enorme cantidad de código antiguo. Esa continuidad explica algunas peculiaridades como `typeof null`, `var` y la igualdad flexible.

1.3 Las tres capas de JavaScript

Cuando decimos “JavaScript”, solemos mezclar tres cosas diferentes:

1. **El lenguaje** define la sintaxis y el comportamiento de valores, objetos, funciones y operadores. Su estándar se llama ECMAScript.
2. **El motor** lee y ejecuta ese código. V8 es un motor usado por Chrome y Node.js; otros entornos pueden utilizar motores diferentes.
3. **El anfitrión** ofrece capacidades externas. El navegador aporta el DOM, eventos y `localStorage`; Node.js aporta archivos, procesos y red.

La especificación ECMAScript define valores como `Map`, `Promise`, `Symbol` y `Array`. Documentos separados definen APIs web como `fetch`, `document` y `setTimeout`. Node.js implementa algunas APIs web y aporta sus propios módulos. Por eso una función puede existir en varios entornos sin pertenecer originalmente al núcleo del lenguaje.

Por eso `document.querySelector()` funciona en una página web pero no es parte del lenguaje, y `readFile()` funciona en Node.js pero no aparece por arte de magia en el navegador. El DOM, la API del navegador que expone `document`, es el tema de la sección 20.10.

```
// Lenguaje: funciona en distintos entornos.
const total = [10, 20, 30].reduce((suma, valor) => suma + valor, 0);

// API del navegador.
document.querySelector("#total").textContent = total;

// API de Node.js.
import { writeFile } from "node:fs/promises";
await writeFile("total.txt", String(total), "utf8");
```

Cuando una API “no existe”, la pregunta productiva es: ¿pertenece al lenguaje, al anfitrión, a una biblioteca instalada o a una versión diferente del entorno?

1.4 Motores y anfitriones diferentes

Entre los motores conocidos están V8, SpiderMonkey y JavaScriptCore. Todos buscan implementar ECMAScript, aunque pueden diferir durante la incorporación de características nuevas, en optimizaciones y en herramientas de diagnóstico.

El mismo motor puede aparecer en anfitriones distintos. V8 ejecuta código dentro de Chrome y también dentro de Node.js, pero Chrome ofrece DOM y Node.js ofrece `node:fs`. Compartir motor no iguala las APIs.

El anfitrión también coordina tareas alrededor del motor. Un bucle de eventos decide cuándo ejecutar callbacks de temporizadores, red o interfaz. JavaScript puede ejecutar una tarea a la vez en un hilo principal y, aun así, coordinar operaciones asincrónicas realizadas por el entorno.

1.5 Del navegador a una plataforma completa

JavaScript nació para agregar comportamiento a páginas web y evolucionó hasta convertirse en un lenguaje de propósito general. Hoy puede participar en toda una aplicación:

- en el **frontend**, responde a la interacción del usuario y presenta información;
- en el **backend**, aplica reglas, protege recursos y accede a datos;
- en herramientas de línea de comandos, automatiza tareas;
- en aplicaciones híbridas, comparte modelos y validaciones entre capas.

El lugar de ejecución importa. El frontend está cerca del usuario, pero su código y sus datos pueden inspeccionarse. Las reglas de seguridad, las credenciales y las decisiones que no deben manipularse pertenecen al servidor.

```
navegador → solicitud → servidor → datos
      ↑               ↓
      └── respuesta ─┘
```

La validación del navegador mejora la experiencia; la validación del servidor conserva la integridad del sistema. Una no reemplaza a la otra.

1.6 Frontend: código cerca del usuario

El frontend puede:

- leer eventos de teclado, ratón o tacto;
- modificar la representación de la página;
- validar rápidamente una entrada;
- almacenar preferencias locales;
- solicitar datos a servicios remotos.

Ese código se descarga en el dispositivo del usuario. No debe contener secretos ni ser la única barrera de autorización. Ocultar un botón no impide llamar a la API correspondiente. El capítulo 20 construye un frontend completo, y la sección 20.11 vuelve sobre por qué una clave no puede esconderse en él.

1.7 Backend: código cerca de datos y reglas

El backend puede:

- autenticar y autorizar;
- aplicar invariantes del negocio;
- acceder a bases de datos y sistemas internos;
- usar credenciales protegidas;
- coordinar operaciones entre usuarios.

Node.js permite escribir ese backend en JavaScript. Usar el mismo lenguaje en ambos extremos facilita compartir conocimientos y algunos contratos, pero no elimina la frontera de seguridad ni convierte en confiables los datos que cruzan la red.

1.8 Módulos y dependencias

Un programa real se divide en módulos. Los módulos ECMAScript declaran explícitamente qué exportan e importan:

```
// precios.js
export function calcularTotal(precio, cantidad) {
  return precio * cantidad;
}
```



```
// aplicacion.js
import { calcularTotal } from "./precios.js";

console.log(calcularTotal(100, 3));
```

El especificador `./precios.js` señala un módulo local. Nombres como `node:path` señalan módulos integrados de Node.js. Otros nombres pueden referirse a paquetes instalados y resueltos por el entorno o una herramienta de construcción.

La sección 19.4 muestra cómo preparar un proyecto de Node.js para que los archivos `.js` usen esta sintaxis, y el apéndice D construye una aplicación completa repartida en módulos.

Los módulos crean límites de nombres, permiten reutilización y vuelven visibles las dependencias. Una importación no garantiza que el código sea seguro o compatible: las dependencias externas también requieren evaluación, versiones y mantenimiento.

1.9 Qué ocurre al ejecutar

Un motor moderno no se limita a interpretar cada línea de la misma manera. Analiza el código, produce una representación ejecutable y puede optimizar las partes que se usan con frecuencia mediante compilación JIT (*just in time*). Si las suposiciones de una optimización dejan de cumplirse, el motor puede descartarla.

De forma conceptual, el recorrido incluye:

1. **lectura y análisis léxico:** el texto se separa en unidades significativas;
2. **análisis sintáctico:** se comprueba la gramática y se construye una representación del programa;
3. **creación de ámbitos:** se preparan declaraciones y vínculos;
4. **ejecución:** un intérprete o código generado realiza las operaciones;
5. **observación y optimización:** las rutas frecuentes pueden compilarse con supuestos más específicos;
6. **desoptimización:** si los supuestos dejan de cumplirse, el motor vuelve a una representación general.

Un error de sintaxis aparece antes de ejecutar el bloque afectado:

```
// const = 10; // SyntaxError
```

Un error de ejecución surge al alcanzar una operación inválida:

```
const usuario = null;
// usuario.nombre; // TypeError al ejecutar esta línea
```

Un error lógico no necesariamente lanza nada: el programa corre y produce una respuesta equivocada. Las pruebas y ejemplos son esenciales para detectarlo.

La consecuencia práctica no es “programar para el motor”. Es mantener datos y operaciones comprensibles. Primero se escribe código correcto y medible; la optimización prematura suele aumentar complejidad sin demostrar una mejora.

1.10 Interpretación, compilación y transpilación

Las categorías no son excluyentes. Un motor puede interpretar una representación intermedia y compilar partes durante la misma ejecución. “JavaScript es interpretado” describe una visión histórica incompleta; “JavaScript es compilado” también requiere aclarar cuándo y hacia qué representación.

Transpilar suele significar transformar código fuente a otro código fuente de un nivel parecido. Una herramienta puede convertir sintaxis moderna a una forma compatible con entornos antiguos. Eso no agrega automáticamente las APIs que el entorno no posee: una sintaxis puede transformarse, mientras que una funcionalidad ausente necesita un *polyfill* o una estrategia alternativa.

1.11 JavaScript y TypeScript

JavaScript tiene tipado dinámico: una variable no queda asociada para siempre a un tipo, aunque cada valor sí tiene uno.

```
let resultado = 42;
resultado = "cuarenta y dos"; // válido en JavaScript
```

TypeScript agrega un análisis estático que ayuda a detectar inconsistencias antes de ejecutar:

```
let resultado: number = 42;
resultado = "cuarenta y dos"; // error del verificador
```

El navegador y Node.js ejecutan JavaScript. Por eso TypeScript se transforma — habitualmente se dice que se transpila— a JavaScript. Sus tipos mejoran las herramientas y documentan contratos, pero desaparecen en tiempo de ejecución. Los datos que llegan desde un formulario, archivo o red siguen necesitando validación real.

1.12 Dos dimensiones que suelen confundirse

Estático frente a dinámico pregunta cuándo se comprueban tipos:

- estático: una herramienta razona antes de ejecutar;
- dinámico: las comprobaciones principales ocurren con valores durante la ejecución.

Fuerte frente a débil intenta describir cuánto permite el lenguaje mezclar tipos y qué conversiones realiza. No existe una única escala universal. JavaScript conserva distinciones importantes —no puede sumarse directamente `bigint` y `number`, por ejemplo— y también aplica coerciones que sorprenden:

```
"5" + 1; // "51"
"5" - 1; // 4
```

En lugar de etiquetarlo con una sola palabra, conviene aprender las reglas concretas de cada operador y convertir en los bordes. El capítulo 7 recorre esas reglas una por una.

1.13 Los tipos atraviesan fronteras

Un formulario entrega texto:

```
const edadTexto = formulario.elements.edad.value;
```

El frontend puede convertir y mostrar una advertencia. Al serializar JSON, el número viaja sin información de TypeScript. El backend recibe bytes, interpreta JSON y vuelve a validar:

```
function validarSolicitud(datos) {
  if (!Number.isSafeInteger(datos.edad) || datos.edad < 0) {
    throw new TypeError("Edad inválida");
  }

  return datos;
}
```

Una anotación como `datos: Solicitud` expresa lo que el desarrollador espera; no transforma una entrada desconocida en una solicitud válida.

1.14 Modo estricto

El modo estricto elimina o convierte en errores varios comportamientos históricos problemáticos:

```
"use strict";
```

Los módulos ECMAScript son estrictos de forma automática. Entre otros efectos, una asignación a un identificador no declarado lanza error y `this` en una llamada de función simple queda `undefined` en lugar de convertirse en el objeto global, como detalla la sección 14.8.

No reemplaza validación ni estilo, pero ofrece una semántica más segura para código moderno.

1.15 Un método productivo para comenzar

Antes de escribir código, respondé cuatro preguntas:

1. ¿Qué datos entran?
2. ¿Qué resultado debe salir?
3. ¿Qué reglas conectan ambos extremos?
4. ¿Qué puede fallar y cómo lo observaremos?

Agregá dos más cuando la solución cruza entornos:

5. ¿En qué anfitrión se ejecuta cada parte y qué APIs necesita?
6. ¿Qué datos atraviesan una frontera y deben volver a validarse?

Después construí la versión más pequeña que recorra el camino completo. Para la inscripción:

```
const solicitud = { nombre: "Ada", edad: 20 };
const respuesta = evaluarInscripcion(solicitud);
console.log(respuesta);
```

Recién entonces agregá más reglas. Este recorrido vertical entrega evidencia temprana y reduce el costo de corregir una idea equivocada.

1.16 Leer documentación con el modelo de capas

Ante una API, identificá:

- si está definida por ECMAScript, la Web, Node.js o una biblioteca;
- qué versiones del entorno la implementan;
- si es síncrona, devuelve una promesa o usa callbacks;
- qué tipos acepta y produce;
- qué errores o resultados de ausencia forman parte del contrato.

Este hábito evita copiar ejemplos de un anfitrión a otro sin comprender por qué fallan.

1.17 Errores frecuentes

Confundir el lenguaje con el entorno

Buscar `document` en Node.js o `readFile` en el navegador. La pregunta correcta no es «¿existe esta función?», sino «¿a qué capa pertenece?».

Suponer que TypeScript valida datos externos

Una anotación describe lo que el programa espera. Un JSON que llega por la red no sabe nada de esa anotación: hay que comprobarlo en tiempo de ejecución.

Confiar en una validación que corre en el navegador

El código del frontend se descarga en la máquina del usuario y puede modificarse. Sirve para mejorar la experiencia, no para proteger una regla.

Discutir si JavaScript es interpretado o compilado

Las dos descripciones son incompletas. Un motor moderno analiza, ejecuta y compila las rutas frecuentes durante la misma ejecución.

Empezar por las características del lenguaje

Antes de elegir sintaxis conviene saber qué entra, qué sale, qué reglas conectan ambos extremos y qué puede fallar.

1.18 Para recordar

- Lenguaje, motor y anfitrión son capas distintas.
- ECMAScript estandariza el núcleo; las APIs del entorno pertenecen a especificaciones y plataformas adicionales.
- Frontend y backend resuelven responsabilidades diferentes.
- TypeScript comprueba tipos durante el desarrollo; no valida datos externos por sí solo.
- Los motores analizan, ejecutan y pueden optimizar; sintaxis, ejecución y lógica fallan en momentos diferentes.
- Un programa productivo comienza por el contrato de entrada y salida, no por una lista de características del lenguaje.

Capítulo 2. Variables, alcance y expresiones

Idea central

Una variable no es una caja: es un nombre vinculado con un valor durante una parte determinada de la ejecución. Para escribir programas previsibles conviene crear ese vínculo lo más cerca posible de su uso, preferir `const`, reservar `let` para cambios deliberados y elegir nombres que expresen el papel del dato.

Esta conclusión se sostiene en tres ideas:

1. declaración, inicialización y asignación son momentos diferentes;
2. el alcance determina dónde puede utilizarse un nombre y la vida del dato determina cuánto tiempo existe;
3. una expresión produce un valor, y los operadores determinan cómo se construye ese resultado.

2.1 De un valor anónimo a un dato con propósito

Un programa puede operar directamente sobre literales:

```
1500 * 3;
```

El cálculo es válido, pero no comunica qué representan esos números. Al darles nombre aparece la intención:

```
const precioUnitario = 1500;
const cantidad = 3;
const subtotal = precioUnitario * cantidad;
```

Una **declaración** introduce el nombre. La **inicialización** le asigna su primer valor. Una **reasignación** reemplaza posteriormente el valor vinculado.

```
let total;           // declaración; su valor inicial es undefined
total = 100;         // asignación
total = 150;         // reasignación

const iva = 0.21;    // declaración e inicialización juntas
```

`const` exige inicialización inmediata porque el vínculo no podrá cambiar.

2.2 Elegir entre `const`, `let` y `var`

La regla de trabajo es:

1. empezá con `const`;
2. cambiá a `let` solo si la reasignación expresa una evolución real del algoritmo;
3. evitá `var` en código nuevo.

const : el nombre conserva su vínculo

```
const moneda = "ARS";  
// moneda = "USD"; // TypeError
```

`const` no vuelve inmutable el valor referenciado. Si el valor es un objeto, sus propiedades todavía pueden cambiar:

```
const pedido = { estado: "pendiente", total: 1000 };  
pedido.estado = "pagado"; // permitido
```

Lo que no puede hacerse es vincular `pedido` con otro objeto:

```
// pedido = { estado: "nuevo", total: 0 };
```

Esta diferencia —vínculo constante frente a valor inmutable— será esencial al estudiar arrays y objetos, en los capítulos 9 y 10.

let : el cambio forma parte del modelo

```
let saldo = 10_000;  
saldo -= 2_500;  
saldo -= 1_000;
```

`let` es apropiado para acumuladores, índices, estados que avanzan y referencias que se reemplazan. Que pueda cambiar no significa que deba hacerlo desde muchos lugares. Cuanto menor sea su alcance, más fácil será reconstruir su historia.

var : una regla histórica diferente

`var` tiene alcance de función, no de bloque, permite redeclaración y su inicialización se comporta de manera diferente durante la elevación:

```
function ejemplo() {  
  if (true) {  
    var mensaje = "visible en toda la función";  
  }  
  
  console.log(mensaje);  
}
```

Con `let` o `const`, `mensaje` solo existiría dentro del `if`. Comprender `var` sigue siendo útil para leer código antiguo, pero rara vez mejora un diseño nuevo.

2.3 Alcance: desde dónde puede verse un nombre

El **alcance léxico** se deduce de la estructura del código. Un bloque interior puede consultar nombres exteriores, pero el exterior no puede consultar los nombres privados del interior.

```
const recargoGeneral = 0.05;

function cotizar(precio) {
  const subtotal = precio * (1 + recargoGeneral);

  if (subtotal > 10_000) {
    const envio = 0;
    return subtotal + envio;
  }

  return subtotal + 800;
}
```

Aquí aparecen varios niveles:

- `recargoGeneral` está en el alcance exterior;
- `precio` y `subtotal` viven en el alcance de la función;
- `envio` solo existe en el bloque del `if`.

Alcance global

Un nombre global puede consultarse desde muchas partes. Esa comodidad tiene un costo: cualquier cambio puede afectar a consumidores lejanos. Preferí módulos y parámetros explícitos para compartir datos.

En navegadores, algunas declaraciones globales históricas se reflejan en `globalThis` o `window`, pero no todas lo hacen del mismo modo. En módulos de JavaScript, los nombres del archivo tienen alcance de módulo y no se convierten automáticamente en propiedades globales.

Alcance de función

Parámetros y declaraciones internas pertenecen a cada llamada:

```
function sumarImpuesto(precio, tasa) {
  const impuesto = precio * tasa;
  return precio + impuesto;
}

sumarImpuesto(100, 0.21);
sumarImpuesto(200, 0.10);
```

Las dos llamadas crean contextos independientes. El `impuesto` de una no se mezcla con el de la otra.

Alcance de bloque

`let`, `const` y `class` respetan las llaves de `if`, `for`, `while`, `switch` y bloques sueltos:

```
for (let indice = 0; indice < 3; indice += 1) {  
  const posicionHumana = indice + 1;  
  console.log(posicionHumana);  
}  
  
// indice y posicionHumana ya no existen aquí
```

2.4 Sombreado: un nombre puede ocultar a otro

Un bloque puede declarar un nombre que ya existe fuera:

```
const estado = "global";  
  
function informar() {  
  const estado = "local";  
  return estado;  
}
```

Esto es válido y a veces útil, pero puede dificultar la lectura si ambos valores participan en la misma operación. Usá nombres diferentes cuando representen conceptos diferentes.

2.5 Elevación y zona muerta temporal

Las declaraciones se procesan antes de ejecutar el bloque, pero no todas quedan utilizables de la misma forma.

```
// console.log(total); // ReferenceError  
const total = 100;
```

Desde el comienzo del bloque hasta la inicialización, `total` está en la **zona muerta temporal**. No se trata de que el nombre sea desconocido: existe, pero todavía no puede leerse.

Con `var`, la declaración se eleva e inicializa con `undefined`:

```
console.log(totalHistorico); // undefined  
var totalHistorico = 100;
```

Eso puede ocultar el uso prematuro de un dato. Las declaraciones de función completas también se elevan, mientras que una función almacenada en `const` sigue las reglas de `const`.

La práctica más clara es declarar cerca del primer uso y no depender de la elevación para organizar el archivo.

2.6 Vida de un valor y recolección de memoria

El alcance indica dónde puede escribirse un nombre; la **vida** indica durante cuánto tiempo el valor sigue siendo alcanzable. Un valor local suele dejar de ser necesario al terminar la función. El recolector de basura puede liberar su memoria cuando ya no existe ninguna referencia accesible.

Una clausura puede prolongar esa vida:

```
function crearSecuencia() {
  let siguiente = 1;

  return () => siguiente++;
}

const obtenerId = crearSecuencia();
obtenerId(); // 1
obtenerId(); // 2
```

Aunque `crearSecuencia` terminó, la función devuelta conserva acceso a `siguiente`. La sección 14.18 desarrolla este mecanismo.

2.7 Identificadores válidos y nombres útiles

Un identificador puede contener letras Unicode, dígitos, `_` y `$`, pero no puede comenzar con un dígito ni coincidir con una palabra reservada.

```
const año = 2026;
const _interno = true;
const $elemento = null;
// const 2curso = "A";
```

Las convenciones más habituales son:

- `camelCase` para variables y funciones;
- `PascalCase` para clases y constructores;
- mayúsculas con guiones bajos para constantes verdaderamente globales e inmutables, como `MAX_INTENTOS`.

Un nombre describe el papel, no solo el tipo:

```
const datos = 15; // ¿qué datos?
const diasHastaVencimiento = 15; // intención visible
```

Para booleanos funcionan bien prefijos como `es`, `tiene`, `puede` y `debe`. Para funciones, un verbo expresa la acción: `calcularTotal`, `buscarAlumno`, `guardarArchivo`.

2.8 Expresiones y sentencias

Una **expresión** produce un valor:

```
2 + 3
precio * cantidad
edad >= 18
usuario?.nombre ?? "Anónimo"
```

Una **sentencia** realiza una acción estructural, como declarar, decidir o repetir:

```
const total = precio * cantidad;

if (total > limite) {
  console.log("Requiere autorización");
}
```

Una expresión puede aparecer dentro de una sentencia. Reconocer esta diferencia ayuda a leer funciones flecha, operadores condicionales y asignaciones.

2.9 Operadores: aridad, precedencia y asociatividad

Un operador puede ser:

- unario: `!activo`, `typeof valor`, `-cantidad`;
- binario: `a + b`, `x === y`;
- ternario: `condicion ? valorA : valorB`.

La **precedencia** indica qué operador se agrupa primero:

```
2 + 3 * 4; // 14, porque * tiene mayor precedencia
```

La **asociatividad** decide cómo se agrupan operadores del mismo nivel:

```
10 - 3 - 2; // (10 - 3) - 2 = 5
2 ** 3 ** 2; // 2 ** (3 ** 2) = 512
```

La asignación se asocia desde la derecha:

```
let a;
let b;
a = b = 10;
```

Estas dos reglas no son un detalle de sintaxis: son el mecanismo que cualquier evaluador de expresiones tiene que reproducir. El apéndice B lo muestra construyendo uno con dos pilas, donde la precedencia se vuelve una comparación entre números y la asociatividad depende de un `>=`.

No conviene convertir estas reglas en un acertijo. Los paréntesis documentan la intención:

```
const total = (precio * cantidad) - descuento;
const habilitado = esCliente && (tieneSaldo || tieneCredito);
```

2.10 Asignaciones abreviadas e incremento

```
saldo += deposito;
stock -= vendido;
factor *= 2;
indice += 1;
```

El incremento prefijo cambia y luego devuelve; el sufijo devuelve el valor anterior y luego cambia:

```
let n = 5;
const anterior = n++; // anterior = 5, n = 6
const actual = ++n;   // n = 7, actual = 7
```

Cuando el valor producido importa, `n += 1` seguido de una lectura explícita suele ser más fácil de entender.

2.11 Un método productivo para revisar variables

Al leer una función, preguntá por cada nombre:

1. ¿Qué representa?
2. ¿Quién puede cambiarlo?
3. ¿Durante cuánto tiempo debe existir?
4. ¿Podría calcularse en lugar de almacenarse?
5. ¿Su unidad está en el nombre o en el contrato?

```
function calcularDemoraHoras(inicioMs, finMs) {
  const MILISEGUNDOS_POR_HORA = 3_600_000;
  const duracionMs = finMs - inicioMs;
  return duracionMs / MILISEGUNDOS_POR_HORA;
}
```

El nombre evita confundir milisegundos con horas y la constante explica el factor de conversión.

2.12 Errores frecuentes

Crear que `const` vuelve profundo e inmutable al objeto

`const` evita reasignar la variable; las propiedades siguen siendo modificables.

Declarar todo al comienzo

Aleja la creación del uso, amplía innecesariamente el alcance y obliga a mantener estados parciales.

Reutilizar una variable para conceptos distintos

```
let resultado = leerEntrada();
resultado = Number(resultado);
resultado = resultado * 1.21;
```

Es más claro distinguir `entrada`, `precio` y `precioConIva`.

Depender de precedencia difícil de reconocer

Aunque el código sea correcto, los paréntesis pueden evitar una revisión lenta o una modificación incorrecta.

2.13 Para recordar

- Una variable vincula un nombre con un valor dentro de un alcance.
- `const` es la elección inicial; `let` expresa evolución; `var` queda para comprender código histórico.
- Alcance y vida no son lo mismo: una clausura puede mantener vivo un dato local.
- Una expresión produce un valor; los operadores lo construyen según precedencia y asociatividad.
- Los nombres, las unidades y el alcance reducido son herramientas de corrección, no decoración.

Capítulo 3. El tipo `boolean`

Idea central

JavaScript permite usar cualquier valor como condición, pero una condición verdadera no implica que su resultado sea el booleano `true`. Para evitar errores hay que distinguir booleanos de valores *truthy* y *falsy*, y recordar que `&&` y `||` seleccionan operandos mediante cortocircuito.

Esta distinción permite usar los operadores lógicos de forma productiva sin confundir validación, ausencia y selección de valores.

3.1 Dos valores para representar una decisión

El tipo `boolean` tiene solo dos valores:

```
true  
false
```

Las comparaciones producen booleanos:

```
10 > 3;           // true  
5 === "5";        // false  
"ana" !== "Ana";  // true
```

Un nombre booleano debería permitir leer la condición como una frase:

```
const esMayorDeEdad = edad >= 18;  
const tieneCupo = inscriptos < capacidad;  
const puedeIngresar = esMayorDeEdad && tieneCupo;
```

Guardar una condición intermedia no es obligatorio, pero puede volver explícita la regla del dominio.

3.2 Una condición no exige un booleano

`if`, `while`, el operador ternario y los operadores lógicos aceptan cualquier valor. Antes de decidir, JavaScript lo interpreta en contexto booleano:

```
if (nombre) {  
  console.log(`Hola, ${nombre}`);  
}
```

`nombre` puede ser un string. El lenguaje aplica conceptualmente `Boolean(nombre)`.

3.3 La lista completa de valores *falsy*

Los valores cuya conversión booleana da `false` son:

```
false
0
-0
0n
""
null
undefined
NaN
```

En navegadores existe una excepción histórica muy especializada: `document.all` también se comporta como *falsy*. No debe utilizarse para lógica de aplicación.

Todos los demás valores son *truthy*. Algunos sorprenden:

```
Boolean("false"); // true: es una cadena no vacía
Boolean("0");     // true
Boolean([]);      // true
Boolean({});      // true
Boolean(-1);      // true
```

Truthy no significa “verdadero según el significado humano”. Solo significa que la conversión definida por el lenguaje produce `true`.

3.4 Conversión explícita

La forma descriptiva es `Boolean`:

```
Boolean(1); // true
Boolean(0); // false
Boolean("ok"); // true
Boolean(""); // false
```

La doble negación produce el mismo resultado:

```
!!"ok"; // true
!!0;    // false
```

La primera `!` convierte e invierte; la segunda vuelve a invertir. `Boolean(valor)` suele comunicar mejor la intención en código didáctico y de negocio; `!!valor` es una abreviatura común que conviene reconocer.

3.5 Negación con `!`

`!` siempre devuelve un booleano:

```
!"texto"; // false
!0;      // true
```

Una condición negativa puede resultar más difícil de leer, especialmente si se combina con nombres negativos:

```
if (!usuario.noEstaBloqueado) {
  // doble negación conceptual
}
```

Preferí nombres afirmativos:

```
function puedeContinuar(usuario) {
  if (usuario.estaBloqueado) return false;
  return true;
}
```

3.6 &&: avanzar mientras los valores sean *truthy*

El operador AND evalúa de izquierda a derecha:

1. si encuentra un valor falsy, lo devuelve y se detiene;
2. si todos son truthy, devuelve el último.

```
true && true;           // true
"Ana" && 20;             // 20
"" && ejecutar();        // ""; ejecutar no se llama
1 && "ok" && { id: 1 }; // { id: 1 }
```

Este comportamiento se llama **cortocircuito**. Permite proteger una operación:

```
const ciudad = usuario && usuario.direccion && usuario.direccion.ciudad;
```

Hoy suele ser más claro usar encadenamiento opcional:

```
const ciudad = usuario?.direccion?.ciudad;
```

&& también puede ejecutar condicionalmente un efecto:

```
estaListo && iniciar();
```

La forma con `if` suele ser preferible si el efecto es importante o si el lector podría confundir el resultado:

```
if (estaListo) iniciar();
```


3.7 ||: buscar el primer valor *truthy*

OR también evalúa de izquierda a derecha:

1. devuelve el primer operando *truthy*;
2. si ninguno lo es, devuelve el último *falsy*.

```
"Ana" || "Anónimo";      // "Ana"
"" || "Anónimo";         // "Anónimo"
0 || false || null;      // null
configuracionA || configuracionB || configuracionBase;
```

Durante años se utilizó para valores predeterminados:

```
const cantidad = entrada || 1;
```

Pero reemplaza cualquier *falsy*. Si `0` es una cantidad válida, el resultado será incorrecto.

3.8 ??: ausencia nula, no falsedad

La coalescencia nula devuelve el operando derecho solo cuando el izquierdo es `null` o `undefined`:

```
0 ?? 1;           // 0
false ?? true;    // false
"" ?? "texto";    // ""
null ?? "texto";  // "texto"
```

Regla práctica:

- usá `||` cuando querés reemplazar cualquier valor *falsy*;
- usá `??` cuando solo querés reemplazar ausencia.

El capítulo 6 vuelve sobre `??` desde el otro lado del problema: qué significa exactamente que un dato falte.

JavaScript exige paréntesis al mezclar directamente `??` con `&&` o `||`, porque la intención puede ser ambigua:

```
const valor = (preferido || alternativo) ?? predeterminado;
```

3.9 Los operadores lógicos conservan valores

La simetría ayuda a recordarlos:

```
a && b → primer falsy; si no existe, último valor
a || b → primer truthy; si no existe, último valor
```

No son funciones que siempre produzcan `true` o `false`; son operadores de control y selección. Si la API necesita un booleano real, convertí el resultado:

```
const tieneNombre = Boolean(usuario.nombre);
```

3.10 Comparaciones y orden

Los operadores relacionales producen booleanos:

```
3 < 10;  
10 >= 10;  
"Ana" < "Luis";
```

Con operandos del mismo tipo numérico, el sentido suele ser evidente. Con strings, la comparación sigue unidades de código, no el orden lingüístico humano. Con tipos diferentes puede haber coerción numérica:

```
"10" < 2; // false; "10" se convierte en 10
```

Convertí entradas antes de comparar y usé `Intl.Collator` para orden alfabético destinado a personas, como muestra la sección 5.14.

3.11 Igualdad estricta y flexible

La igualdad estricta no convierte tipos:

```
5 === 5; // true  
5 === "5"; // false
```

La igualdad flexible aplica reglas de coerción:

```
5 == "5"; // true  
false == 0; // true  
null == undefined; // true
```

La regla general es usar `===` y `!==`. La comparación `valor == null` es un modismo deliberado que detecta `null` o `undefined`, pero `valor === null || valor === undefined` es más explícito para quien está aprendiendo.

`NaN` es diferente de todos los valores, incluso de sí mismo:

```
NaN === NaN; // false  
Number.isNaN(NaN); // true
```

La sección 7.10 explica por qué `==` produce los resultados de arriba y por qué reconstruir esas conversiones cuesta más que evitarlas.

`Object.is` distingue casos especiales:

```
Object.is(NaN, NaN); // true
Object.is(0, -0);    // false
```

3.12 Combinar condiciones

Una regla puede construirse con condiciones pequeñas:

```
const tieneEdadValida = edad >= 18;
const tieneDocumentacion = Boolean(dni && constancia);
const noEstaSuspendido = !estaSuspendido;

const puedeInscribirse =
  tieneEdadValida &&
  tieneDocumentacion &&
  noEstaSuspendido;
```

Los saltos de línea y nombres intermedios permiten revisar cada parte. No es necesario comprimir una regla compleja en una sola expresión.

3.13 Leyes de De Morgan

Al negar una condición compuesta:

```
!(a && b) equivale a !a || !b
!(a || b) equivale a !a && !b
```

Ejemplo:

```
const noPuedeComprar = !(tieneSaldo && hayStock);
const equivalente = !tieneSaldo || !hayStock;
```

Estas leyes ayudan a transformar condiciones y escribir guardas tempranas; la sección 12.7 las usa con ese fin. Son las mismas identidades del álgebra de Boole que aparecen en la sección C.10, aplicadas allá a circuitos y acá a condiciones de un programa.

3.14 Precedencia y paréntesis

En las operaciones habituales:

```
! se evalúa antes que &&
&& se evalúa antes que ||
|| se evalúa antes que el ternario
```

```
const acceso = esAdmin || esEditor && estaActivo;  
// equivale a: esAdmin || (esEditor && estaActivo)
```

Aunque conozcas la precedencia, agregá paréntesis si representan una regla conceptual:

```
const acceso = esAdmin || (esEditor && estaActivo);
```

3.15 Asignación lógica

Los operadores de asignación lógica actualizan solo cuando corresponde:

```
config.tema ||= "claro";           // si tema es falsy  
config.intentos ??= 3;             // si es null o undefined  
sesion.activa &&= verificar();      // si activa es truthy
```

Son útiles, pero conservan las mismas reglas de `||`, `??` y `&&`. Antes de usarlos, decidí qué valores son válidos en el dominio.

3.16 Errores frecuentes

Interpretar una cadena como su significado humano

```
Boolean("false"); // true
```

Para una entrada textual, definí un parser; la sección 7.12 generaliza esta idea:

```
function leerBooleano(texto) {  
  const normalizado = texto.trim().toLowerCase();  
  if (normalizado === "true") return true;  
  if (normalizado === "false") return false;  
  throw new TypeError("Se esperaba true o false");  
}
```

Usar `||` cuando cero, vacío o `false` son válidos

Usá `??` si solo querés cubrir ausencia.

Esperar que `&&` produzca un booleano

`usuario && usuario.nombre` puede devolver `null`, `undefined`, `""` o el nombre. Convertí si el contrato exige `boolean`.

Ocultar demasiado en una expresión

Una cadena larga de cortocircuitos puede ser compacta y difícil de depurar. Separá reglas y efectos.

3.17 Para recordar

- Solo `true` y `false` son booleanos; todos los valores tienen una interpretación booleana.
- La lista de falsy es pequeña y cerrada; arrays y objetos vacíos son truthy.
- `&&` devuelve el primer falsy o el último valor; `||`, el primer truthy o el último.
- `??` trata únicamente `null` y `undefined` como ausencia.
- Las condiciones importantes merecen nombres, paréntesis y un contrato booleano explícito.

Capítulo 4. Los tipos `number` y `bigint`

Idea central

`number` sirve para la mayoría de los cálculos, pero no representa todos los decimales ni todos los enteros con exactitud; `bigint` representa enteros arbitrariamente grandes, aunque no puede mezclarse libremente con `number`. La elección correcta depende de la precisión, el rango y las operaciones del dominio.

Para trabajar con números de forma segura hay que separar cuatro preguntas:

1. ¿cómo se escribe o se convierte la entrada?
2. ¿qué puede representar el tipo?
3. ¿qué operación y precisión exige el problema?
4. ¿cómo se presenta el resultado sin confundir formato con dato?

4.1 Un único tipo para enteros y decimales

En JavaScript, `number` utiliza el formato IEEE 754 de doble precisión. El mismo tipo representa:

```
42
-7
3.14159
1.5e6
Infinity
NaN
```

No existe un tipo entero pequeño separado. Un valor puede ser entero desde el punto de vista matemático y seguir almacenado como `number`.

```
Number.isInteger(42); // true
Number.isInteger(42.5); // false
```

4.2 Literales y bases

```
const decimal = 42;
const binario = 0b101010;
const octal = 0o52;
const hexadecimal = 0x2a;

decimal === binario; // true
binario === hexadecimal; // true
```

La base cambia cómo escribimos el literal, no el valor matemático. Los separadores numéricos mejoran la lectura:

```
const poblacion = 46_000_000;  
const mascara = 0b1111_0000;
```

La notación exponencial expresa escalas grandes o pequeñas:

```
1e3;    // 1000  
2.5e-3; // 0.0025
```

4.3 Operaciones aritméticas

```
10 + 3; // 13  
10 - 3; // 7  
10 * 3; // 30  
10 / 3; // 3.333...  
10 % 3; // 1  
2 ** 3; // 8
```

El operador `%` produce el resto, no un módulo matemático siempre positivo:

```
-5 % 3; // -2
```

Para normalizar un valor cíclico:

```
function modulo(valor, base) {  
    return ((valor % base) + base) % base;  
}  
  
modulo(-1, 7); // 6
```

La exponenciación se asocia desde la derecha:

```
2 ** 3 ** 2; // 2 ** (3 ** 2) = 512
```

JavaScript exige paréntesis para una base unaria negativa:

```
(-2) ** 2; // 4  
-(2 ** 2); // -4
```

4.4 División y valores especiales

Dividir por cero no lanza una excepción en `number`:

```
1 / 0;    // Infinity
-1 / 0;   // -Infinity
0 / 0;    // NaN
```

NaN significa que una operación numérica no produjo un número válido. Sigue perteneciendo al tipo `number`:

```
typeof NaN; // "number"
```

No se compara consigo mismo:

```
NaN === NaN; // false
```

Para detectarlo usá `Number.isNaN`, no la función global `isNaN`, que primero convierte:

```
Number.isNaN(NaN);    // true
Number.isNaN("hola"); // false
isNaN("hola");         // true, porque Number("hola") es NaN
```

`Number.isFinite` comprueba que un valor ya numérico sea finito:

```
Number.isFinite(10);    // true
Number.isFinite(Infinity); // false
Number.isFinite("10");  // false
```

4.5 El cero negativo

IEEE 754 distingue `0` de `-0`, aunque la igualdad estricta los considera iguales:

```
0 === -0;    // true
Object.is(0, -0); // false
1 / 0;       // Infinity
1 / -0;      // -Infinity
```

En la mayoría de las aplicaciones no importa. Puede ser relevante en cálculos que conservan dirección o signo al aproximarse a cero.

4.6 Los decimales no siempre caben exactamente

Muchos decimales finitos en base diez son periódicos en base dos:

```
0.1 + 0.2; // 0.30000000000000004
```

No es un error exclusivo de JavaScript, sino una consecuencia de la representación finita. No compares cálculos decimales esperando siempre igualdad exacta:


```
function casiIguales(a, b, tolerancia = Number.EPSILON) {  
  return Math.abs(a - b) <= tolerancia * Math.max(1, Math.abs(a), Math.abs(b));  
}
```

`Number.EPSILON` es la distancia entre 1 y el siguiente número representable, no una tolerancia universal. El dominio debe definir una tolerancia coherente con su escala.

Dinero

Para dinero simple, suele ser más seguro guardar unidades mínimas enteras:

```
const precioCentavos = 19_99;  
const cantidad = 3;  
const totalCentavos = precioCentavos * cantidad;
```

Esto no resuelve por sí solo porcentajes, redondeos legales, monedas con distinta cantidad de decimales ni importes gigantes. Sistemas financieros serios suelen usar enteros con reglas explícitas o bibliotecas decimales.

4.7 Enteros seguros

`number` representa exactamente enteros entre:

```
Number.MIN_SAFE_INTEGER;  
Number.MAX_SAFE_INTEGER; // 9007199254740991
```

Fuera de ese rango, enteros diferentes pueden volverse indistinguibles:

```
const limite = Number.MAX_SAFE_INTEGER;  
limite + 1 === limite + 2; // true
```

Compróba entradas que deban ser enteros exactos:

```
Number.isSafeInteger(123); // true
```

Identificadores numéricos largos —tarjetas, códigos, documentos— muchas veces no son cantidades. Guardarlos como strings conserva ceros iniciales y evita operaciones sin sentido.

4.8 Convertir a `number`

`Number` exige que toda la cadena represente un número, ignorando espacios exteriores:

```
Number("42");      // 42
Number(" 42 ");     // 42
Number("");         // 0
Number("42px");     // NaN
Number(null);       // 0
Number(undefined); // NaN
```

El caso de la cadena vacía exige validación previa si el campo es obligatorio.

El `+` unario también convierte, pero es menos descriptivo:

```
+"42"; // 42
```

La sección 7.4 detalla el resto de las conversiones numéricas, incluidas las de arrays y objetos.

`parseInt` y `parseFloat` leen un prefijo válido:

```
parseInt("42px", 10); // 42
parseFloat("3.14kg"); // 3.14
```

Esto es útil cuando el contrato acepta unidades posteriores; es peligroso si esperamos que toda la entrada sea numérica. Indicá siempre la base de `parseInt` cuando el formato la conozca:

```
parseInt("1010", 2); // 10
```

Para convertir otras bases a texto:

```
(42).toString(2); // "101010"
(42).toString(16); // "2a"
```

4.9 Redondear y truncar

```
Math.floor(3.8); // 3, hacia -Infinity
Math.ceil(3.2);  // 4, hacia +Infinity
Math.round(3.5); // 4
Math.trunc(3.8); // 3, elimina la fracción
```

Con negativos, `floor` y `trunc` difieren:

```
Math.floor(-3.2); // -4
Math.trunc(-3.2); // -3
```

Otros métodos frecuentes:

```
Math.abs(-10);      // 10
Math.min(3, 7, 1);  // 1
Math.max(3, 7, 1);  // 7
Math.sqrt(81);      // 9
Math.random();      // valor en [0, 1)
```

Para un entero aleatorio uniforme entre `min` y `max`, inclusive:

```
function enteroAleatorio(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

`Math.random` no es apropiado para seguridad, claves ni sorteos auditables.

4.10 Formatear no es calcular

`toFixed` devuelve una cadena:

```
const texto = (12.5).toFixed(2); // "12.50"
typeof texto;                      // "string"
```

No sigas calculando sobre ese resultado sin una conversión deliberada. Para presentar números según idioma y moneda:

```
const formato = new Intl.NumberFormat("es-AR", {
  style: "currency",
  currency: "ARS"
});

formato.format(1234.5);
```

El valor interno sigue siendo numérico; el formato pertenece al borde de salida.

4.11 Operadores bit a bit

Los operadores bitwise de `number` convierten los operandos a enteros de 32 bits con signo:

```
0b1100 & 0b1010; // 0b1000, AND
0b1100 | 0b1010; // 0b1110, OR
0b1100 ^ 0b1010; // 0b0110, XOR
~0;              // -1
```

Los desplazamientos son:

```
1 << 3;    // 8
8 >> 1;     // 4, conserva el signo
-1 >>> 1;  // desplaza relleno con ceros
```

La conversión a 32 bits puede truncar valores grandes y descartar fracciones:

```
5.9 | 0; // 5, pero no debe usarse como sustituto general de Math.trunc
```

Máscaras de opciones

```
const LEER = 1 << 0;
const ESCRIBIR = 1 << 1;
const BORRAR = 1 << 2;

let permisos = LEER | ESCRIBIR; // activar
const puedeLeer = (permisos & LEER) !== 0; // consultar
permisos &= ~ESCRIBIR; // apagar
permisos ^= BORRAR; // alternar
```

Una máscara es compacta e interoperable. En lógica de negocio, un `Set` de nombres puede ser más legible. Elegí según la representación externa y las operaciones necesarias.

El apéndice C explica de dónde vienen estos operadores: por qué `n` bits ofrecen 2^n combinaciones, cómo se representan los enteros con signo y por qué el ancho fijo de 32 bits no es un capricho de JavaScript.

4.12 `bigint`: enteros sin el límite seguro de `number`

Un literal termina en `n`:

```
const poblacionGalactica = 9_007_199_254_740_993n;
const convertido = BigInt("9007199254740993");
```

Soporta aritmética entera:

```
10n + 2n; // 12n
10n * 2n; // 20n
10n ** 3n; // 1000n
10n / 3n; // 3n: descarta la fracción
```

No puede mezclarse directamente con `number`:

```
// 10n + 2; // TypeError
10n + BigInt(2); // 12n
```

La conversión debe ser consciente del rango. Convertir un `bigint` enorme a `number` puede perder precisión; convertir un decimal no entero a `bigint` lanza error.

```
BigInt(42); // 42n
// BigInt(4.2); // RangeError
```

Las comparaciones relacionales permiten mezclar ambos tipos en ciertos casos:

```
10n < 11; // true
```

La igualdad estricta conserva la diferencia:

```
10n === 10; // false
```

`Math` no acepta `bigint`, el operador `>>>` no está disponible y `JSON.stringify` no lo serializa por defecto:

```
// JSON.stringify({ valor: 10n }); // TypeError
```

Un formato externo debe decidir cómo representarlo, normalmente como string, y cómo reconstruirlo.

4.13 Validar una entrada numérica

```
function leerCantidad(texto) {
  const limpio = texto.trim();

  if (limpio === "") {
    throw new TypeError("La cantidad es obligatoria");
  }

  const cantidad = Number(limpio);

  if (!Number.isSafeInteger(cantidad) || cantidad < 0) {
    throw new RangeError("La cantidad debe ser un entero seguro no negativo");
  }

  return cantidad;
}
```

La validación separa presencia, conversión, clase de número y rango del dominio. El resto del programa puede trabajar con una garantía más fuerte.

4.14 Errores frecuentes

- esperar exactitud decimal sin definir tolerancia o redondeo;
- usar `parseInt` cuando el texto completo debería ser válido;
- considerar `NaN` mediante igualdad;
- confundir `toFixed` con una operación numérica;
- guardar identificadores largos como cantidades;
- usar bitwise sobre valores que necesitan más de 32 bits;
- mezclar `number` y `bigint` sin una conversión justificada.

4.15 Para recordar

- `number` es punto flotante de doble precisión: tiene valores especiales, errores decimales y un rango entero seguro.
- Convertir, validar, calcular y formatear son etapas distintas.
- Los operadores bitwise de `number` trabajan sobre enteros de 32 bits.
- `bigint` amplía los enteros, pero pierde fracciones, compatibilidad con `Math` y serialización JSON directa.
- La precisión correcta es una decisión del dominio, no un detalle que pueda dejarse implícito.

Capítulo 5. El tipo `string` y Unicode

Idea central

Un `string` es una secuencia inmutable de unidades UTF-16, no una lista perfecta de letras visibles. Para procesar texto correctamente hay que elegir la unidad adecuada —unidad de código, punto de código o grafema—, normalizar cuando corresponda y separar comparación técnica de orden lingüístico.

Esta idea explica por qué el texto cotidiano parece sencillo hasta que aparecen acentos combinados, alfabetos diferentes o emoji.

5.1 Crear cadenas

JavaScript admite comillas simples, dobles y plantillas literales:

```
const simple = 'Hola';
const doble = "Hola";
const nombre = "Ana";
const plantilla = `Hola, ${nombre}`;
```

Las comillas simples y dobles tienen el mismo comportamiento. Elegí una convención y dejá que una herramienta de formato la aplique.

Las secuencias de escape representan caracteres especiales:

```
const linea = "primera\nsegunda";
const tabulado = "nombre\tedad";
const comillas = "Dijo: \"hola\"";
const barra = "C:\\datos\\archivo.txt";
```

También pueden escribirse puntos de código:

```
const letra = "\u0041";      // A
const emoji = "\u{1F600}";   // 😄; requiere llaves por superar FFFF
```

5.2 Plantillas literales

Las comillas invertidas permiten interpolar expresiones y escribir varias líneas:

```
const producto = "teclado";
const precio = 25000;

const mensaje = `${producto.toUpperCase()}: ${precio}`;
```

Dentro de `${...}` puede aparecer cualquier expresión, pero una plantilla muy compleja pierde legibilidad. Calculá primero las decisiones importantes:

```
const precioFormateado = formatoMoneda.format(precio);
const mensaje = `${producto}: ${precioFormateado}`;
```

Las plantillas preservan saltos y espacios de la fuente:

```
const bloque = `línea 1
línea 2`;
```

5.3 Los strings son inmutables

No se puede cambiar una posición:

```
const palabra = "casa";
palabra[0] = "C"; // no modifica el string
```

Cada método produce un nuevo valor:

```
const original = " JavaScript ";
const limpio = original.trim();
const mayusculas = limpio.toUpperCase();

original; // " JavaScript "
limpio;   // "JavaScript"
mayusculas; // "JAVASCRIPT"
```

Una variable `let` puede vincularse después con otra cadena, pero ninguna de las cadenas fue modificada internamente.

5.4 Longitud y acceso por índice

```
const texto = "JavaScript";
texto.length; // 10
texto[0];     // "J"
texto.at(-1); // "t"
```

Los índices y `length` operan sobre **unidades de código UTF-16**. Para texto ASCII y muchos caracteres del alfabeto latino, una unidad coincide con lo que esperamos. No es una garantía general.

`charAt` es una API histórica:

```
texto.charAt(0); // "J"
texto[99];       // undefined
texto.charAt(99); // ""
```


`codePointAt` obtiene el punto de código que comienza en una posición:

```
"😊".codePointAt(0); // 128512
String.fromCharCode(128512); // "😊"
```

5.5 Extraer partes

`slice(inicio, fin)` usa un fin exclusivo y acepta índices negativos:

```
const lenguaje = "JavaScript";
lenguaje.slice(0, 4); // "Java"
lenguaje.slice(4);    // "Script"
lenguaje.slice(-6);   // "Script"
```

`substring` también usa fin exclusivo, pero transforma negativos en cero e intercambia los argumentos si están invertidos. `substr` es histórica y debe evitarse en código nuevo. `slice` ofrece el modelo más coherente con arrays.

Recordá que cortar por índices UTF-16 puede dividir un carácter compuesto o un emoji.

5.6 Buscar contenido literal

```
const frase = "Aprender JavaScript con práctica";

frase.includes("JavaScript"); // true
frase.startsWith("Aprender"); // true
frase.endsWith("práctica");   // true
frase.indexOf("JavaScript");  // 9
frase.lastIndexOf("a");       // última posición o -1
```

Estas operaciones son preferibles a una expresión regular cuando la búsqueda es exacta. Comunican mejor la intención y no introducen metacaracteres.

5.7 Transformar y normalizar la forma

```
" hola ".trim();
"hola".toUpperCase();
"HOLA".toLowerCase();
"7".padStart(3, "0"); // "007"
"x".repeat(4);         // "xxxx"
```

`trimStart` y `trimEnd` actúan sobre un solo extremo. Los cambios de mayúsculas pueden depender del idioma:

```
"istanbul".toLocaleUpperCase("tr");
```

No uses `toLowerCase` como sustituto automático de todas las reglas de identidad. Nombres de usuario, rutas, correos y textos humanos tienen contratos diferentes.

5.8 Reemplazar y separar

```
"uno dos uno".replace("uno", "1");    // "1 dos uno"
"uno dos uno".replaceAll("uno", "1"); // "1 dos 1"
"a,b,c".split(",");                  // ["a", "b", "c"]
```

`replace` acepta strings o expresiones regulares y también una función de reemplazo. `split` no es un parser de formatos complejos: un CSV con comillas y comas internas necesita reglas adicionales o una biblioteca.

Para unir un array:

```
["Ana", "Luis", "Sofía"].join(" | ");
```

5.9 Concatenación y coerción

El operador `+` suma números o concatena si aparece una cadena, y la evaluación ocurre de izquierda a derecha:

```
1 + 2;          // 3
"1" + 2;        // "12"
1 + 2 + "3";    // "33"
"1" + 2 + 3;    // "123"
```

Para armar un mensaje, una plantilla evita depender de esa regla:

```
const mensaje = `Total: ${1 + 2}`;
```

La conversión explícita es `String(valor)`, que también acepta `null` y `undefined`, a diferencia de `valor.toString()`:

```
String(42);      // "42"
String(null);    // "null"
String(undefined); // "undefined"
```

La sección 7.8 explica el algoritmo completo de `+` y por qué es el único operador aritmético que puede producir texto.

5.10 Tres unidades diferentes de texto

Consideremos:

```
const emoji = "😄";
```

Visualmente es un símbolo, Unicode le asigna un punto de código y UTF-16 lo almacena con dos unidades sustitutas:

```
emoji.length;           // 2 unidades UTF-16  
[...emoji].length;      // 1 punto de código
```

Ahora consideremos una á formada por a y un acento combinante:

```
const combinada = "a\u0301";  
[...combinada].length; // 2 puntos de código
```

Una persona ve una sola letra. Por lo tanto:

```
unidad UTF-16 ≠ punto de código ≠ grafema visible
```

Un emoji familiar también reúne varios puntos de código mediante selectores y uniones:

```
const familia = "👨👩👧";  
familia.length > 1;           // true  
[...familia].length > 1;      // true
```

5.11 Recorrer puntos de código

Un bucle por índice puede separar pares sustitutos:

```
for (let i = 0; i < "A😄B".length; i += 1) {  
  console.log("A😄B"[i]);  
}
```

`for...of` usa el iterador de strings y conserva puntos de código:

```
for (const caracter of "A😄B") {  
  console.log(caracter); // A, 😄, B  
}
```

El spread y `Array.from` aplican el mismo protocolo:

```
[... "A😄B"];           // ["A", "😄", "B"]  
Array.from("A😄B");     // ["A", "😄", "B"]
```

5.12 Recorrer grafemas con `Intl.Segmenter`

Cuando la unidad es lo que percibe la persona —por ejemplo, limitar un nombre visible—, usá segmentación de grafemas:

```
const segmentador = new Intl.Segmenter("es", {
  granularity: "grapheme"
});

function grafemas(texto) {
  return [...segmentador.segment(texto)]
    .map(parte => parte.segment);
}

grafemas("a\u0301").length; // 2
```

`Intl.Segmenter` también puede segmentar palabras y oraciones según reglas lingüísticas. Su disponibilidad depende del entorno y de los datos internacionales incluidos. Es una alternativa más fiel que `\b` de las expresiones regulares, según discute la sección 18.12.

5.13 Normalización Unicode

Dos cadenas pueden verse iguales y contener secuencias distintas:

```
const compuesta = "á";
const descompuesta = "a\u0301";

compuesta === descompuesta; // false
```

Normalizar las lleva a una forma acordada:

```
compuesta.normalize("NFC") === descompuesta.normalize("NFC"); // true
```

El apéndice A muestra qué ocurre cuando estas dos formas llegan a un tokenizador: producen bytes distintos y, por lo tanto, tokens distintos.

Formas principales:

- `NFC`: composición canónica, frecuente para almacenar y comparar;
- `NFD`: descomposición canónica;
- `NFKC` y `NFKD`: incluyen equivalencias de compatibilidad y pueden cambiar distinciones visuales significativas.

Normalizar es una decisión del dominio. No elimina automáticamente acentos ni resuelve todas las equivalencias de identidad.

5.14 Comparación técnica y comparación humana

La igualdad estricta compara secuencias exactas de unidades:

```
"Ana" === "ana"; // false
```

Los operadores `<` y `>` producen un orden lexicográfico por unidades de código:

```
["2", "10"].toSorted(); // ["10", "2"]
```

Para ordenar números almacenados como texto, convertí o proporcioné un comparador numérico:

```
["2", "10"].toSorted((a, b) => Number(a) - Number(b));
```

Para texto humano:

```
const collator = new Intl.Collator("es", {
  sensitivity: "base",
  numeric: true
});

const ordenados = nombres.toSorted(collator.compare);
```

`sensitivity: "base"` puede ignorar diferencias de mayúsculas y acentos para comparar, según el idioma. No transforma el texto almacenado.

5.15 Tagged templates

Una función puede recibir las partes literales y los valores interpolados:

```
function inspeccionar(partes, ...valores) {
  return { partes, valores };
}

const usuario = "Ana";
const resultado = inspeccionar`Hola ${usuario}, tenés ${3} mensajes`;
```

Las plantillas etiquetadas permiten construir DSL, sanitizar salidas o parametrizar consultas. La seguridad depende de que la función trate cada contexto correctamente; no alcanza con “escapar todo” de una sola manera. La biblioteca del capítulo 21 usa una plantilla etiquetada, `html`, para describir interfaces que se actualizan solas.

`String.raw` devuelve las barras como fueron escritas:

```
String.raw`C:\usuarios\ana`; // "C:\\usuarios\\ana" como contenido visible escapado
```

Es útil para ejemplos de regex y rutas, aunque `node:path` sigue siendo la herramienta adecuada para construir rutas reales.

5.16 Errores frecuentes

- usar `length` como cantidad universal de caracteres visibles;
- cortar por índice y dejar un emoji o acento partido;

- ordenar texto humano con `<` o `sort()` sin comparador;
- convertir a minúsculas sin decidir idioma y reglas de identidad;
- concatenar números y strings esperando suma;
- intentar interpretar CSV, HTML o lenguajes anidados con operaciones de texto demasiado simples;
- normalizar o quitar acentos sin conservar el original cuando tiene valor.

5.17 Para recordar

- Los strings son inmutables y sus índices trabajan con UTF-16.
- `for...of` recorre puntos de código; `Intl.Segmenter` puede recorrer grafemas.
- Normalización, cambio de mayúsculas y comparación lingüística resuelven problemas diferentes.
- Las plantillas literales expresan interpolación; las tagged templates permiten crear protocolos de procesamiento.
- Al procesar texto, primero definí qué unidad y qué equivalencia necesita el usuario.

Capítulo 6. `undefined`, `null` y la ausencia de datos

Idea central

`undefined` suele indicar que JavaScript o una operación todavía no produjo un valor; `null` suele expresar que la aplicación decidió representar una ausencia. Ninguno explica por sí solo por qué falta el dato: un programa productivo define el significado, lo valida en los límites y evita propagar estados ambiguos.

La ausencia aparece en propiedades opcionales, búsquedas sin resultado, parámetros omitidos, formularios incompletos y datos externos. Tratarla como parte del contrato es más seguro que esperar a que una operación falle lejos del origen.

6.1 `undefined`: falta de valor asignado o producido

JavaScript utiliza `undefined` en varias situaciones:

```
let pendiente;
pendiente; // undefined

const persona = {};
persona.telefono; // undefined

function saludar(nombre) {
  return nombre;
}

saludar(); // undefined
```

Una función sin `return` también devuelve `undefined`:

```
function registrar(mensaje) {
  console.log(mensaje);
}

const resultado = registrar("inicio");
// resultado es undefined
```

Esto distingue un efecto de un resultado reutilizable. Si una función promete producir un valor, todos sus caminos deberían respetar ese contrato o señalar claramente la ausencia.

6.2 `null`: ausencia intencional

`null` se escribe explícitamente:

```
const usuarioSeleccionado = null;
```

Puede significar “todavía no se seleccionó”, “la relación fue eliminada” o “la búsqueda concluyó y no encontró nada”. El significado pertenece a la aplicación.

```
function buscarPorLegajo(alumnos, legajo) {  
  return alumnos.find(alumno => alumno.legajo === legajo) ?? null;  
}
```

Aquí `null` no representa un error: es un resultado posible y documentado.

6.3 Una peculiaridad histórica de `typeof`

```
typeof undefined; // "undefined"  
typeof null;      // "object"
```

El resultado de `typeof null` es una incompatibilidad histórica del lenguaje. No significa que `null` sea un objeto utilizable. Para detectarlo:

```
valor === null;
```

Para `undefined`:

```
valor === undefined;  
typeof valor === "undefined";
```

La forma con `typeof` también puede consultar un identificador no declarado sin lanzar `ReferenceError`, aunque no debería usarse para ocultar dependencias globales.

6.4 Ausencia no es falsedad

`null` y `undefined` son falsy, pero comparten esa condición con valores válidos como `0`, `false` y `""`:

```
if (!cantidad) {  
  // entra tanto con ausencia como con cero  
}
```

Si cero es válido, comprobá el estado que realmente importa:

```
if (cantidad === null || cantidad === undefined) {  
  // falta la cantidad  
}
```


La forma `valor == null` detecta exactamente `null` o `undefined` mediante una regla particular de igualdad flexible:

```
null == undefined; // true
0 == null;         // false
"" == null;        // false
```

Puede ser un modismo útil en equipos que lo acuerdan, pero la versión explícita enseña mejor el contrato.

6.5 Coalescencia nula

`??` usa el valor derecho solo ante `null` o `undefined`:

```
const cantidad = entrada.cantidad ?? 1;

0 ?? 1;          // 0
false ?? true;   // false
"" ?? "N/D";     // ""
null ?? "N/D";   // "N/D"
```

Esto contrasta con `||`, que reemplaza cualquier falsy:

```
0 || 1; // 1
```

La asignación nula inicializa solo si falta:

```
config.intentos ??= 3;
```

6.6 Encadenamiento opcional

`?.` detiene una cadena de acceso cuando el valor anterior es nulo:

```
const ciudad = usuario?.direccion?.ciudad;
```

Si `usuario` o `direccion` es `null/undefined`, el resultado es `undefined`. No detiene ante otros falsy:

```
const longitud = "".length; // 0
```

Puede aplicarse a propiedades, índices y llamadas:

```
config.temas?.[0];
observador?.(evento);
```

Usalo solo cuando la ausencia sea válida. Si una propiedad es obligatoria, el encadenamiento opcional puede ocultar un dato roto y retrasar el error:

```
// Si cliente es obligatorio, esto podría silenciar el problema.  
const nombre = pedido.cliente?.nombre;
```

En ese caso, validá el contrato al ingresar el pedido.

6.7 Parámetros predeterminados

Un valor predeterminado de parámetro se aplica ante `undefined`, no ante `null`:

```
function saludar(nombre = "Anónimo") {  
  return `Hola, ${nombre}`;  
}  
  
saludar();           // "Hola, Anónimo"  
saludar(undefined); // "Hola, Anónimo"  
saludar(null);       // "Hola, null"
```

Si `null` también significa ausencia, normalízalo explícitamente:

```
function saludar(nombre) {  
  const visible = nombre ?? "Anónimo";  
  return `Hola, ${visible}`;  
}
```

Los valores predeterminados en desestructuración siguen la misma regla:

```
const { tema = "claro" } = { tema: undefined }; // "claro"  
const { idioma = "es" } = { idioma: null };     // null
```

6.8 Propiedad ausente y propiedad con `undefined`

Estos objetos devuelven lo mismo al acceder, pero no tienen la misma estructura:

```
const ausente = {};  
const presente = { valor: undefined };  
  
ausente.valor; // undefined  
presente.valor; // undefined  
  
Object.hasOwn(ausente, "valor"); // false  
Object.hasOwn(presente, "valor"); // true
```

Esto importa al aplicar parches, validar campos o recorrer claves.

`in` también considera propiedades heredadas:

```
"toString" in {}; // true
Object.hasOwn({}, "toString"); // false
```

6.9 Huecos en arrays

La distinción anterior reaparece en los arrays. Una posición puede no existir, y eso no es lo mismo que contener `undefined`:

```
const conHueco = [1, , 3];
const conUndefined = [1, undefined, 3];

conHueco[1]; // undefined
conUndefined[1]; // undefined
1 in conHueco; // false
1 in conUndefined; // true
```

Algunos métodos saltan los huecos, otros los materializan. Para datos de aplicación, preferí arrays densos y representé la ausencia deliberadamente; la sección 9.4 detalla el comportamiento de cada método.

6.10 Búsquedas y resultados opcionales

APIs diferentes usan convenciones distintas:

```
[10, 20].find(valor => valor > 50); // undefined
[10, 20].indexOf(50); // -1
new Map().get("clave"); // undefined
"texto".match(/numero/); // null
```

No supongas una convención universal. Lee el contrato y convertí el resultado a la forma que usa tu dominio:

```
function buscarConfiguracion(mapa, clave) {
  if (!mapa.has(clave)) return { encontrada: false };
  return { encontrada: true, valor: mapa.get(clave) };
}
```

Este diseño distingue “no existe” de “existe y su valor es `undefined`”.

6.11 JSON, red y bases de datos

JSON tiene `null`, pero no `undefined`:

```
JSON.stringify({ a: null, b: undefined }); // '{"a":null}'
JSON.stringify([null, undefined]); // '[null,null]'
```

En objetos, una propiedad `undefined` se omite; en arrays se serializa como `null`. Esto puede cambiar el significado de un parche:

- propiedad ausente: “no modificar”;
- propiedad con `null`: “borrar el valor”;
- propiedad con dato: “reemplazar”.

Cada API debe documentar su convención. Las bases de datos también tienen semánticas propias para `NULL`; no deben trasladarse automáticamente a JavaScript sin decidir el contrato. La sección 10.18 enumera el resto de lo que JSON no conserva.

6.12 Estados explícitos cuando la ausencia no alcanza

Una única variable con `null` puede mezclar demasiados significados:

```
let datos = null;  
// ¿todavía no se cargaron, no existen o falló la carga?
```

Un estado etiquetado conserva la diferencia:

```
let consulta = { estado: "pendiente" };  
  
consulta = { estado: "lista", datos: [] };  
consulta = { estado: "sin-resultados" };  
consulta = { estado: "error", error };
```

Este patrón evita booleanos paralelos contradictorios y permite manejar cada caso de forma exhaustiva.

6.13 Validar y normalizar en el borde

```
function normalizarPerfil(entrada) {  
  if (!entrada || typeof entrada !== "object") {  
    throw new TypeError("Se esperaba un perfil");  
  }  
  
  const nombre = entrada.nombre?.trim();  
  if (!nombre) throw new TypeError("El nombre es obligatorio");  
  
  const telefono = entrada.telefono == null  
    ? null  
    : String(entrada.telefono).trim();  
  
  return { nombre, telefono };  
}
```

Después de normalizar, el núcleo de la aplicación puede asumir que `nombre` es no vacío y `telefono` es string o `null`, sin repetir comprobaciones ambiguas.

6.14 Errores frecuentes

- usar `if (!valor)` para detectar solo ausencia;
- esperar que un parámetro predeterminado reemplace `null`;
- tratar `typeof null === "object"` como una clasificación útil;
- encadenar `?.` sobre propiedades obligatorias y ocultar datos rotos;
- no distinguir propiedad ausente de propiedad presente con `undefined`;
- serializar `undefined` esperando que JSON lo conserve;
- usar `null` para estados diferentes sin una etiqueta adicional.

6.15 Para recordar

- `undefined` suele surgir por falta de asignación o resultado; `null` suele ser una ausencia elegida por la aplicación.
- Ambos son falsy, pero no deben confundirse con `0`, `false` o `""`.
- `??`, `?.` y los valores predeterminados tienen reglas precisas y diferentes.
- Propiedad ausente, propiedad `undefined` y hueco de array no son exactamente lo mismo.
- Cuando existen varios tipos de ausencia, un estado etiquetado expresa mejor el proceso.

Capítulo 7. Conversión y coerción de tipos

Idea central

Los datos externos deben convertirse y validarse explícitamente una vez; dentro del programa conviene operar con tipos estables y evitar que la coerción decida reglas del dominio. La coerción automática es parte de JavaScript y puede ser útil, pero resulta segura solo cuando conocemos qué conversión aplica cada contexto.

7.1 Conversión explícita y coerción implícita

Una conversión explícita aparece en el código:

```
const cantidad = Number(textoCantidad);
const etiqueta = String(numeroPedido);
const presente = Boolean(valor);
```

Una coerción ocurre porque una operación necesita otro tipo:

```
"Total: " + 10; // convierte 10 a string
"6" - 1;        // convierte "6" a number
if (valor) {}   // convierte valor a boolean
```

La coerción no es aleatoria. Sigue algoritmos definidos por ECMAScript. El problema práctico aparece cuando el lector o la aplicación esperaban otro significado.

7.2 Los bordes del sistema

Formularios, variables de entorno, argumentos de terminal, CSV y muchos atributos HTML llegan como texto. JSON conserva más tipos, pero sigue siendo una entrada no confiable.

```
entrada externa → normalización → conversión → validación → dato interno
```

Ejemplo:

```
function leerPuerto(texto) {
  const limpio = texto.trim();

  if (!/^\d+$/u.test(limpio)) {
    throw new TypeError("El puerto debe contener solo dígitos");
  }

  const puerto = Number(limpio);

  if (!Number.isInteger(puerto) || puerto < 1 || puerto > 65535) {
    throw new RangeError("El puerto debe estar entre 1 y 65535");
  }

  return puerto;
}
```

La regex decide la forma aceptada; `Number` convierte; la condición valida el rango del dominio.

7.3 Convertir a string

```
String(42);           // "42"
String(true);         // "true"
String(null);         // "null"
String(undefined);    // "undefined"
String(10n);          // "10"
String(Symbol("x"));  // "Symbol(x)"
```

Las plantillas y la concatenación también convierten muchos valores:

```
`${42}`;           // "42"
"valor=" + 42;     // "valor=42"
```

`valor.toString()` no funciona con `null` o `undefined` y puede aceptar una base para números:

```
(255).toString(16); // "ff"
```

Los objetos usan protocolos de conversión que desarrolla la sección 8.10: `Symbol.toPrimitive`, `valueOf` y `toString`.

7.4 Convertir a número

`Number` intenta convertir el valor completo:

```

Number("42");      // 42
Number(" 42 ");    // 42
Number("");        // 0
Number(" ");       // 0
Number("3.14");    // 3.14
Number("3.14px");  // NaN
Number(true);      // 1
Number(false);     // 0
Number(null);      // 0
Number(undefined); // NaN

```

Arrays y objetos producen resultados menos intuitivos debido a la conversión previa a primitivo:

```

Number([]);      // 0, porque [] se convierte en ""
Number([5]);     // 5, porque [5] se convierte en "5"
Number([1, 2]);  // NaN
Number({});      // NaN

```

Estos casos sirven para comprender el lenguaje, no para diseñar validaciones.

El `+` unario aplica conversión numérica:

```

+"5"; // 5

```

No acepta `bigint` y es menos explícito que `Number`.

`parseInt` y `parseFloat`

```

parseInt("42px", 10); // 42
parseInt("101", 2);   // 5
parseFloat("3.5kg");  // 3.5

```

Se detienen al encontrar un carácter inválido. Eso puede ser correcto para un formato deliberado y peligroso para una entrada que debería ser completamente numérica.

```

Number("12abc");      // NaN
parseInt("12abc", 10); // 12

```

7.5 Convertir a booleano

La sección 3.3 enumera los ocho valores falsy. `Boolean` devuelve `false` exactamente para ellos:


```
Boolean(0);           // false
Boolean("");          // false
Boolean(null);        // false
Boolean(undefined);   // false
Boolean(NaN);         // false
Boolean("false");     // true
Boolean([]);          // true
```

No uses `Boolean(texto)` para interpretar palabras como "sí", "no", "true" o "false". Eso exige un parser del dominio.

7.6 Convertir a `bigint`

```
BigInt("9007199254740993"); // 9007199254740993n
BigInt(42);                  // 42n
BigInt(true);                // 1n
```

No acepta decimales no enteros ni texto decimal con punto:

```
// BigInt(3.5); // RangeError
// BigInt("3.5"); // SyntaxError
```

Convertir primero un entero grande a `number` puede perder precisión antes de llegar a `BigInt`. Convertí directamente desde el string.

7.7 Contextos de coerción

Una forma productiva de entender el lenguaje es reconocer el tipo que exige cada contexto.

Contexto booleano

```
if (valor) {}
while (valor) {}
valor ? a : b;
!valor;
valor && otro;
valor || otro;
```

Contexto numérico

La mayoría de los operadores aritméticos convierten a número:

```
"8" - "3"; // 5
"8" * "3"; // 24
"8" / 2; // 4
"8" ** 2; // 64
```

Si aparece un `bigint`, ambos operandos deben terminar siendo `bigint` para la aritmética correspondiente.

Contexto de string

Plantillas, `String` y concatenación cuando `+` elige texto:

```
`id:${123}`; // "id:123"
```

Contexto de propiedad

Las claves comunes de objeto se convierten en strings; los símbolos permanecen símbolos:

```
const objeto = {};  
objeto[10] = "diez";  
Object.keys(objeto); // ["10"]
```

7.8 La regla especial de `+`

`+` puede sumar o concatenar. De forma simplificada:

1. convierte objetos a primitivos;
2. si alguno de los primitivos es string, concatena como strings;
3. en caso contrario, realiza suma numérica compatible.

```
1 + 2; // 3  
"1" + 2; // "12"  
1 + "2"; // "12"  
1 + 2 + "3"; // "33"  
"1" + 2 + 3; // "123"
```

La evaluación es de izquierda a derecha. Los paréntesis cambian el primer resultado:

```
"Total: " + (1 + 2); // "Total: 3"
```

`Symbol` no se concatena implícitamente:

```
const id = Symbol("id");  
String(id); // "Symbol(id)"  
// "id=" + id; // TypeError
```

7.9 Comparaciones relacionales

Con dos strings, `<` compara lexicográficamente:

```
"20" < "3"; // true
```

Con tipos diferentes, normalmente hay conversión numérica:

```
"20" < 3; // false
```

Los resultados con `NaN` son falsos:

```
NaN < 3; // false  
NaN >= 3; // false
```

Convertí y validá antes de ordenar valores que llegan como strings.

7.10 Igualdad flexible: por qué sorprende

`==` intenta acercar tipos mediante reglas específicas:

```
0 == false; // true  
"" == false; // true  
"0" == false; // true  
null == undefined; // true  
[] == ""; // true  
[0] == 0; // true
```

Cada resultado puede explicarse, pero obliga a reconstruir varias conversiones. La igualdad estricta evita esa fase:

```
0 === false; // false  
"0" === false; // false
```

Usá `===` y `!==` por defecto. Si elegís `==` para un caso específico, documentá la intención y mantené la expresión local.

7.11 Conversión de objetos a primitivos

Cuando una operación necesita un primitivo, un objeto puede participar mediante:

1. `Symbol.toPrimitive`, si existe;
2. `valueOf` y `toString`, en un orden que depende de la sugerencia de tipo.

```
const temperatura = {
  celsius: 25,
  valueOf() {
    return this.celsius;
  },
  toString() {
    return `${this.celsius} °C`;
  }
};

Number(temperatura); // 25
String(temperatura); // "25 °C"
```

Personalizar coerción puede hacer una API elegante o misteriosa. Métodos explícitos como `aCentavos()` y `formatear()` suelen ser mejores cuando existen varias interpretaciones legítimas. La sección 8.10 muestra el mismo ejemplo desde el lado del protocolo.

7.12 Diseñar parsers del dominio

Una conversión del lenguaje no decide formatos culturales:

```
Number("1,5"); // NaN
```

Si la aplicación acepta coma decimal, debe normalizar bajo un contrato claro y rechazar mezclas ambiguas:

```
function leerDecimales(texto) {
  const limpio = texto.trim();

  if (!/^([+-]?[0-9]+(?:[.,][0-9]*)?)/u.test(limpio)) {
    throw new TypeError("Formato decimal inválido");
  }

  const numero = Number(limpio.replace(",", "."));
  if (!Number.isFinite(numero)) {
    throw new RangeError("Número fuera de rango");
  }

  return numero;
}
```

El separador de miles requiere aún más cuidado: `"1.234"` puede significar mil doscientos treinta y cuatro o uno con fracción.

7.13 Un pipeline de entrada completo

```
function normalizarProducto(entrada) {
  if (!entrada || typeof entrada !== "object") {
    throw new TypeError("Se esperaba un producto");
  }

  const nombre = String(entrada.nombre ?? "").trim();
  if (nombre === "") throw new TypeError("Falta el nombre");

  const precio = Number(entrada.precio);
  if (!Number.isFinite(precio) || precio < 0) {
    throw new RangeError("Precio inválido");
  }

  const stock = Number(entrada.stock);
  if (!Number.isSafeInteger(stock) || stock < 0) {
    throw new RangeError("Stock inválido");
  }

  const activo = leerBooleano(String(entrada.activo));

  return { nombre, precio, stock, activo };
}

function leerBooleano(texto) {
  const valor = texto.trim().toLowerCase();
  if (valor === "true") return true;
  if (valor === "false") return false;
  throw new TypeError("Activo debe ser true o false");
}
```

Después de este límite, ningún cálculo necesita adivinar tipos.

7.14 Errores frecuentes

- convertir una cadena vacía con `Number` y aceptar cero sin querer;
- usar `parseInt` y tolerar basura posterior;
- interpretar `"false"` con `Boolean`;
- comparar números almacenados como strings;
- depender de `+` sin saber si suma o concatena;
- convertir un entero grande a `number` antes de `BigInt`;
- personalizar la coerción de un objeto cuando métodos nombrados serían más claros.

7.15 Para recordar

- Convertir cambia la representación; validar comprueba el contrato. Son pasos diferentes.
- La coerción depende del contexto: booleano, numérico, string o clave de propiedad.

- `+` es especial porque puede sumar o concatenar.
- La igualdad flexible añade reglas de conversión; la estricta conserva los tipos.
- Un sistema productivo normaliza entradas una vez y trabaja internamente con datos estables.

Capítulo 8. El tipo `symbol` y los protocolos del lenguaje

Idea central

Un `symbol` representa una identidad única; usado como clave, evita colisiones, y mediante los símbolos conocidos permite que un objeto participe en protocolos internos del lenguaje. No es una cadena especial ni un mecanismo de privacidad.

Hay dos usos que conviene separar:

1. símbolos creados por la aplicación para obtener claves únicas;
2. símbolos conocidos definidos por JavaScript para personalizar iteración, coerción y otras operaciones.

8.1 Crear una identidad

```
const id = Symbol();  
const idConDescripcion = Symbol("id de alumno");
```

Cada llamada produce un valor diferente:

```
Symbol("id") === Symbol("id"); // false
```

La descripción es información de depuración, no identidad:

```
const clave = Symbol("interno");  
clave.description; // "interno"
```

No se crea con `new`:

```
// new Symbol(); // TypeError
```

`typeof` reconoce el tipo:

```
typeof clave; // "symbol"
```

8.2 Símbolo y string son espacios de claves distintos

Un objeto puede tener una propiedad string y otra symbol con descripciones parecidas:

```
const ID = Symbol("id");

const alumno = {
  id: "visible",
  [ID]: 123
};

alumno.id; // "visible"
alumno[ID]; // 123
```

Esto evita que dos módulos elijan accidentalmente el mismo nombre textual:

```
const METADATOS_MODULO_A = Symbol("metadatos");
const METADATOS_MODULO_B = Symbol("metadatos");
```

Aunque compartan descripción, no colisionan.

8.3 Enumeración y reflexión

Las claves symbol no aparecen en enumeraciones habituales:

```
Object.keys(alumno); // ["id"]
Object.entries(alumno); // [["id", "visible"]]
JSON.stringify(alumno); // '{"id":"visible"}'
```

Pueden descubrirse:

```
Object.getOwnPropertySymbols(alumno); // [ID]
Reflect.ownKeys(alumno); // ["id", ID]
```

Por eso un símbolo reduce interferencias accidentales, pero no oculta información frente a quien posee el objeto. Para privacidad real dentro del lenguaje usá cierres o campos privados de clase.

El spread y `Object.assign` sí copian propiedades symbol propias y enumerables:

```
const copia = { ...alumno };
copia[ID]; // 123
```

8.4 El registro global con `Symbol.for`

`Symbol.for(clave)` consulta un registro y reutiliza la identidad asociada:

```
const uno = Symbol.for("universidad.usuario");
const dos = Symbol.for("universidad.usuario");

uno === dos; // true
```


`Symbol.keyFor` recupera la clave de un símbolo registrado:

```
Symbol.keyFor(unos); // "universidad.usuario"
Symbol.keyFor(Symbol("local")); // undefined
```

Elegí según la coordinación necesaria:

- `Symbol()` para identidad local y aislada;
- `Symbol.for()` para compartir identidad mediante un nombre acordado.

El registro amplía el alcance del acuerdo. Usá prefijos con contexto para evitar que bibliotecas distintas adopten la misma clave por accidente.

8.5 Conversión y límites

La conversión explícita a string funciona:

```
const marca = Symbol("marca");
String(marca); // "Symbol(marca)"
marca.toString(); // "Symbol(marca)"
```

La concatenación implícita lanza error:

```
// "clave=" + marca; // TypeError
```

Un símbolo no se convierte a número y no tiene un orden relacional significativo:

```
// Number(marca); // TypeError
```

Estas restricciones protegen su función de identidad.

8.6 Símbolos conocidos: acuerdos con el lenguaje

JavaScript expone símbolos estáticos como `Symbol.iterator`. No se crean para una aplicación concreta: son claves compartidas que el motor consulta durante determinadas operaciones.

Un objeto implementa un **protocolo** cuando proporciona la propiedad esperada con el contrato esperado.

8.7 `Symbol.iterator`: hacer un objeto iterable

`for...of`, `spread`, desestructuración y constructores como `Array.from` consumen iterables.

El protocolo requiere un método que devuelva un iterador. El iterador tiene `next()`, que devuelve objetos `{ value, done }`.

```
const rango = {
  desde: 3,
  hasta: 5,

  [Symbol.iterator]() {
    let actual = this.desde;
    const fin = this.hasta;

    return {
      next() {
        if (actual <= fin) {
          return { value: actual++, done: false };
        }

        return { value: undefined, done: true };
      }
    };
  }
};

[...rango]; // [3, 4, 5]
```

Un generador, que presenta la sección 14.24, implementa el mismo contrato con menos infraestructura:

```
const rangoSimple = {
  desde: 3,
  hasta: 5,

  *[Symbol.iterator]() {
    for (let n = this.desde; n <= this.hasta; n += 1) {
      yield n;
    }
  }
};
```

Este protocolo es lo que hace funcionar a `for...of`, que presenta la sección 12.15. Arrays, strings, `Map`, `Set` y muchos objetos de plataforma ya son iterables. Un objeto común no lo es:

```
// for (const valor of { a: 1 }) {} // TypeError
```

Se puede recorrer `Object.entries(objeto)` porque ese método produce un array iterable.

8.8 Un iterable puede ofrecer recorridos diferentes

La elección de lo que se produce forma parte de la API:

```

class ListaDeAlumnos {
  #alumnos = [];

  agregar(alumno) {
    this.#alumnos.push(alumno);
  }

  *[Symbol.iterator]() {
    yield* this.#alumnos;
  }
}

```

Podría producir alumnos, ids o pares. El nombre y la documentación deben hacer previsible el recorrido predeterminado.

8.9 Symbol.asyncIterator: valores a través del tiempo

Un iterable asincrónico devuelve promesas o usa un generador asincrónico:

```

const paginas = {
  async *[Symbol.asyncIterator]() {
    let pagina = 1;

    while (true) {
      const resultado = await cargarPagina(pagina);
      if (resultado.items.length === 0) return;

      yield resultado.items;
      pagina += 1;
    }
  }
};

for await (const items of paginas) {
  procesar(items);
}

```

El protocolo expresa una secuencia cuyos elementos requieren espera, como páginas remotas, archivos por fragmentos o eventos. El capítulo 17 explica la asincronía que hay detrás y la sección 19.26 aplica `for await...of` a leer un archivo grande por líneas.

8.10 `Symbol.toPrimitive`: decidir una coerción

```
const dinero = {
  centavos: 1250,

  [Symbol.toPrimitive](hint) {
    if (hint === "number") return this.centavos;
    return `$$${(this.centavos / 100).toFixed(2)}`;
  }
};

Number(dinero); // 1250
String(dinero); // "$12.50"
```

El método recibe una sugerencia:

- "number" para contextos numéricos;
- "string" para conversión explícita a string;
- "default" para operaciones como `+` o igualdad flexible.

Debe devolver un primitivo; devolver un objeto causa `TypeError`.

La coerción implícita debe ser inequívoca. Para `dinero`, devolver centavos ante `Number` puede sorprender a quien esperaba unidades monetarias. Métodos `aCentavos()` y `formatear()` suelen ser más explícitos.

8.11 `Symbol.toStringTag`: describir una clase de objeto

```
const registro = {
  get [Symbol.toStringTag]() {
    return "RegistroDeAlumnos";
  }
};

Object.prototype.toString.call(registro);
// "[object RegistroDeAlumnos]"
```

Muchas APIs integradas usan etiquetas como `Map`, `Set` o `ArrayBuffer`. Es una ayuda descriptiva, no una validación de seguridad.

8.12 `Symbol.hasInstance`: personalizar `instanceof`

```
class EnteroPositivo {
  static [Symbol.hasInstance](valor) {
    return Number.isInteger(valor) && valor > 0;
  }
}

3 instanceof EnteroPositivo; // true
-1 instanceof EnteroPositivo; // false
```

Aunque es posible, una función `esEnteroPositivo(valor)` comunica mejor la intención cuando no existe una relación real de instancias.

8.13 Símbolos vinculados con expresiones regulares

Los métodos de string consultan protocolos:

- `Symbol.match` para `match`;
- `Symbol.matchAll` para `matchAll`;
- `Symbol.replace` para `replace`;
- `Symbol.search` para `search`;
- `Symbol.split` para `split`.

Esto explica por qué esos métodos —que usa el capítulo 18— aceptan objetos `RegExp`, y por qué un objeto especializado podría personalizar la operación.

```
const censor = {
  [Symbol.replace](texto, reemplazo) {
    return texto.replaceAll("secreto", reemplazo);
  }
};

"dato secreto".replace(censor, "***"); // "dato ***"
```

Es una demostración de protocolo; para una operación concreta, una función nombrada puede ser más directa.

8.14 `Symbol.isConcatSpreadable`

`Array.prototype.concat` normalmente expande arrays y conserva otros objetos como una sola posición. Esta clave puede modificar la decisión:

```
const grupo = {
  0: "a",
  1: "b",
  length: 2,
  [Symbol.isConcatSpreadable]: true
};

[0].concat(grupo); // [0, "a", "b"]
```

El objeto debe tener índices y `length` coherentes con el comportamiento esperado.

8.15 `Symbol.species`

Algunas clases integradas consultan `Symbol.species` para decidir qué constructor utilizar en resultados derivados:

```
class MiArray extends Array {
  static get [Symbol.species]() {
    return Array;
  }
}

const valores = new MiArray(1, 2, 3);
const dobles = valores.map(x => x * 2);

dobles instanceof MiArray; // false
dobles instanceof Array;   // true
```

Es un mecanismo avanzado de interoperabilidad entre subclases y métodos que crean colecciones.

8.16 `Symbol.unscopables`

Controla qué propiedades quedan excluidas dentro de la sentencia histórica `with`. El modo estricto y los módulos prohíben `with`, por lo que este símbolo existe principalmente para compatibilidad del lenguaje. No debería orientar diseños nuevos.

8.17 Un ejemplo integrador

```
function crearRegistro() {
  const entradas = new Map();
  const VERSION = Symbol("version interna");

  return {
    [VERSION]: 1,

    guardar(clave, valor) {
      entradas.set(clave, valor);
    },

    obtener(clave) {
      return entradas.get(clave);
    },

    get [Symbol.toStringTag]() {
      return "Registro";
    },

    *[Symbol.iterator]() {
      yield* entradas.entries();
    }
  };
}

const registro = crearRegistro();
registro.guardar("tema", "oscuro");

for (const [clave, valor] of registro) {
  console.log(clave, valor);
}
```

El símbolo local evita colisión, la clausura encapsula el `Map`, la etiqueta mejora la descripción y el iterador integra el objeto con el lenguaje.

8.18 Errores frecuentes

- creer que dos símbolos con la misma descripción son iguales;
- tratar la descripción como una clave recuperable;
- usar símbolos como seguridad o privacidad;
- esperar que `Object.keys` o JSON los incluyan;
- concatenarlos implícitamente con strings;
- implementar un protocolo sin respetar exactamente su contrato;
- personalizar coerción o `instanceof` cuando una función nombrada sería más clara.

8.19 Para recordar

- `Symbol()` crea identidad única; la descripción no participa de la igualdad.

- `Symbol.for()` coordina identidades mediante un registro compartido.
- Una clave `symbol` evita colisiones y enumeración ordinaria, pero puede descubrirse.
- Los símbolos conocidos son puntos de extensión que implementan protocolos.
- Aprendé primero `Symbol.iterator`; los demás se entienden como variantes del mismo acuerdo entre objeto y lenguaje.

Parte II. Modelar colecciones y entidades

Capítulo 9. Arrays y matrices

Idea central

Un array representa una secuencia ordenada y mutable; elegir la operación adecuada depende de si queremos consultar, transformar o cambiar esa secuencia. La productividad aparece cuando distinguimos métodos mutantes de no mutantes, evitamos arrays dispersos y controlamos las referencias compartidas en estructuras anidadas.

9.1 Cuándo elegir un array

Un array es adecuado cuando:

- el orden importa;
- cada elemento ocupa una posición;
- se recorrerá la colección completa o por rangos;
- pueden existir valores repetidos;
- las operaciones principales son agregar, filtrar, ordenar o transformar.

```
const notas = [8, 6, 10, 7];
```

Si la operación dominante es encontrar un elemento por una clave estable, un Map o un índice auxiliar puede ser más apropiado que recorrer siempre el array.

9.2 Crear arrays

El literal es la forma habitual:

```
const vacio = [];  
const lenguajes = ["JavaScript", "C#", "Python"];  
const mezcla = [1, "dos", true, null];
```

JavaScript permite mezclar tipos, pero una colección homogénea suele ser más fácil de procesar y documentar.

`Array.of` crea un array con sus argumentos:

```
Array.of(3);           // [3]  
Array.of(1, 2, 3);     // [1, 2, 3]
```

Esto contrasta con el constructor de un solo número:

```
Array(3); // array con longitud 3 y tres huecos
```

`Array.from` convierte iterables o estructuras similares a arrays y puede transformar al mismo tiempo:

```
Array.from("A😊B"); // ["A", "😊", "B"]
Array.from({ length: 5 }, (_, indice) => indice + 1);
// [1, 2, 3, 4, 5]
```

`Array.isArray` distingue arrays de otros objetos:

```
Array.isArray([]); // true
typeof []; // "object"
```

9.3 Índices y `length`

Los índices enteros comienzan en cero:

```
const colores = ["rojo", "verde", "azul"];

colores[0]; // "rojo"
colores[2]; // "azul"
colores[99]; // undefined
colores.length; // 3
```

`at` admite posiciones negativas:

```
colores.at(-1); // "azul"
colores.at(-2); // "verde"
```

Asignar una posición modifica el array:

```
colores[1] = "amarillo";
```

`length` no es solo una propiedad informativa. Reducirla elimina elementos; ampliarla crea huecos:

```
const valores = [1, 2, 3];
valores.length = 1; // [1]
valores.length = 4; // [1, <3 huecos>]
```

No uses esta capacidad como operación cotidiana. Métodos con nombres expresan mejor la intención.

9.4 Arrays densos y dispersos

Un hueco es una posición inexistente, diferente de una posición con `undefined`:

```
const disperso = [1, , 3];
const explicito = [1, undefined, 3];

1 in disperso; // false
1 in explicito; // true
```

Algunos métodos como `map` saltan los huecos; `for...of` produce `undefined` para ellos. Estas diferencias dificultan el razonamiento. Preferí arrays densos creados con valores explícitos.

9.5 Extraer rangos con `slice`

`slice(inicio, fin)` devuelve un nuevo array y no incluye `fin`:

```
const valores = [10, 20, 30, 40, 50];

valores.slice(1, 4); // [20, 30, 40]
valores.slice(2);    // [30, 40, 50]
valores.slice(-2);   // [40, 50]
valores.slice();      // copia superficial
```

Los elementos no se clonan. Si son objetos, ambas colecciones contienen las mismas referencias.

9.6 Agregar y quitar en los extremos

Estos métodos modifican el array:

```
const cola = ["a", "b"];

cola.push("c"); // agrega al final; devuelve nueva longitud
cola.pop();     // quita y devuelve el último
cola.unshift("z"); // agrega al comienzo
cola.shift();   // quita y devuelve el primero
```

`push` y `pop` permiten una pila LIFO: el último elemento que entra es el primero que sale. `push` y `shift` permiten una cola FIFO, aunque quitar del comienzo obliga a reindexar elementos y no escala bien para colas enormes. El apéndice B usa esas dos operaciones para construir un evaluador de expresiones aritméticas.

9.7 `splice`: cambiar una región

`splice(inicio, cantidad, ...nuevos)` modifica el original y devuelve lo eliminado:

```
const letras = ["a", "b", "c", "d"];
const eliminadas = letras.splice(1, 2, "x", "y");

letras; // ["a", "x", "y", "d"]
eliminadas; // ["b", "c"]
```

No lo confundas con `slice`. Si querés una variante sin mutación para reemplazar una posición:

```
const indice = 1;
const nuevo = [
  ...letras.slice(0, indice),
  "reemplazo",
  ...letras.slice(indice + 1)
];
```

9.8 Buscar y comprobar

```
const numeros = [10, 20, 30, 20];

numeros.includes(20); // true
numeros.indexOf(20); // 1
numeros.lastIndexOf(20); // 3
numeros.find(n => n > 15); // 20
numeros.findIndex(n => n > 15); // 1
numeros.some(n => n < 0); // false
numeros.every(n => n > 0); // true
```

`includes` puede encontrar `NaN`, mientras que `indexOf` no:

```
[NaN].includes(NaN); // true
[NaN].indexOf(NaN); // -1
```

`find` devuelve `undefined` si no hay resultado. Si los elementos pueden ser literalmente `undefined`, usá `findIndex` o un resultado etiquetado para distinguir los casos.

9.9 Tres formas fundamentales de recorrer

for tradicional

Permite controlar índice, dirección y paso:

```
for (let indice = 0; indice < numeros.length; indice += 1) {
  console.log(indice, numeros[indice]);
}
```

Es la mejor opción cuando el índice participa en el algoritmo o el recorrido no es lineal.

for...of

Recorre valores:

```
for (const numero of numeros) {  
  console.log(numero);  
}
```

Para índice y valor:

```
for (const [indice, numero] of numeros.entries()) {  
  console.log(indice, numero);  
}
```

for...in

Recorre nombres de propiedades enumerables, incluidos los heredados. No es la herramienta para valores de arrays:

```
for (const clave in numeros) {  
  console.log(clave); // strings como "0", "1"...  
}
```

Una propiedad adicional o una modificación del prototipo puede aparecer en el recorrido. Usá `for...of` o métodos de array.

9.10 Transformar con `map`

`map` produce un nuevo array con un resultado por elemento:

```
const dobles = numeros.map(numero => numero * 2);
```

El callback recibe valor, índice y array original:

```
const etiquetas = numeros.map(  
  (numero, indice) => `${indice + 1}: ${numero}`  
);
```

No uses `map` solo para efectos si descartás su resultado. Para imprimir, enviar o registrar, `for...of` o `forEach` comunica mejor.

Este capítulo presenta `map`, `filter` y `reduce` como métodos de array: qué devuelven y qué no modifican. El capítulo 15 los retoma como piezas de composición, con implementaciones didácticas que muestran cómo están hechos por dentro.

9.11 Seleccionar con `filter`

```
const aprobados = alumnos.filter(alumno => alumno.nota >= 6);
```

El callback se interpreta como condición. El array resultante conserva referencias a los mismos objetos seleccionados; no los clona.

9.12 Acumular con `reduce`

`reduce` combina elementos en un acumulador:

```
const suma = numeros.reduce(
  (acumulado, numero) => acumulado + numero,
  0
);
```

El valor inicial define el tipo y cubre el array vacío. Sin valor inicial, el primer elemento se usa como acumulador y un array vacío lanza `TypeError`.

Puede construir objetos, mapas o agrupaciones, aunque un bucle puede ser más legible si el acumulador tiene muchas reglas:

```
const porEstado = pedidos.reduce((grupos, pedido) => {
  const lista = grupos[pedido.estado] ?? [];

  return {
    ...grupos,
    [pedido.estado]: [...lista, pedido]
  };
}, {});
```

Crear copias en cada iteración puede ser costoso. La inmutabilidad es una herramienta, no una obligación de producir el algoritmo menos eficiente. Un acumulador local mutable que no escapa puede ser claro y seguro.

9.13 Efectos con `forEach`

```
alumnos.forEach(alumno => console.log(alumno.nombre));
```

`forEach` siempre devuelve `undefined` y no admite `break` ni `continue`. Tampoco espera callbacks asíncronicos: ignora la promesa que devuelve la función y sigue de largo. Para recorrer con `await` hay que usar `for...of`, si el orden importa, o `map` más `Promise.all`, si las operaciones son independientes. La sección 17.8 desarrolla esa diferencia.

9.14 Aplanar y combinar

```
[[1, 2], [3, 4]].flat();           // [1, 2, 3, 4]
[1, [2, [3]]].flat(2);            // [1, 2, 3]
[1, 2].flatMap(n => [n, n]);       // [1, 1, 2, 2]
[1, 2].concat([3, 4]);            // [1, 2, 3, 4]
```

El spread también combina:

```
const combinado = [...a, ...b];
```

`flat` solo aplanar hasta la profundidad indicada y conserva las referencias de los valores internos.

9.15 Ordenar e invertir

`sort` modifica el array y, sin comparador, convierte a string:

```
[2, 10, 1].sort(); // [1, 10, 2]
```

Para números:

```
numeros.sort((a, b) => a - b);
```

El comparador debe devolver un valor negativo, cero o positivo. Para objetos:

```
alumnos.sort((a, b) => a.nota - b.nota);
```

Las variantes no mutantes son `toSorted`, `toReversed` y `toSpliced`:

```
const ordenados = numeros.toSorted((a, b) => a - b);
const invertidos = numeros.toReversed();
```

Para nombres humanos, usá `Intl.Collator`, como explica la sección 5.14. La sección 15.12 vuelve sobre `toSorted` desde el punto de vista de la inmutabilidad.

9.16 Referencias y copias superficiales

```
const original = [{ valor: 1 }];
const alias = original;
const copia = [...original];

alias === original; // true
copia === original; // false
copia[0] === original[0]; // true
```


La copia tiene otra estructura exterior, pero comparte los objetos internos. Es el mismo fenómeno que la sección 10.11 analiza para objetos. Para actualizar un elemento sin modificar el original:

```
const actualizado = original.map((item, indice) =>
  indice === 0 ? { ...item, valor: 2 } : item
);
```

9.17 Desestructuración

```
const [primero, segundo] = numeros;
const [cabeza, ...cola] = numeros;
const [, omitidoElPrimero, tercero = 0] = numeros;
```

Permite intercambiar variables:

```
let izquierda = "A";
let derecha = "B";
[izquierda, derecha] = [derecha, izquierda];
```

También puede anidarse, pero demasiada profundidad hace frágil el código frente a cambios de forma.

9.18 Matrices y filas compartidas

Una matriz es un array de arrays:

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6]
];

matriz[1][2]; // 6
```

No crees todas las filas con la misma referencia:

```
const incorrecta = Array(3).fill(Array(3).fill(0));
incorrecta[0][0] = 1;
// cambia la primera columna de todas las filas
```

Creá una fila por iteración:

```
const correcta = Array.from(
  { length: 3 },
  () => Array(3).fill(0)
);
```

Recorrido:

```
for (let fila = 0; fila < correcta.length; fila += 1) {
  for (let columna = 0; columna < correcta[fila].length; columna += 1) {
    console.log(correcta[fila][columna]);
  }
}
```

No todas las filas tienen que medir lo mismo; si el algoritmo exige una matriz rectangular, validalo.

9.19 Caso integrador: estadísticas de notas

```
function resumirNotas(notas) {
  if (!Array.isArray(notas) || notas.length === 0) {
    throw new TypeError("Se necesita al menos una nota");
  }

  const invalidas = notas.filter(
    nota => !Number.isFinite(nota) || nota < 0 || nota > 10
  );

  if (invalidas.length > 0) {
    throw new RangeError(`Notas inválidas: ${invalidas.join(", ")}`);
  }

  const suma = notas.reduce((total, nota) => total + nota, 0);

  return {
    cantidad: notas.length,
    promedio: suma / notas.length,
    minima: Math.min(...notas),
    maxima: Math.max(...notas),
    aprobadas: notas.filter(nota => nota >= 6).length,
    ordenadas: notas.toSorted((a, b) => a - b)
  };
}
```

La entrada se valida una vez, los resultados tienen nombres y el array original no cambia.

9.20 Errores frecuentes

- confundir `slice` con `splice`;
- ordenar números sin comparador;
- usar `for...in` para valores;
- esperar que `spread` clone objetos internos;
- crear matrices con filas compartidas;
- usar `map` para efectos o `forEach` con `await` esperando secuencia;
- omitir el valor inicial de `reduce` sin considerar el array vacío;

- crear huecos mediante índices lejanos o cambios de `length`.

9.21 Para recordar

- Un array es una secuencia ordenada; sus métodos expresan consultas, transformaciones o mutaciones.
- `slice`, `map`, `filter` y `toSorted` producen arrays nuevos; muchos otros modifican el original.
- Las copias son superficiales salvo que se copie explícitamente cada nivel necesario.
- Evitá arrays dispersos y matrices con referencias de fila compartidas.
- Elegí el recorrido según la tarea: índice, valor, transformación, acumulación o efecto.

Capítulo 10. Objetos, propiedades y referencias

Idea central

Un objeto reúne propiedades para representar una entidad, y las variables que lo contienen guardan referencias a esa entidad. Para usar objetos con seguridad hay que distinguir clave de valor, identidad de contenido y copia superficial de copia profunda.

10.1 Cuándo elegir un objeto

Un objeto es una buena representación cuando un dato tiene campos conocidos con significados diferentes:

```
const alumno = {  
  legajo: 12345,  
  nombre: "Ana",  
  regular: true  
};
```

El orden de las propiedades no suele ser la operación principal. Consultamos por nombre, no por posición. Si las claves son datos dinámicos o de cualquier tipo, `Map` puede expresar mejor la colección; la sección 11.13 compara ambas opciones.

10.2 Crear objetos

El literal es la forma habitual:

```
const vacio = {};  
  
const producto = {  
  codigo: "TEC-01",  
  descripcion: "Teclado",  
  precio: 25000  
};
```

Las claves literales se interpretan como strings, salvo las calculadas con símbolos:

```
const ejemplo = {  
  10: "diez",  
  "con espacios": true  
};  
  
Object.keys(ejemplo); // ["10", "con espacios"]
```

10.3 Punto y corchetes

La notación de punto requiere un identificador conocido al escribir el programa:

```
producto.precio;  
producto.precio = 26000;
```

Los corchetes evalúan una expresión y permiten claves dinámicas:

```
const campo = "precio";  
producto[campo]; // 26000  
  
producto["con espacios"] = "valor";
```

No confundas:

```
producto.campo; // busca literalmente "campo"  
producto[campo]; // busca el valor de la variable campo
```

10.4 Crear, actualizar y eliminar propiedades

```
producto.stock = 10;  
producto.precio = 27000;  
delete producto.stock;
```

`delete` elimina la propiedad, no asigna `undefined`. En objetos con una forma estable, agregar y quitar campos constantemente puede complicar el contrato. A veces conviene conservar una propiedad opcional con `null`, si ese es el modelo acordado.

10.5 Existencia de propiedades

```
Object.hasOwn(producto, "precio"); // true  
"precio" in producto; // true
```

`in` considera la cadena de prototipos; `Object.hasOwn` solo las propiedades propias.

Leer una propiedad ausente devuelve `undefined`:

```
producto.categoria; // undefined
```

Para acceso anidado opcional:

```
const ciudad = alumno.direccion?.ciudad ?? "Sin informar";
```

No uses `?` para ocultar una propiedad obligatoria. Valida la entidad al construirla o recibirla.

10.6 Formas abreviadas

Si nombre de variable y propiedad coinciden:

```
const nombre = "Ana";
const edad = 20;
const persona = { nombre, edad };
```

Los métodos tienen sintaxis abreviada:

```
const contador = {
  valor: 0,
  incrementar() {
    this.valor += 1;
    return this.valor;
  }
};
```

Una función flecha no crea su propio `this` y no debe usarse como reemplazo automático de un método:

```
const incorrecto = {
  valor: 1,
  leer: () => this.valor
};
```

10.7 Claves calculadas

```
const prefijo = "nota";
const indice = 1;

const registro = {
  [`${prefijo}${indice}`]: 8
};
```

Las claves calculadas son útiles al construir índices o adaptar datos, pero una proliferación de campos dinámicos puede indicar que corresponde un `Map` o una colección anidada.

10.8 Recorrer propiedades

```
Object.keys(producto); // claves string propias y enumerables
Object.values(producto); // valores
Object.entries(producto); // pares [clave, valor]
```

```
for (const [clave, valor] of Object.entries(producto)) {  
  console.log(clave, valor);  
}
```

`Reflect.ownKeys` también incluye símbolos y claves no enumerables. La enumerabilidad y los descriptores son mecanismos avanzados; para modelos comunes, los literales producen propiedades propias, enumerables, modificables y configurables.

10.9 Orden de propiedades

JavaScript define un orden de enumeración: índices enteros válidos primero en orden numérico, luego strings en orden de inserción y finalmente símbolos en orden de inserción. Aun así, un objeto modela campos, no una secuencia. Si el orden es esencial para la operación, usá un array o `Map`.

10.10 Identidad y asignación por referencia

```
const original = { nombre: "Ana" };  
const alias = original;  
  
alias.nombre = "Beatriz";  
original.nombre; // "Beatriz"
```

Ambas variables contienen una referencia al mismo objeto.

La igualdad compara identidad:

```
({ x: 1 }) === ({ x: 1 }); // false  
  
const a = { x: 1 };  
const b = a;  
a === b; // true
```

JavaScript no incluye una igualdad profunda general porque “mismo contenido” depende del dominio: ¿importa el orden de arrays?, ¿los prototipos?, ¿fechas?, ¿símbolos?, ¿ciclos?

10.11 Copia superficial con spread

```
const copia = { ...original };  
copia === original; // false
```

El spread copia propiedades propias y enumerables, incluidas claves symbol, pero solo una capa:

```
const configuracion = {
  tema: "claro",
  usuario: { nombre: "Ana" }
};

const copia = { ...configuracion };
copia.usuario === configuracion.usuario; // true
```

Modificar `copia.usuario.nombre` también afecta al original.

Para una actualización anidada sin mutar:

```
const actualizada = {
  ...configuracion,
  usuario: {
    ...configuracion.usuario,
    nombre: "Beatriz"
  }
};
```

Solo se copian las ramas que cambian; las demás pueden compartirse de manera segura si no se mutan.

10.12 Precedencia del spread

Las propiedades posteriores reemplazan anteriores:

```
const base = { tema: "claro", idioma: "es" };

const configuracion = {
  ...base,
  tema: "oscuro"
};
```

El orden inverso perdería el valor personalizado:

```
const incorrecta = {
  tema: "oscuro",
  ...base // vuelve a "claro"
};
```

10.13 Object.assign

```
const combinado = Object.assign({}, base, { tema: "oscuro" });
```

El primer argumento es el destino y se modifica. El spread suele ser más legible para crear un objeto nuevo. Ambos realizan copia superficial y activan getters al leer valores del origen.

10.14 Copia profunda con `structuredClone`

```
const clon = structuredClone(configuracion);
clon.usuario === configuracion.usuario; // false
```

Puede clonar muchos valores integrados, referencias repetidas y estructuras cíclicas. No clona funciones, símbolos como valores ni todos los objetos de plataforma, y los prototipos personalizados no necesariamente se conservan como espera una clase.

No copies profundamente por rutina. Puede ser costoso y esconder un diseño con demasiado estado compartido. Copiá según el límite que realmente necesite aislamiento.

El truco `JSON.parse(JSON.stringify(objeto))` pierde `undefined`, símbolos, `bigint`, fechas como objetos, `Map`, `Set`, valores no finitos y ciclos. No es un clon general.

10.15 Desestructuración

```
const { nombre, edad } = persona;
```

Renombrar una variable local:

```
const { nombre: nombreCompleto } = persona;
```

Valor predeterminado ante `undefined`:

```
const { idioma = "es" } = configuracion;
```

Resto de propiedades:

```
const { id, ...datosEditables } = alumno;
```

Anidada:

```
const {
  direccion: { ciudad }
} = alumno;
```

La sección 9.17 presenta la forma equivalente para arrays y la sección 14.12 la usa en parámetros.

La sección 9.17 presenta la forma equivalente para arrays y la sección 14.12 la usa en parámetros.

La desestructuración anidada falla si `direccion` falta. Se puede proporcionar un objeto predeterminado:

```
const { direccion: { ciudad } = {} } = alumno;
```

Pero `ciudad` quedará `undefined`; si es obligatoria, corresponde validar.

10.16 Parámetros desestructurados

```
function presentar({ nombre, edad = 0 }) {  
  return `${nombre} tiene ${edad} años`;  
}
```

Para permitir una llamada sin argumento:

```
function configurar({ tema = "claro" } = {}) {  
  return { tema };  
}
```

Esta forma es cómoda para opciones, pero puede esconder que falta una entidad obligatoria. No agregues `= {}` automáticamente a todos los parámetros.

10.17 Métodos estáticos útiles

```
Object.fromEntries([  
  ["nombre", "Ana"],  
  ["edad", 20]  
]);  
  
Object.freeze(producto);  
Object.seal(producto);  
Object.preventExtensions(producto);
```

`freeze` impide cambios en las propiedades directas, pero es superficial:

```
const congelado = Object.freeze({ interior: { valor: 1 } });  
congelado.interior.valor = 2; // el objeto interior no está congelado
```

En modo estricto, algunas modificaciones prohibidas lanzan error; en otros contextos pueden fallar silenciosamente. La inmutabilidad por convención y una arquitectura clara siguen siendo necesarias.

10.18 JSON como formato, no como objeto JavaScript completo

```
const texto = JSON.stringify(producto);  
const recuperado = JSON.parse(texto);
```

JSON admite objetos, arrays, strings, números finitos, booleanos y `null`. No conserva métodos, prototipos, `undefined`, símbolos, `Map`, `Set`, `BigInt` ni referencias compartidas. Al leer, validá de nuevo la estructura: la sección 19.30 muestra cómo hacerlo con un archivo. El capítulo 22 lo trata como formato de intercambio entre programas.

10.19 Caso integrador: actualizar un perfil

```
function actualizarPerfil(perfil, cambios) {
  if (!Object.hasOwn(cambios, "nombre") &&
      !Object.hasOwn(cambios, "preferencias")) {
    return perfil;
  }

  const actualizado = { ...perfil };

  if (Object.hasOwn(cambios, "nombre")) {
    const nombre = cambios.nombre.trim();
    if (!nombre) throw new TypeError("Nombre vacío");
    actualizado.nombre = nombre;
  }

  if (Object.hasOwn(cambios, "preferencias")) {
    actualizado.preferencias = {
      ...perfil.preferencias,
      ...cambios.preferencias
    };
  }

  return actualizado;
}
```

La función distingue propiedades ausentes, valida los cambios y copia solamente los niveles modificados.

10.20 Errores frecuentes

- creer que asignar copia el objeto;
- comparar contenido con `===`;
- esperar copia profunda de spread u `Object.assign`;
- poner una clave dinámica con punto y leer literalmente otro campo;
- invertir el orden de los spreads;
- desestructurar un camino opcional sin valor predeterminado o validación;
- usar JSON como clon universal;
- creer que `freeze` congela todo el grafo.

10.21 Para recordar

- Un objeto representa campos nombrados; una variable guarda una referencia a él.
- Punto sirve para claves conocidas; corchetes, para claves calculadas.
- Identidad no equivale a igualdad de contenido.
- Spread, `Object.assign` y `freeze` actúan superficialmente.
- La actualización inmutable copia el camino modificado y puede compartir el resto.

Capítulo 11. Map, Set y colecciones especializadas

Idea central

Map modela asociaciones dinámicas y **Set** modela pertenencia sin repetidos. Ambos aceptan valores de cualquier tipo, preservan el orden de inserción y ofrecen una API de colección más explícita que un objeto o un array usados fuera de su propósito.

La elección se resume así:

```
entidad con campos conocidos → Object
secuencia ordenada           → Array
clave dinámica → valor       → Map
pertenencia única            → Set
```

11.1 Map: pares clave-valor

```
const alumnosPorLegajo = new Map();

alumnosPorLegajo.set(101, { nombre: "Ana" });
alumnosPorLegajo.set(102, { nombre: "Luis" });
```

Operaciones fundamentales:

```
alumnosPorLegajo.get(101); // { nombre: "Ana" }
alumnosPorLegajo.has(103); // false
alumnosPorLegajo.size;     // 2
alumnosPorLegajo.delete(102); // true si existía
alumnosPorLegajo.clear();   // elimina todos
```

set devuelve el propio mapa, por lo que puede encadenarse:

```
const estados = new Map()
  .set("P", "pendiente")
  .set("A", "aprobado");
```

11.2 Crear un Map desde pares

```
const mapa = new Map([
  ["tema", "oscuro"],
  ["idioma", "es"]
]);
```

Cada elemento exterior debe ser iterable con al menos dos posiciones. `Object.entries` permite convertir un objeto con claves string:

```
const desdeObjeto = new Map(Object.entries({ a: 1, b: 2 }));
```

La conversión inversa funciona si las claves pueden convertirse apropiadamente en propiedades:

```
const objeto = Object.fromEntries(desdeObjeto);
```

Si las claves son objetos, la conversión a objeto común pierde su identidad como claves y no es equivalente.

11.3 Claves de cualquier tipo

```
const porObjeto = new Map();
const boton = { id: "guardar" };

porObjeto.set(boton, { clicks: 0 });
porObjeto.get(boton); // funciona con la misma referencia
porObjeto.get({ id: "guardar" }); // undefined
```

La igualdad de claves sigue una comparación de identidad para objetos y una semántica similar a `SameValueZero` para primitivos. `NaN` puede funcionar como clave y `0/-0` se consideran la misma.

11.4 Ausencia y valores `undefined`

`get` devuelve `undefined` tanto cuando falta la clave como cuando su valor es `undefined`:

```
const mapa = new Map([["presente", undefined]]);

mapa.get("presente"); // undefined
mapa.get("ausente"); // undefined
```

Usá `has` para distinguir:

```
mapa.has("presente"); // true
mapa.has("ausente"); // false
```

11.5 Recorrer un `Map`

El iterador predeterminado produce pares en orden de inserción:

```
for (const [clave, valor] of estados) {  
  console.log(clave, valor);  
}
```

También existen:

```
estados.keys();  
estados.values();  
estados.entries();
```

Los tres devuelven iteradores, no arrays. Se pueden materializar:

```
const claves = [...estados.keys()];
```

`forEach` usa el orden (valor, clave, mapa), diferente al orden visual de los pares:

```
estados.forEach((valor, clave) => {  
  console.log(clave, valor);  
});
```

11.6 Patrones productivos con Map

Crear un índice

```
const porLegajo = new Map(  
  alumnos.map(alumno => [alumno.legajo, alumno])  
);
```

Una búsqueda pasa de recorrer el array a consultar por clave. El costo de construir el índice se justifica cuando habrá muchas búsquedas o actualizaciones.

Contar frecuencias

```
function contar(valores) {  
  const frecuencias = new Map();  
  
  for (const valor of valores) {  
    frecuencias.set(valor, (frecuencias.get(valor) ?? 0) + 1);  
  }  
  
  return frecuencias;  
}
```

Agrupar

```
function agruparPor(valores, obtenerClave) {
  const grupos = new Map();

  for (const valor of valores) {
    const clave = obtenerClave(valor);
    const grupo = grupos.get(clave) ?? [];
    grupo.push(valor);
    grupos.set(clave, grupo);
  }

  return grupos;
}
```

El array local se muta dentro de la función, pero el estado queda encapsulado y se devuelve al final. Es exactamente la mutación local controlada que describe la sección 15.5: puede ser mucho más eficiente que copiar cada grupo en cada iteración y, vista desde afuera, la función sigue siendo pura.

11.7 Actualización inmutable de un Map

```
const actualizado = new Map(original);
actualizado.set(clave, valor);
```

La estructura exterior es nueva, pero claves y valores internos mantienen sus referencias. Igual que con arrays y objetos, se trata de una copia superficial.

11.8 Set: valores únicos

```
const etiquetas = new Set(["js", "node", "js"]);

etiquetas.size; // 2
etiquetas.add("web");
etiquetas.has("node"); // true
etiquetas.delete("js");
etiquetas.clear();
```

`add` devuelve el propio conjunto y puede encadenarse.

Un `Set` mantiene el orden de la primera inserción. Agregar nuevamente un valor no lo mueve.

11.9 Eliminar duplicados

```
const unicos = [...new Set([1, 2, 2, 3, 1])];
// [1, 2, 3]
```


Con objetos, la unicidad usa identidad:

```
const a = { id: 1 };
const b = { id: 1 };

new Set([a, b]).size; // 2
new Set([a, a]).size; // 1
```

Para deduplicar por una propiedad, indexá por esa clave:

```
const porId = new Map(items.map(item => [item.id, item]));
const unicosPorId = [...porId.values()];
```

Decidí qué aparición gana. El ejemplo conserva la última para cada id.

11.10 Recorrer un Set

```
for (const etiqueta of etiquetas) {
  console.log(etiqueta);
}
```

`values()` y `keys()` son equivalentes por compatibilidad con `Map`. `entries()` produce pares `[valor, valor]`.

11.11 Operaciones de conjuntos

Podemos expresar unión, intersección, diferencia y diferencia simétrica de forma portable:

```
function union(a, b) {
  return new Set([...a, ...b]);
}

function interseccion(a, b) {
  return new Set([...a].filter(valor => b.has(valor)));
}

function diferencia(a, b) {
  return new Set([...a].filter(valor => !b.has(valor)));
}

function diferenciaSimetrica(a, b) {
  return union(diferencia(a, b), diferencia(b, a));
}
```

Para elegir qué conjunto recorrer en una intersección grande, conviene usar el más pequeño.

Subconjunto:

```
function esSubconjunto(a, b) {  
  return [...a].every(valor => b.has(valor));  
}
```

11.12 Actualización inmutable de un Set

```
const actualizado = new Set(original);  
actualizado.add(nuevoValor);
```

La copia conserva referencias a objetos internos. Si los objetos se modifican, ambos conjuntos observan el cambio.

11.13 Elegir entre objeto y Map

Preferí un objeto cuando:

- representa una entidad con campos conocidos;
- necesitás notación literal y desestructuración;
- el formato natural de intercambio es JSON;
- las claves son nombres de propiedades.

Preferí Map cuando:

- las claves nacen de los datos;
- no son solo strings o símbolos;
- consultás `size`, agregás y eliminás con frecuencia;
- querés iteración directa de pares;
- necesitás distinguir claramente la colección de una entidad.

No elijas solo por una afirmación genérica de rendimiento. Medí si el volumen realmente lo exige; la semántica correcta suele ser el criterio principal.

11.14 Elegir entre array y Set

Preferí array cuando:

- importan posición y duplicados;
- transformás toda la secuencia;
- necesitás índices y rangos.

Preferí Set cuando:

- la pregunta principal es pertenencia;
- los duplicados no tienen significado;

- agregás y quitás miembros.

Un `Set` no reemplaza un array para ordenar, acceder por índice o representar varias apariciones.

11.15 WeakMap: asociar datos sin retener las claves

`WeakMap` acepta objetos y símbolos no registrados como claves. No impide que una clave objeto sea recolectada si no queda otra referencia fuerte:

```
const metadatos = new WeakMap();

let elemento = { id: 1 };
metadatos.set(elemento, { seleccionado: true });
metadatos.get(elemento); // { seleccionado: true }

elemento = null;
// La entrada podrá desaparecer cuando el recolector lo determine.
```

No es iterable y no tiene `size`, porque exponer sus claves impediría una semántica previsible de recolección. Sirve para metadatos privados, cachés ligados a la vida de objetos y asociaciones que no deben prolongar esa vida.

11.16 WeakSet: marcar objetos sin retenerlos

```
const procesados = new WeakSet();
const tarea = {};

procesados.add(tarea);
procesados.has(tarea); // true
```

Comparte las limitaciones de no enumeración y claves débiles. Es adecuado para registrar pertenencia de objetos mientras esos objetos existan.

11.17 Caso integrador: inscripciones

```
function indexarInscripciones(inscripciones) {
  const porLegajo = new Map();
  const cursos = new Set();
  const duplicados = new Set();

  for (const inscripcion of inscripciones) {
    const { legajo, curso } = inscripcion;

    if (porLegajo.has(legajo)) duplicados.add(legajo);
    porLegajo.set(legajo, { ...inscripcion });
    cursos.add(curso);
  }

  return {
    porLegajo,
    cursos,
    duplicados,
    cantidadUnica: porLegajo.size
  };
}
```

El contrato decide que la última inscripción reemplaza a las anteriores y conserva una lista de legajos repetidos para informar el problema.

11.18 Errores frecuentes

- usar `get` sin `has` cuando `undefined` es un valor posible;
- convertir un `Map` con claves objeto a un objeto común y esperar equivalencia;
- creer que `Set` elimina objetos con contenido igual;
- usar un objeto como diccionario dinámico sin considerar prototipos y tipos de clave;
- usar `Map` para una entidad fija y perder claridad de campos;
- esperar que una copia de `Map` o `Set` clone sus valores;
- intentar enumerar `WeakMap` o depender del momento de recolección.

11.19 Para recordar

- `Map` expresa asociaciones dinámicas; `Set`, pertenencia única.
- Ambos preservan inserción y comparan objetos por identidad.
- `has` distingue ausencia de un valor `undefined` almacenado.
- Las conversiones con arrays y objetos son útiles, pero no siempre conservan la semántica de las claves.
- `WeakMap` y `WeakSet` vinculan información con la vida de objetos sin volverla enumerable.

Parte III. Organizar el comportamiento

Capítulo 12. Estructuras de control

Idea central

Una estructura de control debe mostrar por qué el programa toma un camino y cómo garantiza que una repetición termina. Las guardas mantienen visible el caso normal, cada bucle hace explícito su progreso y `switch` se reserva para seleccionar entre valores discretos.

12.1 Del flujo lineal a una decisión

Sin estructuras de control, las instrucciones se ejecutan de arriba hacia abajo:

```
const subtotal = precio * cantidad;
const impuesto = subtotal * 0.21;
const total = subtotal + impuesto;
```

Una condición introduce un desvío:

```
if (cantidad > stock) {
  console.log("No hay stock suficiente");
}
```

JavaScript interpreta la expresión entre paréntesis en contexto booleano. Para reglas importantes, una comparación explícita comunica mejor el motivo que un valor `truthy` ambiguo.

12.2 `if`: ejecutar bajo una condición

```
if (nota >= 6) {
  estado = "aprobado";
}
```

Las llaves son recomendables incluso con una sola sentencia. Evitan errores al agregar una segunda línea y hacen visible el bloque.

Una condición puede tener nombre:

```
const alcanzaAprobacion = nota >= 6;

if (alcanzaAprobacion) {
  estado = "aprobado";
}
```

No crees una variable por cada comparación trivial; hazlo cuando el nombre explica una regla.

12.3 `else`: dos caminos excluyentes

```
if (stock >= cantidad) {  
  confirmarPedido();  
} else {  
  informarFaltante();  
}
```

El operador condicional produce un valor:

```
const estado = stock >= cantidad ? "disponible" : "agotado";
```

Es apropiado para una selección breve. Ternarios anidados suelen ser difíciles de leer:

```
const categoria = nota >= 8  
  ? "promoción"  
  : nota >= 6  
    ? "aprobación"  
    : "desaprobación";
```

Un `if` con retornos puede expresar mejor esa clasificación.

12.4 Encadenar rangos con `else if`

```
function categoria(nota) {  
  if (nota >= 8) {  
    return "promoción";  
  } else if (nota >= 6) {  
    return "aprobación";  
  } else {  
    return "desaprobación";  
  }  
}
```

Las condiciones se evalúan en orden y solo se ejecuta la primera verdadera. Por eso los rangos deben colocarse de más restrictivo a más general.

Este orden es incorrecto:

```
function categoriaIncorrecta(nota) {  
  if (nota >= 6) return "aprobación";  
  if (nota >= 8) return "promoción"; // nunca se alcanza para 8 o más  
  return "desaprobación";  
}
```

12.5 Guardas y retorno temprano

Una guarda elimina un caso que impide continuar:

```
function procesarPedido(pedido) {  
  if (!pedido) return { ok: false, error: "Pedido ausente" };  
  if (pedido.items.length === 0) {  
    return { ok: false, error: "Carrito vacío" };  
  }  
  if (!pedido.cliente) {  
    return { ok: false, error: "Falta el cliente" };  
  }  
  
  const total = calcularTotal(pedido.items);  
  return { ok: true, total };  
}
```

Sin guardas, el camino válido quedaría dentro de varios niveles. El retorno temprano reduce el estado mental necesario: después de cada guarda sabemos qué condición ya se cumple.

Una guarda puede devolver un resultado esperado o lanzar un error si se rompió el contrato. La sección 13.5 desarrolla esa diferencia.

12.6 Condiciones anidadas

El anidamiento es útil cuando una decisión solo tiene sentido dentro de otra:

```
if (usuario.estaAutenticado) {  
  if (usuario.esAdmin) {  
    mostrarPanelAdministrativo();  
  }  
}
```

Puede combinarse:

```
if (usuario.estaAutenticado && usuario.esAdmin) {  
  mostrarPanelAdministrativo();  
}
```

No toda anidación debe aplanarse. Si las ramas representan pasos jerárquicos con acciones diferentes, conservar la estructura puede ser más claro.

12.7 Negar condiciones compuestas

Las leyes de De Morgan, que presentó la sección 3.13, ayudan a escribir guardas:

```
!(a && b) = !a || !b  
!(a || b) = !a && !b
```



```
function validarCompra(tieneSaldo, hayStock) {
  if (!tieneSaldo || !hayStock) {
    return { ok: false };
  }

  return { ok: true };
}
```

equivale a rechazar cuando no se cumple `tieneSaldo && hayStock`.

12.8 switch: seleccionar por un mismo valor

```
function etiquetaEstado(estado) {
  switch (estado) {
    case "P":
      return "Pendiente";
    case "A":
      return "Aprobado";
    case "R":
      return "Rechazado";
    default:
      return "Desconocido";
  }
}
```

`switch` compara el valor con cada `case` usando igualdad estricta. Es apropiado para códigos, comandos, estados o variantes discretas. Para rangos y condiciones heterogéneas, `if` comunica mejor.

12.9 break, return y fall-through

Si un caso no termina con `break`, `return` o `throw`, la ejecución continúa en el siguiente:

```
function tipoDeDia(dia) {
  switch (dia) {
    case "sábado":
    case "domingo":
      return "fin de semana";
    default:
      return "día hábil";
  }
}
```

Aquí el *fall-through* agrupa dos entradas. Cuando sea intencional pero no tan evidente, agregá un comentario. El olvido accidental de `break` puede ejecutar lógica de otro caso.

12.10 Alcance dentro de `switch`

Los `case` no crean bloques independientes. Dos declaraciones con el mismo nombre pueden colisionar:

```
switch (tipo) {  
  case "usuario": {  
    const resultado = cargarUsuario();  
    usar(resultado);  
    break;  
  }  
  case "curso": {  
    const resultado = cargarCurso();  
    usar(resultado);  
    break;  
  }  
}
```

Las llaves crean un alcance por caso.

12.11 Repetir: elegir el bucle por la pregunta

La elección práctica:

- `for...of`: recorrer valores de una colección iterable;
- `for`: controlar índice, rango o paso;
- `while`: repetir hasta que cambie una condición;
- `do...while`: ejecutar primero y decidir después;
- métodos de array: producir una transformación declarativa.

12.12 `while`: repetir mientras se cumpla

```
let saldo = 1000;  
  
while (saldo >= 250) {  
  saldo -= 250;  
}
```

Antes de ejecutar, identificá:

1. estado inicial: `saldo = 1000`;
2. condición: `saldo >= 250`;
3. progreso: en cada vuelta baja 250;
4. estado de salida: `saldo < 250`.

Un bucle infinito suele perder el progreso:

```
let intentos = 0;

while (intentos < 3) {
  ejecutar();
  // falta intentos += 1
}
```

Los bucles infinitos pueden ser deliberados en servidores o consumidores de eventos, pero necesitan un mecanismo externo de cancelación, espera y manejo de fallos.

12.13 do...while: al menos una ejecución

```
let entrada;

do {
  entrada = solicitarEntrada();
} while (!esValida(entrada));
```

Es apropiado para menús, reintentos interactivos o procesos donde la primera acción debe ocurrir antes de evaluar. No lo uses si el cuerpo podría ser inválido desde el inicio.

12.14 for: inicio, condición y actualización

```
for (let indice = 0; indice < 5; indice += 1) {
  console.log(indice);
}
```

El orden real es:

```
inicialización
→ condición
→ cuerpo
→ actualización
→ condición...
```

Un while equivalente:

```
let indice = 0;

while (indice < 5) {
  console.log(indice);
  indice += 1;
}
```

Elegí for cuando esas tres piezas pertenecen a la misma idea de recorrido.

Recorrer hacia atrás y con paso

```
for (let i = valores.length - 1; i >= 0; i -= 1) {
  console.log(valores[i]);
}

for (let par = 0; par <= 10; par += 2) {
  console.log(par);
}
```

Los límites y el operador (< frente a <=) merecen pruebas en el primer y último elemento.

12.15 for...of: valores de un iterable

```
for (const alumno of alumnos) {
  console.log(alumno.nombre);
}
```

Funciona con arrays, strings, Map, Set, generadores y cualquier objeto que implemente Symbol.iterator, el protocolo que describe la sección 8.7. Para secuencias asíncronicas existe for await...of, que presenta la sección 17.9.

Para Map:

```
for (const [clave, valor] of mapa) {
  console.log(clave, valor);
}
```

Para índices de un array:

```
for (const [indice, alumno] of alumnos.entries()) {
  console.log(indice, alumno.nombre);
}
```

12.16 for...in: nombres de propiedades

```
for (const clave in objeto) {
  if (Object.hasOwn(objeto, clave)) {
    console.log(clave, objeto[clave]);
  }
}
```

Incluye propiedades enumerables heredadas, por eso suele requerir Object.hasOwn. Para objetos comunes, Object.keys, values o entries y for...of son más explícitos. No uses for...in para valores de arrays.

12.17 break: terminar el bucle

```
let encontrado = null;

for (const alumno of alumnos) {
  if (alumno.legajo === buscado) {
    encontrado = alumno;
    break;
  }
}
```

Para este caso concreto, `find` expresa mejor el resultado. `break` es útil cuando el recorrido incluye lógica que no cabe naturalmente en un método de colección.

12.18 continue: saltar a la siguiente vuelta

```
for (const linea of lineas) {
  if (linea.trim() === "") continue;
  procesar(linea);
}
```

En un `while`, verificá que `continue` no salte la actualización:

```
let i = 0;

while (i < valores.length) {
  const valor = valores[i];
  i += 1; // progreso antes de cualquier continue

  if (valor == null) continue;
  procesar(valor);
}
```

12.19 Bucles anidados y etiquetas

`break` termina el bucle más cercano. Una etiqueta puede señalar uno exterior:

```
buscar:
for (let fila = 0; fila < matriz.length; fila += 1) {
  for (let columna = 0; columna < matriz[fila].length; columna += 1) {
    if (matriz[fila][columna] === objetivo) {
      break buscar;
    }
  }
}
```

Las etiquetas son válidas y poco frecuentes. Muchas veces una función con `return` permite una salida más clara y devuelve el resultado encontrado.

12.20 Evitar modificar una colección durante el recorrido

Quitar elementos mientras se avanza puede saltar posiciones:

```
for (let i = 0; i < valores.length; i += 1) {  
  if (valores[i] < 0) valores.splice(i, 1);  
}
```

Después de `splice`, el siguiente elemento ocupa el índice actual, pero `i` avanza. La sección 9.7 describe ese método en detalle. Alternativas:

```
const noNegativos = valores.filter(valor => valor >= 0);
```

o recorrer hacia atrás cuando la mutación sea necesaria.

12.21 Caso integrador: procesar un lote

```
function procesarLineas(lineas) {  
  const resultados = [];  
  
  for (const [numero, lineaOriginal] of lineas.entries()) {  
    const linea = lineaOriginal.trim();  
  
    if (linea === "") continue;  
    if (linea === "FIN") break;  
  
    const separador = linea.indexOf(":");  
  
    if (separador === -1) {  
      resultados.push({  
        linea: numero + 1,  
        ok: false,  
        error: "Falta el separador"  
      });  
      continue;  
    }  
  
    resultados.push({  
      linea: numero + 1,  
      ok: true,  
      clave: linea.slice(0, separador).trim(),  
      valor: linea.slice(separador + 1).trim()  
    });  
  }  
  
  return resultados;  
}
```

El bucle muestra los tres desvíos: ignorar, terminar o procesar. Los resultados negativos esperados se conservan como datos.

12.22 Errores frecuentes

- confundir asignación `=` con comparación `===`;
- ordenar mal condiciones que representan rangos;
- anidar todas las reglas en lugar de usar guardas;
- olvidar `break` en un `switch`;
- crear un `while` sin progreso;
- colocar el progreso después de un `continue` posible;
- usar `for...in` para valores de array;
- modificar un array hacia adelante y saltar elementos;
- usar un ternario anidado para lógica con varias decisiones.

12.23 Para recordar

- Las guardas eliminan casos inválidos y dejan visible el camino principal.
- `if` expresa condiciones generales; `switch`, variantes discretas de un mismo valor.
- Todo bucle necesita estado inicial, condición, progreso y salida.
- `for...of` recorre valores; `for...in`, propiedades enumerables.
- `break` y `continue` deben simplificar el recorrido, no ocultar su terminación.

Capítulo 13. Errores y excepciones

Idea central

Una excepción representa que una operación no pudo cumplir su contrato; debe capturarse únicamente en la capa que sabe recuperarse, agregar contexto o traducirla. Los resultados negativos esperables se modelan como datos; los fallos excepcionales se propagan sin perder su causa.

13.1 Un error cambia el flujo normal

```
const configuracion = JSON.parse(texto);
iniciar(configuracion);
```

Si `JSON.parse` encuentra sintaxis inválida, lanza una excepción. `iniciar` no se ejecuta. La excepción sube por la pila hasta encontrar un `catch` o llegar al entorno de ejecución.

llamada actual → función que llamó → capa superior → entorno

Esta propagación evita que cada función intermedia tenga que comprobar y reenviar manualmente el mismo fallo.

13.2 try...catch

```
try {
  const datos = JSON.parse(texto);
  usar(datos);
} catch (error) {
  console.error("No se pudo leer la configuración", error);
}
```

`try` intenta ejecutar el bloque. Si se lanza una excepción síncrona, el control salta al `catch`. Las líneas posteriores al fallo dentro del `try` no se ejecutan.

El parámetro puede omitirse cuando no se necesita:

```
function esJsonValido(texto) {
  try {
    JSON.parse(texto);
    return true;
  } catch {
    return false;
  }
}
```


No conviertas todo error en `false` si quien llama necesita conocer la causa.

13.3 El objeto `Error`

```
const error = new Error("No se pudo guardar el pedido");

error.name;    // "Error"
error.message; // mensaje
error.stack;   // pila, dependiente del entorno
```

Lanzá objetos `Error`, no strings:

```
throw new Error("Falta el archivo de configuración");
```

Una string lanzada puede capturarse, pero no ofrece una interfaz consistente para nombre, pila y causa.

13.4 `throw`: declarar que el contrato no puede cumplirse

```
function dividir(dividendo, divisor) {
  if (divisor === 0) {
    throw new RangeError("El divisor no puede ser cero");
  }

  return dividendo / divisor;
}
```

En JavaScript numérico, dividir por cero normalmente devuelve `Infinity`. La función decide imponer un contrato más estricto porque su dominio lo necesita.

`throw` acepta cualquier expresión, pero mantener objetos `Error` como convención simplifica el manejo.

13.5 Resultado esperado frente a excepción

Buscar un alumno puede no encontrarlo:

```
function buscarAlumno(alumnos, legajo) {
  return alumnos.find(alumno => alumno.legajo === legajo) ?? null;
}
```

El resultado `null` es una variante normal. En cambio, recibir un legajo con formato imposible puede romper el contrato:

```
function buscarAlumno(alumnos, legajo) {
  if (!Number.isSafeInteger(legajo) || legajo <= 0) {
    throw new TypeError("Legajo inválido");
  }

  return alumnos.find(alumno => alumno.legajo === legajo) ?? null;
}
```

Preguntá: ¿quien llama espera decidir con frecuencia sobre este resultado? Si sí, modelalo como dato. ¿La función no puede mantener una garantía que prometía? Una excepción puede ser adecuada.

13.6 Tipos integrados de error

Algunos errores frecuentes:

- `TypeError`: valor u operación de tipo incompatible;
- `RangeError`: valor fuera del rango permitido;
- `ReferenceError`: identificador no disponible;
- `SyntaxError`: texto o código con sintaxis inválida;
- `URIError`: codificación o decodificación URI inválida;
- `AggregateError`: varios errores reunidos.

```
try {
  const datos = JSON.parse(texto);
} catch (error) {
  if (error instanceof SyntaxError) {
    informarJsonInvalido(error.message);
  } else {
    throw error;
  }
}
```

No dependas solo del texto del mensaje, que puede cambiar o localizarse. Tipos, códigos y propiedades estables son mejores para decisiones.

13.7 La pila de llamadas

```
function nivelTres() {  
  throw new Error("fallo");  
}  
  
function nivelDos() {  
  nivelTres();  
}  
  
function nivelUno() {  
  nivelDos();  
}  
  
nivelUno();
```

La pila registra cómo se llegó al punto del error. Capturar y crear un error nuevo sin causa puede perder parte de ese diagnóstico.

Esa pila no es una metáfora: es la estructura de datos que describe el apéndice B, administrada por el motor. La sección 16.1 muestra cómo crece con cada llamada recursiva y por qué tiene un límite.

13.8 Errores personalizados

```
class SaldoInsuficienteError extends Error {  
  constructor({ disponible, requerido }, options) {  
    super("El saldo no alcanza para completar la operación", options);  
    this.name = "SaldoInsuficienteError";  
    this.disponible = disponible;  
    this.requerido = requerido;  
  }  
}
```

```
function debitar(cuenta, importe) {  
  if (cuenta.saldo < importe) {  
    throw new SaldoInsuficienteError({  
      disponible: cuenta.saldo,  
      requerido: importe  
    });  
  }  
  
  return { ...cuenta, saldo: cuenta.saldo - importe };  
}
```

El tipo permite tratar este caso sin analizar una string. Las propiedades ofrecen datos estructurados para interfaz, registro y pruebas.

13.9 Conservar la causa

Una capa puede traducir un error técnico a uno de dominio:

```
class ConfiguracionError extends Error {
  constructor(mensaje, options) {
    super(mensaje, options);
    this.name = "ConfiguracionError";
  }
}

function interpretarConfiguracion(texto) {
  try {
    return JSON.parse(texto);
  } catch (cause) {
    throw new ConfiguracionError(
      "El archivo de configuración no es válido",
      { cause }
    );
  }
}
```

`error.cause` mantiene el fallo original. Agregar contexto no debería borrar evidencia.

13.10 Capturar solo lo que podemos manejar

Un `try` demasiado amplio no permite saber qué operación produjo el error esperado:

```
try {
  const datos = JSON.parse(texto);
  const normalizados = normalizar(datos);
  await guardar(normalizados);
  notificar(normalizados);
} catch (error) {
  // ¿falló el JSON, la validación, el disco o la notificación?
}
```

Reducí la zona o distinguí errores por tipo y código. Una captura útil realiza al menos una de estas tareas:

- recupera con una alternativa válida;
- agrega contexto y relanza;
- traduce a un error del dominio;
- registra en una frontera del proceso;
- convierte el error en una respuesta externa apropiada.

Capturar y silenciar deja al programa continuar con un estado posiblemente incompleto.

13.11 Relanzar

```
try {
  ejecutar();
} catch (error) {
  if (error instanceof ErrorEsperado) {
    recuperar(error);
  } else {
    throw error;
  }
}
```

Usá `throw error`, no `throw new Error(error.message)`, si no necesitás envolverlo; la segunda forma reemplaza identidad y pila.

13.12 finally: limpieza garantizada

```
const recurso = await abrirRecurso();

try {
  await usarRecurso(recurso);
} finally {
  await recurso.close();
}
```

`finally` se ejecuta si el `try` termina normalmente, retorna o lanza. También se ejecuta después de un `catch`.

```
function ejemplo() {
  try {
    return "resultado";
  } finally {
    console.log("limpieza");
  }
}
```

No retornes desde `finally`: ese retorno puede reemplazar el resultado o silenciar una excepción.

`try...finally` puede usarse sin `catch` para limpiar y dejar que el error continúe.

13.13 Promesas y `async/await`

Una promesa rechazada se captura si se espera dentro del `try`:

```
try {
  const datos = await cargarDatos();
  usar(datos);
} catch (error) {
  manejar(error);
}
```

Crear una promesa sin esperarla permite que el `try` termine antes del rechazo:

```
try {
  cargarDatos(); // no se espera ni se devuelve
} catch (error) {
  // no captura un rechazo posterior
}
```

Corregí con `await` o devolví la promesa a quien deba manejarla.

La forma equivalente con promesas:

```
cargarDatos()
  .then(usar)
  .catch(manejar)
  .finally(limpiar);
```

No mezcles estilos sin una razón clara; una cadena olvidada o un `await` ausente puede producir rechazos no manejados. El capítulo 17 desarrolla el modelo completo de `async` y `await`.

13.14 La sutileza de `fetch`

`fetch` rechaza ante fallos de red o cancelación, pero normalmente resuelve ante HTTP 404 o 500; la sección 22.8 muestra por qué, a partir de cómo llega una respuesta HTTP. Hay que comprobar el estado:

```
async function cargarUsuario(id) {
  const respuesta = await fetch(`/api/usuarios/${id}`);

  if (!respuesta.ok) {
    throw new Error(`HTTP ${respuesta.status}`);
  }

  return respuesta.json();
}
```

Una API de aplicación puede traducir estados:

```

class UsuarioNoEncontradoError extends Error {}

async function cargarUsuario(id) {
  const respuesta = await fetch(`/api/usuarios/${id}`);

  if (respuesta.status === 404) {
    throw new UsuarioNoEncontradoError(`No existe el usuario ${id}`);
  }

  if (!respuesta.ok) {
    throw new Error(`Error del servicio: ${respuesta.status}`);
  }

  return respuesta.json();
}

```

13.15 Errores concurrentes

`Promise.all` rechaza con el primer rechazo observado:

```

await Promise.all(tareas.map(ejecutar));

```

Las demás tareas no se cancelan automáticamente. Si necesitas conocer todos los resultados:

```

const resultados = await Promise.allSettled(
  tareas.map(ejecutar)
);

```

Cada elemento indica `fulfilled` con `value` o `rejected` con `reason`. `AggregateError` aparece en APIs como `Promise.any` cuando todas las alternativas fallan. La sección 17.7 compara los cuatro combinadores.

13.16 Mensaje al usuario y diagnóstico técnico

No expongas automáticamente `error.stack`, rutas, consultas o datos internos. Separá:

```

registrarError({
  operacion: "crear-pedido",
  pedidoId,
  error
});

const respuesta = {
  ok: false,
  mensaje: "No pudimos completar el pedido. Intentá nuevamente."
};

```

El registro necesita contexto técnico; la interfaz necesita una acción comprensible y un identificador de incidente cuando corresponda.

13.17 No usar excepciones como un `if` sofisticado

Esto oculta una condición esperable:

```
try {  
  if (!hayStock) throw new Error("sin stock");  
  reservar();  
} catch {  
  mostrarSinStock();  
}
```

Si `sin stock` es una variante normal, modelala directamente:

```
function intentarReserva(hayStock) {  
  if (!hayStock) return { ok: false, motivo: "sin-stock" };  
  return { ok: true };  
}
```

Una excepción puede seguir siendo válida si una función inferior prometía reservar y no pudo cumplir; el diseño depende de la capa y el contrato.

13.18 Caso integrador

```
class PedidoInvalidoError extends Error {
  constructor(mensaje, options) {
    super(mensaje, options);
    this.name = "PedidoInvalidoError";
  }
}

async function procesarPedido(texto) {
  let pedido;

  try {
    pedido = JSON.parse(texto);
  } catch (cause) {
    throw new PedidoInvalidoError("JSON inválido", { cause });
  }

  validarPedido(pedido);

  try {
    const respuesta = await enviarPedido(pedido);

    if (!respuesta.ok) {
      throw new Error(`HTTP ${respuesta.status}`);
    }

    return await respuesta.json();
  } catch (cause) {
    throw new Error(`No se pudo enviar el pedido ${pedido.id}`, { cause });
  }
}
```

Cada `try` tiene un propósito concreto y agrega el contexto disponible en esa etapa.

13.19 Errores frecuentes

- lanzar strings;
- capturar `Error` y continuar sin recuperación;
- envolver un error sin `cause` y perder diagnóstico;
- usar excepciones para decisiones normales;
- poner demasiadas operaciones dentro de un mismo `try`;
- retornar desde `finally`;
- olvidar `await` o no devolver una promesa;
- suponer que `fetch` rechaza ante todos los estados HTTP;
- mostrar detalles internos al usuario final.

13.20 Para recordar

- Una excepción indica que una operación no pudo cumplir su contrato.
- Los casos negativos esperables son datos; los fallos inesperados se propagan.
- Capturá solo donde puedas recuperar, traducir, agregar contexto o registrar.
- Conservá la causa original y usá `finally` para limpieza.
- En asincronía, una promesa solo entra al `try...catch` si se espera o se devuelve correctamente.

Capítulo 14. Funciones

Idea central

Una función convierte una operación en una unidad con nombre, entradas, resultado y contrato. Cuanto más explícitas sean sus dependencias y más acotada su responsabilidad, más fácil será reutilizarla, probarla y combinarla.

14.1 Definir no es ejecutar

```
function sumar(a, b) {  
  return a + b;  
}
```

La declaración crea la función. La llamada la ejecuta:

```
const resultado = sumar(2, 3); // 5
```

Sin paréntesis obtenemos el valor función:

```
const operacion = sumar;  
operacion(10, 20); // 30
```

Esta separación permite pasar funciones como datos.

14.2 Anatomía de una función

```
function calcularPrecioFinal(precio, descuento) {  
  const rebaja = precio * descuento;  
  return precio - rebaja;  
}
```

- `calcularPrecioFinal`: nombre;
- `precio, descuento`: parámetros;
- cuerpo entre llaves: instrucciones;
- `return`: salida y finalización de la llamada.

Un contrato posible:

```
precio finito no negativo + descuento entre 0 y 1  
→ precio final finito no negativo
```

La sintaxis no expresa todo el contrato; nombres, validaciones, tipos, documentación y pruebas lo completan.

14.3 Parámetros y argumentos

Los parámetros son nombres locales de la definición; los argumentos son valores de una llamada:

```
function presentar(nombre, edad) {  
  return `${nombre} tiene ${edad} años`;  
}  
  
presentar("Ana", 20);
```

JavaScript no exige la cantidad exacta:

```
presentar("Ana");           // edad es undefined  
presentar("Ana", 20, 99); // el argumento adicional se ignora
```

Que el lenguaje lo permita no significa que el contrato deba aceptarlo.

14.4 Declaraciones de función

```
function duplicar(numero) {  
  return numero * 2;  
}
```

La declaración completa se eleva, por lo que puede llamarse antes en el mismo alcance:

```
duplicar(5);  
  
function duplicar(numero) {  
  return numero * 2;  
}
```

Esto permite colocar la función principal antes de detalles auxiliares. No hace falta depender de la elevación si el orden natural ya es claro.

14.5 Expresiones de función

```
const duplicar = function (numero) {  
  return numero * 2;  
};
```

La variable sigue las reglas de `const`; no puede usarse antes de la inicialización.

Una expresión puede tener nombre interno:

```
const factorial = function calcular(n) {
  if (n <= 1) return 1;
  return n * calcular(n - 1);
};
```

El nombre mejora pilas de error y permite recursión sin depender del nombre exterior.

14.6 Funciones flecha

```
const duplicar = numero => numero * 2;
```

Formas:

```
const constante = () => 42;
const sumar = (a, b) => a + b;
const procesar = valor => {
  const normalizado = Number(valor);
  return normalizado * 2;
};
```

Para devolver un objeto literal de forma implícita, hacen falta paréntesis:

```
const crearAlumno = nombre => ({ nombre, activo: true });
```

Sin paréntesis, las llaves se interpretan como cuerpo.

14.7 Flechas y funciones tradicionales no son idénticas

Las flechas:

- no crean su propio `this`;
- no tienen `arguments` propio;
- no pueden llamarse con `new`;
- no tienen `prototype` para instancias;
- no pueden ser generadores.

Son excelentes para callbacks que deben conservar el contexto exterior:

```
const temporizador = {
  segundos: 0,
  iniciar() {
    setInterval(() => {
      this.segundos += 1;
    }, 1000);
  }
};
```

La flecha usa el `this` de `iniciar`.

Para un método que recibe `this` del objeto, usá sintaxis de método o función tradicional:

```
const cuenta = {
  saldo: 100,
  depositar(importe) {
    this.saldo += importe;
  }
};
```

14.8 `this` depende de la llamada

En una función tradicional, `this` no queda determinado solo por dónde fue escrita:

```
const persona = {
  nombre: "Ana",
  saludar() {
    return `Hola, ${this.nombre}`;
  }
};

persona.saludar(); // this es persona
```

Separar el método puede perder el receptor:

```
const saludar = persona.saludar;
// saludar(); // this es undefined en modo estricto
```

`bind` crea una función con receptor fijado:

```
const saludarAna = persona.saludar.bind(persona);
```

`call` y `apply` invocan inmediatamente con un `this` elegido:

```
persona.saludar.call({ nombre: "Luis" });
```

No uses `this` cuando parámetros explícitos harían la dependencia más clara.

14.9 Valores predeterminados

```
function saludar(nombre, saludo = "Hola") {
  return `${saludo}, ${nombre}`;
}
```

El predeterminado se aplica ante argumento omitido o `undefined`, no `null`. Puede depender de parámetros anteriores:

```
function crearRango(inicio, fin = inicio, paso = 1) {
  // ...
}
```

La expresión predeterminada se evalúa en cada llamada, lo que evita compartir accidentalmente un array creado allí:

```
function agregar(valor, lista = []) {
  lista.push(valor);
  return lista;
}
```

Cada llamada sin segundo argumento recibe un array nuevo.

14.10 Parámetros rest

```
function sumar(...numeros) {
  return numeros.reduce((total, numero) => total + numero, 0);
}

sumar(1, 2, 3); // 6
```

El rest debe ser el último parámetro y siempre es un array. Reemplaza muchos usos del objeto histórico `arguments`.

```
function registrar(nivel, ...mensajes) {
  console.log(nivel, mensajes.join(" "));
}
```

14.11 arguments

Las funciones tradicionales tienen un objeto similar a array:

```
function cantidadDeArgumentos() {
  return arguments.length;
}
```

No es un array real, aunque es iterable en entornos modernos. Rest ofrece un contrato visible, funciona en flechas y da directamente un array.

14.12 Parámetros desestructurados

```
function calcularTotal({ precio, cantidad = 1, descuento = 0 }) {
  return precio * cantidad * (1 - descuento);
}
```

La llamada se vuelve descriptiva:

```
calcularTotal({ precio: 100, descuento: 0.1, cantidad: 2 });
```

También arrays:

```
function distancia([x1, y1], [x2, y2]) {  
  return Math.hypot(x2 - x1, y2 - y1);  
}
```

La desestructuración no sustituye la validación. Una entrada `null` falla antes de entrar al cuerpo.

14.13 Pasaje de argumentos: siempre por valor

Con primitivos, la función recibe una copia del valor:

```
function duplicar(numero) {  
  numero *= 2;  
}  
  
let cantidad = 3;  
duplicar(cantidad);  
cantidad; // 3
```

Con objetos, el valor copiado es una referencia. La función puede modificar el mismo objeto:

```
function cumplirAnios(persona) {  
  persona.edad += 1;  
}  
  
const ana = { edad: 20 };  
cumplirAnios(ana);  
ana.edad; // 21
```

Reemplazar la referencia local no reemplaza la exterior:

```
function reemplazar(persona) {  
  persona = { edad: 0 };  
}  
  
reemplazar(ana);  
ana.edad; // sigue 21
```

Decir “los objetos se pasan por referencia” es una simplificación que confunde este último caso. Se pasa por valor una referencia.

14.14 return: resultado y salida

```
function absoluto(numero) {  
  if (numero >= 0) return numero;  
  return -numero;  
}
```

Una función sin `return`, o con `return;`, devuelve `undefined`.

`console.log` no reemplaza el retorno:

```
function dobleIncorrecto(n) {  
  console.log(n * 2);  
}  
  
const valor = dobleIncorrecto(3); // undefined
```

El valor mostrado no puede componerse en otro cálculo.

14.15 Inserción automática después de return

Un salto inmediatamente después de `return` puede terminar la sentencia:

```
function incorrecta() {  
  return  
  {  
    ok: true  
  };  
}
```

Devuelve `undefined`. La llave del objeto debe comenzar en la misma línea o ir entre paréntesis:

```
function correcta() {  
  return {  
    ok: true  
  };  
}
```

14.16 Devolver varios datos

Un objeto ofrece nombres:

```
function analizar(numeros) {
  return {
    minimo: Math.min(...numeros),
    maximo: Math.max(...numeros)
  };
}

const { minimo, maximo } = analizar([3, 1, 7]);
```

Un array es adecuado si las posiciones tienen una convención breve y estable:

```
function cocienteYResto(a, b) {
  return [Math.trunc(a / b), a % b];
}

const [cociente, resto] = cocienteYResto(10, 3);
```

Para contratos públicos, los nombres suelen evolucionar mejor.

14.17 Alcance de función y alcance léxico

```
const tasa = 0.21;

function conImpuesto(precio) {
  const impuesto = precio * tasa;
  return precio + impuesto;
}
```

La función consulta `tasa` en el lugar donde fue definida, no en el lugar desde donde se llama, según la regla de alcance léxico que presentó la sección 2.3. Esa es la base de las clausuras.

14.18 Clausuras

```
function crearMultiplicador(factor) {
  return numero => numero * factor;
}

const duplicar = crearMultiplicador(2);
const triplicar = crearMultiplicador(3);
```

Las funciones devueltas conservan su propio `factor`.

Una clausura puede mantener estado:

```
function crearContador(inicial = 0) {
  let valor = inicial;

  return {
    incrementar() {
      valor += 1;
      return valor;
    },
    leer() {
      return valor;
    }
  };
}
```

`valor` no es accesible directamente. Cada llamada a `crearContador` crea una variable independiente.

La clausura conserva la variable, no una fotografía:

```
function ejemplo() {
  let valor = 1;
  const leer = () => valor;
  valor = 2;
  return leer;
}

ejemplo()(); // 2
```

14.19 Clausuras y bucles

`let` crea una vinculación por iteración:

```
const funciones = [];

for (let i = 0; i < 3; i += 1) {
  funciones.push(() => i);
}

funciones.map(fn => fn()); // [0, 1, 2]
```

Con `var`, todas compartirían la misma variable final y devolverían `3`. Esta diferencia fue una razón importante para adoptar `let`, como anticipó la sección 2.2.

14.20 Funciones como valores

```
const operaciones = {
  sumar: (a, b) => a + b,
  restar: (a, b) => a - b
};

operaciones.sumar(3, 2);
```

Pueden almacenarse en arrays, objetos, mapas y pasarse a otras funciones.

14.21 Callbacks y funciones de orden superior

Una función de orden superior recibe o devuelve funciones:

```
function aplicar(operacion, a, b) {
  return operacion(a, b);
}

aplicar((a, b) => a * b, 3, 4); // 12
```

El callback puede ejecutarse inmediatamente, varias veces o en el futuro. El contrato debe aclararlo, especialmente si puede ser asíncronico. El capítulo 15 construye sobre esta idea toda una forma de organizar transformaciones.

14.22 Recursividad

```
function factorial(n) {
  if (!Number.isInteger(n) || n < 0) {
    throw new RangeError("n debe ser un entero no negativo");
  }

  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
```

Toda recursión necesita caso base y reducción. Para secuencias lineales grandes, un bucle evita límites de pila. El capítulo 16 la aplica a árboles.

14.23 Funciones asíncronicas

Una función `async` siempre devuelve una promesa:

```

async function cargarUsuario(id) {
  const respuesta = await fetch(`/api/usuarios/${id}`);
  if (!respuesta.ok) throw new Error(`HTTP ${respuesta.status}`);
  return respuesta.json();
}

const usuario = await cargarUsuario(10);

```

Un `return` valor se convierte en una promesa resuelta; un `throw`, en una rechazada. `await` pausa esa función, no todo el proceso. El capítulo 17 desarrolla el tema.

14.24 Generadores

Una función generadora puede pausar y producir varios valores:

```

function* rango(inicio, fin) {
  for (let valor = inicio; valor <= fin; valor += 1) {
    yield valor;
  }
}

const iterador = rango(1, 3);
iterador.next(); // { value: 1, done: false }
[...rango(1, 3)]; // [1, 2, 3]

```

El cuerpo no se ejecuta al crear el iterador; avanza bajo demanda. Un generador implementa el protocolo iterable de la sección 8.7 sin escribirlo a mano, y por eso reaparece en la evaluación diferida de la sección 15.18, en los recorridos de árbol de la sección 16.9 y en el recorrido de directorios de la sección 19.23.

Los generadores asincrónicos combinan `async function*`, `await` y `yield`, y se consumen con `for await...of`.

14.25 Diseñar una función productiva

Una buena función:

- tiene un nombre que expresa una acción o cálculo;
- realiza una responsabilidad principal;
- recibe sus dependencias relevantes;
- devuelve datos reutilizables en lugar de solo imprimir;
- no modifica argumentos sin que el contrato lo anuncie;
- valida en el nivel correcto;
- mantiene una interfaz pequeña;
- permite comprobar casos normales, límites y errores.

Ejemplo con dependencia explícita:

```
function crearServicioDeUsuarios({ repositorio, reloj, generarId }) {
  return {
    async registrar(datos) {
      const usuario = {
        id: generarId(),
        creadoEn: reloj.ahora(),
        ...datos
      };

      await repositorio.guardar(usuario);
      return usuario;
    }
  };
}
```

Las dependencias pueden sustituirse en pruebas sin depender de variables globales.

14.26 Errores frecuentes

- confundir la función con su ejecución;
- olvidar `return`;
- usar una flecha como método esperando un `this` propio;
- mutar un argumento sin avisar;
- depender de demasiados globales;
- crear parámetros opcionales que ocultan errores;
- escribir una función enorme con decisiones y efectos mezclados;
- olvidar `await` o no devolver la promesa;
- usar recursión sin reducción o caso base.

14.27 Para recordar

- Definir una función crea un valor; llamarla ejecuta una nueva invocación.
- Declaraciones, expresiones y flechas comparten capacidades, pero difieren en elevación, `this`, `arguments` y construcción.
- JavaScript pasa todos los argumentos por valor; para objetos, ese valor es una referencia.
- Una clausura conserva acceso al alcance léxico y puede encapsular configuración o estado.
- Una función productiva tiene contrato, dependencias explícitas y resultado comprobable.

Capítulo 15. Programación funcional y pipelines de datos

Idea central

La programación funcional organiza una solución como composición de transformaciones, favorece datos inmutables y concentra los efectos en los bordes. En JavaScript no es una obligación de pureza absoluta: es un conjunto de herramientas para reducir estados implícitos y volver comprobable cada paso.

15.1 Funciones como datos

La base es que una función puede almacenarse, pasarse y devolverse:

```
const duplicar = numero => numero * 2;

function aplicar(funcion, valor) {
  return funcion(valor);
}

aplicar(duplicar, 5); // 10
```

Una función que recibe o devuelve funciones es de orden superior:

```
function mayorQue(limite) {
  return valor => valor > limite;
}

const mayorQueDiez = mayorQue(10);
```

Las funciones especializadas conservan configuración mediante clausuras.

15.2 Pureza: misma entrada, mismo resultado

Una función pura:

1. devuelve el mismo resultado para las mismas entradas;
2. no cambia estado observable fuera de ella.

```
function calcularIva(precio, tasa) {
  return precio * tasa;
}
```

Esta función depende de estado externo y produce resultados diferentes con la misma entrada:

```
let tasaActual = 0.21;

function calcularIvaImplicito(precio) {
  return precio * tasaActual;
}
```

Pasar la tasa vuelve visible la dependencia.

Otros efectos incluyen:

- modificar argumentos o variables globales;
- escribir archivos o bases de datos;
- enviar una solicitud;
- leer el reloj o generar azar;
- imprimir;
- lanzar una excepción observable.

Los efectos no son malos: una aplicación debe interactuar con el mundo. El objetivo es saber dónde ocurren.

15.3 Núcleo funcional, bordes imperativos

```
const texto = await readFile(ruta, "utf8"); // efecto de entrada
const datos = JSON.parse(texto);           // transformación, puede fallar
const resumen = resumir(datos);            // núcleo puro
await writeFile(salida, JSON.stringify(resumen)); // efecto de salida
```

El núcleo puro puede probarse con valores en memoria. Los bordes son responsables de recursos, errores e integración.

15.4 Inmutabilidad

En lugar de cambiar un valor compartido, se produce una nueva versión:

```
const alumno = { nombre: "Ana", nota: 7 };

const actualizado = {
  ...alumno,
  nota: 8
};
```

Con arrays:

```
const numeros = [1, 2, 3];
const conCuatro = [...numeros, 4];
const dobles = numeros.map(numero => numero * 2);
```


La inmutabilidad aporta:

- historial de estados más fácil de reconstruir;
- menos interferencia entre consumidores;
- comparación por identidad para detectar cambios;
- pruebas sin preparación y limpieza compartida.

No exige clonar profundamente todo. Se copia el camino modificado y se comparten ramas que nadie mutará; la sección 16.12 muestra esa técnica sobre un árbol y el apéndice D la aplica a una aplicación completa.

15.5 Mutación local controlada

Esta función es pura desde afuera aunque use un acumulador mutable local:

```
function indexarPorId(items) {  
  const indice = new Map();  
  
  for (const item of items) {  
    indice.set(item.id, item);  
  }  
  
  return indice;  
}
```

La pureza observable importa más que prohibir toda asignación interna. Para la misma entrada y sin mutarla, devuelve un mapa equivalente y no cambia estado exterior.

15.6 map: una salida por entrada

```
const preciosConIva = precios.map(precio => precio * 1.21);
```

Con objetos:

```
const alumnosConEstado = alumnos.map(alumno => ({  
  ...alumno,  
  estado: alumno.nota >= 6 ? "aprobado" : "desaprobado"  
}));
```

El capítulo 9 presentó estos métodos desde el array; acá interesa cómo están hechos y cómo se combinan. El capítulo 9 presentó estos métodos desde el array; acá interesa cómo están hechos y cómo se combinan. Una implementación didáctica:

```
function map(valores, transformar) {
  const resultado = [];

  for (let i = 0; i < valores.length; i += 1) {
    resultado.push(transformar(valores[i], i, valores));
  }

  return resultado;
}
```

El contrato garantiza orden y cantidad, no que el callback sea puro.

15.7 filter: conservar lo que cumple

```
const mayores = alumnos.filter(alumno => alumno.edad >= 18);
```

Implementación:

```
function filter(valores, predicado) {
  const resultado = [];

  for (let i = 0; i < valores.length; i += 1) {
    const valor = valores[i];
    if (predicado(valor, i, valores)) resultado.push(valor);
  }

  return resultado;
}
```

El predicado se interpreta en contexto booleano. Para reglas críticas, devolvé un booleano claro.

15.8 reduce: acumular cualquier estructura

Suma:

```
const total = importes.reduce(
  (suma, importe) => suma + importe,
  0
);
```

Conteo por categoría:

```
const conteos = productos.reduce((mapa, producto) => {
  const anterior = mapa.get(producto.categoria) ?? 0;
  mapa.set(producto.categoria, anterior + 1);
  return mapa;
}, new Map());
```

Implementación didáctica:

```
function reduce(valores, combinar, inicial) {
  let acumulador = inicial;

  for (let i = 0; i < valores.length; i += 1) {
    acumulador = combinar(acumulador, valores[i], i, valores);
  }

  return acumulador;
}
```

`reduce` es poderoso y puede ocultar intención. Si el acumulador tiene muchas mutaciones y bifurcaciones, un bucle y variables nombradas pueden ser más fáciles de mantener.

15.9 Consultas especializadas

```
alumnos.find(alumno => alumno.legajo === 10);
alumnos.findIndex(alumno => alumno.legajo === 10);
alumnos.some(alumno => alumno.nota >= 8);
alumnos.every(alumno => alumno.asistencia >= 75);
```

`some` se detiene al primer `true`; `every`, al primer `false`. Son preferibles a reducir booleanos porque expresan la pregunta y permiten cortocircuito.

15.10 `forEach`: efecto, no transformación

```
alumnos.forEach(alumno => registrar(alumno));
```

Usalo cuando el objetivo sea ejecutar un efecto por elemento. Si necesitás un array, `map`; si necesitás control de `break` o `await` secuencial, `for...of`.

15.11 `flatMap`: transformar a cero, uno o varios elementos

```
const materias = alumnos.flatMap(alumno => alumno.materias);
```

Puede filtrar y transformar en un paso:

```
const errores = filas.flatMap((fila, indice) => {
  const error = validar(fila);
  return error ? [{ fila: indice + 1, error }] : [];
});
```

Solo aplanar un nivel.

15.12 Ordenar sin mutar

La sección 9.15 comparó `sort` con `toSorted`. Acá interesa la segunda: devuelve un array nuevo y deja intacto el original.

La sección 9.15 comparó `sort` con `toSorted`. Acá interesa la segunda: devuelve un array nuevo y deja intacto el original.

```
const porNota = alumnos.toSorted((a, b) => a.nota - b.nota);
```

Para varios criterios:

```
const porCursoYNombre = alumnos.toSorted((a, b) => {  
  const porCurso = a.curso.localeCompare(b.curso, "es");  
  if (porCurso !== 0) return porCurso;  
  return a.nombre.localeCompare(b.nombre, "es");  
});
```

Un comparador debe ser coherente: comparar el mismo par debe mantener el sentido, y elementos equivalentes deben producir cero.

15.13 Encadenar un pipeline

```
const totalDeAprobados = alumnos  
  .filter(alumno => alumno.nota >= 6)  
  .map(alumno => alumno.nota)  
  .reduce((suma, nota) => suma + nota, 0);
```

Se lee como una secuencia:

```
seleccionar → transformar → acumular
```

Cada etapa debería tener una responsabilidad. Si el callback es complejo, nombralo:

```
const estaAprobado = alumno => alumno.nota >= 6;  
const obtenerNota = alumno => alumno.nota;  
const sumar = (a, b) => a + b;  
  
const total = alumnos  
  .filter(estaAprobado)  
  .map(obtenerNota)  
  .reduce(sumar, 0);
```

No extraigas una función si el nombre no agrega información y solo obliga a saltar por el archivo.

15.14 Composición de funciones

```
const recortar = texto => texto.trim();
const minusculas = texto => texto.toLowerCase();
const quitarEspacios = texto => texto.replaceAll(" ", "-");

const slug = texto => quitarEspacios(minusculas(recortar(texto)));
```

Una utilidad de composición de izquierda a derecha:

```
const pipe = (...funciones) => valor =>
  funciones.reduce((actual, funcion) => funcion(actual), valor);

const crearSlug = pipe(recortar, minusculas, quitarEspacios);
```

Para funciones con errores, asincronía o varios parámetros, una utilidad demasiado genérica puede ocultar el flujo. La composición explícita sigue siendo válida.

15.15 Negar y especializar predicados

```
const negar = predicado => valor => !predicado(valor);

const esPar = numero => numero % 2 === 0;
const esImpar = negar(esPar);
```

Combinar:

```
const y = (...predicados) => valor =>
  predicados.every(predicado => predicado(valor));

const esAdulto = persona => persona.edad >= 18;
const estaActivo = persona => persona.activo;
const puedeIngresar = y(esAdulto, estaActivo);
```

Mantén estas abstracciones si el vocabulario del dominio las vuelve legibles.

15.16 Equivalencias aproximadas con LINQ

Para quien viene de C#:

LINQ	JavaScript
Where	filter
Select	map
Aggregate	reduce
Any	some
All	every
FirstOrDefault	find con undefined
SelectMany	flatMap
Skip	slice(inicio)
Take	slice(0, cantidad)
Distinct	new Set(valores)
OrderBy	toSorted(comparador)

Ejemplos:

```
const suma = numeros.reduce((a, b) => a + b, 0);

const promedio = numeros.length === 0
  ? null
  : suma / numeros.length;

const pagina = items.slice(desde, desde + cantidad);

const distintos = [...new Set(valores)];
```

Las equivalencias son semánticas, no idénticas. LINQ sobre `IEnumerable` suele ser diferido; los métodos de arrays de JavaScript son inmediatos.

15.17 Evaluación inmediata y arrays intermedios

```
const resultado = datos
  .filter(condicion)
  .map(transformacion)
  .slice(0, 10);
```

`filter` recorre y crea un array; `map` vuelve a recorrer y crea otro; `slice` crea un tercero. Para colecciones habituales, el costo puede ser irrelevante y la claridad valiosa.

Si una medición demuestra un problema, un solo bucle evita intermedios:

```
const resultado = [];

for (const dato of datos) {
  if (!condicion(dato)) continue;
  resultado.push(transformacion(dato));
  if (resultado.length === 10) break;
}
```

La versión también deja de recorrer al obtener diez resultados, algo que el pipeline de arrays no logra antes de filtrar y mapear todo.

15.18 Evaluación diferida con generadores

```
function* filtrar(iterable, predicado) {
  for (const valor of iterable) {
    if (predicado(valor)) yield valor;
  }
}

function* transformar(iterable, funcion) {
  for (const valor of iterable) {
    yield funcion(valor);
  }
}

function* tomar(iterable, cantidad) {
  if (cantidad <= 0) return;

  let usados = 0;
  for (const valor of iterable) {
    yield valor;
    usados += 1;
    if (usados === cantidad) return;
  }
}
```

```
const pares = filtrar(rango(1, 1_000_000), n => n % 2 === 0);
const cuadrados = transformar(pares, n => n * n);
const primeros = [...tomar(cuadrados, 5)];
```

Solo se producen los valores consumidos. La sección 14.24 presenta los generadores y la sección 19.23 los usa para recorrer un árbol de directorios sin construir la lista completa. La evaluación diferida agrega complejidad; usala para secuencias grandes, infinitas o costosas, no como ritual.

15.19 Callbacks reciben más de un argumento

```
["10", "10", "10"].map(parseInt);
```

Puede producir `[10, NaN, 2]` porque `map` pasa `(valor, indice)` y `parseInt` interpreta el segundo argumento como base.

```
["10", "10", "10"].map(texto => parseInt(texto, 10));
```

No pases una función existente como callback sin comprobar que su firma sea compatible.

15.20 Flechas y `return`

```
const correcto = numeros.map(numero => numero * 2);

const tambienCorrecto = numeros.map(numero => {
  return numero * 2;
});

const incorrecto = numeros.map(numero => {
  numero * 2;
}); // [undefined, ...]
```

Las llaves eliminan el retorno implícito.

15.21 Caso integrador

```
function resumirVentas(ventas) {
  const validas = ventas.filter(venta =>
    Number.isFinite(venta.precio) &&
    Number.isSafeInteger(venta.cantidad) &&
    venta.cantidad > 0
  );

  const detalle = validas.map(venta => ({
    id: venta.id,
    vendedor: venta.vendedor,
    total: venta.precio * venta.cantidad
  }));

  const totalGeneral = detalle.reduce(
    (suma, venta) => suma + venta.total,
    0
  );

  const porVendedor = detalle.reduce((mapa, venta) => {
    mapa.set(
      venta.vendedor,
      (mapa.get(venta.vendedor) ?? 0) + venta.total
    );
    return mapa;
  }, new Map());

  return {
    validas: detalle,
    descartadas: ventas.length - validas.length,
    totalGeneral,
    porVendedor
  };
}
```

Las etapas separan validación, proyección y agregación. La mutación del `Map` es local y no cambia la entrada.

15.22 Errores frecuentes

- llamar “pura” a una función que lee reloj, azar o estado global;
- copiar solo el array y luego mutar objetos compartidos;
- usar `reduce` para cualquier problema aunque oculte la intención;
- olvidar `return` en una flecha con llaves;
- pasar callbacks con una firma incompatible, como `parseInt` directo a `map`;
- suponer evaluación diferida en métodos de array;
- construir muchos intermedios en una ruta medida como crítica;
- intentar eliminar todos los efectos en lugar de aislarlos.

15.23 Para recordar

- Pureza e inmutabilidad reducen dependencias y cambios invisibles.
- Los efectos son necesarios; concentrarlos en los bordes vuelve comprobable el núcleo.
- `map`, `filter`, `reduce`, `find`, `some` y `every` responden preguntas diferentes.
- Los métodos de array son inmediatos; generadores y otros iterables permiten evaluación diferida.
- Elegí la abstracción más clara y optimizá después de medir.

Capítulo 16. Recursividad y árboles binarios

Idea central

La recursividad resulta natural cuando un problema contiene versiones más pequeñas de sí mismo. Para que sea correcta, cada llamada debe acercarse a un caso base; para que sea productiva, la forma de la función debe reflejar la estructura de los datos y mantener explícitas sus invariantes.

Los árboles son el ejemplo central: cada nodo contiene un valor y referencias a subárboles que obedecen la misma definición.

16.1 Qué ocurre en una llamada recursiva

Una función puede llamarse a sí misma:

```
function cuentaRegresiva(numero) {  
  if (numero < 0) return;  
  
  console.log(numero);  
  cuentaRegresiva(numero - 1);  
}
```

Cada llamada crea un marco en la pila con sus parámetros y variables locales. La llamada actual queda suspendida hasta que termine la siguiente.

```
cuentaRegresiva(2)  
  cuentaRegresiva(1)  
    cuentaRegresiva(0)  
      cuentaRegresiva(-1) → termina
```

La pila tiene un límite. Una recursión sin fin o con demasiada profundidad produce un error de tamaño máximo de pila. Es la misma pila de llamadas que recorre una excepción al propagarse, según describe la sección 13.7.

16.2 Las tres preguntas obligatorias

Antes de escribir una función recursiva:

1. **Caso base:** ¿qué entrada puedo resolver sin otra llamada?
2. **Reducción:** ¿cómo garantizo que la siguiente entrada es más pequeña o más cercana al final?
3. **Combinación:** ¿cómo se integra el resultado pequeño en la respuesta actual?

Factorial:

```
function factorial(n) {
  if (!Number.isSafeInteger(n) || n < 0) {
    throw new RangeError("n debe ser un entero no negativo");
  }

  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
```

- caso base: `0!` y `1!` valen `1`;
- reducción: `n - 1`;
- combinación: multiplicar `n` por el resultado menor.

16.3 Recursión sobre una secuencia

```
function sumar(numeros, indice = 0) {
  if (indice === numeros.length) return 0;
  return numeros[indice] + sumar(numeros, indice + 1);
}
```

Funciona, pero un bucle es más directo y no consume un marco por elemento:

```
function sumarIterativo(numeros) {
  let total = 0;

  for (const numero of numeros) {
    total += numero;
  }

  return total;
}
```

La recursividad no es “mejor” por ser más abstracta. Es especialmente útil cuando la estructura de la entrada también es recursiva.

16.4 Estructuras anidadas

```
const documento = {
  titulo: "Curso",
  secciones: [
    {
      titulo: "Unidad 1",
      secciones: [
        { titulo: "Tema 1", secciones: [] }
      ]
    }
  ]
};
```

Cada sección contiene secciones. Una función puede aplicar la misma operación en cada nivel:

```
function contarSecciones(seccion) {  
  return 1 + seccion.secciones.reduce(  
    (total, hija) => total + contarSecciones(hija),  
    0  
  );  
}
```

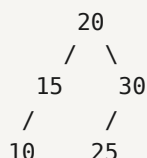
El array vacío funciona como base implícita: su suma adicional es cero.

16.5 Definir un nodo binario

```
function Nodo(valor, menor = null, mayor = null) {  
  return { valor, menor, mayor };  
}
```

Un nodo tiene como máximo dos hijos. `null` representa un subárbol vacío.

```
const raiz = Nodo(  
  20,  
  Nodo(15, Nodo(10)),  
  Nodo(30, Nodo(25))  
);
```



16.6 El invariante de un árbol binario de búsqueda

Para cada nodo:

- todos los valores del subárbol `menor` se comparan antes que el valor del nodo;
- todos los valores del subárbol `mayor` se comparan después;
- ambos subárboles cumplen la misma regla.

Hay que decidir qué hacer con duplicados:

- rechazarlos;
- ubicarlos siempre en una rama;
- almacenar una frecuencia;
- permitir varios registros bajo la misma clave.

Sin una política consistente, buscar, insertar y eliminar dejarán de concordar.

16.7 Recorrido inorden

```
function recorrerEnOrden(nodo, resultado = []) {
  if (nodo === null) return resultado;

  recorrerEnOrden(nodo.menor, resultado);
  resultado.push(nodo.valor);
  recorrerEnOrden(nodo.mayor, resultado);

  return resultado;
}

recorrerEnOrden(raiz); // [10, 15, 20, 25, 30]
```

El caso base es el árbol vacío. El orden es:

```
subárbol menor → nodo → subárbol mayor
```

En un árbol de búsqueda válido, produce los valores ordenados.

16.8 Preorden y postorden

Preorden procesa el nodo antes de sus hijos:

```
function preorden(nodo, resultado = []) {
  if (nodo === null) return resultado;

  resultado.push(nodo.valor);
  preorden(nodo.menor, resultado);
  preorden(nodo.mayor, resultado);
  return resultado;
}
```

Es útil para copiar o serializar conservando la raíz antes de las ramas.

Postorden procesa hijos antes del nodo:

```
function postorden(nodo, resultado = []) {
  if (nodo === null) return resultado;

  postorden(nodo.menor, resultado);
  postorden(nodo.mayor, resultado);
  resultado.push(nodo.valor);
  return resultado;
}
```

Sirve cuando el resultado del padre depende de ambos hijos o cuando se liberan estructuras desde las hojas.

16.9 Recorridos como generadores

Un generador evita construir un array completo y permite cortar el consumo:

```
function* valoresEnOrden(nodo) {
  if (nodo === null) return;

  yield* valoresEnOrden(nodo.menor);
  yield nodo.valor;
  yield* valoresEnOrden(nodo.mayor);
}

for (const valor of valoresEnOrden(raiz)) {
  console.log(valor);
}
```

La recursión sigue creando marcos por profundidad, pero los valores se producen de manera diferida, según el mecanismo que describe la sección 15.18.

16.10 Buscar descartando una rama

```
function compararNumeros(a, b) {
  return a - b;
}

function buscar(nodo, valor, comparar = compararNumeros) {
  if (nodo === null) return null;

  const orden = comparar(valor, nodo.valor);

  if (orden === 0) return nodo.valor;

  return orden < 0
    ? buscar(nodo.menor, valor, comparar)
    : buscar(nodo.mayor, valor, comparar);
}
```

Cada comparación elige una sola rama. En un árbol equilibrado con n nodos, la profundidad típica es proporcional a $\log_2(n)$. En el peor caso puede ser n .

Versión iterativa:

```
function buscarIterativo(raiz, valor, comparar = compararNumeros) {
  let actual = raiz;

  while (actual !== null) {
    const orden = comparar(valor, actual.valor);

    if (orden === 0) return actual.valor;
    actual = orden < 0 ? actual.menor : actual.mayor;
  }

  return null;
}
```

La versión iterativa mantiene profundidad constante en la pila y refleja que solo seguimos un camino.

16.11 Insertar modificando el árbol

```
function insertarMutable(nodo, valor, comparar = compararNumeros) {
  if (nodo === null) return Nodo(valor);

  if (comparar(valor, nodo.valor) < 0) {
    nodo.menor = insertarMutable(nodo.menor, valor, comparar);
  } else {
    nodo.mayor = insertarMutable(nodo.mayor, valor, comparar);
  }

  return nodo;
}
```

Es importante reasignar la rama al resultado recursivo: cuando llega a `null`, la llamada devuelve el nuevo nodo.

```
let arbol = null;

for (const valor of [20, 15, 10, 30, 25]) {
  arbol = insertarMutable(arbol, valor);
}
```

Si el árbol estaba vacío, también hay que reasignar la raíz.

16.12 Insertar sin modificar la versión anterior

```
function insertar(nodo, valor, comparar = compararNumeros) {
  if (nodo === null) return Nodo(valor);

  if (comparar(valor, nodo.valor) < 0) {
    return {
      ...nodo,
      menor: insertar(nodo.menor, valor, comparar)
    };
  }

  return {
    ...nodo,
    mayor: insertar(nodo.mayor, valor, comparar)
  };
}
```

Solo se copian los nodos del camino. El otro subárbol se comparte porque no cambia:

```
const nuevo = insertar(raiz, 12);

nuevo !== raiz;           // true
nuevo.mayor === raiz.mayor; // true, rama compartida
nuevo.menor !== raiz.menor; // true, camino modificado
```

Este **compartir estructural** permite conservar versiones sin clonar todo el árbol. Es la generalización de la actualización inmutable que presentaron las secciones 10.11 y 15.4.

16.13 Altura y cantidad

```
function cantidad(nodo) {
  if (nodo === null) return 0;
  return 1 + cantidad(nodo.menor) + cantidad(nodo.mayor);
}

function altura(nodo) {
  if (nodo === null) return 0;
  return 1 + Math.max(altura(nodo.menor), altura(nodo.mayor));
}
```

Ambas funciones deben visitar todos los nodos: su costo es lineal en la cantidad de elementos. La profundidad de la recursión depende de la altura.

16.14 Mínimo y máximo

El menor valor está en la rama izquierda más profunda:

```
function minimo(nodo) {
  if (nodo === null) return null;

  let actual = nodo;
  while (actual.menor !== null) actual = actual.menor;
  return actual.valor;
}
```

El máximo sigue la rama derecha. Estas operaciones aprovechan el invariante y no necesitan recorrer todo.

16.15 Eliminar un valor

La eliminación tiene tres casos:

1. nodo hoja: devolver `null`;
2. un solo hijo: devolver ese hijo;
3. dos hijos: reemplazar por un sucesor y eliminar ese sucesor de su rama original.

```
function eliminar(nodo, valor, comparar = compararNumeros) {
  if (nodo === null) return null;

  const orden = comparar(valor, nodo.valor);

  if (orden < 0) {
    return { ...nodo, menor: eliminar(nodo.menor, valor, comparar) };
  }

  if (orden > 0) {
    return { ...nodo, mayor: eliminar(nodo.mayor, valor, comparar) };
  }

  if (nodo.menor === null) return nodo.mayor;
  if (nodo.mayor === null) return nodo.menor;

  const sucesor = minimo(nodo.mayor);

  return {
    valor: sucesor,
    menor: nodo.menor,
    mayor: eliminar(nodo.mayor, sucesor, comparar)
  };
}
```

Esta versión supone que la política de duplicados y el comparador hacen identificable el sucesor. Para registros con claves no únicas, el contrato debe precisar qué elimina.

16.16 Validar el invariante

No alcanza con comparar cada nodo solo con sus hijos. Un valor puede violar un límite heredado desde un ancestro:

```
function esArbolDeBusqueda(
  nodo,
  comparar = compararNumeros,
  minimoPermitido = null,
  maximoPermitido = null
) {
  if (nodo === null) return true;

  if (minimoPermitido !== null &&
    comparar(nodo.valor, minimoPermitido) < 0) {
    return false;
  }

  if (maximoPermitido !== null &&
    comparar(nodo.valor, maximoPermitido) >= 0) {
    return false;
  }

  return esArbolDeBusqueda(
    nodo.menor,
    comparar,
    minimoPermitido,
    nodo.valor
  ) && esArbolDeBusqueda(
    nodo.mayor,
    comparar,
    nodo.valor,
    maximoPermitido
  );
}
```

La función adopta la política “menores a la izquierda y mayores o iguales a la derecha”. Los límites deben adaptarse a la política real y a claves que puedan coincidir con los centinelas; una implementación genérica puede usar indicadores separados en lugar de `null`.

16.17 Comparadores y datos compuestos

```
const compararPorLegajo = (a, b) => a.legajo - b.legajo;

let alumnos = null;
alumnos = insertar(alumnos, { legajo: 20, nombre: "Ana" }, compararPorLegajo);
alumnos = insertar(alumnos, { legajo: 10, nombre: "Luis" }, compararPorLegajo);
```

Buscar requiere un valor comparable:

```
buscar(alumnos, { legajo: 10 }, compararPorLegajo);
```

El comparador debe ser consistente y definir un orden total apropiado. Si dos registros tienen el mismo legajo, el árbol necesita una política de duplicados.

16.18 Encapsular el árbol

```
function crearArbol(comparar = compararNumeros) {
  let raiz = null;

  return {
    insertar(valor) {
      raiz = insertar(raiz, valor, comparar);
      return this;
    },

    buscar(valor) {
      return buscar(raiz, valor, comparar);
    },

    eliminar(valor) {
      raiz = eliminar(raiz, valor, comparar);
      return this;
    },

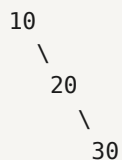
    valores() {
      return [...valoresEnOrden(raiz)];
    },

    get cantidad() {
      return cantidad(raiz);
    }
  };
}
```

La clausura de la sección 14.18 conserva la raíz y garantiza que todas las operaciones usen el mismo comparador. Nadie de afuera puede reemplazarla ni dejar el árbol a mitad de camino.

16.19 Equilibrio y complejidad

Insertar valores ya ordenados crea una cadena:



La búsqueda pierde la ventaja y cuesta hasta n comparaciones. Un orden mezclado puede producir una altura cercana a $\log_2(n)$.

Árboles AVL y rojo-negro realizan rotaciones para mantener balance. Implementarlos excede este capítulo, pero dejan una lección: la complejidad prometida depende de mantener no solo el orden, sino también una forma suficientemente equilibrada.

Para muchos programas JavaScript, `Map` es la estructura productiva para búsquedas por clave. Implementar un árbol es valioso para comprender recursividad, invariantes, orden y complejidad; se elige en producción cuando sus capacidades específicas justifican el costo.

16.20 Recorridos iterativos con pila

Para árboles de profundidad externa o potencialmente enorme, una pila explícita evita desbordar la pila de llamadas:

```
function inordenIterativo(raiz) {
  const resultado = [];
  const pendientes = [];
  let actual = raiz;

  while (actual !== null || pendientes.length > 0) {
    while (actual !== null) {
      pendientes.push(actual);
      actual = actual.menor;
    }

    actual = pendientes.pop();
    resultado.push(actual.valor);
    actual = actual.mayor;
  }

  return resultado;
}
```

La estructura explícita reproduce los marcos que la recursión guardaba implícitamente. El apéndice B recorre el camino inverso: resuelve con dos pilas explícitas un problema —evaluar una expresión aritmética— que también admite una solución recursiva.

16.21 Depurar una recursión

Registra entrada, profundidad y caso base:

```
function buscarConTraza(nodo, valor, profundidad = 0) {
  console.log({ profundidad, actual: nodo?.valor ?? null });

  if (nodo === null || nodo.valor === valor) return nodo;

  return valor < nodo.valor
    ? buscarConTraza(nodo.menor, valor, profundidad + 1)
    : buscarConTraza(nodo.mayor, valor, profundidad + 1);
}
```

Probá:

- estructura vacía;
- un nodo;
- valor en la raíz;
- valor en cada rama;
- valor ausente;
- árbol degenerado;
- duplicados según la política elegida.

16.22 Errores frecuentes

- no definir caso base;
- hacer una llamada que no reduce el problema;
- olvidar retornar el resultado recursivo;
- no reasignar una rama después de insertar o eliminar;
- mezclar políticas de duplicados;
- verificar solo padres e hijos y no límites de ancestros;
- asumir búsqueda logarítmica sin controlar equilibrio;
- usar recursión profunda con entrada externa no limitada.

16.23 Para recordar

- Recursión correcta significa caso base, reducción y combinación.
- La estructura recursiva de un árbol se refleja en la función que lo recorre.
- Un árbol de búsqueda necesita una política de orden y duplicados compartida por todas sus operaciones.
- Inmutabilidad puede lograrse copiando el camino y compartiendo ramas no modificadas.
- El equilibrio determina si buscar cuesta aproximadamente $\log n$ o se degrada a n .

Parte IV. Aplicar e integrar

Capítulo 17. Programación asíncrona con `async` y `await`

Idea central

JavaScript ejecuta una sola cosa por vez, así que una operación que tarda no puede esperarse bloqueando: la función que la pide se suspende, el resto del programa sigue, y esa función retoma cuando el dato llega. `async` y `await` expresan esa suspensión con la misma forma que tendría un programa secuencial, y por eso el manejo de errores vuelve a ser `try/catch`.

Lo que cambia respecto de un programa sincrónico no es la sintaxis sino el momento: entre el `await` y su resultado pueden ocurrir otras cosas. Diseñar bien la asincronía consiste, casi siempre, en decidir qué operaciones deben esperarse en fila y cuáles pueden arrancar juntas.

17.1 El problema: un solo hilo

Hay operaciones que tardan: pedir datos a un servidor, leer un archivo, esperar un segundo. JavaScript tiene **un solo hilo** de ejecución, así que no puede quedarse parado esperando. Si lo hiciera, la página entera se congela: no responde a un clic, no anima nada, no repinta.

La solución es no esperar bloqueando. La función que pide el dato se **suspende**, el resto del programa sigue corriendo, y cuando el dato llega la función retoma exactamente donde estaba.

Conviene separar dos ideas que suelen mezclarse. El motor ejecuta una tarea por vez; el anfitrión, en cambio, puede tener varias operaciones en curso —una descarga, una lectura de disco, un temporizador— y avisar cuando cada una termina. La sección 1.4 describió ese reparto entre motor y anfitrión. La sección 20.11 muestra el orden exacto en que el navegador procesa el código actual, las promesas y los temporizadores.

17.2 Una promesa es un valor que todavía no llegó

Las funciones que tardan no devuelven el dato: devuelven una **promesa** de ese dato.

Una promesa tiene tres estados y solo cambia una vez:

Estado	Significado
pendiente	la operación está en curso
cumplida	terminó bien y tiene un valor
rechazada	falló y tiene un motivo

```
const promesa = fetch("/api/usuarios");

console.log(promesa); // Promise { <pending> }
```

Imprimir la promesa no imprime el dato. La promesa es el comprobante, no el resultado.

17.3 Las dos palabras

await espera a que la promesa termine y devuelve el valor:

```
const respuesta = await fetch("/api/usuarios");
```

async marca la función que contiene **await**. Sin ella, **await** es un error de sintaxis:

```
async function cargar() {
  const respuesta = await fetch("/api/usuarios");
  const usuarios = await respuesta.json();
  return usuarios;
}
```

Dos consecuencias, y son todo lo que hay que recordar:

- Una función **async** **siempre devuelve una promesa**. Por eso, para usar su resultado desde afuera, también hay que ponerle **await**.
- **await** **no bloquea el programa**: suspende solamente a *esa* función.

```
const usuarios = await cargar(); // el await de afuera
```

Un **return** valor dentro de una función **async** produce una promesa cumplida con ese valor; un **throw**, una promesa rechazada.

Regla: si una función tiene un **await** adentro, va marcada **async**. Si una función es **async**, al llamarla le va un **await**. Se contagia hacia arriba.

17.4 La forma que se usa siempre

El grueso del código asíncronico real tiene esta forma:

```

async function mostrarUsuario(id) {
  const respuesta = await fetch(`/api/usuarios/${id}`);
  const usuario = await respuesta.json();
  document.querySelector("#nombre").textContent = usuario.nombre;
}

```

Dos detalles de `fetch` que conviene saber de memoria:

1. Hacen falta **dos `await`**: el primero espera la respuesta, el segundo espera y convierte el cuerpo, porque `json()` también devuelve una promesa.
2. **Un 404 no falla.** `fetch` solo rechaza si se cayó la red o se canceló el pedido. Hay que revisar `ok` a mano:

```

if (!respuesta.ok) throw new Error(`Error ${respuesta.status}`);

```

La sección 13.14 desarrolla esta sutileza y muestra cómo traducir un estado HTTP a un error del dominio; la 22.8 explica por qué `fetch` se comporta así.

17.5 Errores: `try/catch` de siempre

Si la promesa se rechaza, `await` lanza una excepción. Se atrapa como cualquier otra:

```

async function mostrarUsuario(id) {
  const cartel = document.querySelector("#estado");
  cartel.textContent = "Cargando...";

  try {
    const respuesta = await fetch(`/api/usuarios/${id}`);
    if (!respuesta.ok) throw new Error(`Error ${respuesta.status}`);

    const usuario = await respuesta.json();
    cartel.textContent = usuario.nombre;
  } catch (error) {
    cartel.textContent = `No se pudo cargar: ${error.message}`;
  } finally {
    document.querySelector("#boton").disabled = false; // siempre se ejecuta
  }
}

```

Un solo `try` cubre todos los `await` que haya adentro. Eso es cómodo y también es la trampa que describe la sección 13.10: si el bloque abarca demasiadas operaciones, el `catch` ya no sabe cuál falló.

La equivalencia con el estilo de promesas encadenadas es directa:

```

cargarDatos()
  .then(usr)
  .catch(manejar)
  .finally(limpiar);

```

Las dos formas hacen lo mismo. Conviene elegir una por unidad de código: mezclarlas es la manera más común de perder un rechazo.

17.6 En paralelo: `Promise.all`

Este es el error más frecuente de todos.

```
// MAL: tres segundos
const usuarios = await obtenerUsuarios(); // 1 s
const productos = await obtenerProductos(); // 1 s
const ventas = await obtenerVentas(); // 1 s
```

Los tres pedidos son independientes; no hay razón para hacerlos en fila:

```
// BIEN: un segundo
const [usuarios, productos, ventas] = await Promise.all([
  obtenerUsuarios(),
  obtenerProductos(),
  obtenerVentas()
]);
```

Lo que cambia es *cuándo arranca cada uno*. Llamar a la función arranca el trabajo; `await` solamente espera el resultado. Dentro del arreglo, las tres arrancan juntas.

Regla: si un `await` no usa el resultado del `await` anterior, están mal puestos.

17.7 Cuando `all` no alcanza

`Promise.all` rechaza en cuanto una de las promesas rechaza, y las demás siguen corriendo sin que nadie las mire. Hay tres combinadores más:

Combinador	Termina cuando	Resultado
<code>all</code>	todas se cumplen, o una rechaza	array de valores, o el primer rechazo
<code>allSettled</code>	todas terminan, pase lo que pase	array de estados
<code>race</code>	termina la primera, se cumpla o rechace	ese valor o ese rechazo
<code>any</code>	se cumple la primera	ese valor, o <code>AggregateError</code> si fallan todas

```
const resultados = await Promise.allSettled([guardarA(), guardarB()]);
// cada uno: { status: "fulfilled", value } | { status: "rejected", reason }
```

`allSettled` es el adecuado cuando cada operación vale por sí misma y querés un informe completo. La sección 13.15 muestra cómo recorrer esos estados.

17.8 `await` dentro de un bucle

Uno por vez, cuando importa el orden o cada paso depende del anterior:

```
for (const id of ids) {  
  const usuario = await obtenerUsuario(id);  
  console.log(usuario.nombre);  
}
```

Todos juntos, cuando son independientes, que es lo habitual:

```
const usuarios = await Promise.all(ids.map(id => obtenerUsuario(id)));
```

forEach no funciona. Ignora la promesa que devuelve el callback, así que no espera nada:

```
ids.forEach(async id => {  
  const usuario = await obtenerUsuario(id);  
  console.log(usuario.nombre); // se imprime más tarde, sin orden garantizado  
});  
  
console.log("listo"); // se imprime primero, sin ningún dato
```

Con `async`: `for...of` o `map` más `Promise.all`. Nunca `forEach`.

Lanzar miles de pedidos a la vez tampoco es gratis. Cuando la cantidad de operaciones depende de los datos, conviene limitar la concurrencia; la sección 19.38 discute ese límite para archivos.

17.9 Secuencias que llegan por partes

Una promesa representa **un** valor futuro. Cuando lo que llega es una secuencia —páginas de un listado, líneas de un archivo, eventos— el protocolo adecuado es el iterable asíncronico, que se consume con `for await...of`:

```
for await (const linea of lineas) {  
  procesar(linea);  
}
```

La sección 8.9 explica el protocolo y la sección 19.26 lo aplica a leer un archivo grande sin cargarlo entero en memoria.

17.10 `await` en el nivel superior de un módulo

Dentro de un módulo ECMAScript, `await` puede usarse fuera de toda función:

```
import { readFile } from "node:fs/promises";

const configuracion = JSON.parse(await readFile("config.json", "utf8"));
```

El módulo queda suspendido hasta que la promesa termina, y los módulos que lo importan esperan con él. Es cómodo para cargar configuración al arrancar y desaconsejable para trabajo largo, porque demora a todo el que dependa de ese módulo.

17.11 Errores frecuentes

Error	Síntoma	Solución
Olvidar <code>await</code>	sale <code>Promise { <pending> }</code> en vez del valor	agregar <code>await</code>
Olvidar el segundo <code>await</code> en <code>fetch</code>	el cuerpo nunca llega	<code>await respuesta.json()</code>
<code>await</code> en serie sobre cosas independientes	tarda N veces más	<code>Promise.all</code>
<code>forEach</code> con <code>async</code>	el código sigue de largo sin esperar	<code>for...of</code> , o <code>map</code> más <code>all</code>
No revisar <code>respuesta.ok</code>	un 404 se procesa como dato válido	<code>if (!respuesta.ok) throw ...</code>
<code>await</code> fuera de una función <code>async</code>	error de sintaxis, salvo en el nivel superior de un módulo	marcar la función con <code>async</code>
Llamar una <code>async</code> sin <code>await</code> ni <code>.catch</code>	el error se pierde en silencio	esperar la llamada o devolver la promesa
Mezclar <code>await</code> con <code>.then</code> en la misma función	rechazos sin manejar	elegir un estilo

17.12 Para recordar

- Una promesa es un valor que todavía no llegó; tiene tres estados y solo cambia una vez.
- `async` convierte el resultado de una función en una promesa; `await` suspende esa función, no el programa.
- El manejo de errores vuelve a ser `try/catch/finally`, con las mismas reglas del capítulo 13.

- Llamar arranca el trabajo y `await` solo espera: de esa distinción salen `Promise.all` y la diferencia entre tres segundos y uno.
- `all`, `allSettled`, `race` y `any` responden preguntas distintas sobre un conjunto de operaciones.
- Para una secuencia de valores en el tiempo, el protocolo es `for await...of`, no una promesa.

```
async function tarea() {
  try {
    const a = await pedirA(); // en fila: b necesita a
    const b = await pedirB(a);
    const [c, d] = await Promise.all([pedirC(), pedirD()]); // juntos
    return { a, b, c, d };
  } catch (error) {
    console.error(error);
  } finally {
    limpiar();
  }
}

await tarea();
```

Capítulo 18. Expresiones regulares

Idea central

Una expresión regular describe un conjunto de secuencias de texto; se construye combinando átomos, cantidades, alternativas y posiciones. Es productiva para buscar, validar formas, extraer y reemplazar patrones locales, pero no sustituye un parser ni valida el significado del dominio.

La forma más segura de diseñarla es progresiva:

```
ejemplos válidos e inválidos
→ piezas literales
→ clases y cantidades
→ agrupación
→ anclas
→ pruebas y casos límite
```

18.1 Antes de una regex: búsqueda literal

No todo problema de texto necesita un lenguaje de patrones:

```
texto.includes("error");
texto.startsWith("TUP-");
texto.endsWith(".md");
texto.indexOf(":");
texto.replaceAll(" ", "-");
```

Estas operaciones son claras cuando la secuencia es fija. Usá una regex cuando deben variar caracteres, cantidad, posición o alternativas.

18.2 Crear un objeto `RegExp`

Literal:

```
const patron = /javascript/i;
```

Constructor:

```
const patronDinamico = new RegExp("javascript", "i");
```

Ambos producen objetos:

```
typeof patron;           // "object"
patron instanceof RegExp; // true
```

El literal se analiza al cargar el código y es preferible para un patrón fijo. El constructor permite incorporar datos en tiempo de ejecución, pero las barras invertidas atraviesan primero la sintaxis del string:

```
const literal = /\d+/u;
const construido = new RegExp("\\d+", "u");
```

18.3 Texto dinámico: escapar antes de interpolar

Si el usuario busca `a.b`, el punto no debería significar “cualquier carácter”. Hay que escapar los metacaracteres:

```
function escaparRegex(texto) {
  return texto.replace(/[\.*+?^$()|[\]\]/gu, "\\$&");
}

const termino = "a.b";
const patron = new RegExp(escaparRegex(termino), "giu");
```

El escape depende del contexto. Insertar dentro de una clase `[]` puede exigir reglas diferentes de insertar como patrón general.

18.4 Caracteres literales

```
/casa/u.test("la casa azul"); // true
```

La regex busca en cualquier posición salvo que agreguemos anclas. Las mayúsculas importan sin el flag `i`.

18.5 Metacaracteres

Tienen significado especial fuera de clases:

```
. ^ $ * + ? ( ) [ ] { } | \
```

Para buscar uno literalmente, se escapa:

```
/\./u.test("archivo.txt");
/\/u.test("¿qué?");
/\\/u.test("C:\\datos");
```

Dentro de `[]`, las reglas cambian: `-`, `]`, `^` y `\` pueden necesitar atención según su posición.

18.6 El punto

`.` coincide con cualquier carácter salvo terminadores de línea en el modo habitual:

```
/c.sa/u.test("casa"); // true
/c.sa/u.test("cosa"); // true
```

El flag `s` activa *dotAll* y permite incluir saltos:

```
/inicio.*fin/su.test("inicio\nfin"); // true
```

Una clase negada suele expresar mejor un límite que `.*?:`

```
/"[^"]*" /u;
```

18.7 Clases de caracteres

```
/[abc]/u;           // a, b o c
/[0-9]/u;           // dígito ASCII
/[a-zA-Z]/u;        // letra ASCII inglesa
/[^0-9]/u;          // cualquier carácter excepto dígito ASCII
```

El `^` niega solo si aparece al comienzo de la clase. Un guion al final o escapado se interpreta literalmente:

```
/[A-Z-]/u;
```

Los rangos siguen puntos de código, no categorías lingüísticas completas. `[A-z]` incluye caracteres entre `Z` y `a` que no son letras; no lo uses como abreviatura de ambos alfabetos.

18.8 Clases abreviadas

```
\d  dígito ASCII
\D  no dígito
\w  letra ASCII, dígito o guion bajo
\W  no carácter de palabra
\s  espacio o separador reconocido
\S  no espacio
```

```
/\d+/u.test("Legajo 123"); // true
```

`\w` no representa todas las letras Unicode, por las razones que desarrolla la sección 5.10. Para alfabetos generales, usá propiedades Unicode con `u`:

```
/\p{L}+/u; // una o más letras Unicode
/\p{N}+/u; // números Unicode
/\p{Script=Greek}+/u;
```

La negación usa `\P{...}`.

18.9 Cuantificadores

Se aplican al átomo anterior:

```
x?      cero o una x
x*      cero o más x
x+      una o más x
x{3}    exactamente tres x
x{2,5}  entre dos y cinco x
x{2,}   dos o más x
```

```
/ab+/u;      // a seguida de una o más b
/(ab)+/u;    // una o más secuencias "ab"
/colou?r/u;   // u opcional
/\d{2,4}/u;   // entre dos y cuatro dígitos
```

`*` y `?` permiten cero coincidencias. Un patrón puede coincidir con la cadena vacía, lo que tiene consecuencias al repetir búsquedas globales.

18.10 Cuantificadores codiciosos y perezosos

Por defecto intentan consumir lo máximo y retroceden si hace falta:

```
const texto = "<b>uno</b><i>dos</i>";
texto.match(/<.*>/u)?.[0]; // puede abarcar todo
```

Agregar `?` los vuelve perezosos:

```
texto.match(/<.*?>/u)?.[0]; // "<b>"
```

Para este caso, una clase con límite explícito es más precisa:

```
texto.match(/<[>]*>/u)?.[0];
```

Ninguna de estas expresiones convierte a regex en un parser HTML correcto; atributos, comentarios y contenido especial requieren una herramienta estructural.

18.11 Anclas: posiciones, no caracteres

`^` representa el comienzo y `$` el final:

```
/^hola/u.test("hola mundo"); // true
/mundo$/u.test("hola mundo"); // true
```

Para validar el texto completo:

```
const legajo = /^d{5}$/u;

legajo.test("12345"); // true
legajo.test("X12345"); // false
legajo.test("123456"); // false
```

Sin anclas, `/d{5}/` solo pregunta si existe una subsecuencia de cinco dígitos.

Con flag `m`, `^` y `$` también operan al inicio y final de cada línea. No confundas texto completo con cada línea.

18.12 Frontera de palabra

`\b` coincide entre una posición de palabra y una de no palabra:

```
/\bgato\b/u.test("un gato negro"); // true
/\bgato\b/u.test("gatopardo"); // false
```

Su idea de “palabra” está vinculada a `\w` y no cubre de manera intuitiva todos los idiomas. Para segmentación lingüística, `Intl.Segmenter` es más apropiado, según muestra la sección 5.12.

18.13 Agrupar

Los paréntesis agrupan una secuencia para aplicar cantidad o alternativas:

```
/(ab)+/u;
/^(rojo|verde|azul)$/u;
```

Sin grupo:

```
/rojo|verde$/u;
```

se interpreta como `rojo` en cualquier posición o `verde` al final. Agrupar hace visible el alcance de la alternativa.

18.14 Capturas

Los grupos también guardan la parte coincidente:

```
const fecha = "28/08/2026";
const coincidencia = fecha.match(/^(\d{2})\/(\d{2})\/(\d{4})$/u);

coincidencia?.[0]; // texto completo
coincidencia?.[1]; // "28"
coincidencia?.[2]; // "08"
coincidencia?.[3]; // "2026"
```

Un grupo no capturante organiza sin agregar una posición:

```
/(?:https?|ftp):\/\/\/u;
```

Usalo cuando no necesitarás el contenido del grupo.

18.15 Grupos con nombre

```
const patronFecha = /^(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})$/u;
const resultado = "28/08/2026".match(patronFecha);

if (resultado) {
  const { dia, mes, anio } = resultado.groups;
  console.log({ dia, mes, anio });
}
```

Los nombres resisten mejor cambios de orden y documentan la extracción.

18.16 Referencias a capturas

Una referencia posterior exige repetir el mismo texto:

```
/\b(\p{L}+)\s+\1\b/giu;
```

Con nombre:

```
/\b(?<palabra>\p{L}+)\s+\k<palabra>\b/giu;
```

Puede detectar palabras consecutivas repetidas. El flag `i` aplica comparación sin distinguir mayúsculas según las reglas del motor.

18.17 Lookahead y lookbehind

Comprueban contexto sin consumirlo.

Lookahead positivo:

```
/\d+(?=\s*ARS)/u; // número seguido de ARS
```

Lookahead negativo:

```
/^(?!admin$)[a-z]+$ /u; // palabra minúscula excepto admin
```

Lookbehind positivo:

```
/(?<=\$)\d+(?:\.\d+)? /u; // número precedido por $
```

Lookbehind negativo:

```
/(?<!\d)-\d+ /u; // signo menos no precedido por dígito
```

Son potentes y pueden reducir legibilidad. A veces capturar el contexto y procesarlo después es más portable y fácil de depurar.

18.18 Flags

Flag	Efecto
g	todas las coincidencias y uso de <code>lastIndex</code>
i	ignora diferencias de mayúsculas según reglas del motor
m	anclas por línea
s	. incluye terminadores de línea
u	semántica Unicode y sintaxis Unicode estricta
y	coincidencia adherida a <code>lastIndex</code>
d	índices de los rangos coincidentes

```
const patron = /hola/giu;  
patron.flags; // "giu" en un orden canónico  
patron.global; // true  
patron.source; // "hola"
```

El flag `u` debería ser habitual en patrones modernos que procesan texto Unicode.

18.19 test

```
/error/iu.test("ERROR de conexión"); // true
```

Devuelve un booleano. Con `g` o `y`, el objeto conserva `lastIndex`:

```
const global = /a/g;

global.test("a"); // true; lastIndex pasa a 1
global.test("a"); // false; comienza desde 1
```

Para comprobaciones independientes, evita `g` o reiniciá deliberadamente el estado.

18.20 exec

```
const patronNumero = /\d+/gu;
const coincidencia = patronNumero.exec("A12 B34");

coincidencia[0]; // "12"
coincidencia.index; // 1
patronNumero.lastIndex; // posición posterior
```

Con `g`, llamadas sucesivas recorren coincidencias. Si el patrón puede coincidir con vacío, asegúrá progreso para evitar bucles infinitos en código manual.

18.21 match y matchAll

Sin `g`, `match` devuelve detalle de la primera coincidencia:

```
"A12 B34".match(/(?<numero>\d+)/u);
```

Con `g`, devuelve solo textos completos:

```
"A12 B34".match(/\d+/gu); // ["12", "34"]
```

`matchAll` devuelve un iterador con detalles para cada coincidencia y requiere `g`:

```
const resultados = [
  ..."A12 B34".matchAll(/(?<numero>\d+)/gu)
];
```

Es la forma cómoda de obtener grupos e índices de todas las apariciones.

18.22 replace

Con referencias:

```
"28/08/2026".replace(
  /^( \d{2}) \d{2} \d{4}$/,
  "$3-$2-$1"
); // "2026-08-28"
```

Con grupos nombrados:

```
"28/08/2026".replace(
  /^(?<d>\d{2})\/(?<m>\d{2})\/(?<a>\d{4})$/u,
  "$<a>- $<m>- $<d>"
);
```

Con callback:

```
"precios: 10 y 20".replace(/\d+/gu, texto =>
  String(Number(texto) * 2)
);
```

Sin `g`, solo reemplaza la primera coincidencia.

18.23 search y split

```
"abc 123".search(/\d/u); // 4
"uno, dos; tres".split(/\s*[;,]\s*/u);
// ["uno", "dos", "tres"]
```

Si `split` contiene grupos capturantes, los separadores capturados pueden aparecer en el resultado. Usá `(?:...)` si no los necesitás.

18.24 Construir una validación paso a paso

Código ABC-1234:

```
tres letras ASCII mayúsculas → [A-Z]{3}
guion literal                → -
cuatro dígitos ASCII         → \d{4}
texto completo               → ^[A-Z]{3}-\d{4}$
```

```
const codigo = /^[A-Z]{3}-\d{4}$/u;
```

Casos de prueba:

```
const casos = new Map([
  ["ABC-1234", true],
  ["abc-1234", false],
  ["AB-1234", false],
  ["ABC-12345", false],
  ["XABC-1234", false]
]);

for (const [texto, esperado] of casos) {
  console.assert(codigo.test(texto) === esperado, texto);
}
```

18.25 Validar forma y luego significado

Esta regex comprueba una fecha con dos dígitos por componente:

```
/^\d{2}\/\d{2}\/\d{4}$/u;
```

También acepta 99/99/0000. Después de extraer:

```
function leerFecha(texto) {
  const match = texto.match(
    /^(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})$/u
  );

  if (!match) return null;

  const dia = Number(match.groups.dia);
  const mes = Number(match.groups.mes);
  const anio = Number(match.groups.anio);
  const fecha = new Date(Date.UTC(anio, mes - 1, dia));

  const valida =
    fecha.getUTCFullYear() === anio &&
    fecha.getUTCMonth() === mes - 1 &&
    fecha.getUTCDate() === dia;

  return valida ? fecha : null;
}
```

Regex valida forma; el código valida calendario.

18.26 Email: no exagerar

Una comprobación práctica puede detectar errores obvios:

```
const formaBasica = /^[^\s@]+@[^\s@]+\.[^\s@]+$/u;
```

No prueba que la dirección exista ni cubre necesariamente todas las variantes permitidas por estándares. La verificación real es enviar un mensaje o usar un flujo de confirmación. Ajusta la sintaxis a lo que la aplicación realmente admite, sin prometer validación universal.

18.27 Patrones útiles

Normalizar espacios:

```
texto.trim().replace(/\s+/gu, " ");
```

Extraer números:


```
[...texto.matchAll(/[-]?d+(?:[.,]d+)?/gu)]
  .map(match => match[0]);
```

Hashtags Unicode:

```
[...texto.matchAll(/#(?<etiqueta>[\p{L}\p{N}_]+)/gu)]
  .map(match => match.groups.etiqueta);
```

Hora de 24 horas:

```
/^(?:[01]\d|2[0-3]):[0-5]\d$/u;
```

Nombre y número:

```
const linea = /^(?<nombre>\p{L}+(?:[ '\-]\p{L}+)*)\s*:\s*(?<valor>\d+)\$/u;
```

El patrón refleja un contrato concreto; no pretende reconocer todos los nombres del mundo.

18.28 Rendimiento y retroceso excesivo

Algunos motores exploran alternativas mediante retroceso. Patrones ambiguos con cuantificadores anidados pueden crecer de forma extrema ante entradas que casi coinciden:

```
/(a+)+$/u;
```

Con muchas `a` seguidas de un carácter inválido, puede requerir gran cantidad de intentos. Para entradas externas:

- evitá cuantificadores anidados ambiguos;
- usá clases y límites más específicos;
- limitá el tamaño de entrada;
- medí patrones críticos con casos adversos;
- no aceptes patrones arbitrarios de usuarios sin aislamiento y límites.

Una regex corta no es necesariamente barata.

18.29 Regex y autómatas

En su forma teórica, una expresión regular describe un lenguaje regular que puede reconocerse con un autómata finito. JavaScript agrega extensiones como referencias a capturas y lookahead que superan parte de esa definición clásica.

El modelo útil sigue siendo:

estado actual + próximo símbolo → nuevo estado

Las anclas restringen posiciones, las clases aceptan conjuntos de símbolos, los cuantificadores repiten transiciones y `|` ofrece caminos alternativos.

Un autómata finito tiene una cantidad fija de estados, y esa es exactamente la limitación de la sección siguiente: para saber qué `)` cierra cuál `(` hay que llevar la cuenta de los grupos abiertos, y esa cuenta no tiene tope. La memoria que falta es una pila, y el apéndice B la construye.

18.30 Cuándo no usar regex

No es la herramienta principal para:

- JSON: `JSON.parse`;
- HTML o XML general: parser estructural;
- CSV completo: parser con comillas y saltos;
- código fuente: tokenizador y parser;
- estructuras anidadas arbitrariamente;
- validación semántica de fechas, dominios o identidades.

Puede participar en una fase pequeña, como tokenizar un fragmento o comprobar una forma antes del parser. La sección 19.32 muestra ese reparto sobre un archivo de registro: la regex reconoce la forma de cada línea y el código decide qué hacer con las que no coinciden.

18.31 Errores frecuentes

- olvidar anclas al validar el texto completo;
- no escapar texto dinámico;
- confundir `\d` y `\w` con todas las categorías Unicode;
- capturar grupos que solo servían para agrupar;
- usar `g` con `test` y olvidar `lastIndex`;
- esperar grupos detallados de `match` global;
- usar `.*` cuando existe un delimitador específico;
- validar solo forma y asumir significado;
- crear patrones vulnerables a retroceso excesivo;
- usar regex como parser de una gramática anidada.

18.32 Para recordar

- Una regex describe posibilidades; no “entiende” el significado del texto.

- Construí desde átomos, cantidades, grupos y posiciones, guiado por ejemplos.
- Anclas convierten una búsqueda en validación completa; capturas convierten coincidencias en datos.
- Flags y métodos cambian estado y forma del resultado; `g` merece atención especial.
- Regex es excelente para patrones locales y peligrosa cuando reemplaza parsers, validación semántica o límites de seguridad.

Capítulo 19. Gestión de archivos con Node.js

Idea central

Una operación de archivos atraviesa cuatro capas: una ruta identifica el recurso, el sistema entrega bytes, una codificación puede convertirlos en texto y un formato puede transformar ese texto o esos bytes en datos. Un programa productivo controla cada capa, evita sobrescrituras y eliminaciones accidentales, y elige entre lectura completa, manejadores o streams según el tamaño y la concurrencia.

```
ruta → archivo → bytes → codificación → texto → formato → datos
```

Confundir capas produce errores como interpretar un PDF con `readFile(..., "utf8")`, asumir que `.json` garantiza JSON válido o creer que una ruta relativa parte del archivo JavaScript.

19.1 Archivo, directorio, ruta y metadatos

- **Archivo:** secuencia de bytes con metadatos.
- **Directorio:** estructura que asocia nombres con entradas del sistema.
- **Ruta:** expresión que permite localizar una entrada.
- **Metadatos:** tamaño, tipo, permisos, fechas y otra información del sistema.

La extensión es parte del nombre. Ayuda a inferir un formato, pero no transforma ni valida el contenido.

```
datos.json puede contener texto inválido  
imagen.jpg puede no ser JPEG  
informe.txt puede contener cualquier secuencia de bytes
```

19.2 Node.js como intermediario

El lenguaje JavaScript no define acceso general al disco. Node.js aporta módulos como:

```
import { readFile } from "node:fs/promises";  
import path from "node:path";
```

El navegador restringe el sistema de archivos y ofrece APIs diferentes por seguridad.

19.3 Tres estilos de API

Promesas

```
import { readFile } from "node:fs/promises";

const texto = await readFile("datos.txt", "utf8");
```

Es el estilo principal de este capítulo: no bloquea el hilo mientras espera y se integra con `async/await`, según el modelo que desarrolla el capítulo 17.

Callbacks

```
import { readFile } from "node:fs";

readFile("datos.txt", "utf8", (error, texto) => {
  if (error) {
    console.error(error);
    return;
  }

  console.log(texto);
});
```

Sigue presente en APIs y código histórico.

Sincrónico

```
import { readFileSync } from "node:fs";

const texto = readFileSync("datos.txt", "utf8");
```

Bloquea el hilo hasta terminar. Puede ser razonable en un script corto, durante el arranque o antes de aceptar trabajo concurrente. En un servidor puede detener todas las solicitudes mientras el disco responde.

19.4 Preparar un módulo ejecutable

Podés usar un archivo `.mjs`:

```
node programa.mjs
```

O un proyecto con `package.json`:

```
{
  "type": "module"
}
```

Entonces los archivos `.js` usan `import`. Los proyectos CommonJS utilizan `require`; no mezcles formatos sin comprender cómo los carga Node.js.

19.5 Rutas relativas y directorio de trabajo

```
await readFile("datos/entrada.txt", "utf8");
```

La ruta se interpreta desde `process.cwd()`, el directorio de trabajo del proceso:

```
console.log(process.cwd());
```

No necesariamente coincide con la carpeta del módulo. Puede cambiar según desde dónde se ejecute:

```
node herramientas/procesar.mjs
```

19.6 Ruta relativa al módulo

```
import path from "node:path";
import { fileURLToPath } from "node:url";

const archivoActual = fileURLToPath(import.meta.url);
const directorioActual = path.dirname(archivoActual);
const rutaDatos = path.join(directorioActual, "datos", "entrada.txt");
```

Elegí la base según el contrato:

- `process.cwd()` para una herramienta que trabaja sobre el proyecto invocado;
- `import.meta.url` para recursos ubicados junto al módulo;
- una ruta recibida explícitamente para mayor control y prueba.

19.7 Construir rutas con `node:path`

```
path.join("datos", "2026", "alumnos.json");
path.resolve("datos", "entrada.txt");
path.dirname(ruta);
path.basename(ruta);
path.extname(ruta);
path.parse(ruta);
path.format({ dir: "/tmp", name: "informe", ext: ".txt" });
```

`join` combina segmentos y normaliza. `resolve` construye una ruta absoluta procesando de derecha a izquierda hasta encontrar una base absoluta o usar el directorio de trabajo.

No concatenes separadores manualmente:

```
// const ruta = carpeta + "/" + archivo;
const ruta = path.join(carpeta, archivo);
```

19.8 Diferencias entre sistemas

Windows y sistemas tipo Unix difieren en:

- separadores (\ frente a /);
- raíces y letras de unidad;
- sensibilidad habitual a mayúsculas;
- caracteres permitidos;
- permisos y enlaces;
- convenciones de fin de línea.

`path` usa las reglas de la plataforma actual. `path.posix` y `path.win32` permiten manipular expresiones de una plataforma específica cuando se procesa un formato externo.

No cambies mayúsculas ni reemplaces caracteres de una ruta sin conocer el sistema y el contrato.

19.9 Crear directorios

```
import { mkdir } from "node:fs/promises";

await mkdir("salidas", { recursive: true });
```

Con `recursive`, crea padres faltantes y no falla si el directorio ya existe. Sin esa opción, un padre ausente produce error y una entrada existente puede producir `EEXIST`.

19.10 Escribir texto

```
import { writeFile } from "node:fs/promises";

await writeFile("salida.txt", "Hola\n", "utf8");
```

Por defecto, `writeFile` crea o reemplaza. Si sobrescribir es peligroso, pedilo de forma explícita:

```
await writeFile("salida.txt", "Hola\n", {
  encoding: "utf8",
  flag: "wx"
});
```

`wx` crea de manera exclusiva y falla con `EEXIST` si ya existe. Evita la carrera de “comprobar y luego crear”.

Crear un archivo vacío:

```
await writeFile("vacio.txt", "", { flag: "wx" });
```

19.11 Agregar sin borrar

```
import { appendFile } from "node:fs/promises";

await appendFile("eventos.log", "inicio\n", "utf8");
```

Varias operaciones concurrentes sobre el mismo archivo no forman automáticamente una transacción. Los límites de atomicidad dependen del sistema, el tamaño y el modo de apertura. Si el orden es esencial, serializá escrituras o utilizá un almacenamiento diseñado para concurrencia.

19.12 Bytes, Buffer y texto

Sin codificación, `readFile` devuelve un `Buffer`:

```
const bytes = await readFile("imagen.png");

Buffer.isBuffer(bytes); // true
bytes.length;           // cantidad de bytes
```

Un `Buffer` es una vista de bytes de Node.js. Puede convertirse:

```
const texto = bytes.toString("utf8");
const buffer = Buffer.from("mañana", "utf8");
```

Con codificación en `readFile`, Node.js devuelve string:

```
const texto = await readFile("datos.txt", "utf8");
```

19.13 Unicode no es UTF-8

Unicode asigna puntos de código. UTF-8 define cómo codificarlos en bytes. JavaScript representa strings internamente mediante UTF-16.

```
const texto = "😊";

texto.length; // 2 unidades UTF-16
Buffer.byteLength(texto, "utf8"); // 4 bytes UTF-8
[...texto].length; // 1 punto de código
```


No supongas un byte por carácter ni que `string.length` mide almacenamiento. La sección 5.10 desarrolla la diferencia entre unidad de código, punto de código y grafema, y el apéndice A la aprovecha para construir un tokenizador sobre bytes.

19.14 Codificación incorrecta

Si bytes UTF-8 se decodifican como otra codificación, aparecen caracteres corruptos. No existe una detección universal infalible; varias codificaciones pueden interpretar cualquier byte y producir texto aparentemente válido.

El origen debe declarar el contrato. Para decodificación UTF-8 estricta:

```
const decodificador = new TextDecoder("utf-8", { fatal: true });
const texto = decodificador.decode(bytes);
```

Con `fatal`, una secuencia inválida lanza en vez de insertar el carácter de reemplazo.

19.15 BOM

Algunos archivos comienzan con una marca de orden de bytes. UTF-8 no la necesita, pero puede incluir `EF BB BF`. Algunas herramientas la eliminan o interpretan; otras la conservan como `U+FEFF` al inicio.

Si el primer encabezado de un CSV o JSON parece tener un carácter invisible, inspecciona el BOM. No elimines automáticamente cualquier `U+FEFF` interior, porque puede formar parte real del contenido.

19.16 Fines de línea

Convenciones comunes:

```
Unix/macOS moderno: \n
Windows:            \r\n
```

Para dividir líneas de ambos:

```
const lineas = texto.split(/\r?\n/u);
```

Eso puede dejar una última línea vacía si el archivo termina con salto. Decidí si representa contenido o solo terminación.

Para escribir con la convención de la plataforma:

```
import os from "node:os";

const contenido = lineas.join(os.EOL);
```

En repositorios, suele acordarse `\n` independientemente del sistema para reducir diferencias.

19.17 Leer y transformar un archivo pequeño

```
async function numerarLineas(origen, destino) {
  const texto = await readFile(origen, "utf8");

  const numerado = texto
    .split(/\r?\n/u)
    .map((linea, indice) => `${indice + 1}: ${linea}`)
    .join("\n");

  await writeFile(destino, numerado, {
    encoding: "utf8",
    flag: "wx"
  });
}
```

Es simple y correcto cuando el contenido completo cabe cómodamente en memoria.

19.18 Escritura más segura mediante archivo temporal

Reemplazar un archivo crítico directamente puede dejarlo incompleto si el proceso falla durante la escritura. Un patrón común:

1. escribir un temporal en el mismo directorio;
2. cerrar y, si el nivel de garantía lo exige, sincronizar;
3. renombrar el temporal sobre el destino.

```
import { rename } from "node:fs/promises";

async function escribirReemplazo(ruta, contenido) {
  const temporal = `${ruta}.tmp-${process.pid}`;

  await writeFile(temporal, contenido, {
    encoding: "utf8",
    flag: "wx"
  });

  await rename(temporal, ruta);
}
```

La atomicidad y reemplazo de `rename` dependen del sistema; debe ocurrir en el mismo volumen. Una implementación robusta también limpia temporales, evita colisiones y considera permisos y sincronización.

19.19 Listar un directorio

```
import { readdir } from "node:fs/promises";

const nombres = await readdir("datos");
```

Con tipos:

```
const entradas = await readdir("datos", { withFileTypes: true });

for (const entrada of entradas) {
  if (entrada.isFile()) console.log("archivo", entrada.name);
  if (entrada.isDirectory()) console.log("directoriot", entrada.name);
  if (entrada.isSymbolicLink()) console.log("enlace", entrada.name);
}
```

Un `Dirent` evita consultar `stat` para la clasificación básica de cada entrada.

19.20 Metadatos con `stat` y `lstat`

```
import { stat, lstat } from "node:fs/promises";

const info = await stat(ruta);

info.isFile();
info.isDirectory();
info.size;
info.mtime;
```

`stat` sigue un enlace simbólico; `lstat` informa sobre el enlace mismo. Esta diferencia es importante al recorrer o eliminar árboles, porque seguir enlaces puede salir del directorio esperado o crear ciclos.

Los metadatos pueden cambiar inmediatamente después de consultarlos. No los trates como una garantía permanente.

19.21 Comprobar existencia sin crear una carrera

Esto tiene una ventana:

```
comprobar → otro proceso cambia el archivo → operar
```

Intentá la operación y manejá el código esperado:

```

async function leerOpcional(ruta) {
  try {
    return await readFile(ruta, "utf8");
  } catch (error) {
    if (error.code === "ENOENT") return null;
    throw error;
  }
}

```

`access` existe, pero suele servir para diagnósticos o comprobaciones informativas, no para autorizar una operación posterior.

19.22 Códigos de error frecuentes

- `ENOENT`: ruta inexistente;
- `EACCES` o `EPERM`: permiso insuficiente;
- `EEXIST`: ya existe;
- `EISDIR`: se esperaba archivo y era directorio;
- `ENOTDIR`: un segmento no era directorio;
- `EMPTY`: directorio no vacío;
- `EXDEV`: movimiento entre volúmenes no soportado por `rename`;
- `EMFILE`: demasiados descriptores abiertos.

```

try {
  await operacion();
} catch (error) {
  if (error.code === "ENOENT") {
    // recuperación específica
  } else {
    throw error;
  }
}

```

No dependas solo del mensaje, que cambia entre plataformas.

19.23 Recorrer directorios recursivamente

```
async function* recorrer(directorio) {
  const entradas = await readdir(directorio, { withFileTypes: true });

  for (const entrada of entradas) {
    const ruta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      yield* recorrer(ruta);
    } else if (entrada.isFile()) {
      yield ruta;
    }
  }
}
```

El generador —sección 14.24— produce a medida que recorre, sin construir la lista completa en memoria. La política ignora los enlaces simbólicos; hay que decidir deliberadamente si seguirlos, detectar ciclos y limitar la profundidad.

19.24 Buscar por nombre o extensión

```
async function buscarMarkdown(raiz) {
  const resultados = [];

  for await (const ruta of recorrer(raiz)) {
    if (path.extname(ruta).toLowerCase() === ".md") {
      resultados.push(ruta);
    }
  }

  return resultados;
}
```

La extensión no valida el contenido, pero puede filtrar candidatos. Para patrones complejos se usan globbers con reglas claras sobre separadores, archivos ocultos y enlaces.

19.25 Buscar texto en archivos pequeños

```
async function buscarTexto(ruta, patron) {
  const texto = await readFile(ruta, "utf8");

  return texto
    .split(/\r?\n/u)
    .flatMap((linea, indice) =>
      patron.test(linea)
        ? [{ numero: indice + 1, linea }]
        : []
    );
}
```

Si `patron` tiene flag `g`, `test` conserva `lastIndex` y puede alternar resultados. Para comprobar cada línea, eliminá `g`, cloná el patrón apropiadamente o reiniciá su estado.

19.26 Leer archivos grandes por líneas

```
import { createReadStream } from "node:fs";
import { createInterface } from "node:readline";

async function contarCoincidencias(ruta, patron) {
  const entrada = createReadStream(ruta, { encoding: "utf8" });
  const lineas = createInterface({
    input: entrada,
    crlfDelay: Infinity
  });

  let cantidad = 0;

  for await (const linea of lineas) {
    if (patron.test(linea)) cantidad += 1;
  }

  return cantidad;
}
```

`readline` conserva fragmentos hasta completar una línea y entiende `\r\n` con `crlfDelay: Infinity`.

19.27 Un fragmento de stream no es una línea

Un stream entrega chunks según buffers y disponibilidad, no según límites semánticos:

```
chunk 1: "primera\nsegu"
chunk 2: "nda\ntercera"
```

Tampoco debe dividirse un carácter UTF-8 manualmente sin un decodificador que conserve bytes incompletos. APIs de texto del stream y `readline` resuelven parte de ese trabajo.

19.28 Copiar con streams y `pipeline`

```
import { createReadStream, createWriteStream } from "node:fs";
import { pipeline } from "node:stream/promises";

await pipeline(
  createReadStream(origen),
  createWriteStream(destino, { flags: "wx" })
);
```

`pipeline` propaga errores y coordina cierre y contrapresión. Es preferible a conectar eventos manualmente para un flujo lineal.

La **contrapresión** evita que el productor lea mucho más rápido de lo que el consumidor puede escribir o procesar.

19.29 Manejadores de archivo

```
import { open } from "node:fs/promises";

const archivo = await open(ruta, "r");

try {
  const buffer = Buffer.alloc(100);
  const { bytesRead } = await archivo.read(buffer, 0, 100, 0);
  usar(buffer.subarray(0, bytesRead));
} finally {
  await archivo.close();
}
```

Un `FileHandle` permite lecturas parciales, posiciones explícitas, sincronización y varias operaciones sobre la misma apertura. Siempre debe cerrarse, incluso ante error.

Modos comunes:

- `r`: lectura; debe existir;
- `r+`: lectura y escritura; debe existir;
- `w`: escritura; crea o trunca;
- `wx`: escritura exclusiva;
- `a`: agregar; crea si falta;
- `ax`: agregar de forma exclusiva.

Elegir el modo equivocado puede borrar contenido.

19.30 JSON: sintaxis y estructura

```
async function leerJson(ruta) {
  const texto = await readFile(ruta, "utf8");
  const datos = JSON.parse(texto);

  if (!datos || typeof datos !== "object" || Array.isArray(datos)) {
    throw new TypeError("Se esperaba un objeto JSON");
  }

  return datos;
}
```

`JSON.parse` comprueba sintaxis. La aplicación debe validar propiedades, tipos, rangos y relaciones; la sección 10.18 enumera lo que el formato no conserva y la sección 6.11 discute qué significa una propiedad ausente frente a una propiedad en `null`.

Escribir:

```
await writeFile(
  ruta,
  `${JSON.stringify(datos, null, 2)}\n`,
  { encoding: "utf8", flag: "wx" }
);
```

JSON no admite comentarios, comas finales, `undefined`, símbolos, `bigint`, `Map`, `Set` ni referencias cíclicas sin una representación personalizada.

19.31 CSV no es simplemente `split(",")`

Una línea puede contener:

```
123,"Pérez, Ana","texto con ""comillas"""
```

El formato real permite separadores dentro de comillas, comillas escapadas y a veces saltos dentro de un campo. Usá una biblioteca CSV probada y configurá:

- delimitador;
- codificación;
- encabezados;
- política de filas mal formadas;
- conversión y validación por columna.

Para un formato didáctico deliberadamente simple, documentá que no acepta comillas ni separadores internos.

19.32 Extraer datos de un log

```
const patron = /^(?<fecha>\S+)\s+(?<nivel>INFO|WARN|ERROR)\s+(?<mensaje>.*)$/u;

function interpretarLinea(linea) {
  const match = linea.match(patron);
  if (!match) return { ok: false, linea };

  return {
    ok: true,
    ...match.groups
  };
}
```

El capítulo 18 explica esta expresión regular, con sus grupos con nombre y su ancla de línea completa. Un parser de línea puede devolver éxitos y errores como datos para que una fila inválida no detenga todo el lote; los fallos de lectura, en cambio, siguen siendo excepciones de la operación, según la distinción de la sección 13.5.

19.33 Formatos binarios

PDF, DOCX, imágenes y audio no deben convertirse enteros a UTF-8. Se leen como bytes y se procesan con bibliotecas que entienden su formato.

```
const contenido = await readFile("informe.pdf");
```

Una firma inicial puede ayudar a detectar un candidato, pero validar un formato completo requiere interpretar su estructura.

19.34 Copiar

```
import { copyFile, cp } from "node:fs/promises";
import { constants } from "node:fs";

await copyFile(origen, destino);
await copyFile(origen, destino, constants.COPYFILE_EXCL);

await cp(origenDir, destinoDir, {
  recursive: true,
  errorOnExist: true,
  force: false
});
```

Definí si se reemplaza, si se siguen enlaces, qué metadatos se preservan y qué ocurre ante un fallo parcial.

19.35 Renombrar y mover

```
import { rename } from "node:fs/promises";

await rename(origen, destino);
```

Dentro del mismo sistema de archivos suele ser una operación atómica sobre el nombre. Entre volúmenes puede fallar con `EXDEV`; una estrategia de copiar y eliminar debe contemplar que la copia termine y sea verificable antes de borrar el origen.

19.36 Eliminar

```
import { unlink, rmdir, rm } from "node:fs/promises";

await unlink(rutaArchivo);
await rmdir(directorioVacio);
await rm(arbol, { recursive: true });
```

La eliminación recursiva es peligrosa. Antes de usarla:

1. resolvé una ruta absoluta;
2. verificá que esté dentro de una raíz permitida;
3. rechazá raíz, carpeta de trabajo y destinos demasiado amplios;
4. ofrecé simulación o papelera cuando sea una herramienta de usuario;
5. registrá exactamente qué se eliminará;
6. tratá enlaces según una política explícita.

No dependas de una variable vacía, glob o concatenación para identificar un destino destructivo.

19.37 Impedir que una ruta escape

```
function resolverDentroDe(base, entrada) {
  const raiz = path.resolve(base);
  const candidata = path.resolve(raiz, entrada);
  const relativa = path.relative(raiz, candidata);

  if (relativa === "" ||
      relativa.startsWith(`.${path.sep}`) ||
      relativa === ".." ||
      path.isAbsolute(relativa)) {
    throw new Error("Ruta fuera del directorio permitido");
  }

  return candidata;
}
```

La condición `relativa === ""` rechaza la propia raíz si la operación no debe apuntarle. Ajustala si leer la raíz es válido.

Esta comprobación textual no resuelve por sí sola enlaces simbólicos, cambios concurrentes ni diferencias de mayúsculas del sistema. Para entradas hostiles se necesitan controles adicionales y privilegios mínimos.

19.38 Concurrencia y límites

Procesar miles de archivos con `Promise.all` los abre casi simultáneamente y puede producir `EMFILE`:

```
await Promise.all(rutas.map(procesar));
```

La alternativa secuencial es segura pero puede ser lenta:

```
for (const ruta of rutas) {  
  await procesar(ruta);  
}
```

Un pool con concurrencia limitada equilibra recursos y rendimiento. El límite adecuado depende del disco, la red, el tamaño y la operación. La sección 17.6 explica por qué la primera versión es rápida y la sección 17.8 por qué la segunda es segura.

No ejecutes escrituras concurrentes sobre el mismo destino sin coordinación. El orden de finalización puede diferir del orden de inicio.

19.39 Programa integrador: analizar una carpeta

```
async function analizarCarpeta(raiz) {
  const resumen = {
    archivos: 0,
    bytes: 0,
    lineas: 0,
    errores: []
  };

  for await (const ruta of recorrer(raiz)) {
    try {
      const info = await stat(ruta);
      resumen.archivos += 1;
      resumen.bytes += info.size;

      if (path.extname(ruta).toLowerCase() === ".txt") {
        const texto = await readFile(ruta, "utf8");
        resumen.lineas += texto === ""
          ? 0
          : texto.split(/\r?\n/u).length;
      }
    } catch (error) {
      resumen.errores.push({
        ruta,
        codigo: error.code ?? "DESCONOCIDO",
        mensaje: error.message
      });
    }
  }

  return resumen;
}
```

Esta primera versión prioriza claridad. Para producción habría que decidir tamaño máximo para lectura completa, seguimiento de enlaces, concurrencia, cancelación y política ante errores.

19.40 Criterios para elegir una técnica

Necesidad	Técnica inicial
archivo pequeño completo	<code>readFile</code>
texto grande por líneas	<code>createReadStream</code> + <code>readline</code>
copiar flujo	pipeline
leer posiciones específicas	<code>FileHandle</code>
script corto de arranque	API síncrona, si bloquear es aceptable
muchas rutas	iterador y concurrencia limitada
crear sin reemplazar	flag exclusivo <code>x</code>
reemplazar dato crítico	temporal + <code>rename</code> , con garantías documentadas

19.41 Errores frecuentes

- creer que extensión y formato son lo mismo;
- asumir que una ruta relativa parte del módulo;
- concatenar separadores manualmente;
- omitir codificación y esperar string;
- suponer un byte por carácter;
- sobrescribir con `writeFile` sin intención;
- comprobar existencia y luego actuar como si nada pudiera cambiar;
- leer archivos gigantes completos;
- creer que un chunk es una línea;
- dividir cualquier CSV por comas;
- seguir enlaces sin política;
- lanzar concurrencia sin límite;
- capturar y silenciar todos los códigos;
- eliminar recursivamente una ruta no validada.

19.42 Para recordar

- Ruta, bytes, codificación y formato son capas separadas.
- La ruta relativa depende del directorio de trabajo; los recursos del módulo requieren otra base.
- `readFile` simplifica archivos pequeños; streams y manejadores controlan tamaño y acceso parcial.

- Los flags de apertura expresan si crear, truncar, agregar o exigir exclusividad.
- Seguridad y corrección exigen validar rutas, limitar concurrencia y tratar sobrescritura y eliminación como decisiones explícitas.

Parte V. Construir para la Web

Capítulo 20. Desarrollo web: del documento a una aplicación del clima

Idea central

Una página web reparte el trabajo entre tres tecnologías: HTML describe qué es cada parte, CSS decide cómo se presenta y JavaScript responde a lo que ocurre y actualiza el documento. Separar esas responsabilidades permite cambiar una sin romper las otras, y entender cómo el navegador las combina es la base para construir una aplicación que consulta un servicio externo sin recargar la página.

Los capítulos anteriores construyeron el lenguaje: valores, funciones, objetos, arrays, errores y asincronía. Este lo lleva al navegador, que es su anfitrión original según la sección 1.3. El recorrido termina en una aplicación completa: escribís una ciudad, elegís la ubicación correcta y consultás temperatura, sensación térmica, humedad, viento y condiciones meteorológicas. Usa archivos sencillos y una API pública; no requiere un framework ni una clave para el uso educativo propuesto.

En lenguaje cotidiano decimos «consultar el clima». Técnicamente mostraremos el **tiempo meteorológico actual**; el clima describe patrones de períodos prolongados. Conservamos «app del clima» como nombre del proyecto.

20.1 Qué ocurre cuando abrimos una página

Internet es la infraestructura que conecta redes de computadoras. La **Web** es uno de los servicios que funciona sobre esa infraestructura: permite acceder a recursos enlazados mediante direcciones y protocolos compartidos. El correo electrónico también puede utilizar Internet; Internet y Web no son sinónimos.

En una visita típica a un sitio:

```
Usuario escribe una dirección
↓
Navegador solicita un recurso por HTTP o HTTPS
↓
Servidor responde con HTML
↓
Navegador interpreta HTML y solicita CSS, JS e imágenes referenciadas
↓
Navegador organiza, dibuja y hace interactiva la página
```

El **cliente** es quien solicita el recurso; en este caso, el navegador. El **servidor** recibe solicitudes y entrega respuestas. No necesariamente es una computadora dedicada: durante el desarrollo tu propia computadora puede cumplir ambos papeles.

Una URL identifica un recurso. En `https://ejemplo.com/apuntes/html.html`, `https` es el esquema, `ejemplo.com` es el nombre del servidor y `/apuntes/html.html` es la ruta. Más adelante agregaremos parámetros de consulta después de `?`.

El navegador no muestra literalmente el archivo HTML: lo interpreta. Construye estructuras internas, determina estilos, calcula tamaños y posiciones y dibuja el resultado. Si JavaScript modifica la página, puede actualizar esa representación sin pedir un documento completo nuevo.

Las tres responsabilidades

Tecnología	Responsabilidad	En la app del clima
HTML	Contenido y significado	Un formulario, un título y una lista de mediciones
CSS	Presentación del contenido	Colores, espacios, tipografía y distribución
JavaScript	Respuesta a acciones y cambios	Buscar, consultar datos y actualizar resultados

Esta separación organiza el trabajo, pero no es una frontera absoluta. HTML ya trae comportamientos como navegar con un enlace, enviar un formulario y validar un campo obligatorio. CSS responde a estados como el foco y a condiciones como el ancho disponible. JavaScript permite coordinar lógica más general.

20.2 HTML: origen y problema que resuelve

HTML significa **HyperText Markup Language**, lenguaje de marcado de hipertexto. Un lenguaje de marcado usa señales dentro de un texto para describir su estructura y significado. El hipertexto permite relacionar documentos mediante enlaces.

Tim Berners-Lee propuso la Web en CERN en 1989 para facilitar el intercambio de información. Hacia fines de 1990 ya había implementado sus componentes iniciales, entre ellos HTML, HTTP, direcciones URL, un navegador y un servidor. La necesidad era compartir documentos conectados entre computadoras y equipos de trabajo. CERN: nacimiento de la Web y CERN: componentes iniciales.

Imaginá un documento que contiene estas líneas:

```
El tiempo en Córdoba
La temperatura es de 23 grados.
Más información
```

Para una persona, la primera línea podría ser un título. Para un programa, son tres líneas sin significado explícito. HTML permite expresar esa intención:

```
<h1>El tiempo en Córdoba</h1>
<p>La temperatura es de 23 grados.</p>
<a href="https://open-meteo.com/">Más información meteorológica</a>
```

Ahora el navegador sabe que hay un encabezado principal, un párrafo y un enlace. También pueden aprovechar esa estructura tecnologías de asistencia, buscadores y otras herramientas.

HTML es un lenguaje **declarativo**: describís los elementos que necesitás. Por sí solo no expresa un algoritmo general como «consultá este servicio, compará los resultados y repetí la operación».

Contenido, significado y presentación

Consideremos tres decisiones diferentes:

- **Contenido:** el texto «El tiempo en Córdoba».
- **Estructura y significado:** ese texto es el encabezado principal de la página.
- **Presentación:** se dibuja con cierto tamaño, color y espaciado.

`<h1>` expresa la segunda decisión. CSS controla la tercera. No elegimos un encabezado por su tamaño visual: elegimos su nivel por la relación entre las partes del documento.

El navegador incluye estilos predeterminados. Por eso un HTML sin una hoja CSS propia no se ve como texto plano. El autor puede cambiar la presentación conservando el significado.

20.3 Sintaxis y estructura de HTML

Etiquetas, elementos y atributos

```
<p class="resumen">Hoy hay cielo despejado.</p>
```

Parte	Nombre	Función
<code><p class="resumen"></code>	Etiqueta de apertura	Inicia el párrafo y declara un atributo
<code>class="resumen"</code>	Atributo y valor	Asigna una clase que se puede reutilizar
Hoy hay cielo despejado.	Contenido	Texto del párrafo
<code></p></code>	Etiqueta de cierre	Termina el párrafo
El conjunto completo	Elemento	Unidad de la estructura

Los atributos se escriben en la apertura. Usaremos nombres de etiquetas en minúscula y valores entre comillas. Las comillas delimitan el valor; no son parte del texto visible.

Una **clase** identifica un grupo reutilizable. Un **id** identifica un elemento y debe ser único dentro del documento:

```
<p id="estado" class="mensaje destacado">Buscando ciudad...</p>
```

Este párrafo tiene un identificador y dos clases, separadas por un espacio. Las clases no generan un aspecto visual por su nombre: debe existir una regla CSS que las utilice.

Anidamiento y relaciones

```
<section>
  <h2>Condiciones actuales</h2>
  <p>La temperatura es de <strong>23 °C</strong>.</p>
</section>
```

`section` contiene dos elementos hijos: `h2` y `p`. Son hermanos. `strong` es hijo de `p` y descendiente de `section`. Esta estructura en forma de árbol será fundamental para comprender selectores y DOM.

Las etiquetas se cierran en el orden inverso al que se abrieron:

```
<!-- Correcto -->
<p>Temperatura <strong>actual</strong>.</p>

<!-- Incorrecto: las etiquetas se cruzan -->
<p>Temperatura <strong>actual.</p></strong>
```

La sangría facilita la lectura, pero no define quién contiene a quién: lo hacen las etiquetas. El navegador intenta recuperarse de HTML incorrecto; que algo «se vea» no demuestra que esté bien estructurado.

Elementos vacíos y atributos booleanos

Algunos elementos, como `img`, `input`, `meta` y `link`, no admiten contenido ni etiqueta de cierre:

```
<input type="text" required>

```

En HTML no se escribe `</input>` ni `</img>`. La barra final de la notación `<input />` no es necesaria.

`required`, `disabled` y `hidden` son ejemplos de atributos booleanos: **su presencia los activa**. `disabled="false"` igualmente deshabilita el control; para habilitarlo hay que quitar ese atributo. En JavaScript sí podremos usar la propiedad `elemento.disabled =`

false.

Documento mínimo completo

Un documento HTML mínimo, codificado como UTF-8:

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi primera página</title>
</head>
<body>
  <h1>Mi primera página del clima</h1>
  <p>Estoy aprendiendo a estructurar contenido.</p>
</body>
</html>
```

Componente	Explicación
<!doctype html>	Activa el modo de estándares del navegador; no es un elemento visible
html	Elemento raíz que contiene el documento
lang="es"	Declara el idioma principal del contenido
head	Reúne metadatos y referencias a recursos
charset="UTF-8"	Indica la codificación de caracteres
viewport	Permite que el ancho de diseño acompañe el dispositivo en móviles
title	Título del documento, usado en la pestaña y los marcadores
body	Contiene los elementos de la página

`head`, `header` y `h1` tienen propósitos distintos. `head` contiene metadatos; `header` organiza una introducción dentro del cuerpo; `h1` es un encabezado del contenido.

Los documentos HTML son archivos de texto, habitualmente con extensión `.html`.

Espacios, comentarios y caracteres especiales

En el flujo de texto normal, varios espacios o saltos de línea del código suelen colapsar visualmente. No se organiza una página agregando espacios repetidos. Para párrafos usamos `p`; para separación visual, CSS. `pre` conserva espacios y saltos cuando realmente forman parte del contenido.

```
<!-- Este comentario ayuda a quien lee el archivo. -->
<p>La etiqueta <p> representa un párrafo.</p>
<p>HTML & CSS trabajan juntos.</p>
```

<, > y & representan <, > y &. Un comentario no se muestra en la página, pero cualquiera puede verlo al inspeccionar el archivo: no sirve para ocultar secretos.

Referencia de sintaxis: [MDN: estructura de elementos y documentos HTML](#).

20.4 Componentes básicos y HTML con significado

Texto y estructura de página

Elementos	Para qué se usan	Criterio práctico
h1 a h6	Encabezados de distintos niveles	En este proyecto habrá un h1; los apartados principales usarán h2
p	Párrafo	Una idea o unidad de texto
strong, em	Importancia y énfasis	El significado importa más que su aspecto predeterminado
ul, ol, li	Listas sin orden o con secuencia	Usá ol cuando cambiar el orden cambie el sentido
dl, dt, dd	Términos y descripciones o valores	Útiles para «Humedad: 60 %»
header	Introducción de una página o sección	Puede contener título y descripción
nav	Grupo de enlaces de navegación	Reservado para navegación relevante
main	Contenido principal	Un único main visible en esta página
section	Sección temática	Normalmente tiene un encabezado
article	Contenido que puede entenderse por sí mismo	Por ejemplo, una noticia
footer	Información de cierre	Fuentes, autoría o enlaces relacionados
div, span	Agrupaciones sin significado específico	Cuando no hay un elemento semántico adecuado

Un `div` suele iniciar una caja de bloque y un `span` suele participar en una línea de texto. Eso es comportamiento visual predeterminado, modificable con `display` en CSS; no equivale a su significado.

Enlaces, rutas e imágenes

```
<a href="https://open-meteo.com/">Visitar Open-Meteo</a>
<a href="acerca.html">Acerca de este proyecto</a>
<a href="#fuentes">Ir a las fuentes</a>

<h2 id="fuentes">Fuentes de los datos</h2>
```

El primer enlace usa una URL absoluta; el segundo busca un archivo relativo a la ubicación del documento; el tercero apunta a un identificador dentro de la página. El texto del enlace debe anticipar el destino: «Documentación de la API» informa más que «clic aquí».

Una imagen de contenido puede agruparse con su descripción mediante `figure` y `figcaption`:

```
<figure>
  
  <figcaption>Una ciudad que podemos consultar.</figcaption>
</figure>
```

El texto `alt` comunica la información visual relevante cuando la imagen no puede verse. Una imagen exclusivamente decorativa puede llevar `alt=""`. `width` y `height` ayudan a reservar espacio; CSS puede adaptar la imagen a su contenedor. La aplicación final no necesita imágenes para funcionar.

Datos tabulares

Una tabla sirve cuando la relación entre filas y columnas es parte del significado:

```
<table>
  <caption>Temperaturas de práctica, inventadas</caption>
  <thead>
    <tr><th scope="col">Ciudad</th><th scope="col">Temperatura</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">Córdoba</th><td>23 °C</td></tr>
    <tr><th scope="row">Salta</th><td>20 °C</td></tr>
  </tbody>
</table>
```

`tr` define una fila; `th`, una celda de encabezado; `td`, una celda de datos. No usamos tablas para ubicar el menú al lado del contenido: ese problema de presentación se resuelve con CSS.

Formularios y comportamiento nativo

```
<form action="index.html" method="get">
  <label for="ciudad">Ciudad</label>
  <input id="ciudad" name="ciudad" type="search" required minlength="2">
  <button type="submit">Buscar</button>
</form>
```

`label` se conecta al campo porque su `for` coincide con el `id`. Al pulsar la etiqueta se enfoca el campo. `name` identifica el dato en un envío: no cumple el mismo papel que `id`. `type="search"` expresa la finalidad del campo. `required` impide un envío normal vacío y `minlength` agrega una restricción de longitud.

El formulario ya sabe reaccionar al botón y al envío con Enter. Si es válido, `method="get"` envía los valores como parámetros de la URL. `action="index.html"` apunta a nuestro documento de práctica; puede recargarlo con `?ciudad=...`. Todavía no hay un servicio que transforme ese nombre en un resultado.

Después JavaScript interceptará el evento `submit` y gestionará la consulta sin navegar a otro documento. No necesita inventar un botón con un `div` ni detectar manualmente cada pulsación de Enter.

Otros controles importantes:

Elemento o atributo	Función
<code>input type="email"</code>	Campo con semántica y validación básica de correo
<code>input type="number"</code>	Valor numérico, con restricciones como <code>min</code> y <code>max</code>
<code>input type="checkbox"</code>	Elección que puede estar marcada o desmarcada
<code>input type="radio"</code>	Elección excluyente dentro de un grupo con el mismo <code>name</code>
<code>textarea</code>	Texto de varias líneas
<code>select</code> y <code>option</code>	Selección entre opciones
<code>fieldset</code> y <code>legend</code>	Agrupación de campos con un título
<code>button type="button"</code>	Botón que no envía automáticamente el formulario
<code>placeholder</code>	Ejemplo breve dentro del campo; no reemplaza una etiqueta
<code>disabled</code>	Control deshabilitado; no participa del envío nativo

La validación del navegador ayuda al usuario, pero no garantiza la validez de los datos en un servidor. Además, `required` sobre un campo de texto no significa «contiene un nombre real»: una cadena de espacios requiere una comprobación adicional. Es la frontera que describió la sección 1.5: la validación del navegador mejora la experiencia; la del servidor protege el sistema.

20.5 CSS: historia y separación de la presentación

CSS significa **Cascading Style Sheets**, hojas de estilo en cascada. Permite declarar cómo deben presentarse los elementos de un documento.

A medida que la Web creció, aumentó la necesidad de controlar tipografías, colores y distribución. Mezclar decisiones visuales dentro del HTML hacía difícil reutilizar diseños y mantener muchas páginas. Håkon Wium Lie presentó una propuesta de hojas de estilo en 1994; Bert Bos se sumó al desarrollo. CSS1 se convirtió en una recomendación de W3C en diciembre de 1996. [W3C: historia de CSS](#).

Supongamos que cien páginas deben compartir el mismo color de encabezado. Si la decisión está repetida dentro de cada documento, cambiarla exige revisar cien lugares. Si todas enlazan una hoja de estilos, podemos modificar una regla compartida.

La separación también permite cambiar la presentación para una pantalla pequeña o para impresión manteniendo el mismo contenido. No consiste simplemente en tener archivos separados: importa que el HTML describa lo que es cada elemento y que CSS concentre las decisiones visuales.

Tres maneras de incorporar CSS

En un atributo, para un estilo local:

```
<p style="color: teal;">Temperatura actual</p>
```

Dentro del documento, con un elemento `style` en `head`:

```
<style>
  p { color: teal; }
</style>
```

En un archivo externo, nuestra opción para la aplicación:

```
<link rel="stylesheet" href="styles.css">
```

El elemento `link` referencia la hoja desde `head`. En un archivo CSS se escriben reglas directamente, **sin** etiquetas `<style>`.

Un archivo externo facilita compartir y mantener estilos. El estilo en un atributo puede servir en situaciones puntuales, pero repetirlo por toda la página hace más difíciles los cambios.

20.6 Sintaxis, selectores, cascada y herencia

Anatomía de una regla

```
.tarjeta {  
  color: #173342;  
  background-color: white;  
  padding: 1rem;  
}
```

`.tarjeta` es el **selector**: indica a qué elementos se aplica la regla. Entre llaves está el **bloque de declaraciones**. Cada declaración une una **propiedad** y un **valor** mediante `:`; se separan con `;`.

CSS no dice «recorré cada tarjeta y pintala». Declara una condición: los elementos que coinciden con ese selector reciben esas declaraciones cuando la cascada las determina como ganadoras.

Los comentarios CSS se escriben `/* así */`. Si una propiedad o un valor no son válidos, el navegador suele descartar esa declaración. Una llave faltante puede afectar una región más amplia de la hoja.

Selectores esenciales

Tipo	Ejemplo	Qué selecciona
Universal	*	Todos los elementos
Por elemento	p	Todos los párrafos
Por clase	.tarjeta	Elementos cuya clase incluya tarjeta
Por identificador	#estado	El elemento con ese id
Por atributo	input[type="search"]	Campos cuyo atributo coincida
Lista	h1, h2	Los elementos que coincidan con cualquiera de esos selectores
Descendiente	.tarjeta p	Párrafos a cualquier profundidad dentro de una tarjeta
Hijo directo	.tarjeta > p	Párrafos cuyo padre inmediato sea la tarjeta
Hermano siguiente	label + input	Un input inmediatamente después de un label con el mismo padre
Pseudoclase	button:hover	Botones sobre los que está el puntero
Pseudoclase de foco	input:focus-visible	Campos cuyo foco el navegador considera que debe verse
Pseudoelemento	p::first-letter	La primera letra de un párrafo en las condiciones aplicables

No es lo mismo `.tarjeta .destacado` que `.tarjeta.destacado`. El primero busca un descendiente; el segundo selecciona un mismo elemento que tenga ambas clases.

```
<section class="tarjeta destacado">
  <p>Primer párrafo.</p>
  <div><p>Segundo párrafo.</p></div>
</section>
```

Cuando dos reglas quieren definir lo mismo

```
<p class="aviso" id="estado">Buscando ciudad...</p>
```

```
p { color: navy; }
.avisos { color: green; }
#estado { color: darkred; }
```

En este ejemplo gana el rojo oscuro del selector por identificador. Para entenderlo hay que distinguir **cascada** y **especificidad**.

La cascada resuelve las declaraciones que compiten por una propiedad. Considera, entre otros factores, su origen —navegador, usuario o autor—, importancia, capas y especificidad. En el caso introductorio de **declaraciones normales del autor, sin capas ni estilos en atributos**, comparamos especificidad y, si empatan, orden de aparición.

Podemos leer la especificidad de los selectores básicos con tres columnas:

Selector	IDs	Clases, atributos y pseudoclases	Elementos y pseudoelementos
p	0	0	1
.avisos	0	1	0
p.avisos	0	1	1
#estado	1	0	0
#estado.avisos	1	1	0

Se comparan las columnas de izquierda a derecha. **No se suman como un número decimal**. El selector universal y los combinadores, como `>` o el espacio, no agregan peso por sí mismos.

```
.avisos { color: green; }
.avisos { color: purple; }
```

Ahora la especificidad empata: gana la declaración que aparece después. Por eso «siempre gana la última regla» es una explicación incompleta.

Los estilos en atributos tienen una prioridad particular dentro de las declaraciones normales del autor. `!important` modifica la etapa de importancia de la cascada; no es un incremento de especificidad. Este proyecto no utiliza `!important`.

Herencia

```
body {
  color: #173342;
  font-family: system-ui, sans-serif;
}
```

Muchas propiedades de texto se heredan de los ancestros cuando el elemento no obtiene otro valor aplicable. Esto evita repetir la fuente en cada párrafo. En cambio, propiedades como `margin`, `padding` y `border` no se heredan normalmente.

Una declaración aplicada directamente a un hijo no «compite en especificidad» con el color que podría heredar de su padre: la herencia se usa cuando corresponde, después de resolver los valores del propio elemento. Para que los controles sigan la tipografía de la página usaremos explícitamente `font: inherit`.

Referencia para profundizar: [MDN: cascada, especificidad y herencia](#).

20.7 Propiedades CSS por categorías

Las propiedades se organizan según la dimensión de presentación que controlan. Una tarjeta permite observar su efecto sobre un contenido concreto:

```
<section class="tarjeta">
  <h2>Córdoba</h2>
  <p class="temperatura">23 °C</p>
  <p>Dato inventado para practicar estilos.</p>
</section>
```

Valores y unidades: con qué medimos

Unidad o valor	Qué expresa	Uso inicial
px	Píxel CSS, no necesariamente un píxel físico	Bordes finos
%	Proporción de una referencia que depende de la propiedad	Ancho relativo al contenedor
rem	Múltiplo del tamaño de fuente del elemento raíz	Texto, espacios y límites
em	Relativo a la fuente; en font-size, a la del padre	Medidas que acompañan al componente
vw, vh	Porcentaje del ancho o alto del viewport	Tamaños ligados a la ventana
auto	Valor calculado según la propiedad y el contexto	Márgenes para centrar un bloque limitado
0	Cero, generalmente sin unidad	Quitar márgenes o bordes

Si la fuente raíz es de 16 píxeles CSS, `1.5rem` equivale a 24 píxeles CSS. No conviene asumir que todos los usuarios conservan esa configuración. Por eso mantenemos medidas que se adapten al texto.

En móviles, el alto visible cambia cuando aparecen barras del navegador; las unidades `svh`, `lvh` y `dvh` distinguen variantes de ese viewport. Nuestra aplicación deja crecer su contenido y no necesita fijar la altura de la pantalla.

Texto y tipografía: cómo se lee

Propiedad	Ejemplo	Efecto
<code>font-family</code>	<code>system-ui, sans-serif</code>	Familia preferida y alternativa
<code>font-size</code>	<code>1.2rem</code>	Tamaño del texto
<code>font-weight</code>	<code>700</code>	Grosor tipográfico
<code>line-height</code>	<code>1.6</code>	Altura de línea; sin unidad acompaña el tamaño de fuente
<code>text-align</code>	<code>center</code>	Alineación del contenido en línea dentro de la caja
<code>text-decoration</code>	<code>underline</code>	Decoración como subrayado
<code>letter-spacing</code>	<code>0.02em</code>	Separación adicional entre caracteres
<code>overflow-wrap</code>	<code>anywhere</code>	Permite cortar secuencias muy largas para evitar desbordamientos

```
body {  
  font-family: system-ui, sans-serif;  
  font-size: 1rem;  
  line-height: 1.6;  
}  
  
.temperatura {  
  font-size: 3rem;  
  font-weight: 700;  
  line-height: 1.2;  
}
```

El dato principal destaca por tamaño y peso, mientras los párrafos mantienen una altura de línea cómoda. `text-align: center` centra texto y contenido en línea; no es una instrucción universal para centrar cualquier caja.

Colores y superficies: qué se distingue

Propiedad	Ejemplo	Efecto
color	#172d39	Color del texto
background-color	#f3f6f8	Color del fondo
background-image	url("fondo.jpg")	Imagen de fondo decorativa
border	1px solid #748591	Grosor, estilo y color del borde
border-radius	0.75rem	Redondeo de esquinas
box-shadow	0 2px 8px #00000020	Sombra: desplazamientos, desenfoque y color
opacity	0.65	Transparencia de todo el elemento y su contenido

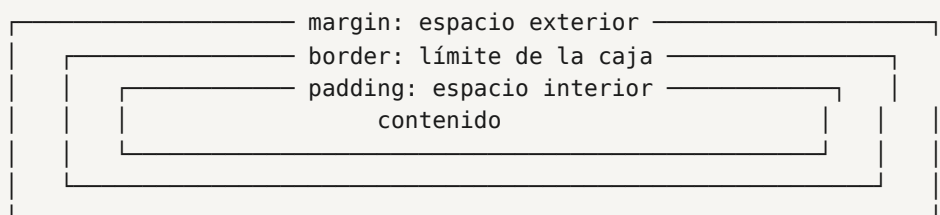
Podés usar nombres (`white`), valores hexadecimales (`#086b70`) o funciones como `rgb(8 107 112)`. En `#00000020`, los dos últimos dígitos representan transparencia alfa.

```
.tarjeta {  
  color: #172d39;  
  background-color: white;  
  border: 1px solid #748591;  
  border-radius: 0.75rem;  
}
```

Un fondo no agrega contenido semántico. Una imagen que comunica información necesita una representación adecuada en HTML, no solamente una imagen decorativa en CSS.

Modelo de caja: cuánto espacio ocupa algo

Cada caja se puede entender en cuatro capas:



Propiedad	Dimensión que controla
width, height	Ancho o alto solicitado
min-width, min-height	Tamaño mínimo permitido
max-width, max-height	Límite superior del tamaño
padding	Espacio entre contenido y borde
margin	Espacio exterior solicitado
box-sizing	Partes incluidas en el ancho o alto
overflow	Tratamiento del contenido que desborda

Con el modelo inicial `content-box`, `width` define el ancho del contenido:

```
.tarjeta {
  width: 300px;
  padding: 20px;
  border: 2px solid;
}
```

Su ancho hasta el borde exterior será $300 + 20 + 20 + 2 + 2 = 344$ **píxeles CSS**, sin contar márgenes. Con `box-sizing: border-box`, los 300 ya incluyen contenido, relleno y bordes: quedan 256 para contenido.

```
* { box-sizing: border-box; }
```

Esta regla facilita razonar sobre las dimensiones. No agrega márgenes al ancho declarado.

Las abreviaturas de `margin` y `padding` siguen esta lógica:

```
.tarjeta {
  padding: 1rem;           /* Los cuatro lados */
  margin: 1rem 2rem;       /* Vertical; horizontal */
  border-width: 1px 2px 3px 4px; /* Arriba; derecha; abajo; izquierda */
}
```

En bloques del flujo normal, ciertos márgenes verticales pueden **colapsar**: dos márgenes de 20 y 30 píxeles no necesariamente producen 50. `padding` no colapsa de esa forma. Más adelante usaremos `gap` entre elementos de Flexbox y Grid para expresar separación de manera directa.

Flujo normal y visibilidad: cómo se colocan las cajas

El navegador organiza inicialmente el contenido en el **flujo normal**. En nuestra escritura horizontal, elementos de bloque como un párrafo suelen ocupar una línea propia y el ancho disponible; los elementos en línea, como `span`, acompañan el texto.

Declaración	Comportamiento principal
<code>display: block</code>	Caja de bloque dentro del flujo
<code>display: inline</code>	Participación en una línea; ancho y alto no se controlan como en un bloque
<code>display: inline-block</code>	Caja dimensionable que participa en una línea
<code>display: flex</code>	Organiza los hijos con Flexbox
<code>display: grid</code>	Organiza los hijos en una cuadrícula
<code>display: none</code>	No genera caja; normalmente también se omite del árbol de accesibilidad
<code>visibility: hidden</code>	Oculta la caja visualmente, pero conserva su espacio

El atributo HTML `hidden` expresa que un elemento no es relevante para el estado actual y normalmente lo oculta. Lo usaremos para resultados todavía inexistentes. Hay que evitar reglas propias de `display` que anulen involuntariamente ese ocultamiento.

No fijas una altura pequeña a una tarjeta con texto variable: un nombre largo, una traducción o el zoom pueden necesitar más espacio. En general, dejá que el contenido determine el alto.

Posicionamiento: cuándo salir del flujo

Valor de position	Uso
<code>static</code>	Posición normal; valor inicial
<code>relative</code>	Conserva su lugar y puede desplazarse; sirve como referencia para descendientes posicionados
<code>absolute</code>	Sale del flujo y se ubica respecto de su bloque contenedor; a menudo un ancestro posicionado
<code>fixed</code>	Sale del flujo; normalmente queda fijado respecto del viewport
<code>sticky</code>	Conserva espacio en el flujo y puede adherirse a un límite durante el desplazamiento

`top`, `right`, `bottom` y `left` definen desplazamientos cuando el contexto de posicionamiento los admite. `z-index` participa en el orden de superposición dentro de contextos de apilamiento; un número enorme no garantiza quedar sobre cualquier cosa de la página.


```
.tarjeta { position: relative; }
.insignia { position: absolute; top: 0.5rem; right: 0.5rem; }
```

Estados e interacción visual

```
button { cursor: pointer; }
button:hover { background-color: #075257; }
button:disabled { opacity: 0.65; cursor: wait; }
:focus-visible { outline: 3px solid #964e0b; outline-offset: 3px; }
```

Una pseudoclase describe un estado; no agrega un elemento nuevo. El foco identifica qué control recibe la interacción del teclado. El contorno de foco debe verse. Quitar `outline` sin reemplazarlo hace difícil saber dónde se encuentra el usuario.

Variables y movimiento opcional

Las **propiedades personalizadas** permiten nombrar decisiones reutilizables:

```
:root { --acento: #086b70; }
button { background-color: var(--acento); }
a { color: var(--acento); }
```

`:root` selecciona el elemento raíz; las propiedades personalizadas suelen heredarse. Al cambiar `--acento`, ambos usos se actualizan.

Para una transición breve, declaramos qué propiedad puede cambiar gradualmente:

```
@media (prefers-reduced-motion: no-preference) {
  button { transition: background-color 150ms ease; }
}
```

`transform` permite trasladar, rotar o escalar visualmente; `transition` interpola cambios; `animation` y `@keyframes` definen secuencias. Son herramientas opcionales. Primero necesitamos una interfaz entendible, y cualquier movimiento debe respetar las preferencias del usuario.

20.8 Distribución y diseño adaptable

Un contenedor que deja respirar al contenido

```
.contenedor {
  width: 100%;
  max-width: 48rem;
  margin-inline: auto;
  padding: 2rem 1rem;
}
```

En una pantalla chica ocupa el ancho disponible. En una grande deja de crecer al llegar al máximo, evitando líneas excesivamente largas. Los márgenes automáticos en el eje en línea lo centran cuando sobra espacio. `margin-inline` es una propiedad lógica: acompaña la dirección de escritura; en nuestro caso se corresponde con los márgenes izquierdo y derecho.

Flexbox: distribuir en un eje principal

```
<div class="fila">
  <input aria-label="Ciudad de práctica" type="search">
  <button type="button">Buscar</button>
</div>
```

Este fragmento pequeño usa un nombre accesible para aislar el ejemplo de distribución; en el formulario completo tendremos una etiqueta visible.

```
.fila {
  display: flex;
  gap: 0.75rem;
  align-items: center;
}

.fila input { flex: 1; min-width: 0; }
.fila button { flex-shrink: 0; }
```

`display: flex` convierte a los hijos directos en elementos flexibles. Con la dirección inicial `row`, el eje principal recorre la fila. `justify-content` distribuye el espacio en ese eje y `align-items` alinea en el transversal. Si cambiamos a `column`, los ejes cambian: no significan siempre horizontal y vertical.

Propiedad	Ejemplo	Función
<code>flex-direction</code>	<code>row</code> o <code>column</code>	Dirección del eje principal
<code>gap</code>	<code>0.75rem</code>	Separación entre elementos
<code>justify-content</code>	<code>space-between</code>	Distribución del espacio sobrante en el eje principal
<code>align-items</code>	<code>center</code> o <code>stretch</code>	Alineación en el eje transversal
<code>flex-wrap</code>	<code>wrap</code>	Permite más de una línea
<code>flex</code>	<code>1</code>	Abreviatura que aquí permite al campo crecer y encogerse
<code>flex-shrink</code>	<code>0</code>	Impide que ese elemento se reduzca por reparto de espacio

`min-width: 0` deja que el campo se reduzca cuando su tamaño mínimo automático podría impedirlo. Es útil con campos y contenido largo.

Grid: organizar filas y columnas

```
.datos {  
  display: grid;  
  grid-template-columns: repeat(3, minmax(0, 1fr));  
  gap: 1rem;  
}  
  
.dato-principal { grid-column: 1 / -1; }
```

`repeat(3, ...)` crea tres columnas. `1fr` reparte una fracción del espacio disponible. `minmax(0, 1fr)` permite reducir cada columna hasta cero antes de considerar su contenido; el texto todavía debe poder ajustarse. `grid-column: 1 / -1` hace que el dato principal abarque desde la primera hasta la última línea de la cuadrícula.

Grid es adecuado para las mediciones de nuestra tarjeta; Flexbox para un campo y su botón. La elección depende de la distribución que queremos expresar, no de que una técnica reemplace universalmente a la otra.

Media queries: cambiar una regla cuando hay espacio

```
.fila { display: flex; flex-direction: column; gap: 0.75rem; }  
.datos { display: grid; grid-template-columns: 1fr; gap: 1rem; }  
  
@media (min-width: 40rem) {  
  .fila { flex-direction: row; }  
  .datos { grid-template-columns: repeat(3, minmax(0, 1fr)); }  
}
```

La versión base apila los controles. La consulta de medios activa otra distribución cuando el viewport alcanza el ancho indicado. Es un enfoque **mobile first**: comenzamos por el espacio reducido y agregamos distribución cuando cabe.

El punto de cambio se elige observando el contenido, no buscando un modelo específico de teléfono. El metadato `viewport` del HTML es parte del conjunto; no adapta por sí solo un diseño de ancho fijo.

Para imágenes de contenido, una base útil es:

```
img { max-width: 100%; height: auto; }
```

20.9 JavaScript y su entorno de ejecución

Hasta ahora el contenido fue escrito por nosotros. Para mostrar datos diferentes según lo que ingrese una persona, necesitamos describir una secuencia de operaciones: leer, validar, buscar, transformar y presentar.

JavaScript es un lenguaje de programación. Su núcleo está estandarizado como **ECMAScript**; la primera edición de ECMA-262 se publicó en 1997. Java y JavaScript son lenguajes distintos. Ecma International: ECMA-262.

Lenguaje, motor y entorno no son lo mismo

Concepto	Responsabilidad	Ejemplos
Lenguaje	Define valores, expresiones, funciones y reglas de ejecución	const, if, arreglos, promesas
Motor	Ejecuta el código JavaScript	V8, SpiderMonkey, JavaScriptCore
Entorno	Proporciona capacidades adicionales al programa	Navegador o un entorno de servidor como Node.js
API del navegador	Expone funciones de la plataforma	DOM, eventos, fetch, temporizadores

`document` no es una palabra del lenguaje JavaScript: es un objeto que el navegador proporciona. En un programa de Node.js, por defecto, no existe el documento de una página ni su DOM. Algunos entornos comparten APIs como `fetch`, pero eso no los vuelve idénticos. La sección 1.3 presenta estas capas en detalle.

En nuestra aplicación, JavaScript se ejecutará **en el navegador del usuario**. El servicio meteorológico corre en servidores externos. No estamos construyendo un servidor de datos propio: consumimos uno existente.

Carga de un script en el documento

El documento puede referenciar un archivo externo desde `head`:

```
<script src="app.js" defer></script>
```

En un script clásico externo, `defer` permite descargar el archivo durante el análisis de HTML y ejecutarlo cuando el documento terminó de analizarse. Así podrá encontrar los elementos del cuerpo. Los scripts clásicos con `defer` conservan el orden del documento. `async` tiene otra finalidad: ejecuta cuando está disponible y no garantiza ese orden.

El atributo `defer` no se aplica del mismo modo a un script clásico escrito directamente dentro del elemento. Otra alternativa son los módulos, con `type="module"`, que ya difieren su ejecución por defecto y permiten importar y exportar código, como describe la sección 1.8. Para empezar usaremos un único archivo clásico externo.

Un mensaje de diagnóstico se escribe con `console.log`:

```
console.log("El archivo JavaScript se ejecutó.");
```

El mensaje aparece en la consola de las herramientas de desarrollador, no como contenido de la página.

20.10 DOM, eventos y una primera página dinámica

El documento como árbol de objetos

DOM significa **Document Object Model**. Es la representación del documento que las APIs permiten consultar y modificar. Al analizar HTML, el navegador construye nodos: elementos, texto y otros tipos.

```
<main>
  <h1>El clima</h1>
  <p id="estado">Todavía no hay una consulta.</p>
</main>
```

Una vista simplificada de ese árbol:

```
document
├── html
│   ├── head
│   └── body
│       └── main
│           ├── h1
│           │   └── texto: El clima
│           └── p#estado
│               └── texto: Todavía no hay una consulta.
```

El archivo fuente y el DOM están relacionados, pero no son la misma cosa. El navegador puede corregir estructuras durante el análisis, y JavaScript puede agregar nodos. Por eso «Ver código fuente» y el panel «Elementos» pueden mostrar información diferente. [MDN: DOM](#).

Buscar y modificar elementos

```
const estado = document.querySelector("#estado");
estado.textContent = "Estamos preparando la consulta.";
estado.classList.add("destacado");
```

`querySelector` recibe un selector CSS y devuelve el primer elemento que coincide, o `null`. `querySelectorAll` devuelve una colección de coincidencias que podemos recorrer. Un resultado `null` puede deberse al selector, a la inexistencia del elemento o al momento de ejecución del script.

Operación	Efecto
<code>elemento.textContent = "..."</code>	Cambia el contenido de texto
<code>campo.value</code>	Lee el valor actual de un control
<code>elemento.classList.add("clase")</code>	Agrega una clase
<code>elemento.classList.remove("clase")</code>	Quita una clase
<code>elemento.classList.toggle("clase", condicion)</code>	Asegura su presencia o ausencia según un booleano
<code>elemento.hidden = true</code>	Activa su estado oculto
<code>campo.disabled = true</code>	Deshabilita el control
<code>document.createElement("option")</code>	Crea un elemento todavía separado del documento
<code>padre.append(hijo)</code>	Inserta un nodo al final del padre
<code>padre.replaceChildren()</code>	Quita los nodos hijos
<code>elemento.remove()</code>	Retira ese elemento del DOM
<code>elemento.focus()</code>	Lleva el foco a un elemento que lo admite

Para texto ingresado por usuarios o recibido de una API usaremos `textContent`. `innerHTML` interpreta marcado: insertar texto no confiable con esa propiedad puede convertirlo en contenido activo. No necesitamos esa interpretación para mostrar una ciudad o una temperatura.

Escuchar eventos

Un evento comunica que ocurrió algo: un clic, una modificación de un campo, un envío o la carga de un recurso. `addEventListener` registra una función para reaccionar.

```
boton.addEventListener("click", () => {
  console.log("Se activó el botón.");
});
```

Aquí `boton` debe ser una referencia obtenida previamente del DOM. Pasamos la función; no escribimos una llamada como segundo argumento. Queremos que se ejecute cuando ocurra el evento.

Muchos eventos se propagan desde el elemento de origen hacia sus ancestros. `evento.target` identifica dónde se originó; `evento.currentTarget`, el elemento cuyo listener se está ejecutando. `preventDefault()` cancela una acción predeterminada cancelable, como navegar al enviar un formulario; **no** detiene por sí mismo la propagación. [MDN: eventos](#).

Primera integración completa sin Internet

El documento del ejemplo contiene un formulario y una región de estado:

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM y eventos</title>
  <script src="dom.js" defer></script>
</head>
<body>
  <h1>Clima de práctica</h1>
  <p>Datos inventados: probá Córdoba o Salta.</p>
  <form id="formulario" action="dom.html" method="get">
    <label for="ciudad-prueba">Ciudad</label>
    <input id="ciudad-prueba" name="ciudad" required minlength="2">
    <button type="submit">Consultar dato de práctica</button>
  </form>
  <p id="mensaje" role="status" aria-live="polite"></p>
</body>
</html>
```

El script conecta el formulario con un catálogo local de datos inventados:

```
const formulario = document.querySelector("#formulario");
const entrada = document.querySelector("#ciudad-prueba");
const mensaje = document.querySelector("#mensaje");

const datosDePractica = {
  "córdoba": { nombre: "Córdoba", temperatura: 23 },
  "salta": { nombre: "Salta", temperatura: 20 }
};

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();
  const nombre = entrada.value.trim().toLowerCase();

  if (nombre.length < 2) {
    mensaje.textContent = "Escribí al menos dos caracteres.";
    return;
  }

  // Solo aceptamos propiedades propias de nuestro pequeño catálogo.
  const dato = Object.hasOwn(datosDePractica, nombre)
    ? datosDePractica[nombre]
    : undefined;

  if (!dato) {
    mensaje.textContent = "No hay un dato de práctica para esa ciudad.";
    return;
  }

  mensaje.textContent = `${dato.nombre}: ${dato.temperatura} °C (dato inventado).`;
});
```

`Object.hasOwn` evita utilizar propiedades heredadas como si fueran ciudades del catálogo. La sección 10.5 explica la diferencia con el operador `in`.

El flujo es el siguiente: el navegador valida las restricciones nativas, emite `submit`, nuestro listener evita la navegación, normaliza la entrada, busca el dato y modifica un nodo de texto. Ninguna de esas modificaciones reescribe el archivo `.html` del disco. Al recargar, se vuelve a comenzar desde el documento original.

`role="status"` y `aria-live="polite"` permiten anunciar cambios a tecnologías de asistencia sin interrumpir de inmediato otras lecturas. No agregan por sí mismos un aspecto visual.

20.11 Asincronía, HTTP, JSON y APIs

De datos locales a un servicio externo

Una **API** es una interfaz mediante la que un programa usa capacidades de otro componente. El DOM es una API del navegador. En este capítulo consumimos una API web: solicitamos datos a un servidor por HTTP.

Necesitamos dos consultas:

```
Texto: «Córdoba»
  ↓ API de geocodificación
Lista de ubicaciones con nombre, región, país, latitud y longitud
  ↓ elección del usuario
Una ubicación concreta
  ↓ API meteorológica con sus coordenadas
Datos meteorológicos
  ↓ JavaScript transforma y actualiza el DOM
Una tarjeta legible
```

Geocodificar es convertir una referencia textual a una ubicación en coordenadas. Una ciudad puede compartir nombre con muchas otras; la selección explícita evita decidir silenciosamente por la persona.

La documentación de Open-Meteo define `/v1/search` para buscar lugares y `/v1/forecast` para consultar condiciones mediante coordenadas. Su geocodificador permite elegir idioma y cantidad de resultados. [API de geocodificación](#) y [API meteorológica](#).

Solicitudes y respuestas

Una solicitud incluye una dirección, un método y, según el caso, encabezados y cuerpo. Usaremos **GET**, destinado a obtener una representación de un recurso. HTTPS agrega protección de la comunicación durante el transporte.

Ejemplo de URL para inspeccionar en el navegador:


```
https://geocoding-api.open-meteo.com/v1/search?
name=Cordoba&count=10&language=es&format=json
```

? inicia los parámetros y & los separa. Cada par une un nombre y un valor. En código no concatenaremos directamente la entrada del usuario: `URLSearchParams` codifica caracteres especiales, espacios y acentos.

```
const parametros = new URLSearchParams({ name: "San Miguel de Tucumán", language: "es"
});
console.log(parametros.toString());
```

Una respuesta tiene un código de estado, encabezados y un cuerpo. Ejemplos de códigos son 200 para éxito, 400 para una solicitud inválida, 404 para un recurso no encontrado, 429 para demasiadas consultas y 500 para un error interno del servidor. El capítulo 22 desarrolla HTTP en detalle: la forma de los mensajes, los métodos y la tabla completa de códigos de estado.

Una ciudad sin coincidencias no necesariamente produce un error HTTP: puede ser una respuesta exitosa sin resultados. Hay que distinguir **fallo de transporte, error HTTP y resultado válido pero vacío**.

JSON es un formato de datos

JSON significa JavaScript Object Notation. Es texto estructurado que puede representar objetos, arreglos, cadenas, números, booleanos y `null`. No contiene funciones ni comentarios, y los nombres de propiedades y las cadenas usan comillas dobles.

Ejemplo didáctico reducido, con valores inventados:

```
{
  "ciudad": "Córdoba",
  "temperatura": 23,
  "unidad": "°C",
  "esEjemplo": true
}
```

No confundas ese texto con un objeto JavaScript ya disponible en memoria. La respuesta viaja como datos que debemos leer y convertir. Tampoco asumimos que la respuesta del servicio use nuestros nombres en español: respetaremos su contrato y mostraremos etiquetas propias en la interfaz. La sección 22.4 detalla qué admite JSON y en qué se diferencia de un objeto de JavaScript.

Promesas y espera sin congelar la página

Una consulta tarda y puede fallar. `fetch()` devuelve una **promesa**, un objeto que representa un resultado futuro: pendiente, cumplido o rechazado.

```

async function obtenerEjemplo() {
  const respuesta = await fetch(
    "https://geocoding-api.open-meteo.com/v1/search?name=Salta&count=1&language=es"
  );

  if (!respuesta.ok) {
    throw new Error(`Error HTTP ${respuesta.status}`);
  }

  const datos = await respuesta.json();
  console.log(datos);
}

obtenerEjemplo().catch((error) => console.error(error.message));

```

`async` hace que la función devuelva una promesa. `await` suspende la continuación de **esa función asíncrona** hasta que se resuelva la promesa esperada; no congela el navegador. Si la promesa se rechaza, `await` produce una excepción que podemos capturar. `.catch(...)` registra qué hacer ante un rechazo.

Hay dos esperas: obtener la respuesta y leer su cuerpo como JSON. `fetch` no rechaza automáticamente por cada estado HTTP de error; debemos revisar `respuesta.ok`, que indica un estado entre 200 y 299. [MDN: uso de Fetch](#). El capítulo 17 desarrolla el modelo de promesas y la sección 22.8 explica por qué hacen falta dos esperas.

El ciclo de eventos

En el hilo principal de una página, JavaScript ejecuta una tarea a la vez. El navegador coordina entradas, red, temporizadores y oportunidades de dibujo. Cuando una operación asíncrona tiene un resultado, su continuación se programa para ejecutarse cuando corresponde; las continuaciones de promesas se procesan como microtareas.

Una tarea larga que hace cálculos sin ceder puede bloquear la interfaz. Usar `await fetch(...)` permite esperar la red sin mantener ocupado ese hilo. `async` no vuelve automáticamente paralelo cualquier cálculo. [MDN: modelo de ejecución](#).

El siguiente ejemplo muestra el orden relativo entre ejecución inmediata, microtareas y temporizadores:

```

console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");

```

El resultado es **A, D, C, B**. Primero termina el código actual; luego se procesa la continuación de la promesa; el temporizador se ejecuta en una tarea posterior. Cero milisegundos no significa «ahora mismo».

Errores y limpieza

```
async function consultar() {
  try {
    // Acá va una operación que puede fallar, posiblemente con await.
  } catch (error) {
    // Informamos el problema de una forma útil.
    console.error(error.message);
  } finally {
    // Liberamos recursos o restablecemos controles.
  }
}
```

`throw new Error("...")` interrumpe el camino normal. `catch` maneja una excepción y `finally` permite ejecutar limpieza tanto si hubo éxito como si hubo fallo. En la app, el botón siempre debe volver a habilitarse. El capítulo 13 desarrolla estas reglas.

Usaremos `AbortController` para cancelar una espera de red demasiado larga. Un temporizador pedirá la cancelación después de 12 segundos y `clearTimeout` lo retirará si la operación termina antes. No es una garantía de reloj exacto: la ejecución de temporizadores depende del entorno.

CORS y claves

Los navegadores aplican reglas a las consultas entre orígenes diferentes. **CORS** permite que un servidor indique qué accesos desde otros orígenes admite. El origen combina esquema, host y puerto.

Si una API no permite el acceso desde tu página, agregar `mode: "no-cors"` no te habilita a leer el JSON: normalmente obtendrías una respuesta opaca. La solución debe respetar la configuración del servicio o usar un backend autorizado que actúe como intermediario.

Todo el JavaScript enviado al navegador puede inspeccionarse. Una clave secreta no se protege escondiéndola en `app.js`. El acceso abierto de Open-Meteo evita necesitar una clave en este ejemplo educativo; sus condiciones limitan el servicio gratuito al uso no comercial, incluyen límites de consultas y requieren atribución. La app conserva los créditos. Para otro tipo de publicación hay que consultar sus condiciones vigentes. [Open-Meteo: condiciones y licencia de datos](#).

20.12 Proyecto integrado: estructura y presentación

La integración reúne un documento semántico, una hoja de estilos y cinco bloques de JavaScript. Cada etapa incorpora una responsabilidad al recorrido de búsqueda y consulta.

Comportamiento de la aplicación

1. La persona escribe una ciudad.

2. La aplicación busca hasta diez coincidencias.
3. La persona distingue las ubicaciones por nombre, región y país.
4. Elige una y activa «Ver clima».
5. La aplicación muestra las condiciones actuales y la hora de los datos en la zona consultada.
6. Durante una consulta se informa el progreso y se impiden envíos simultáneos.
7. Si falta información o hay un problema, aparece un mensaje y se puede reintentar.

Estado de la aplicación	Qué ve la persona	Qué puede hacer
Inicial	Campo y explicación	Escribir una ciudad
Buscando lugares	Mensaje de carga	Esperar a que termine o alcance el límite de tiempo
Sin coincidencias	Mensaje específico	Corregir la búsqueda
Lugares disponibles	Lista con región y país	Elegir una ubicación
Consultando el tiempo	Mensaje de carga	Esperar
Resultado disponible	Mediciones y fecha	Consultar otra ubicación o buscar otra ciudad
Error	Explicación del problema	Reintentar

Esta tabla es parte del diseño del programa. Si solo imaginamos el resultado exitoso, dejamos sin definir qué ocurre durante el resto del recorrido.

Estructura de archivos

La aplicación se organiza en tres archivos con responsabilidades diferentes:

```
app-clima/  
├─ index.html  → estructura y contenido  
├─ styles.css  → presentación  
└─ app.js      → comportamiento y acceso a datos
```

El navegador ejecuta JavaScript y obtiene datos de un servicio externo. No hay librerías, compilación ni un servidor de aplicación propio.

HTML: estructura de la interfaz

El archivo `index.html` define la interfaz completa:

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Buscá una ciudad y consultá sus condiciones
meteorológicas actuales.">
  <title>El clima en tu ciudad</title>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
</head>
<body>
  <main class="contenedor">
    <header>
      <p class="etiqueta">Una ventana al tiempo</p>
      <h1>El clima en tu ciudad</h1>
      <p>Buscá una ciudad, elegí la ubicación y consultá el tiempo actual.</p>
    </header>

    <form id="form-busqueda" class="tarjeta" action="index.html" method="get">
      <fieldset id="campos-busqueda">
        <legend>1. Buscá una ciudad</legend>
        <label for="ciudad">Nombre de la ciudad (obligatorio)</label>
        <p id="ayuda-ciudad" class="ayuda">Escribí al menos 2 caracteres. Por ejemplo:
Córdoba.</p>
        <div class="fila">
          <input id="ciudad" name="ciudad" type="search" required
            minlength="2" maxlength="100" placeholder="Córdoba"
            autocomplete="off" aria-describedby="ayuda-ciudad">
          <button type="submit">Buscar ciudad</button>
        </div>
      </fieldset>
    </form>

    <p id="estado" class="estado" role="status" aria-live="polite" aria-atomic="true">
      Ingresá una ciudad para comenzar.
    </p>

    <form id="form-ubicacion" class="tarjeta" action="index.html" method="get" hidden>
      <fieldset id="campos-ubicacion">
        <legend>2. Elegí la ubicación</legend>
        <label for="ubicacion">Ciudades encontradas</label>
        <div class="fila">
          <select id="ubicacion" name="ubicacion" required></select>
          <button type="submit">Ver clima</button>
        </div>
      </fieldset>
    </form>

    <section id="resultado" class="tarjeta" aria-labelledby="nombre-ciudad" hidden>
      <p class="etiqueta">Condiciones actuales</p>
      <h2 id="nombre-ciudad" tabindex="-1"></h2>
      <p id="descripcion"></p>
      <dl class="datos">
        <div class="dato-principal">
          <dt>Temperatura</dt>
          <dd id="temperatura" class="temperatura"></dd>
        </div>

```

```

        <div><dt>Sensación térmica</dt><dd id="sensacion"></dd></div>
        <div><dt>Humedad</dt><dd id="humedad"></dd></div>
        <div><dt>Viento</dt><dd id="viento"></dd></div>
    </dl>
    <p class="ayuda">Hora de los datos: <time id="actualizacion"></time>.</p>
    <p class="ayuda">Condiciones estimadas a partir de modelos meteorológicos.</p>
</section>

<noscript><p>Activá JavaScript para buscar ciudades y consultar el clima.</p>
</noscript>

<footer>
    <p>Datos meteorológicos de <a href="https://open-meteo.com/">Open-Meteo</a>.
        Ubicaciones de <a href="https://www.geonames.org/">GeoNames</a>.
        Datos bajo <a href="https://creativecommons.org/licenses/by/4.0/">CC BY 4.0</a>.
    </p>
    <p>Proyecto educativo con HTML, CSS y JavaScript.</p>
</footer>
</main>
</body>
</html>

```

El documento separa las responsabilidades de la interfaz. El primer formulario captura la búsqueda. El segundo contiene la selección de una ubicación y empieza oculto. El `section` del resultado tiene espacios para los datos y también empieza oculto. La lista `dl` relaciona cada medición con su valor.

Los `id` son los puntos de conexión con JavaScript; las clases se usan principalmente para CSS. El nombre de una clase no controla automáticamente una función ni viceversa. Un cambio de `id` requiere actualizar el selector correspondiente en el script.

`aria-describedby` conecta la ayuda con el campo. `aria-labelledby` da nombre a la sección usando su encabezado. `tabindex="-1"` permite enfocar el título del resultado desde JavaScript sin agregarlo al recorrido habitual de Tab. `aria-atomic="true"` solicita anunciar el mensaje de estado como una unidad.

`noscript` explica la dependencia de JavaScript. Esta aplicación necesita JS para obtener resultados: el HTML ofrece estructura y controles nativos, pero no estamos implementando una alternativa de búsqueda en servidor.

CSS: presentación de la aplicación

El archivo `styles.css` define los estilos de la aplicación:

```

/* Base: dimensiones predecibles y colores compartidos. */
* { box-sizing: border-box; }

:root {
  --fondo: #f3f6f8;
  --superficie: #ffffff;
  --texto: #172d39;
  --secundario: #4f6470;
  --acento: #086b70;
  --borde: #748591;
  --error: #a32424;
}

body {
  margin: 0;
  background-color: var(--fondo);
  color: var(--texto);
  font-family: system-ui, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
}

.contenedor {
  width: 100%;
  max-width: 48rem;
  margin-inline: auto;
  padding: 2rem 1rem;
}

header { margin-bottom: 2rem; }
h1, h2 { line-height: 1.2; overflow-wrap: anywhere; }
h1 { font-size: 2.3rem; margin: 0.5rem 0 1rem; }
h2 { font-size: 1.6rem; }

.etiqueta {
  color: var(--acento);
  font-weight: 700;
  margin: 0;
}

.tarjeta {
  background-color: var(--superficie);
  border: 1px solid var(--borde);
  border-radius: 1rem;
  padding: 1.25rem;
  margin-bottom: 1rem;
}

fieldset { border: 0; padding: 0; margin: 0; min-width: 0; }
legend { font-weight: 700; padding: 0; margin-bottom: 1rem; }
label { display: block; font-weight: 600; margin-bottom: 0.35rem; }
.ayuda, footer { color: var(--secundario); font-size: 0.9rem; }
.ayuda { margin: 0.35rem 0 0.75rem; }

.fila { display: flex; flex-direction: column; gap: 0.75rem; }

input, select, button {
  font: inherit;
}

```

```

    min-height: 3rem;
    border-radius: 0.5rem;
    padding: 0.65rem 0.8rem;
}

input, select {
    width: 100%;
    min-width: 0;
    color: var(--texto);
    background-color: var(--superficie);
    border: 1px solid var(--borde);
}

button {
    border: 0;
    background-color: var(--acento);
    color: white;
    font-weight: 700;
    cursor: pointer;
}

button:hover:not(:disabled) { background-color: #075257; }
button:disabled { opacity: 0.65; cursor: wait; }
input:disabled, select:disabled { opacity: 0.7; }
a { color: var(--acento); }

:--focus-visible { outline: 3px solid #964e0b; outline-offset: 3px; }
#nombre-ciudad:focus { outline: 3px solid #964e0b; outline-offset: 3px; }

.estado { min-height: 1.6em; margin: 1rem 0; }
.estado.es-error { color: var(--error); font-weight: 600; }

.datos { display: grid; grid-template-columns: 1fr; gap: 1rem; }
.datos > div { padding-top: 0.5rem; border-top: 1px solid #d6dfe4; }
dt { color: var(--secundario); }
dd { margin: 0.2rem 0 0; font-size: 1.2rem; font-weight: 600; }
.temperatura { font-size: 3.2rem; line-height: 1.2; }
.dato-principal { grid-column: 1 / -1; }
footer { margin-top: 2rem; }

/* Primero la pantalla chica; ampliamos cuando hay espacio. */
@media (min-width: 40rem) {
    .fila { flex-direction: row; align-items: stretch; }
    .fila input, .fila select { flex: 1; }
    .fila button { flex-shrink: 0; }
    .datos { grid-template-columns: repeat(3, minmax(0, 1fr)); }
}

```

La hoja se lee en grupos: base, contenedor, tarjetas, formularios, estados, mediciones y adaptación. `font: inherit` hace que los controles sigan la tipografía general. `min-width: 0` permite que los campos se reduzcan dentro de las filas. `:not(:disabled)` evita aplicar el estado de puntero activo a un botón deshabilitado.

La tarjeta exterior no recibe `display: grid`; se lo damos a `.datos`, su lista interna. Así no anulamos accidentalmente el `hidden` del contenedor de resultados.

20.13 Proyecto integrado: JavaScript paso a paso

El archivo `app.js` se compone de los cinco bloques siguientes, en este orden. En conjunto constituyen el comportamiento completo de la aplicación.

Las funciones iniciales preparan operaciones reutilizables; los listeners del último bloque conectan esas operaciones con la interfaz. Que una función esté definida no significa que ya se haya ejecutado.

Bloque 1: referencias al DOM y estado

Referencias y estado:

```
// BLOQUE 1: referencias al DOM y estado de la aplicación.
const formBusqueda = document.querySelector("#form-busqueda");
const camposBusqueda = document.querySelector("#campos-busqueda");
const entradaCiudad = document.querySelector("#ciudad");
const estado = document.querySelector("#estado");
const formUbicacion = document.querySelector("#form-ubicacion");
const camposUbicacion = document.querySelector("#campos-ubicacion");
const selectorUbicacion = document.querySelector("#ubicacion");
const resultado = document.querySelector("#resultado");
const nombreCiudad = document.querySelector("#nombre-ciudad");
const descripcion = document.querySelector("#descripcion");
const temperatura = document.querySelector("#temperatura");
const sensacion = document.querySelector("#sensacion");
const humedad = document.querySelector("#humedad");
const viento = document.querySelector("#viento");
const actualizacion = document.querySelector("#actualizacion");

let ciudades = [];
let ocupada = false;
```

Guardamos referencias para no repetir las búsquedas del DOM en cada función. `ciudades` conserva los resultados de la última búsqueda; `ocupada` indica que hay una consulta en curso. Ambas usan `let` porque les asignaremos nuevos valores.

Una referencia declarada con `const` puede apuntar a un elemento cuyo texto cambia: no se reasigna el nombre, se modifica el objeto. Esto relaciona el DOM con lo que aprendimos sobre objetos.

Bloque 2: operaciones de interfaz

Funciones de interfaz:

```
// BLOQUE 2: pequeñas operaciones de interfaz.
function mostrarEstado(mensaje, esError = false) {
  estado.textContent = mensaje;
  estado.classList.toggle("es-error", esError);
}

function establecerCarga(valor) {
  ocupada = valor;
  camposBusqueda.disabled = valor;
  camposUbicacion.disabled = valor;
}

function nombreCompleto(ciudad) {
  return [ciudad.name, ciudad.admin1, ciudad.country]
    .filter(Boolean)
    .join(", ");
}

function mostrarCiudades() {
  selectorUbicacion.replaceChildren();

  for (const ciudad of ciudades) {
    const opcion = document.createElement("option");
    opcion.value = String(ciudad.id);
    opcion.textContent = nombreCompleto(ciudad);
    selectorUbicacion.append(opcion);
  }

  formUbicacion.hidden = false;
}
```

`mostrarEstado` concentra los mensajes y su clase visual. El parámetro opcional `esError` permite reutilizarla para estados normales o fallidos. JavaScript decide el estado; CSS define cómo se ve la clase.

`establecerCarga` actualiza la variable de control y deshabilita ambos grupos de campos. Así ninguna búsqueda puede empezar mientras otra ocupa el flujo. El guardado `ocupada` también protege los listeners ante invocaciones repetidas.

`nombreCompleto` arma una cadena a partir de datos que pueden estar incompletos. `.filter(Boolean)` conserva los valores que se interpretan como verdaderos: aquí quita propiedades faltantes y cadenas vacías. `.join(", ")` intercala comas entre los valores restantes.

`mostrarCiudades` vacía las opciones anteriores y crea nodos nuevos. Cada `option` muestra una etiqueta legible y guarda el identificador numérico convertido a cadena. El valor de un `select` se obtiene como texto; esa conversión será importante al encontrar la ciudad elegida.

Bloque 3: acceso a datos

Funciones de acceso a la red:

```
// BLOQUE 3: acceso a la red con errores y tiempo límite.
async function pedirJSON(url) {
  const controlador = new AbortController();
  const temporizador = setTimeout(() => controlador.abort(), 12000);

  try {
    const respuesta = await fetch(url, { signal: controlador.signal });

    if (!respuesta.ok) {
      if (respuesta.status === 429) {
        throw new Error("El servicio recibió demasiadas consultas. Esperá un momento y reintentá.");
      }
      throw new Error(`El servicio respondió con un error (${respuesta.status}). Reintentá más tarde.`);
    }

    const datos = await respuesta.json();
    if (datos === null || typeof datos !== "object" || Array.isArray(datos)) {
      throw new Error("El servicio devolvió un formato de datos inesperado.");
    }
    return datos;
  } catch (error) {
    if (error.name === "AbortError") {
      throw new Error("La consulta tardó demasiado. Volvé a intentarlo.");
    }
    if (error instanceof TypeError) {
      throw new Error("No se pudo conectar con el servicio. Revisá tu conexión e intentá de nuevo.");
    }
    if (error instanceof SyntaxError) {
      throw new Error("El servicio devolvió una respuesta que no se pudo leer.");
    }
    throw error;
  } finally {
    clearTimeout(temporizador);
  }
}

async function buscarCiudades(nombre) {
  const parametros = new URLSearchParams({
    name: nombre,
    count: "10",
    language: "es",
    format: "json"
  });
  const datos = await pedirJSON(`https://geocoding-api.open-meteo.com/v1/search?${parametros}`);

  if (datos.error) {
    throw new Error("No se pudo completar la búsqueda de ciudades.");
  }

  const encontradas = datos.results ?? [];
  if (!Array.isArray(encontradas)) {
    throw new Error("El servicio devolvió una lista de ciudades inválida.");
  }
}
```

```

return encontradas.filter((ciudad) =>
  ciudad && Number.isInteger(ciudad.id) && typeof ciudad.name === "string" &&
  Number.isFinite(ciudad.latitude) && Number.isFinite(ciudad.longitude)
);
}

async function consultarClima(ciudad) {
  const parametros = new URLSearchParams({
    latitude: String(ciudad.latitude),
    longitude: String(ciudad.longitude),
    current:
      "temperature_2m,apparent_temperature,relative_humidity_2m,weather_code,wind_speed_10m",
    temperature_unit: "celsius",
    wind_speed_unit: "kmh",
    timezone: "auto",
    timeformat: "unixtime"
  });
  const datos = await pedirJSON(`https://api.open-meteo.com/v1/forecast?${parametros}`);

  if (datos.error || !Number.isFinite(datos.current?.temperature_2m) ||
    !Number.isFinite(datos.current?.time) || typeof datos.timezone !== "string") {
    throw new Error("No hay datos meteorológicos suficientes para esa ubicación.");
  }

  return datos;
}

```

El acceso a los datos tiene tres capas:

Función	Entrada	Salida o resultado
pedirJSON	Una URL	Promesa de un objeto leído, o un error comprensible
buscarCiudades	Nombre escrito	Promesa de un arreglo de ubicaciones utilizables
consultarClima	Una ubicación elegida	Promesa de datos meteorológicos con mínimos comprobados

La primera función resuelve problemas comunes de transporte. Las otras conocen los parámetros y la forma de datos que necesita cada operación. Ninguna modifica la tarjeta del resultado: eso tendrá su propia función.

`AbortController` crea un controlador y una señal. `fetch` observa esa señal; el temporizador invoca `abort()` si se alcanza el tiempo límite. `instanceof` reconoce clases de errores, como fallos de red o JSON inválido. La sección 13.6 enumera esas clases. `finally` limpia el temporizador aun cuando se lanza un error.

La respuesta puede ser JSON válido y aun así tener una forma inesperada. Por eso comprobamos que exista un objeto y que la búsqueda entregue un arreglo. Filtramos ubicaciones sin identificador, nombre o coordenadas utilizables.

Si `results` no está presente, `?? []` permite tratarlo como una colección vacía. En cambio, la ausencia de una temperatura actual o de una marca temporal impide mostrar un resultado completo, por lo que emitimos un error.

Los parámetros meteorológicos solicitan temperatura y sensación en Celsius, humedad, código de condición y viento en km/h. Usamos `timeformat=unixtime` para recibir un instante en segundos, y `timezone=auto` para conocer la zona correspondiente. Las condiciones actuales proceden de modelos meteorológicos; no se presentan como una medición en vivo de una estación. Contrato de Open-Meteo.

Bloque 4: presentación de los datos

Funciones de formato y presentación:

```
// BLOQUE 4: transformar datos en información legible.
function describirClima(codigo) {
  if (codigo === 0) return "Despejado";
  if (codigo === 1) return "Mayormente despejado";
  if (codigo === 2) return "Parcialmente nublado";
  if (codigo === 3) return "Cubierto";
  if ([45, 48].includes(codigo)) return "Niebla";
  if ([51, 53, 55].includes(codigo)) return "Llovizna";
  if ([56, 57].includes(codigo)) return "Llovizna helada";
  if ([61, 63, 65].includes(codigo)) return "Lluvia";
  if ([66, 67].includes(codigo)) return "Lluvia helada";
  if ([71, 73, 75, 77].includes(codigo)) return "Nieve";
  if ([80, 81, 82].includes(codigo)) return "Chaparrones";
  if ([85, 86].includes(codigo)) return "Chaparrones de nieve";
  if (codigo === 95) return "Tormenta";
  if ([96, 99].includes(codigo)) return "Tormenta con granizo";
  return "Condición no disponible";
}

function formatearMedida(valor, unidad) {
  if (!Number.isFinite(valor)) return "No disponible";
  return `${valor.toLocaleString("es-AR", { maximumFractionDigits: 1 })} ${unidad}`;
}

function mostrarClima(ciudad, datos) {
  const actual = datos.current;
  const fecha = new Date(actual.time * 1000);
  const fechaLocal = new Intl.DateTimeFormat("es-AR", {
    dateStyle: "medium",
    timeStyle: "short",
    timeZone: datos.timezone
  }).format(fecha);

  nombreCiudad.textContent = nombreCompleto(ciudad);
  descripcion.textContent = describirClima(actual.weather_code);
  temperatura.textContent = formatearMedida(actual.temperature_2m, "°C");
  sensacion.textContent = formatearMedida(actual.apparent_temperature, "°C");
  humedad.textContent = formatearMedida(actual.relative_humidity_2m, "%");
  viento.textContent = formatearMedida(actual.wind_speed_10m, "km/h");
  actualizacion.dateTime = fecha.toISOString();
  actualizacion.textContent = `${fechaLocal} (${datos.timezone})`;
  resultado.hidden = false;
}

```

La API utiliza códigos meteorológicos. `describirClima` los convierte a textos breves; agrupa algunas intensidades para mantener el ejemplo legible y conserva una alternativa si aparece un código desconocido. No supone que cualquier código diferente de cero signifique lluvia. La tabla oficial está en la [documentación de códigos de Open-Meteo](#).

`formatearMedida` distingue un número de un valor ausente. `Number.isFinite(0)` es verdadero, de modo que **0 °C se muestra correctamente**. Si falta una medición secundaria aparece «No disponible», sin inventar un cero. `toLocaleString` adapta la escritura del número y limita su parte decimal.

`Date` recibe milisegundos; por eso multiplicamos los segundos por 1000. No sumamos manualmente un desfase horario. `Intl.DateTimeFormat` presenta ese mismo instante usando la zona devuelta por la consulta, aunque la computadora del usuario esté en otro país.

El elemento `time` conserva una representación estándar en `dateTime` y muestra texto localizado. Esa fecha corresponde al dato meteorológico; no es simplemente la hora en que la persona hizo clic.

`mostrarClima` asigna cada valor al nodo correcto y revela el resultado al final. `textContent` permite mostrar los nombres externos como texto. **Mostrar un resultado** es transformar datos en una representación; no requiere reconstruir todo el documento.

Bloque 5: coordinación de eventos

Listeners de los formularios:

```
// BLOQUE 5: conectar los formularios con las funciones.
formBusqueda.addEventListener("submit", async (evento) => {
  evento.preventDefault();
  if (ocupada) return;

  const nombre = entradaCiudad.value.trim();
  if (nombre.length < 2) {
    mostrarEstado("Escribí al menos 2 caracteres, sin contar los espacios de los extremos.", true);
    entradaCiudad.focus();
    return;
  }

  establecerCarga(true);
  resultado.hidden = true;
  formUbicacion.hidden = true;
  selectorUbicacion.replaceChildren();
  ciudades = [];
  mostrarEstado("Buscando ciudades...");

  try {
    ciudades = await buscarCiudades(nombre);
    if (ciudades.length === 0) {
      mostrarEstado("No encontramos esa ciudad. Revisá el nombre o probá una ciudad cercana.");
    } else {
      mostrarCiudades();
      mostrarEstado(`Coincidencias: ${ciudades.length}. Elegí una ubicación y presioná Ver clima.`);
    }
  } catch (error) {
    mostrarEstado(error.message, true);
  } finally {
    establecerCarga(false);
    if (ciudades.length > 0) selectorUbicacion.focus();
    else entradaCiudad.focus();
  }
});

selectorUbicacion.addEventListener("change", () => {
  resultado.hidden = true;
  mostrarEstado("Ubicación elegida. Presioná Ver clima para consultar sus datos.");
});

formUbicacion.addEventListener("submit", async (evento) => {
  evento.preventDefault();
  if (ocupada) return;

  const ciudad = ciudades.find((item) => String(item.id) === selectorUbicacion.value);
  if (!ciudad) {
    mostrarEstado("Primero buscá y elegí una ciudad.", true);
    return;
  }

  establecerCarga(true);
  resultado.hidden = true;
  mostrarEstado(`Consultando el clima de ${nombreCompleto(ciudad)}...`);
});
```



```

try {
  const datos = await consultarClima(ciudad);
  mostrarClima(ciudad, datos);
  mostrarEstado("Consulta completada. Podés elegir otra ubicación o buscar una nueva ciudad.");
} catch (error) {
  mostrarEstado(error.message, true);
} finally {
  establecerCarga(false);
  if (!resultado.hidden) nombreCiudad.focus();
  else selectorUbicacion.focus();
}
});

```

El primer listener valida y busca ubicaciones. Oculta el resultado anterior y limpia la selección, para que una búsqueda nueva no conserve opciones de la anterior. Distingue una colección vacía de un error de red.

El listener de `change` oculta un resultado que ya no corresponde a la ubicación seleccionada. El último listener recupera la ciudad mediante su `id`, consulta el tiempo y muestra la respuesta. Comparamos cadenas porque `selectorUbicacion.value` devuelve texto.

En ambos recorridos `finally` libera los controles. El foco pasa a la selección disponible o al encabezado del resultado; si algo falla, vuelve a un control útil para reintentar. Como se bloquean los dos formularios durante cada consulta, no hay dos respuestas simultáneas intentando reemplazar el mismo resultado. Una versión más avanzada podría permitir nuevas búsquedas y cancelar las anteriores.

20.14 Reconstruir una consulta completa

Volvamos al comportamiento, sin mirar todo el código a la vez:

```

submit del formulario de búsqueda
  → preventDefault()
  → leer value y aplicar trim()
  → validar longitud
  → activar carga y limpiar datos anteriores
  → await buscarCiudades(nombre)
  → crear opciones o informar que no hay coincidencias
  → finally: desactivar carga

submit del formulario de ubicación
  → preventDefault()
  → encontrar la ciudad por su identificador
  → activar carga
  → await consultarClima(ciudad)
  → transformar valores y actualizar el DOM
  → finally: desactivar carga

```

En una consulta exitosa intervienen las tres tecnologías. HTML aporta el formulario y los espacios de resultado; CSS presenta controles, estados y mediciones; JavaScript conecta eventos, datos y DOM. La solicitud meteorológica se procesa en un servidor externo, y la presentación se actualiza en el navegador.

No estamos guardando las búsquedas en una base de datos ni conservándolas al recargar. Las variables representan el estado de esta ejecución. Persistir una ciudad favorita sería un requisito nuevo que podemos agregar más adelante.

Cuánto se transfiere y cuándo

No hacemos una solicitud por cada letra escrita. Una búsqueda válida genera una consulta de lugares; cada activación de «Ver clima» genera otra consulta meteorológica. Si hay un error antes de obtener las coordenadas, no iniciamos la consulta del tiempo.

La app envía al geocodificador el nombre que se busca y al servicio meteorológico las coordenadas de la ciudad elegida. No solicita la ubicación del dispositivo: la aplicación nunca sabe dónde está quien la usa, solo qué ciudad eligió.

20.15 Herramientas de diagnóstico

El DOM, la cascada y las solicitudes de red se pueden observar mediante las herramientas del navegador. Cada panel muestra una parte distinta del comportamiento de la aplicación.

Paneles del navegador

Panel	Información disponible	Finalidad
Elementos o Inspector	DOM, clases, atributos y estilos	Identificar elementos y su estado visible
Estilos calculados	Valores aplicados y modelo de caja	Reconocer las declaraciones ganadoras y las dimensiones
Consola	Errores y valores de diagnóstico	Localizar fallos de ejecución
Red o Network	Solicitudes, estados, tiempos y respuestas	Identificar la comunicación con servicios
Vista adaptable	Distintos anchos de viewport	Observar los cambios de distribución

20.16 Errores frecuentes

Elegir un elemento por su aspecto

`h1` no es «texto grande» ni `table` una grilla para ubicar cajas. El elemento se elige por su significado; el aspecto lo decide CSS.

Crear que `disabled="false"` habilita un control

Los atributos booleanos se activan por su sola presencia. Para habilitar hay que quitar el atributo, o asignar `elemento.disabled = false` desde JavaScript.

Leer la especificidad como un número decimal

Las columnas se comparan de izquierda a derecha y no se suman: ninguna cantidad de clases supera a un identificador. Y cuando la especificidad empata, gana la última declaración, no «siempre la última».

Confundir `.tarjeta .destacado` con `.tarjeta.destacado`

El espacio es un combinador: busca un descendiente. Sin espacio, las dos clases tienen que estar en el mismo elemento.

Anular `hidden` con una regla de `display`

Un `display: grid` aplicado al contenedor puede mostrar un elemento que el HTML declaró oculto. Por eso la aplicación le da la grilla a la lista interna y no a la tarjeta.

Mostrar texto externo con `innerHTML`

`innerHTML` interpreta marcado, y un nombre recibido de una API puede contener etiquetas. Para mostrar texto, `textContent`.

Crear que `preventDefault()` detiene la propagación

Cancela la acción predeterminada, como navegar al enviar un formulario. El evento sigue subiendo por los ancestros.

Suponer que `setTimeout(..., 0)` ejecuta enseguida

Primero termina el código actual y después se procesan las microtareas de las promesas. El temporizador llega en una tarea posterior.

Olvidar `respuesta.ok`

`fetch` solo rechaza cuando falla el transporte. Un 404 o un 500 llegan como respuestas normales, y el código tiene que revisarlas.

Esconder una clave en el JavaScript del navegador

Todo lo que se envía al navegador puede leerse. Un secreto necesita un servidor propio que haga de intermediario.

Resolver CORS con `mode: "no-cors"`

No habilita a leer la respuesta: la vuelve opaca. El acceso lo concede la configuración del servicio, no una opción del cliente.

Síntomas y cómo investigarlos

Síntoma	Causa probable	Qué revisar
No aparecen estilos	Ruta incorrecta o archivo sin guardar	href, nombre real y solicitud de styles.css
El formulario recarga después de completar el proyecto	El listener no se registró o hay un error de JS	Carga de app.js, consola y preventDefault
«Cannot read properties of null»	No se encontró un elemento	id, selector y defer
Un resultado continúa oculto	El flujo no llegó al renderizado	Respuesta de red, errores y hidden
Botones que nunca se habilitan	Falta una salida de estado de carga	Bloques finally y temporizador
Ciudad correcta, país incorrecto	Se consultó una coincidencia distinta	Opción seleccionada y coordenadas enviadas
Hora desplazada	Se interpretó un instante con otra zona	Segundos frente a milisegundos e Intl
Failed to fetch	Problema de red, CORS u otra restricción	Consola y panel de red; no asumir una causa única
Identifier has already been declared	Se copiaron bloques dos veces	Declaraciones repetidas o ejemplos mezclados

El diagnóstico relaciona un síntoma observable con su causa en el documento, los estilos, el código o la comunicación de red.

20.17 Para recordar

- HTML expresa contenido y significado; CSS, presentación; JavaScript, respuesta a eventos y cambios. La frontera no es absoluta, pero organiza el trabajo.
- El navegador no muestra el archivo HTML: lo interpreta, construye el DOM y lo dibuja. JavaScript modifica ese árbol, nunca el archivo.

- La cascada decide qué declaración gana según origen, importancia, especificidad y orden; la herencia completa lo que el elemento no define.
- Flexbox distribuye en un eje; Grid, en filas y columnas. Un diseño *mobile first* parte del espacio chico y agrega distribución con media queries.
- Un formulario ya sabe enviar, validar y responder a Enter. JavaScript intercepta `submit` en lugar de reinventar ese comportamiento.
- Una consulta a un servicio distingue tres resultados: fallo de transporte, error HTTP y respuesta válida pero vacía.
- Cada estado de la aplicación —cargando, sin coincidencias, error, resultado— es parte del diseño, no un detalle que se resuelve después.
- Las herramientas del navegador muestran el DOM, los estilos ganadores y las solicitudes de red: todo diagnóstico parte de un síntoma observable.

Capítulo 21. Arrow.js: del contador al inventario

Idea central

Una interfaz reactiva refleja el estado de la aplicación: los eventos cambian los datos y Arrow.js actualiza el HTML que depende de ellos. El programa deja de ocuparse de modificar el DOM en cada paso, como hacía la aplicación del capítulo 20, y pasa a declarar qué parte de la pantalla depende de qué dato.

Vamos a construir un inventario para contar objetos de un local. Cada producto tendrá nombre, cantidad y botones para sumar o restar. Después incorporaremos búsqueda mientras escribimos, un formulario de alta y eliminación.

Trabajaremos siempre sobre **un único archivo `inventario.html`**, con el CSS y el JavaScript de la aplicación incrustados. La única dependencia externa es **Arrow.js 1.0.6**, cargada desde un CDN: hace falta conexión. Los datos viven en memoria; al recargar la página se recuperan los valores iniciales.

21.1 Un contador reactivo

Necesitamos mostrar una cantidad y modificarla al tocar dos botones.

Creá `inventario.html`, copiá este código y ábrilo en el navegador:

```

<!doctype html>
<html lang="es">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Inventario con Arrow.js</title>
<style>
* { box-sizing: border-box; }
body { max-width: 520px; margin: 24px auto; padding: 0 12px; font: 16px system-ui;
background: #f4f6f8; color: #243042; }
button, input { font: inherit; min-height: 44px; border-radius: 8px; }
button { min-width: 44px; border: 0; background: #e4eaf2; color: inherit; cursor:
pointer; touch-action: manipulation; flex-shrink: 0; }
button:disabled { opacity: .4; cursor: default; }
.tarjeta { display: flex; align-items: center; gap: 8px; margin: 10px 0; padding: 12px;
border-radius: 12px; background: white; }
.tarjeta strong { flex: 1; min-width: 0; overflow-wrap: anywhere; }
output { min-width: 2ch; text-align: center; font-variant-numeric: tabular-nums; }
</style>
<body>
  <h1>Inventario</h1>
  <main id="app"></main>

  <script type="module">
import { reactive, html } from 'https://cdn.jsdelivr.net/npm/@arrow-
js/core@1.0.6/dist/index.mjs';

const producto = reactive({ nombre: 'Tornillos', cantidad: 3 });

html`
<article class="tarjeta">
  <strong>${() => producto.nombre}</strong>
  <output>${() => producto.cantidad}</output>
  <button disabled="${() => producto.cantidad === 0}"
    @click="${() => producto.cantidad--}">-</button>
  <button @click="${() => producto.cantidad++}">+</button>
</article>
` (document.getElementById('app'));
</script>
</body>
</html>

```

Si tu entorno no permite abrir módulos desde un archivo local, serví la carpeta con Live Server o un servidor HTTP local.

El estado está en `producto`. `reactive` permite observar sus cambios. Aunque usamos `const`, las propiedades del objeto se pueden modificar: lo que no podemos hacer es reasignar la variable `producto`. Es la distinción de la sección 2.2 entre vínculo constante y valor inmutable.

La plantilla describe los elementos de la tarjeta. La sintaxis `html` seguida de comillas invertidas es una *plantilla etiquetada* de JavaScript, como las de la sección 5.15: `html` recibe las partes del texto y las expresiones `${...}`. En Arrow, el resultado es una plantilla que podemos montar en un elemento. [Plantillas etiquetadas en MDN](#).

La llamada del final la monta dentro de `<main id="app">`:

```
(document.getElementById('app'));
```

Esa línea continúa la expresión `html` anterior; no se ejecuta por separado.

La lectura reactiva aparece dentro de una función:

```
${() => producto.cantidad}
```

Arrow registra la lectura de `producto.cantidad` y actualiza esa parte de la plantilla cuando cambia. En cambio, `${producto.cantidad}` entrega el valor leído en ese momento, sin establecer esa lectura reactiva. [Plantillas de Arrow.js](#).

El evento cambia el dato:

```
@click="${() => producto.cantidad++}"
```

`@click` conecta el botón con una función. La función se ejecuta al hacer clic o tocarlo. El número se actualiza como consecuencia del cambio en el estado.

El atributo `disabled` también depende del estado: cuando la cantidad llega a cero, el botón de restar queda deshabilitado.

21.2 Convertir la tarjeta en un componente

Necesitamos reutilizar la tarjeta con distintos productos. Encapsulamos su plantilla en `Contador`, que recibe un producto.

Conservá el HTML y el CSS. **Reemplazá todo el contenido de** `<script type="module">` por lo siguiente:

```
import { reactive, html, component } from 'https://cdn.jsdelivr.net/npm/@arrow-js/core@1.0.6/dist/index.mjs';

const producto = reactive({ nombre: 'Tornillos', cantidad: 3 });

const Contador = component(producto => html`
<article class="tarjeta">
  <strong>${() => producto.nombre}</strong>
  <output>${() => producto.cantidad}</output>
  <button
    disabled="${() => producto.cantidad === 0}"
    @click="${() => producto.cantidad--}">-</button>
  <button
    @click="${() => producto.cantidad++}">+</button>
</article>
`);

html`${Contador(producto)}$(document.getElementById('app'));
```


`component` convierte la función en un componente de Arrow. La función recibe sus datos y devuelve una plantilla; `Contador(producto)` se incorpora dentro de otra plantilla. Componentes de Arrow.js.

En este ejemplo, el estado vive fuera del componente. La tarjeta recibe ese producto y sus botones modifican su cantidad. No crea una copia ni vuelve a poner el contador en cero.

21.3 Una lista de contadores

Ahora necesitamos contar varios tipos de objetos.

Conservá el `import` y el componente `Contador`. **Reemplazá la declaración de `producto`** por este estado:

```
const estado = reactive({
  productos: [
    { id: 1, nombre: 'Tornillos', cantidad: 3 },
    { id: 2, nombre: 'Tuercas', cantidad: 8 },
    { id: 3, nombre: 'Arandelas', cantidad: 0 }
  ]
});
```

Reemplazá el montaje del final del script por este:

```
html`
  <section>
    ${() => estado.productos.map(p => Contador(p).key(p.id))}
  </section>
`(document.getElementById('app'));
```

`map` recorre el array y produce un componente por producto. Por ejemplo, el objeto de Tornillos se transforma en `Contador(p)`; el de Tuercas, en otro contador.

La función exterior, `${() => ...}`, permite actualizar la lista cuando cambie la colección. Los objetos anidados del estado también son reactivos: cada tarjeta puede observar la cantidad de su producto.

`key(p.id)` identifica a cada producto durante las actualizaciones de la lista. Usamos un identificador estable; su posición puede cambiar cuando filtremos o eliminemos elementos.

21.4 Búsqueda en caliente

Con muchos productos necesitamos encontrar uno mientras escribimos. El texto buscado será otra propiedad del estado.

Agregá `busqueda: ''`, como primera propiedad del objeto que pasás a `reactive`. El estado completo queda así:

```
const estado = reactive({
  busqueda: '',
  productos: [
    { id: 1, nombre: 'Tornillos', cantidad: 3 },
    { id: 2, nombre: 'Tuercas', cantidad: 8 },
    { id: 3, nombre: 'Arandelas', cantidad: 0 }
  ]
});
```

Después de `Contador`, agregá esta función:

```
function visibles() {
  const texto = estado.busqueda.trim().toLowerCase();
  return estado.productos.filter(p => p.nombre.toLowerCase().includes(texto));
}
```

`filter` obtiene los productos cuyo nombre contiene el texto buscado. `trim` quita espacios de los extremos y `toLowerCase` permite buscar sin distinguir mayúsculas. La búsqueda sigue distinguiendo las tildes.

Reemplazá el montaje final por este:

```
html`
<input type="search"
  placeholder="Buscar productos"
  .value="${() => estado.busqueda}"
  @input="${e => estado.busqueda = e.target.value}">
<section>
  ${() => visibles().map(p => Contador(p).key(p.id))}
</section>
<p role="status">${() => visibles().length ? '' : 'No hay productos para mostrar.'}</p>
` (document.getElementById('app'));
```

Agregá estas reglas al final de `<style>`:

```
input { width: 100%; min-width: 0; padding: 10px; border: 1px solid #b8c2ce; }
.barra { display: flex; gap: 8px; }
```

El campo tiene dos conexiones explícitas:

Conexión	Dirección	Efecto
<code>.value="\${() => estado.busqueda}"</code>	Estado → campo	El campo refleja el texto del estado.
<code>@input="\${e => estado.busqueda = e.target.value}"</code>	Campo → estado	Cada edición del campo actualiza el texto buscado.

El punto de `.value` indica una vinculación con la **propiedad** del elemento. Usamos esa propiedad para sincronizar el valor actual del campo, incluso después de que el usuario lo haya editado. Las conexiones de propiedades y eventos están documentadas en la [API de Arrow.js](#).

La lista llama a `visibles()` dentro de una expresión reactiva. Durante esa llamada se leen la búsqueda y los productos; sus cambios actualizan el resultado mostrado.

Filtrar no elimina productos de `estado.productos`. La lista filtrada contiene referencias a los mismos objetos: sus cantidades se conservan cuando una tarjeta desaparece y vuelve a mostrarse. Es el comportamiento de las copias superficiales que describe la sección 9.16.

21.5 Agregar productos desde un diálogo

Necesitamos ingresar un nombre y una cantidad inicial sin abandonar la lista. El botón **Nuevo**, junto al buscador, abrirá un formulario.

Vamos a encapsular los campos y sus acciones en el componente `Formulario`. Su estado `borrador` será local: escribir no modifica el inventario. Al confirmar, el componente entrega los datos mediante la función `onAgregar`.

Después de `estado`, agregá:

```
let siguienteId = 4;
```

Los identificadores 1, 2 y 3 ya pertenecen a los productos iniciales. `siguienteId` asignará los siguientes sin reutilizar los de productos eliminados durante esta sesión.

Antes del montaje de la aplicación, definí el componente:

```

const Formulario = component(acciones => {
  const borrador = reactive({ nombre: '', cantidad: '0' });

  function guardar(evento) {
    evento.preventDefault();
    const nombre = borrador.nombre.trim();
    if (!nombre) return;

    acciones.onAgregar({ nombre, cantidad: Number(borrador.cantidad) });
    cerrar(evento);
  }

  function cerrar(evento) {
    evento.currentTarget.closest('dialog').close();
  }

  return html`
    <form @submit=${guardar}>
      <h2 id="titulo-alta">Nuevo producto</h2>
      <label>Nombre
        <input required autofocus
          .value=${() => borrador.nombre}>
        @input=${e => borrador.nombre = e.target.value}>
      </label>
      <label>Cantidad inicial
        <input type="number" min="0" step="1" required inputmode="numeric"
          .value=${() => borrador.cantidad}>
        @input=${e => borrador.cantidad = e.target.value}>
      </label>
      <div class="barra">
        <button type="button" @click=${cerrar}>Cancelar</button>
        <button class="primario" type="submit"
          disabled=${() => !borrador.nombre.trim()}>Agregar</button>
      </div>
    </form>
  `;
});

```

Formulario recibe un objeto acciones con la función onAgregar. guardar prepara los datos y llama a esa función; cerrar encuentra el diálogo que contiene al formulario y lo cierra. Cancelar solo cierra, sin enviar datos. El componente no accede a estado.productos ni asigna identificadores.

Después del componente, agregá estas funciones de la aplicación:

```

function abrir() {
  dialogo.replaceChildren();
  html`${Formulario({ onAgregar: agregar })}`(dialogo);
  dialogo.showModal();
}

function agregar(datos) {
  estado.productos.push({ id: siguienteId++, ...datos });
  estado.búsqueda = '';
}

```

abrir vacía el diálogo y monta una nueva instancia de Formulario, con nombre vacío y cantidad cero. Le pasa agregar como onAgregar y luego abre el diálogo. agregar recibe los datos, asigna el identificador, incorpora el producto con push y vacía la búsqueda. El componente se encarga de cerrar el diálogo.

Los valores leídos mediante e.target.value son texto. Por eso el borrador empieza con '0' y Number convierte la cantidad antes de guardarla. La sección 7.4 detalla esa conversión.

Reemplazá el montaje final por este bloque completo, incluida la declaración de dialogo del final:

```
html`
<div class="barra">
  <input type="search"
    placeholder="Buscar productos"
    .value="${() => estado.búsqueda}"
    @input="${e => estado.búsqueda = e.target.value}">
  <button class="primario" @click="${abrir}">Nuevo</button>
</div>
<section>
  ${() => visibles().map(p => Contador(p).key(p.id))}
</section>
<p role="status">${() => visibles().length ? '' : 'No hay productos para mostrar.'}</p>
<dialog></dialog>
` (document.getElementById('app'));

const dialogo = document.querySelector('dialog');
```

dialogo se busca después del montaje, cuando el elemento ya existe. Los eventos que usan esa variable se ejecutan más tarde, al interactuar con la aplicación.

showModal() abre el diálogo sobre la página y bloquea la interacción con el fondo. close() lo cierra; el navegador también admite Escape. autofocus señala el campo inicial. El elemento dialog en MDN.

El formulario usa el mismo binding que el buscador. .value muestra el borrador local y @input lo actualiza. Como el borrador se crea dentro de component, cada nueva instancia tiene su propio estado.

La cantidad es obligatoria, entera y no negativa mediante required, step="1" y min="0". Al intentar enviar el formulario, el navegador comprueba esas restricciones. El botón Agregar permanece deshabilitado si el nombre está vacío o solo tiene espacios.

Agregá estas reglas al CSS:

```
.primario { background: #245ac2; color: white; padding: 0 14px; }
dialog { width: min(92vw, 380px); border: 0; border-radius: 14px; padding: 20px; }
dialog::backdrop { background: #0006; }
label { display: grid; gap: 6px; margin: 12px 0; }
```

21.6 Eliminar un producto

Nos falta quitar los productos que cargamos por error. Cada tarjeta tendrá una `×` al comienzo.

Antes del montaje, agregá esta función:

```
function eliminar(id) {  
  estado.productos = estado.productos.filter(p => p.id !== id);  
}
```

`filter` produce un array con todos los productos excepto el que tiene ese identificador. Al asignarlo a `estado.productos`, la lista se actualiza.

Dentro de la plantilla de `Contador`, inmediatamente después de `<article class="tarjeta">`, agregá:

```
<button class="eliminar"  
  @click="{() => eliminar(producto.id)}">×</button>
```

Agregá esta regla al CSS:

```
.eliminar { background: #fde9e7; color: #9a2823; }
```

El botón envía el identificador del producto. Dos productos pueden tener el mismo nombre y seguir siendo elementos diferentes.

21.7 Código final completo

Este es el contenido final de `inventario.html`:

```

<!doctype html>
<html lang="es">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Inventario con Arrow.js</title>
<style>
* { box-sizing: border-box; }
body { max-width: 520px; margin: 24px auto; padding: 0 12px; font: 16px system-ui;
background: #f4f6f8; color: #243042; }
button, input { font: inherit; min-height: 44px; border-radius: 8px; }
button { min-width: 44px; border: 0; background: #e4eaf2; color: inherit; cursor:
pointer; touch-action: manipulation; flex-shrink: 0; }
button:disabled { opacity: .4; cursor: default; }
.tarjeta { display: flex; align-items: center; gap: 8px; margin: 10px 0; padding: 12px;
border-radius: 12px; background: white; }
.tarjeta strong { flex: 1; min-width: 0; overflow-wrap: anywhere; }
output { min-width: 2ch; text-align: center; font-variant-numeric: tabular-nums; }
input { width: 100%; min-width: 0; padding: 10px; border: 1px solid #b8c2ce; }
.barra { display: flex; gap: 8px; }
.primario { background: #245ac2; color: white; padding: 0 14px; }
dialog { width: min(92vw, 380px); border: 0; border-radius: 14px; padding: 20px; }
dialog::backdrop { background: #0006; }
label { display: grid; gap: 6px; margin: 12px 0; }
.eliminar { background: #fde9e7; color: #9a2823; }
</style>
<body>
  <h1>Inventario</h1>
  <main id="app"></main>

<script type="module">
import { reactive, html, component } from 'https://cdn.jsdelivr.net/npm/@arrow-
js/core@1.0.6/dist/index.mjs';

const estado = reactive({
  busqueda: '',
  productos: [
    { id: 1, nombre: 'Tornillos', cantidad: 3 },
    { id: 2, nombre: 'Tuercas', cantidad: 8 },
    { id: 3, nombre: 'Arandelas', cantidad: 0 }
  ]
});

let siguienteId = 4;

const Contador = component(producto => html`
<article class="tarjeta">
  <button class="eliminar"
    @click="${() => eliminar(producto.id)}">x</button>
  <strong>${() => producto.nombre}</strong>
  <output>${() => producto.cantidad}</output>
  <button disabled="${() => producto.cantidad === 0}"
    @click="${() => producto.cantidad--}">-</button>
  <button
    @click="${() => producto.cantidad++}">+</button>
</article>
`);

const Formulario = component(acciones => {

```

```

const borrador = reactive({ nombre: '', cantidad: '0' });

function guardar(evento) {
  evento.preventDefault();
  const nombre = borrador.nombre.trim();
  if (!nombre) return;

  acciones.onAgregar({ nombre, cantidad: Number(borrador.cantidad) });
  cerrar(evento);
}

function cerrar(evento) {
  evento.currentTarget.closest('dialog').close();
}

return html`
<form @submit=${guardar}>
  <h2 id="titulo-alta">Nuevo producto</h2>
  <label>Nombre
    <input required autofocus
      .value=${() => borrador.nombre}
      @input=${e => borrador.nombre = e.target.value}>
  </label>
  <label>Cantidad inicial
    <input type="number" min="0" step="1" required inputmode="numeric"
      .value=${() => borrador.cantidad}
      @input=${e => borrador.cantidad = e.target.value}>
  </label>
  <div class="barra">
    <button type="button" @click=${cerrar}>Cancelar</button>
    <button class="primario" type="submit"
      disabled=${() => !borrador.nombre.trim()}>Agregar</button>
  </div>
</form>
`;
});

function visibles() {
  const texto = estado.búsqueda.trim().toLowerCase();
  return estado.productos.filter(p => p.nombre.toLowerCase().includes(texto));
}

function abrir() {
  dialogo.replaceChildren();
  html`${Formulario({ onAgregar: agregar })}`(dialogo);
  dialogo.showModal();
}

function agregar(datos) {
  estado.productos.push({ id: siguienteId++, ...datos });
  estado.búsqueda = '';
}

function eliminar(id) {
  estado.productos = estado.productos.filter(p => p.id !== id);
}

html`
<div class="barra">

```



```

<input type="search"
  placeholder="Buscar productos"
  .value="${() => estado.búsqueda}"
  @input="${e => estado.búsqueda = e.target.value}">
<button class="primario" @click="${abrir}">Nuevo</button>
</div>
<section>
  ${() => visibles().map(p => Contador(p).key(p.id))}
</section>
<p role="status">${() => visibles().length ? '' : 'No hay productos para mostrar.'}</p>
<dialog></dialog>
\ (document.getElementById('app'));

const dialogo = document.querySelector('dialog');
</script>
</body>
</html>

```

21.8 Patrones para reutilizar

Necesidad	Patrón del inventario
Guardar datos que modifican la pantalla	reactive({ ... })
Mostrar un dato que cambia	\${() => producto.cantidad}
Responder a un evento	@click="\${funcion}"
Vincular un campo con el estado	.value y @input
Reutilizar una parte de la interfaz	component(funcionQueDevuelveUnaPlantilla)
Encapsular una edición pendiente	borrador local dentro de Formulario
Comunicar datos a la aplicación	acciones.onAgregar(datos)
Mostrar una colección	map(p => Contador(p).key(p.id))
Obtener una vista filtrada	filter dentro de una lectura reactiva
Agregar un elemento	estado.productos.push(nuevoProducto)
Quitar un elemento	Reasignar el array con filter

21.9 Errores frecuentes

Leer el estado sin envolverlo en una función

`${producto.cantidad}` entrega el valor de ese momento y no vuelve a leerlo. Para que la plantilla se actualice, la lectura va dentro de una función: `${() => producto.cantidad}`.

Ejecutar el manejador en lugar de pasarlo

`@click="{eliminar(producto.id)}"` llama a `eliminar` mientras se arma la plantilla. El evento necesita una función que se ejecute después: `@click="{() => eliminar(producto.id)}"`.

Omitir la clave de los elementos de una lista

Sin `.key(p.id)`, Arrow no puede saber qué tarjeta corresponde a qué producto cuando la lista se filtra o se reordena. La clave tiene que ser estable: la posición no sirve, porque cambia.

Guardar el texto de un campo como si fuera un número

`e.target.value` siempre es un string, incluso en un `input type="number"`. La conversión con `Number` se hace al confirmar, no en cada tecla.

Filtrar para borrar

`visibles()` no modifica la colección: devuelve una vista. Eliminar de verdad exige reasignar `estado.productos` con el resultado de `filter`.

Buscar un elemento antes de montarlo

`document.querySelector("dialog")` devuelve `null` si se ejecuta antes de que la plantilla que contiene el diálogo esté montada.

Abrir el archivo con doble clic y esperar que los módulos carguen

Algunos navegadores restringen los módulos cuando la página se abre como archivo local. Un servidor HTTP local resuelve la diferencia.

21.10 Para recordar

- `reactive` crea un estado observable; cada lectura dentro de una función de la plantilla queda registrada y se vuelve a calcular cuando ese dato cambia.
- La plantilla `html` es una plantilla etiquetada: describe la interfaz y se monta en un elemento.
- `@evento` conecta un manejador; `.propiedad` vincula una propiedad del elemento; un atributo con función, como `disabled`, se recalcula solo.
- `component` convierte una función que devuelve una plantilla en una pieza reutilizable, que recibe sus datos y comunica acciones mediante funciones.
- Una lista reactiva es un `map` dentro de una lectura reactiva, con una clave estable por elemento.
- El estado de un borrador vive dentro del componente que lo edita; la aplicación solo recibe el resultado confirmado.

- Filtrar produce una vista; agregar y quitar modifican la colección, y la pantalla sigue a los datos.

Capítulo 22. Cómo consumir una API REST

Idea central

Consumir una API es sostener una conversación de pedido y respuesta sobre HTTP: el cliente arma un mensaje con método, URL, cabeceras y cuerpo, y el servidor contesta con un código de estado, cabeceras y datos, casi siempre en JSON. Quien entiende la forma de esos mensajes puede probar cualquier API a mano, leer lo que devuelve y escribir un cliente que no confunda un error con un resultado.

Una aplicación web casi nunca trabaja sola. Pide datos a otros programas y les envía datos: el clima, los usuarios, los pagos, un modelo de IA. Esa conversación entre programas ocurre a través de una API, y en la web casi siempre viaja sobre HTTP. El capítulo 20 ya la usó para consultar el tiempo; este la desarma pieza por pieza.

22.1 El protocolo HTTP

HTTP (HyperText Transfer Protocol) es un acuerdo sobre cómo se hablan dos programas. Uno pregunta y el otro responde. El que pregunta es el cliente. El que responde es el servidor.

El navegador es un cliente HTTP. También lo son `curl`, la extensión REST Client y la función `fetch` de JavaScript. Todos hablan el mismo idioma, así que el servidor no sabe ni le importa quién le pregunta.

Pedido y respuesta

Toda conversación HTTP tiene la misma forma:

1. El cliente envía un pedido (request).
2. El servidor lo procesa.
3. El servidor devuelve una respuesta (response).
4. La conversación termina.

No hay otra forma de iniciar una charla. El servidor nunca le habla al cliente por su cuenta: solo responde.

HTTP no tiene memoria

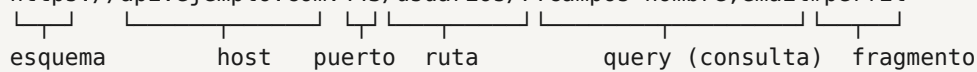
HTTP es un protocolo sin estado (stateless). Cada pedido es independiente. El servidor no recuerda el pedido anterior.

Si el servidor necesita saber quién sos, cada pedido tiene que decirlo. Por eso existen las cookies y los tokens de autenticación: son datos que el cliente vuelve a enviar en cada pedido.

La dirección: la URL

Cada pedido apunta a una URL. La URL dice qué recurso querés y dónde está.

```
https://api.ejemplo.com:443/usuarios/7?campos=nombre,email#perfil
```



esquema host puerto ruta query (consulta) fragmento

Cada parte cumple una función:

- esquema: el protocolo; `https` es HTTP cifrado
- host: el nombre del servidor
- puerto: la “puerta” del servidor; si no se escribe, vale 80 para `http` y 443 para `https`
- ruta: qué recurso pedís dentro del servidor
- query: parámetros extra en forma `clave=valor`, separados por `&`
- fragmento: una marca para el navegador; nunca viaja al servidor

Cómo es un pedido por dentro

Un mensaje HTTP es texto con una estructura fija. Este es un pedido real, tal como viaja:

```
GET /users/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
Accept: application/json
User-Agent: curl/8.5.0
```

Tiene cuatro partes:

1. La línea de pedido: método (`GET`), ruta (`/users/1`) y versión del protocolo (`HTTP/1.1`).
2. Las cabeceras (headers): una por línea, en forma `Nombre: valor`.
3. Una línea vacía que marca el fin de las cabeceras.
4. El cuerpo (body), opcional. Un `GET` no lleva cuerpo.

Un pedido que envía datos sí lleva cuerpo:

```
POST /users HTTP/1.1
Host: jsonplaceholder.typicode.com
Content-Type: application/json
Content-Length: 47

{"name": "Ana Pérez", "email": "ana@mail.com"}
```

La cabecera `Content-Type` le avisa al servidor en qué formato viene el cuerpo. Sin ella, el servidor no sabe cómo leerlo.

Cómo es una respuesta por dentro

La respuesta tiene la misma estructura. Solo cambia la primera línea:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 509
Cache-Control: max-age=43200

{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz"
}
```

La línea de estado tiene la versión, el código de estado (200) y un texto descriptivo (OK). El código es para el programa; el texto es para las personas.

Cabeceras frecuentes

Las cabeceras son metadatos: información sobre el mensaje, no el mensaje en sí.

Cabecera	Va en	Para qué sirve
Host	pedido	a qué servidor va el pedido; es obligatoria
Accept	pedido	qué formato de respuesta acepta el cliente
Content-Type	ambos	en qué formato está el cuerpo
Content-Length	ambos	cuántos bytes tiene el cuerpo
Authorization	pedido	credenciales, por ejemplo Bearer <token>
User-Agent	pedido	qué programa hace el pedido
Location	respuesta	la URL del recurso recién creado o de una redirección
Cache-Control	respuesta	cuánto tiempo se puede reutilizar la respuesta

Los nombres de cabecera no distinguen mayúsculas de minúsculas. `content-type` y `Content-Type` son la misma cabecera.

Versiones de HTTP

HTTP/1.1 manda los mensajes como texto, uno tras otro. HTTP/2 y HTTP/3 los empaquetan en binario y envían varios a la vez por la misma conexión. Lo que cambia es el transporte. Los métodos, las cabeceras y los códigos de estado son los mismos, así que todo lo de este capítulo vale para las tres versiones.

22.2 Qué es una API REST

Una API (Application Programming Interface) es la forma en que un programa ofrece sus servicios a otros programas. Una API web los ofrece a través de HTTP.

REST (Representational State Transfer) es un estilo para diseñar esas APIs. No es un protocolo ni una librería. Es un conjunto de convenciones que hace que todas las APIs se parezcan, así que si aprendés a usar una, ya sabés usar casi todas.

Las ideas de REST

REST se apoya en cinco ideas:

1. Todo es un recurso. Un usuario, un producto o un pedido de compra son recursos.
2. Cada recurso tiene una URL. La URL es un sustantivo: `/users/1`, no `/obtenerUsuario?id=1`.
3. El método HTTP es el verbo. `GET /users/1` lee el usuario; `DELETE /users/1` lo borra.
4. Lo que viaja es una representación del recurso, casi siempre en JSON.
5. Cada pedido se entiende solo, sin depender de pedidos anteriores.

Colecciones y elementos

Las URLs de una API REST forman una jerarquía fácil de adivinar:

URL	Qué representa
<code>/users</code>	la colección de todos los usuarios
<code>/users/1</code>	el usuario con id 1
<code>/users/1/posts</code>	los posts del usuario 1
<code>/posts?userId=1</code>	los posts filtrados por usuario, con query

Las colecciones van en plural. Los filtros, el orden y la paginación van en la query.

Las acciones REST

REST usa los métodos de HTTP como acciones sobre los recursos. Las cuatro operaciones básicas de datos (crear, leer, actualizar, borrar, conocidas como CRUD) se traducen así:

Acción	Método	URL	Cuerpo del pedido	Respuesta típica
listar	GET	/users	no	200 con un arreglo
leer uno	GET	/users/1	no	200 con el objeto
crear	POST	/users	el nuevo recurso	201 con el recurso creado
reemplazar	PUT	/users/1	el recurso completo	200 con el recurso
modificar	PATCH	/users/1	solo los campos que cambian	200 con el recurso
borrar	DELETE	/users/1	no	200 o 204 sin cuerpo

La diferencia entre `PUT` y `PATCH` importa. `PUT` reemplaza el recurso entero: si omitís un campo, el servidor lo pierde. `PATCH` cambia solo lo que enviás.

Seguro e idempotente

Dos propiedades ayudan a razonar sobre cada método:

- seguro: el método no cambia nada en el servidor; solo lee
- idempotente: repetir el pedido deja el servidor igual que hacerlo una vez

Método	Seguro	Idempotente
GET	sí	sí
PUT	no	sí
PATCH	no	no necesariamente
DELETE	no	sí
POST	no	no

Esto tiene consecuencias prácticas. Si un `GET` falla por la red, podés reintentarlo sin miedo. Si un `POST` falla, reintentarlo puede crear el recurso dos veces. Por eso los botones de “Pagar” se desactivan después del primer clic.

22.3 Los códigos de estado

El código de estado es un número de 3 cifras. Resume qué pasó con el pedido. El primer dígito indica la familia:

Familia	Significado	Quién tiene que actuar
1xx	información provisoria	nadie, es raro verlos
2xx	éxito	nadie, salió bien
3xx	redirección	el cliente, que debe ir a otra URL
4xx	error del cliente	el cliente, que pidió algo mal
5xx	error del servidor	el servidor, que falló

Los que vas a ver todos los días:

Código	Texto	Cuándo aparece
200	OK	el pedido salió bien y hay cuerpo
201	Created	se creó un recurso, típico de POST
204	No Content	salió bien, pero no hay cuerpo, típico de DELETE
301	Moved Permanently	el recurso se mudó para siempre a otra URL
304	Not Modified	tu copia en caché sigue siendo válida
400	Bad Request	el pedido está mal formado, por ejemplo JSON inválido
401	Unauthorized	falta autenticarse o el token no sirve
403	Forbidden	estás autenticado, pero no tenés permiso
404	Not Found	el recurso no existe
409	Conflict	el pedido choca con el estado actual, por ejemplo email repetido
422	Unprocessable Content	el JSON es válido, pero los datos no pasan la validación
429	Too Many Requests	hiciste demasiados pedidos; esperá
500	Internal Server Error	el servidor falló por un error propio
503	Service Unavailable	el servidor está caído o sobrecargado

Una regla para leerlos: si es `4xx`, revisá tu pedido; si es `5xx`, el problema no es tuyo.

22.4 JSON como formato de intercambio

Cliente y servidor pueden estar escritos en lenguajes distintos. Necesitan un formato de texto que los dos entiendan. Ese formato es JSON (JavaScript Object Notation).

JSON nació de la sintaxis de los objetos de JavaScript, pero hoy lo lee y escribe cualquier lenguaje.

```
{
  "id": 1,
  "name": "Leanne Graham",
  "activo": true,
  "saldo": 1520.5,
  "telefono": null,
  "roles": ["admin", "editor"],
  "address": {
    "city": "Gwenborough",
    "zipcode": "92998-3874"
  }
}
```

Los tipos de JSON

JSON tiene solo seis tipos de valor:

- cadena: "texto", siempre con comillas dobles
- número: 42, -3.14, 1e6
- booleano: true o false
- nulo: null
- arreglo: [1, 2, 3]
- objeto: {"clave": "valor"}

Diferencias con un objeto de JavaScript

JSON se parece a JavaScript, pero es más estricto:

- las claves van siempre entre comillas dobles
- las cadenas no aceptan comillas simples ni invertidas
- no se permite una coma después del último elemento
- no hay comentarios
- no existen undefined, funciones, NaN ni Infinity
- no hay tipo fecha; las fechas viajan como cadenas, por ejemplo "2026-09-23T14:30:00Z"

La sección 10.18 enumera lo que un objeto pierde al pasar por JSON, y la 19.30 muestra cómo validar un JSON leído de un archivo.

Convertir entre texto y objeto

Por la red solo viaja texto. JavaScript tiene dos funciones para pasar de uno a otro:

```

const usuario = { name: "Ana", edad: 30 };

// Objeto → texto JSON (para enviar)
const texto = JSON.stringify(usuario);
console.log(texto); // {"name":"Ana","edad":30}

// Con sangría, para leerlo mejor
console.log(JSON.stringify(usuario, null, 2));

// Texto JSON → objeto (al recibir)
const deVuelta = JSON.parse(texto);
console.log(deVuelta.name); // Ana

// Un JSON inválido lanza una excepción
try {
  JSON.parse("{name: 'Ana'}");
} catch (e) {
  console.log("JSON inválido:", e.message);
}

```

A convertir un objeto en texto se lo llama serializar. Al camino inverso, deserializar.

22.5 La API de práctica: JSONPlaceholder

JSONPlaceholder es una API REST falsa y gratuita pensada para practicar. Está en <https://jsonplaceholder.typicode.com> y no pide registro.

Ofrece varios recursos: `users`, `posts`, `comments`, `todos`, `albums` y `photos`. Vamos a trabajar con `users`.

Un usuario tiene esta forma:

```

{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": { "lat": "-37.3159", "lng": "81.1496" }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}

```

Hay un detalle importante. JSONPlaceholder acepta `POST`, `PUT`, `PATCH` y `DELETE`, y responde como si hubiera funcionado. Pero no guarda nada. Si creás un usuario y después lo pedís, no existe. Es perfecto para practicar sin romper nada.

22.6 Probar la API con REST Client en VS Code

Antes de escribir código conviene probar la API a mano. Así ves exactamente qué responde y separás los problemas de la API de los problemas de tu programa.

REST Client es una extensión de VS Code que envía pedidos HTTP escritos en un archivo de texto. La ventaja sobre otras herramientas es que el archivo queda en el proyecto, versionado con Git, y sirve como documentación.

Instalación

1. Abrí la vista de extensiones con `Ctrl+Shift+X`.
2. Buscá “REST Client”, del autor Huachao Mao.
3. Instalala.
4. Creá un archivo con extensión `.http`, por ejemplo `usuarios.http`.

El primer pedido

Escribí esto en el archivo:

```
GET https://jsonplaceholder.typicode.com/users/1
```

Encima de la línea aparece un enlace **Send Request**. Al usarlo, se abre un panel con la respuesta completa: línea de estado, cabeceras y cuerpo. También podés usar `Ctrl+Alt+R`.

Fijate que el archivo tiene la misma forma que el mensaje HTTP que vimos en la sección 22.1. REST Client no inventa una sintaxis: escribís HTTP.

Un archivo completo con las acciones REST

Cada pedido se separa del siguiente con `###`. Las variables se declaran con `@` y se usan con `{{ }}`.

```

@base = https://jsonplaceholder.typicode.com
@json = application/json

### Listar todos los usuarios
GET {{base}}/users
Accept: {{json}}

### Leer un usuario
GET {{base}}/users/1
Accept: {{json}}

### Usuario que no existe (esperamos 404)
GET {{base}}/users/999

### Filtrar con query: los posts del usuario 1
GET {{base}}/posts?userId=1

### Crear un usuario
POST {{base}}/users
Content-Type: {{json}}

{
  "name": "Ana Pérez",
  "username": "anap",
  "email": "ana@mail.com"
}

### Reemplazar un usuario completo
PUT {{base}}/users/1
Content-Type: {{json}}

{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "nuevo@mail.com"
}

### Modificar solo el email
PATCH {{base}}/users/1
Content-Type: {{json}}

{
  "email": "otro@mail.com"
}

### Borrar un usuario
DELETE {{base}}/users/1

```

Tres reglas de sintaxis que evitan la mayoría de los errores:

- las cabeceras van justo debajo de la línea del pedido, sin líneas vacías en el medio
- entre las cabeceras y el cuerpo va exactamente una línea vacía
- el cuerpo JSON tiene que ser JSON válido, con comillas dobles

Qué observar en cada respuesta

Probá cada pedido y mirá:

- el GET a `/users` devuelve `200` y un arreglo de 10 usuarios
- el GET a `/users/999` devuelve `404` y un objeto vacío `{}`
- el POST devuelve `201 Created` y el usuario con un `id` nuevo: 11
- el DELETE devuelve `200` y un objeto vacío
- todas las respuestas traen `Content-Type: application/json; charset=utf-8`

Encadenar pedidos

REST Client permite nombrar un pedido y usar su respuesta en el siguiente. Esto sirve, por ejemplo, para crear un recurso y después consultarlo con el `id` que devolvió el servidor.

```
# @name nuevo
POST {{base}}/users
Content-Type: {{json}}

{ "name": "Ana Pérez" }

### Usar el id que devolvió el POST
GET {{base}}/users/{{nuevo.response.body.id}}
```

Con JSONPlaceholder este segundo pedido devuelve `404`, porque el usuario 11 nunca se guardó. Con una API real, devolvería el usuario recién creado.

22.7 Lo mismo desde la terminal con cURL

cURL es un cliente HTTP de línea de comandos. Viene instalado en Linux, macOS y Windows 10 o posterior. Es la forma universal de compartir un pedido HTTP: casi toda documentación de APIs trae ejemplos con `curl`.

En Windows, usá `curl.exe` en PowerShell. El nombre `curl` solo puede apuntar a otro comando de PowerShell.

Las opciones que vas a usar

Opción	Qué hace
-X MÉTODO	elige el método; si no se indica, es GET
-H "Nombre: valor"	agrega una cabecera
-d 'datos'	envía un cuerpo; si no indicás método, pasa a ser POST
-i	muestra las cabeceras de la respuesta junto al cuerpo
-I	pide solo las cabeceras (método HEAD)
-v	muestra todo: pedido y respuesta completos
-s	modo silencioso, sin barra de progreso
-o archivo	guarda el cuerpo en un archivo
-w "%{http_code}"	imprime datos del pedido, como el código de estado

Las acciones REST con cURL

```
# Listar usuarios
curl https://jsonplaceholder.typicode.com/users

# Leer uno, mostrando las cabeceras de la respuesta
curl -i https://jsonplaceholder.typicode.com/users/1

# Ver el diálogo completo: lo que envía (>) y lo que recibe (<)
curl -v https://jsonplaceholder.typicode.com/users/1

# Crear
curl -X POST https://jsonplaceholder.typicode.com/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Ana Pérez", "email": "ana@mail.com"}'

# Reemplazar
curl -X PUT https://jsonplaceholder.typicode.com/users/1 \
  -H "Content-Type: application/json" \
  -d '{"id": 1, "name": "Leanne Graham", "email": "nuevo@mail.com"}'

# Modificar un campo
curl -X PATCH https://jsonplaceholder.typicode.com/users/1 \
  -H "Content-Type: application/json" \
  -d '{"email": "otro@mail.com"}'

# Borrar, mostrando solo el código de estado
curl -s -o /dev/null -w "%{http_code}\n" \
  -X DELETE https://jsonplaceholder.typicode.com/users/1
```


La barra invertida `\` al final de la línea continúa el comando en la línea siguiente. En PowerShell se usa el acento grave ``` en su lugar.

Leer el modo detallado

La salida de `curl -v` muestra el mensaje HTTP tal como lo vimos en la sección 22.1:

```
> GET /users/1 HTTP/2
> Host: jsonplaceholder.typicode.com
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/2 200
< content-type: application/json; charset=utf-8
< cache-control: max-age=43200
<
{
  "id": 1,
  "name": "Leanne Graham",
  ...
```

Las líneas con `>` son lo que envió cURL. Las líneas con `<` son lo que respondió el servidor. Es la mejor forma de ver HTTP “desnudo”.

Formatear el JSON con jq

cURL muestra el cuerpo tal cual llega. Para leerlo mejor o extraer datos se usa `jq`, un procesador de JSON para la terminal:

```
# Formatear con colores
curl -s https://jsonplaceholder.typicode.com/users/1 | jq

# Solo el nombre
curl -s https://jsonplaceholder.typicode.com/users/1 | jq '.name'

# Nombre y email de todos los usuarios
curl -s https://jsonplaceholder.typicode.com/users | jq '.[ ] | {name, email}'
```

22.8 Consumir la API desde JavaScript con fetch

`fetch` es la función estándar de JavaScript para hacer pedidos HTTP. Existe en el navegador y en Node.js desde la versión 18, sin instalar nada.

`fetch` devuelve una promesa. Por eso la combinamos con `async/await`. El capítulo 17 explica ese modelo.

El pedido más simple

```
// usuarios.mjs – ejecutar con: node usuarios.mjs
const respuesta = await fetch("https://jsonplaceholder.typicode.com/users/1");
const usuario = await respuesta.json();

console.log(usuario.name); // Leanne Graham
```

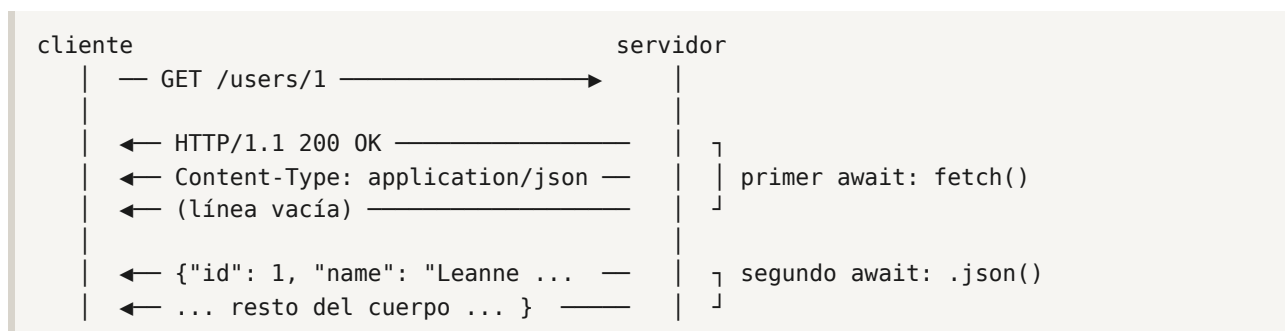
El archivo usa la extensión `.mjs` para que Node lo trate como módulo y permita `await` fuera de una función.

Hay dos `await`, y no es casualidad. Esa es la idea central de esta sección.

Primero llegan las cabeceras, después el cuerpo

Una respuesta HTTP llega en dos partes: primero la línea de estado y las cabeceras, después el cuerpo. El cuerpo puede ser grande y tardar en llegar. `fetch` refleja esa realidad en dos pasos:

1. `await fetch(url)` se resuelve apenas llegan las cabeceras. Te da un objeto `Response` con el estado y las cabeceras, pero el cuerpo todavía se está descargando.
2. `await respuesta.json()` espera a que llegue el cuerpo completo, lo lee como texto y lo convierte en objeto con `JSON.parse`.



Esto permite decidir qué hacer antes de descargar el cuerpo. Por ejemplo, si el estado es un error o si el formato no es el esperado.

Leer el estado y las cabeceras

El objeto `Response` tiene todo lo que llegó antes del cuerpo:

```
const respuesta = await fetch("https://jsonplaceholder.typicode.com/users/1");

// La línea de estado
console.log(respuesta.status); // 200
console.log(respuesta.statusText); // OK (en HTTP/2 puede venir vacío)
console.log(respuesta.ok); // true si el código está entre 200 y 299

// Una cabecera puntual; el nombre no distingue mayúsculas
console.log(respuesta.headers.get("content-type"));
// application/json; charset=utf-8

// Todas las cabeceras
for (const [nombre, valor] of respuesta.headers) {
  console.log(`${nombre}: ${valor}`);
}

// Recién ahora leemos el cuerpo
const usuario = await respuesta.json();
console.log(usuario);
```

El cuerpo se lee una sola vez

El cuerpo es un flujo de datos (stream). Una vez consumido, no se puede volver a leer.

```
const respuesta = await fetch("https://jsonplaceholder.typicode.com/users/1");

const datos = await respuesta.json();
console.log(respuesta.bodyUsed); // true

await respuesta.text(); // TypeError: el cuerpo ya fue leído
```

`Response` ofrece varias formas de leer el cuerpo. Elegís una según el `Content-Type`:

- `json()`: parsea JSON y devuelve un objeto
- `text()`: devuelve el cuerpo como cadena
- `blob()`: devuelve datos binarios, por ejemplo una imagen
- `arrayBuffer()`: devuelve los bytes crudos

fetch no falla con un 404

Esta es la trampa más común. `fetch` solo rechaza la promesa cuando no pudo hablar con el servidor: sin red, dominio inexistente, conexión cortada. Si el servidor responde 404 o 500, para `fetch` la conversación salió bien. Te toca a vos revisar el estado. La sección 13.14 muestra cómo traducir ese estado a un error del dominio.

```
const respuesta = await fetch("https://jsonplaceholder.typicode.com/users/999");

console.log(respuesta.ok); // false
console.log(respuesta.status); // 404
// No se lanzó ninguna excepción
```

Por eso conviene una función que revise el estado y el formato antes de leer el cuerpo:

```
async function pedirJSON(url, opciones = {}) {
  const respuesta = await fetch(url, opciones);

  // 1. ¿Salió bien?
  if (!respuesta.ok) {
    throw new Error(`HTTP ${respuesta.status} en ${url}`);
  }

  // 2. ¿Hay cuerpo? Un 204 no trae nada
  if (respuesta.status === 204) return null;

  // 3. ¿Es JSON?
  const tipo = respuesta.headers.get("content-type") ?? "";
  if (!tipo.includes("application/json")) {
    throw new Error(`Esperaba JSON y llegó ${tipo}`);
  }

  // 4. Recién ahora leemos el cuerpo
  return respuesta.json();
}

try {
  const usuario = await pedirJSON("https://jsonplaceholder.typicode.com/users/999");
  console.log(usuario);
} catch (error) {
  console.error("No se pudo obtener el usuario:", error.message);
}
```

Enviar datos: el segundo parámetro de fetch

Para cualquier método que no sea GET, `fetch` recibe un objeto de opciones. Ahí se indican el método, las cabeceras y el cuerpo.

```
const base = "https://jsonplaceholder.typicode.com";

// Crear
const creado = await pedirJSON(`${base}/users`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Ana Pérez", email: "ana@mail.com" }),
});
console.log(creado); // { name: 'Ana Pérez', email: 'ana@mail.com', id: 11 }

// Modificar
const modificado = await pedirJSON(`${base}/users/1`, {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email: "otro@mail.com" }),
});
console.log(modificado.email); // otro@mail.com

// Borrar
const borrado = await fetch(`${base}/users/1`, { method: "DELETE" });
console.log(borrado.status); // 200
```

Dos detalles:

- `body` recibe texto, no un objeto; por eso pasamos el objeto por `JSON.stringify`
- la cabecera `Content-Type` le avisa al servidor que ese texto es JSON

Armar URLs con parámetros

Concatenar la query a mano es frágil: un espacio o una tilde rompen la URL. La clase `URL` se encarga de codificar los valores:

```
const url = new URL("https://jsonplaceholder.typicode.com/posts");
url.searchParams.set("userId", 1);
url.searchParams.set("_limit", 3);

console.log(url.toString());
// https://jsonplaceholder.typicode.com/posts?userId=1&_limit=3

const posts = await pedirJSON(url);
```

Varios pedidos a la vez

Si los pedidos no dependen entre sí, no hace falta esperarlos de a uno. `Promise.all` los lanza juntos y espera a que terminen todos, como explica la sección 17.6:

```
const [usuario, posts] = await Promise.all([
  pedirJSON(`${base}/users/1`),
  pedirJSON(`${base}/posts?userId=1`),
]);

console.log(`${usuario.name} escribió ${posts.length} posts`);
```

22.9 Ejemplo integrador: el clima en la terminal con Ink

Vamos a juntar todo en una app de terminal. El usuario escribe una ciudad y la app muestra el clima actual.

Las APIs de Open-Meteo

Open-Meteo es un servicio de clima gratuito y sin clave de acceso. Es el mismo que consulta la aplicación del capítulo 20, que además detalla sus condiciones de uso; acá lo usamos desde la terminal. Su documentación está en la página de la [API de pronóstico de Open-Meteo](#). Vamos a usar dos de sus APIs, una detrás de la otra.

La primera convierte un nombre de ciudad en coordenadas. Es la API de geocodificación:

```
GET https://geocoding-api.open-meteo.com/v1/search?
name=Tucumán&count=1&language=es&format=json
```

Devuelve algo así (recortado):

```
{
  "results": [
    {
      "name": "San Miguel de Tucumán",
      "latitude": -26.82414,
      "longitude": -65.2226,
      "country": "Argentina",
      "admin1": "Tucumán"
    }
  ]
}
```

Si no encuentra la ciudad, la respuesta no trae la propiedad `results`.

La segunda recibe las coordenadas y devuelve el clima. En `current` le pedimos qué variables queremos:

```
GET https://api.open-meteo.com/v1/forecast?
latitude=-26.82&longitude=-65.22&current=temperature_2m,relative_humidity_2m,apparent_tem
perature,weather_code,wind_speed_10m&daily=temperature_2m_max,temperature_2m_min&timezone
=auto&forecast_days=1
```

La respuesta trae los valores en `current` y sus unidades en `current_units`; lo mismo ocurre con `daily` y `daily_units`. Recortada a lo que usa la aplicación:

```
{
  "current_units": {
    "temperature_2m": "°C",
    "relative_humidity_2m": "%",
    "apparent_temperature": "°C",
    "wind_speed_10m": "km/h"
  },
  "current": {
    "time": "2026-09-23T15:00",
    "temperature_2m": 24.3,
    "relative_humidity_2m": 41,
    "apparent_temperature": 23.1,
    "weather_code": 2,
    "wind_speed_10m": 11.5
  },
  "daily_units": {
    "temperature_2m_max": "°C",
    "temperature_2m_min": "°C"
  },
  "daily": {
    "temperature_2m_max": [27.8],
    "temperature_2m_min": [14.2]
  }
}
```

`weather_code` es un número de la escala de la Organización Meteorológica Mundial. Lo traducimos a texto con una tabla.

Probá los dos pedidos con REST Client o con cURL antes de programar. Así sabés qué forma tienen los datos.

Qué es Ink

Ink es una librería que usa React para construir interfaces de terminal. El apéndice D la presenta con más detalle y construye con ella una agenda completa. En lugar de `<div>` y `<p>`, usás `<Box>` y `<Text>`. El resto es React normal: componentes, props, `useState` y efectos. Cuando el estado cambia, Ink redibuja la terminal.

Preparar el proyecto

Necesitás Node.js 22 o posterior: es la versión mínima que exige la versión actual de Ink.

```
mkdir clima-cli
cd clima-cli
npm init -y
npm install ink react ink-text-input ink-spinner
npm install --save-dev tsx
```

Cada paquete cumple una función:

- `ink` y `react`: la base de la interfaz
- `ink-text-input`: un campo de texto para la terminal
- `ink-spinner`: una animación de “cargando”
- `tsx`: ejecuta archivos `.jsx` directamente, sin paso de compilación

Editá `package.json` para agregar `"type": "module"` y el script de inicio:

```
{
  "name": "clima-cli",
  "type": "module",
  "scripts": {
    "start": "tsx clima.jsx"
  }
}
```

Dejá las dependencias que `npm install` agregó al archivo.

El código

La app tiene cuatro estados posibles: pidiendo la ciudad, cargando, mostrando el clima y mostrando un error. Cada estado dibuja una pantalla distinta.

```

// clima.jsx
import React, { useState } from "react";
import { render, Box, Text, useApp, useInput } from "ink";
import TextInput from "ink-text-input";
import Spinner from "ink-spinner";

// — Acceso a las APIs —————

async function pedirJSON(url) {
  const respuesta = await fetch(url);

  if (!respuesta.ok) {
    throw new Error(`El servidor respondió ${respuesta.status}`);
  }

  const tipo = respuesta.headers.get("content-type") ?? "";
  if (!tipo.includes("application/json")) {
    throw new Error(`Esperaba JSON y llegó ${tipo}`);
  }

  return respuesta.json();
}

async function buscarCiudad(nombre) {
  const url = new URL("https://geocoding-api.open-meteo.com/v1/search");
  url.searchParams.set("name", nombre);
  url.searchParams.set("count", 1);
  url.searchParams.set("language", "es");
  url.searchParams.set("format", "json");

  const datos = await pedirJSON(url);

  if (!datos.results?.length) {
    throw new Error(`No encontré ninguna ciudad llamada "${nombre}"`);
  }

  return datos.results[0];
}

async function obtenerClima(latitud, longitud) {
  const url = new URL("https://api.open-meteo.com/v1/forecast");
  url.searchParams.set("latitude", latitud);
  url.searchParams.set("longitude", longitud);
  url.searchParams.set(
    "current",
    "temperature_2m,relative_humidity_2m,apparent_temperature,weather_code,wind_speed_10m"
  );
  url.searchParams.set("daily", "temperature_2m_max,temperature_2m_min");
  url.searchParams.set("timezone", "auto");
  url.searchParams.set("forecast_days", 1);

  return pedirJSON(url);
}

// — Traducción del código de clima —————

const DESCRIPCIONES = {

```



```

0: "☀ Despejado",
1: "☀ Mayormente despejado",
2: "☁ Parcialmente nublado",
3: "☁ Nublado",
45: "☁ Niebla",
48: "☁ Niebla con escarcha",
51: "☁ Llovizna leve",
53: "☁ Llovizna",
55: "☁ Llovizna intensa",
61: "☁ Lluvia leve",
63: "☁ Lluvia",
65: "☁ Lluvia intensa",
71: "☁ Nevada leve",
73: "☁ Nevada",
75: "☁ Nevada intensa",
80: "☁ Chaparrones leves",
81: "☁ Chaparrones",
82: "☁ Chaparrones violentos",
95: "☁ Tormenta",
96: "☁ Tormenta con granizo",
99: "☁ Tormenta con granizo fuerte",
};

function describir(codigo) {
    return DESCRIPCIONES[codigo] ?? `Código ${codigo}`;
}

// — Componentes —

function Dato({ etiqueta, valor, unidad }) {
    return (
        <Box>
            <Box width={20}>
                <Text dimColor>{etiqueta}</Text>
            </Box>
            <Text>
                {valor} {unidad}
            </Text>
        </Box>
    );
}

function TarjetaClima({ lugar, clima }) {
    const actual = clima.current;
    const u = clima.current_units;
    const region = [lugar.admin1, lugar.country].filter(Boolean).join(", ");

    return (
        <Box flexDirection="column" borderStyle="round" borderColor="cyan" paddingX={2}>
            <Text bold color="cyan">{lugar.name}</Text>
            <Text dimColor>{region}</Text>
            <Text> </Text>
            <Text>{describir(actual.weather_code)}</Text>
            <Text> </Text>
            <Dato etiqueta="Temperatura" valor={actual.temperature_2m} unidad=
{u.temperature_2m} />
            <Dato etiqueta="Sensación térmica" valor={actual.apparent_temperature} unidad=
{u.apparent_temperature} />
            <Dato etiqueta="Humedad" valor={actual.relative_humidity_2m} unidad=

```

```

{u.relative_humidity_2m} />
    <Dato etiqueta="Viento" valor={actual.wind_speed_10m} unidad={u.wind_speed_10m} />
    <Dato
        etiqueta="Mín. / Máx. hoy"
        valor={` ${clima.daily.temperature_2m_min[0]} /
${clima.daily.temperature_2m_max[0]} `}
        unidad={clima.daily_units.temperature_2m_max}
    />
    </Box>
);
}

function App() {
    const { exit } = useApp();
    const [ciudad, setCiudad] = useState("");
    const [estado, setEstado] = useState("pidiendo"); // pidiendo | cargando | mostrando |
error
    const [resultado, setResultado] = useState(null);
    const [error, setError] = useState("");

    // Después de mostrar un resultado: "q" sale, cualquier otra tecla consulta otra ciudad
    useInput(
        (input, key) => {
            if (input === "q" || key.escape) {
                exit();
                return;
            }
            setCiudad("");
            setEstado("pidiendo");
        },
        { isActive: estado === "mostrando" || estado === "error" }
    );

    async function consultar(nombre) {
        if (!nombre.trim()) return;

        setEstado("cargando");
        try {
            const lugar = await buscarCiudad(nombre.trim());
            const clima = await obtenerClima(lugar.latitude, lugar.longitude);
            setResultado({ lugar, clima });
            setEstado("mostrando");
        } catch (e) {
            setError(e.message);
            setEstado("error");
        }
    }

    return (
        <Box flexDirection="column" padding={1}>
            <Text bold>🌍 Clima actual · datos de Open-Meteo</Text>
            <Text></Text>

            {estado === "pidiendo" && (
                <Box>
                    <Text color="green">Ciudad: </Text>
                    <TextInput value={ciudad} onChange={setCiudad} onSubmit={consultar} />
                </Box>
            )}
        </Box>
    );
}

```

```

    {estado === "cargando" && (
      <Text>
        <Text color="yellow"><Spinner type="dots" /></Text>
        {" "}Consultando el clima de {ciudad}...
      </Text>
    )}

    {estado === "mostrando" && <TarjetaClima {...resultado} />}

    {estado === "error" && <Text color="red">✖ {error}</Text>}

    {(estado === "mostrando" || estado === "error") && (
      <Box marginTop={1}>
        <Text dimColor>Tocá cualquier tecla para otra ciudad, o "q" para salir.</Text>
      </Box>
    )}
  </Box>
);
}

render(<App />);

```

Ejecutarla

```
npm start
```

La salida se ve así:

 Clima actual · datos de Open-Meteo

San Miguel de Tucumán
Tucumán, Argentina

 Parcialmente nublado

Temperatura	24.3 °C
Sensación térmica	23.1 °C
Humedad	41 %
Viento	11.5 km/h
Mín. / Máx. hoy	14.2 / 27.8 °C

Tocá cualquier tecla para otra ciudad, o "q" para salir.

Qué conceptos del capítulo aparecen en la app

Cada parte de la app usa algo que vimos antes:

- `new URL` y `searchParams` arman la query sin errores de codificación
- `pedidoJSON` revisa `respuesta.ok` porque `fetch` no falla con un 404
- `pedidoJSON` lee la cabecera `content-type` antes de leer el cuerpo

- el primer `await` trae estado y cabeceras; el segundo, en `.json()`, trae el cuerpo
- los dos pedidos van uno detrás del otro porque el segundo necesita las coordenadas del primero
- las unidades salen de la respuesta (`current_units`), no están escritas en el código
- los errores de red y de datos terminan en el mismo `catch` y se muestran en pantalla
- `pedirJSON` resuelve el mismo problema que su homónima del capítulo 20, que además agrega un tiempo límite con `AbortController` y traduce cada tipo de fallo a un mensaje para la persona

Cómo se extiende

La estructura admite varias mejoras con cambios pequeños, porque los datos ya llegan con su forma y sus unidades. Con `forecast_days=3`, cada arreglo de `daily` trae tres posiciones y el pronóstico de los próximos días sale de recorrerlas. Con `temperature_unit=fahrenheit` cambia la unidad y `current_units` la informa sola, sin tocar el código que presenta los datos. Pedir cinco resultados de geocodificación y ofrecerlos con `ink-select-input` resuelve las ciudades homónimas, el mismo problema que el capítulo 20 resuelve con un `select`. Y `performance.now()`, tomado antes y después de cada `await`, mide cuánto tarda cada pedido.

22.10 Errores frecuentes

Crear que `fetch` falla con un 404

Solo rechaza cuando no pudo hablar con el servidor. Un 404 o un 500 llegan como respuestas normales, con `ok` en `false`.

Leer el cuerpo dos veces

El cuerpo es un flujo. Después de `json()`, un `text()` lanza `TypeError`: hay que elegir una forma de lectura.

Enviar un objeto en `body`

`body` recibe texto. El objeto se pasa por `JSON.stringify` y se declara `Content-Type: application/json`.

Usar `PUT` para cambiar un campo

`PUT` reemplaza el recurso entero, y los campos omitidos se pierden. Para modificar una parte existe `PATCH`.

Reintentar un `POST` como si fuera un `GET`

`GET`, `PUT` y `DELETE` son idempotentes; `POST` no. Repetir un `POST` que falló por la red puede crear el recurso dos veces.

Concatenar la query a mano

Un espacio, una tilde o un `&` en el valor rompen la URL. `URL` y `URLSearchParams` codifican cada parámetro.

Suponer que un `POST` a `JSONPlaceholder` guardó algo

La API responde como si hubiera funcionado, pero no persiste nada. Es una herramienta de práctica, no una base de datos.

Leer un `4xx` como una falla del servidor

La familia `4xx` señala un problema en el pedido: método, URL, cabeceras o datos. La `5xx` es la que indica que falló el servidor.

22.11 Para recordar

- HTTP es una conversación de pedido y respuesta entre un cliente y un servidor. Los dos mensajes tienen la misma forma: una primera línea, cabeceras, una línea vacía y un cuerpo opcional.
- HTTP no tiene memoria: cada pedido lleva todo lo que el servidor necesita saber, incluida la identidad de quien pregunta.
- REST organiza una API en recursos con URL propia. `GET`, `POST`, `PUT`, `PATCH` y `DELETE` son las acciones sobre esos recursos, y conocer cuáles son seguras e idempotentes dice cuáles pueden reintentarse.
- El código de estado dice qué pasó; su primer dígito dice de quién es el problema.
- JSON es el formato en que viajan los datos: más estricto que un objeto de JavaScript, sin comentarios, fechas ni `undefined`.
- Para probar una API sin programar, REST Client en VS Code o cURL en la terminal. Los dos muestran el mensaje HTTP tal como viaja.
- `fetch` usa dos `await`: uno para el estado y las cabeceras, otro para el cuerpo. Y `respuesta.ok` hay que revisarlo siempre, porque `fetch` no lo hace.

Apéndices

Apéndice A. Byte Pair Encoding: construir un tokenizador

Idea central

Byte Pair Encoding (BPE) convierte texto en una secuencia de tokens al comenzar con unidades universales y aprender, en orden, fusiones de pares frecuentes. El modelo aprendido no es solo un diccionario: es una lista ordenada de reglas que debe aplicarse de la misma forma al codificar y deshacerse al decodificar.

Este apéndice integra conceptos del libro:

- strings, Unicode y UTF-8;
- arrays, Map y conteos;
- funciones puras y clases;
- recursividad;
- archivos JSON;
- invariantes y pruebas.

La implementación es didáctica. Permite comprender el mecanismo, observarlo y experimentar; no intenta reemplazar un tokenizador de producción.

A.1 El problema: convertir texto en números

Los modelos y muchos algoritmos numéricos no operan directamente sobre palabras visibles. Necesitan identificadores:

```
"hola mundo" → [391, 82, 1042]
```

La conversión debe resolver requisitos en tensión:

1. representar cualquier texto;
2. no crear un vocabulario imposible de mantener;
3. producir secuencias razonablemente cortas;
4. ser reversible o conservar la información necesaria;
5. aplicar exactamente el mismo modelo durante entrenamiento e inferencia.

A.2 Por qué no usar solo palabras

Un token por palabra parece natural:

```
"casa" → 1
"casas" → 2
"casita" → 3
```

Pero aparecen:

- formas flexionadas;
- errores de escritura;
- nombres propios;
- URLs, código y números;
- idiomas diferentes;
- palabras que no estaban en el corpus.

Un vocabulario de palabras crece mucho y todavía necesita un token desconocido.

A.3 Por qué no usar solo caracteres o bytes

Unidades pequeñas representan cualquier texto, pero alargan la secuencia. Una palabra frecuente se repite como varios pasos en lugar de una unidad reutilizable.

BPE busca un compromiso:

```
unidades pequeñas universales
+ composiciones frecuentes aprendidas
= cobertura completa con secuencias más compactas
```

A.4 Texto, puntos de código y UTF-8

Un string JavaScript usa unidades UTF-16. Para obtener una base finita y universal, esta implementación lo codifica como UTF-8:

```
const encoder = new TextEncoder();
const decoder = new TextDecoder("utf-8", { fatal: true });

const bytes = [...encoder.encode("mañana")];
const recuperado = decoder.decode(Uint8Array.from(bytes));
```

La sección 5.10 explica esas tres unidades de texto y la sección 19.13 muestra la misma distinción del lado de los archivos. Cada byte es un entero entre 0 y 255, así que los ids base son:

```
0, 1, 2, ... 255
```

Un byte no equivale a un carácter. ñ ocupa más de un byte en UTF-8 y un emoji suele ocupar cuatro. BPE puede aprender a fusionar esos bytes si aparecen con frecuencia.

A.5 Qué es un token

En este modelo, un token es un id que representa:

- un byte base; o
- la concatenación de dos tokens anteriores.

```
token 256 = token 97 + token 110
```

Si 97 representa a y 110 representa n en UTF-8/ASCII, el nuevo token representa an.

Una regla posterior puede usar el token 256:

```
token 257 = token 98 + token 256 → "ban"
```

El vocabulario forma un grafo de composiciones dirigido hacia unidades anteriores.

A.6 La idea del entrenamiento

Dada una secuencia:

```
banana banana
```

la convertimos en bytes y repetimos:

1. contar pares adyacentes;
2. elegir el más frecuente;
3. asignarle un id nuevo;
4. reemplazar todas sus apariciones no superpuestas;
5. guardar la regla;
6. volver a contar sobre la secuencia resultante.

El proceso se detiene al alcanzar un número máximo de fusiones o cuando ningún par alcanza la frecuencia mínima.

A.7 Contar pares

Una clave de texto permite usar Map:

```
function clavePar(izquierda, derecha) {
  return `${izquierda},${derecha}`;
}

function contarPares(ids) {
  const conteos = new Map();

  for (let indice = 0; indice < ids.length - 1; indice += 1) {
    const clave = clavePar(ids[indice], ids[indice + 1]);
    conteos.set(clave, (conteos.get(clave) ?? 0) + 1);
  }

  return conteos;
}
```

Para:

```
[1, 2, 1, 2, 3]
```

cuenta:

```
(1,2) → 2
(2,1) → 1
(2,3) → 1
```

La representación de clave es suficiente para ids enteros. Es el patrón de conteo de frecuencias de la sección 11.6, con una clave compuesta. Una implementación optimizada podría evitar crear strings en cada par.

A.8 Elegir el par más frecuente

```
function parMasFrecuente(ids) {
  const conteos = contarPares(ids);
  let mejorPar = null;
  let mejorFrecuencia = 0;

  for (const [clave, frecuencia] of conteos) {
    if (frecuencia > mejorFrecuencia) {
      mejorPar = clave.split(",").map(Number);
      mejorFrecuencia = frecuencia;
    }
  }

  return mejorPar === null
    ? null
    : { par: mejorPar, frecuencia: mejorFrecuencia };
}
```

El desempate es determinista: `Map` conserva el orden de primera inserción y solo reemplazamos al encontrar una frecuencia estrictamente mayor. Por lo tanto, gana el par que apareció primero entre los máximos.

La política de desempate forma parte del modelo. Otra implementación que elija distinto aprenderá reglas diferentes aunque use el mismo corpus.

A.9 Fusionar sin superposición

```
function fusionar(ids, [izquierda, derecha], nuevoId) {
  const resultado = [];

  for (let indice = 0; indice < ids.length; ) {
    const coincide =
      ids[indice] === izquierda &&
      ids[indice + 1] === derecha;

    if (coincide) {
      resultado.push(nuevoId);
      indice += 2;
    } else {
      resultado.push(ids[indice]);
      indice += 1;
    }
  }

  return resultado;
}
```

```
fusionar([1, 2, 1, 2, 1], [1, 2], 256);
// [256, 256, 1]
```

El índice avanza dos posiciones ante una fusión. Así las coincidencias no se superponen.

Para `[1, 1, 1]` y el par `[1, 1]`, el resultado es `[256, 1]`, no una fusión doble que reutilice el elemento central.

A.10 Representar una regla

```
const regla = {
  izquierda: 97,
  derecha: 110,
  id: 256
};
```

El id nuevo será `256 + cantidadDeFusiones`. La lista queda contigua y ordenada. Una regla solo puede referirse a bytes o a tokens creados antes.

Invariantes:

```
id >= 256
izquierda < id
derecha < id
id de la regla en posición i = 256 + i
```

Estos invariantes permiten localizar una regla mediante `fusiones[id - 256]` y garantizan que la expansión recursiva termina.

A.11 Clase completa

```
class BPE {
  constructor() {
    this.fusiones = [];
  }

  train(texto, maximoFusiones = 100, frecuenciaMinima = 2) {
    if (!Number.isSafeInteger(maximoFusiones) || maximoFusiones < 0) {
      throw new RangeError("maximoFusiones debe ser no negativo");
    }

    if (!Number.isSafeInteger(frecuenciaMinima) || frecuenciaMinima < 2) {
      throw new RangeError("frecuenciaMinima debe ser al menos 2");
    }

    let ids = [...new TextEncoder().encode(texto)];
    this.fusiones = [];

    for (let paso = 0; paso < maximoFusiones; paso += 1) {
      const candidato = parMasFrecuente(ids);

      if (!candidato || candidato.frecuencia < frecuenciaMinima) break;

      const id = 256 + this.fusiones.length;
      const [izquierda, derecha] = candidato.par;

      this.fusiones.push({ izquierda, derecha, id });
      ids = fusionar(ids, candidato.par, id);
    }

    return ids;
  }

  encode(texto) {
    let ids = [...new TextEncoder().encode(texto)];

    for (const regla of this.fusiones) {
      ids = fusionar(
        ids,
        [regla.izquierda, regla.derecha],
        regla.id
      );
    }

    return ids;
  }

  decode(ids) {
    const bytes = [];

    for (const id of ids) {
      bytes.push(...this.#expandir(id));
    }

    return new TextDecoder("utf-8", { fatal: true })
      .decode(Uint8Array.from(bytes));
  }
}
```

```

#expandir(id) {
  if (!Number.isSafeInteger(id) || id < 0) {
    throw new RangeError(`Token inválido: ${id}`);
  }

  if (id < 256) return [id];

  const regla = this.fusiones[id - 256];

  if (!regla || regla.id !== id) {
    throw new RangeError(`Token desconocido: ${id}`);
  }

  return [
    ...this.#expandir(regla.izquierda),
    ...this.#expandir(regla.derecha)
  ];
}
}

```

A.12 Entrenamiento paso a paso

```

function entrenarConTraza(texto, maximoFusiones = 10) {
  let ids = [...new TextEncoder().encode(texto)];
  const reglas = [];

  for (let paso = 0; paso < maximoFusiones; paso += 1) {
    const candidato = parMasFrecuente(ids);
    if (!candidato || candidato.frecuencia < 2) break;

    const id = 256 + reglas.length;
    const antes = ids.length;

    ids = fusionar(ids, candidato.par, id);
    reglas.push({
      id,
      izquierda: candidato.par[0],
      derecha: candidato.par[1]
    });

    console.log({
      paso: paso + 1,
      par: candidato.par,
      frecuencia: candidato.frecuencia,
      antes,
      despues: ids.length,
      id
    });
  }

  return { ids, reglas };
}

```

La traza permite observar que una fusión frecuente reduce la longitud y que los conteos deben recalcularse después de cada cambio.

A.13 Por qué el orden importa

Supongamos:

```
256 = (97, 110)    → "an"  
257 = (98, 256)    → "ban"  
258 = (257, 256)   → "banan"
```

La regla 257 no puede aplicarse antes de crear 256. Ordenar las reglas por frecuencia, texto o ids internos cambiaría la tokenización.

encode debe recorrer exactamente en el orden aprendido. Guardar solo un diccionario de bytes finales sin las prioridades no siempre permite reconstruir esa conducta.

A.14 Decodificación recursiva

Un token base devuelve un byte. Un token compuesto expande sus dos hijos:

```
expandir(258)  
→ expandir(257) + expandir(256)  
→ expandir(98) + expandir(256) + expandir(97) + expandir(110)  
→ bytes originales
```

El caso base es `id < 256` y el caso recursivo expande los dos hijos: la estructura de la sección 16.2, aplicada a un grafo de composiciones. El invariante «cada regla referencia tokens anteriores» impide ciclos y garantiza la terminación.

Una implementación más eficiente puede precalcular la secuencia de bytes de cada regla al cargar el modelo y evitar expandir repetidamente.

A.15 La propiedad fundamental

Para cualquier string que pueda codificarse como UTF-8:

```
decode(encode(texto)) === texto
```

```

const bpe = new BPE();
bpe.train("banana banana mañana mañana 😊😊", 50);

for (const texto of [
  "",
  "banana",
  "mañana",
  "😊 banana",
  "texto nunca visto"
]) {
  const ids = bpe.encode(texto);
  const recuperado = bpe.decode(ids);

  console.assert(recuperado === texto, {
    texto,
    ids,
    recuperado
  });
}

```

Un texto no visto sigue siendo representable porque los 256 tokens base cubren cualquier secuencia de bytes. Tal vez use más tokens, pero no necesita un token desconocido.

A.16 Otras propiedades para comprobar

```

function validarModelo(bpe) {
  for (let indice = 0; indice < bpe.fusiones.length; indice += 1) {
    const regla = bpe.fusiones[indice];
    const idEsperado = 256 + indice;

    if (regla.id !== idEsperado) {
      throw new Error(`Id fuera de secuencia: ${regla.id}`);
    }

    if (regla.izquierda >= regla.id || regla.derecha >= regla.id) {
      throw new Error(`Referencia futura en token ${regla.id}`);
    }
  }
}

```

También:

- entrenar con texto vacío no crea reglas;
- ninguna fusión aumenta la longitud;
- una regla aprendida tiene frecuencia inicial al menos igual al mínimo;
- ids desconocidos se rechazan;
- entrenar de nuevo reinicia el modelo;
- mismo corpus, límites y desempate producen las mismas reglas.

A.17 Guardar el modelo

```
import { writeFile } from "node:fs/promises";

async function guardarModelo(ruta, bpe) {
  validarModelo(bpe);

  const datos = {
    version: 1,
    algoritmo: "bpe-didactico",
    base: "utf-8-bytes",
    fusiones: bpe.fusiones
  };

  await writeFile(
    ruta,
    `${JSON.stringify(datos, null, 2)}\n`,
    { encoding: "utf8", flag: "wx" }
  );
}
```

`version`, `algoritmo` y `base` evitan interpretar silenciosamente un modelo con convenciones diferentes.

A.18 Cargar y validar

```
import { readFile } from "node:fs/promises";

async function cargarModelo(ruta) {
  const datos = JSON.parse(await readFile(ruta, "utf8"));

  if (datos.version !== 1 ||
      datos.algoritmo !== "bpe-didactico" ||
      datos.base !== "utf-8-bytes" ||
      !Array.isArray(datos.fusiones)) {
    throw new TypeError("Modelo BPE incompatible");
  }

  const bpe = new BPE();
  bpe.fusiones = datos.fusiones.map((regla, indice) => {
    const id = 256 + indice;

    if (!regla ||
        regla.id !== id ||
        !Number.isSafeInteger(regla.izquierda) ||
        !Number.isSafeInteger(regla.derecha) ||
        regla.izquierda < 0 ||
        regla.derecha < 0 ||
        regla.izquierda >= id ||
        regla.derecha >= id) {
      throw new TypeError(`Regla inválida en la posición ${indice}`);
    }

    return {
      izquierda: regla.izquierda,
      derecha: regla.derecha,
      id
    };
  });

  validarModelo(bpe);
  return bpe;
}
```

No basta con comprobar que `fusiones` sea un array. Los archivos externos no son confiables y una referencia futura podría crear una expansión inválida. Es el mismo principio de la sección 19.30: `JSON.parse` valida sintaxis, no significado.

A.19 Medir la tokenización

```
function estadísticas(bpe, texto) {
  const bytes = new TextEncoder().encode(texto).length;
  const tokens = bpe.encode(texto).length;

  return {
    bytes,
    tokens,
    bytesPorToken: tokens === 0 ? 0 : bytes / tokens
  };
}
```

Más bytes por token indica una secuencia más corta para ese texto, pero no es una métrica completa de calidad. También importan tamaño del vocabulario, distribución, idioma, preprocesamiento y costo del modelo.

Compará corpus:

- repetitivo;
- prosa natural;
- código fuente;
- texto con emoji;
- texto de un dominio diferente al entrenamiento.

A.20 Complejidad de la versión didáctica

En cada fusión:

1. se recorre la secuencia para contar;
2. se recorre el mapa para elegir;
3. se recorre la secuencia para fusionar.

Con muchas fusiones y un corpus grande, repetir estos recorridos es costoso. `encode` también recorre el texto una vez por regla, incluso si una regla no aparece.

Tokenizadores de producción usan índices, estructuras de prioridad, pretokenización y algoritmos especializados. La versión didáctica privilegia que cada paso sea visible y verificable.

A.21 Decisiones simplificadas

Este apéndice omite o fija:

- normalización Unicode;
- tratamiento especial de espacios;
- división inicial en palabras o fragmentos;

- tokens reservados;
- límites de vocabulario basados en frecuencia y tamaño;
- entrenamiento sobre varios documentos sin permitir fusiones entre límites;
- desempates compatibles con modelos externos;
- codificación optimizada;
- procesamiento distribuido;
- límites ante archivos o modelos hostiles.

Cada decisión cambia los tokens. Dos implementaciones llamadas BPE no son necesariamente compatibles.

A.22 Separar documentos

Concatenar corpus permite que un par cruce el final de un documento y el inicio del siguiente. Si eso no tiene significado, se necesita una frontera que no pueda fusionarse o contar pares documento por documento.

Una estrategia didáctica:

```
function contarParesEnDocumentos(documentos) {
  const total = new Map();

  for (const ids of documentos) {
    for (const [clave, cantidad] of contarPares(ids)) {
      total.set(clave, (total.get(clave) ?? 0) + cantidad);
    }
  }

  return total;
}
```

Después de elegir una regla, se fusiona dentro de cada documento por separado.

A.23 Tokens especiales

Un sistema puede reservar ids para comienzo, fin, relleno o separación. No deben confundirse con bytes ni generarse accidentalmente mediante fusiones.

Una organización posible:

0..255	bytes
256..N	fusiones
N+1...	especiales registrados en el modelo

Otra reserva ids especiales antes de las fusiones. Lo importante es que el formato documento rangos y que codificador y decodificador compartan la convención.

A.24 Experimentos productivos

1. Mostrá en cada paso el par, su representación en bytes y la reducción lograda.
2. Cambiá el criterio de desempate y compará modelos.
3. Entrená con `banana banana` y dibujá el árbol de cada token compuesto.
4. Compará NFC y NFD para palabras con acentos.
5. Evitá fusiones entre documentos.
6. Agregá tokens especiales sin colisionar ids.
7. Precalculá bytes por token al cargar.
8. Limitá profundidad y tamaño durante la validación de modelos externos.
9. Escribí pruebas generativas de ida y vuelta con strings aleatorios.
10. Medí dónde se consume el tiempo antes de optimizar.

A.25 Modelo mental final

```
ENTRENAR
texto
→ bytes
→ contar pares
→ elegir par
→ crear token
→ fusionar
→ repetir
→ guardar reglas ordenadas

CODIFICAR
texto
→ bytes
→ aplicar reglas en orden
→ ids

DECODIFICAR
ids
→ expandir tokens hasta bytes
→ decodificar UTF-8
→ texto
```

A.26 Para recordar

- Los bytes ofrecen cobertura universal; las fusiones aprenden unidades frecuentes.
- Una regla crea un token a partir de dos tokens anteriores.
- El orden y el desempate son parte del modelo.
- La expansión recursiva termina gracias al orden de ids y referencias.
- `decode(encode(texto)) == texto` es la propiedad central, pero no la única validación necesaria.

- Comprender primero la versión simple permite reconocer qué optimizan y qué convenciones agregan los tokenizadores reales.

Apéndice B. Evaluar expresiones mediante pilas

Idea central

Una expresión aritmética puede evaluarse de izquierda a derecha si se guardan en pilas los valores y los operadores que todavía no pueden resolverse. La prioridad de los operadores determina cuándo se retira una operación pendiente; los paréntesis funcionan como fronteras que aíslan cada grupo.

Este apéndice conecta varios conceptos del libro:

- clases y encapsulamiento;
- arrays usados como estructuras de datos;
- recorridos de izquierda a derecha;
- condiciones y ciclos;
- funciones como valores;
- invariantes y validación de entradas.

La implementación es deliberadamente pequeña. Evalúa números positivos o negativos escritos como literales y los operadores binarios `+`, `-`, `*` y `/`. No pretende ser un parser completo de JavaScript.

B.1 El problema: respetar la estructura de la expresión

Evaluar esta expresión estrictamente de izquierda a derecha produce un resultado incorrecto:

```
3 + 4 * 2
```

La multiplicación debe resolverse antes que la suma, por lo que el resultado correcto es 11, no 14. Los paréntesis agregan otra regla:

```
3 + 4 * (2 - 1) → 7
```

El algoritmo necesita conservar información mientras recorre los tokens:

1. los números se convierten en valores;
2. los operadores quedan pendientes;
3. un operador nuevo puede obligar a resolver el anterior;
4. un paréntesis de cierre resuelve todo el grupo que abrió el paréntesis;
5. al llegar al final se resuelven las operaciones que todavía quedan.

La estructura adecuada para ese comportamiento es una pila: el último elemento que se guarda es el primero que se retira. La sección 2.9 presentó la precedencia y la asociatividad como reglas de lectura; acá se vuelven un mecanismo.

B.2 Implementar una pila

Una pila expone dos operaciones principales:

- `push`, que agrega un elemento en el tope;
- `pop`, que retira y devuelve el elemento del tope.

También conviene poder consultar el tope sin retirarlo y saber si la pila está vacía. La clase oculta cómo se representa la estructura: el resto del programa solo depende de ese contrato.

```
class Stack {
  constructor() {
    this.elementos = [];
    this.contador = 0;
  }

  push(elemento) {
    this.elementos[this.contador++] = elemento;
  }

  pop() {
    return this.elementos[--this.contador];
  }

  get top() {
    return this.elementos[this.contador - 1];
  }

  get empty() {
    return this.contador === 0;
  }
}
```

El contador representa la próxima posición disponible. Por eso también coincide con la cantidad de elementos almacenados:

```
elementos: [A, B, C, _, _]
contador:      3
               ↑ próxima posición disponible
```

En `push`, el posincremento usa primero el valor actual de `contador` como índice y después lo incrementa. En `pop`, el predecremento reduce primero el contador y usa el nuevo valor para acceder al último elemento almacenado.

Esta versión supone que la pila se utiliza correctamente: no controla una extracción cuando está vacía ni impone una capacidad máxima. El array comienza vacío y crece automáticamente a medida que se agregan elementos.

En una pila no se accede a un elemento arbitrario. La única entrada válida es el tope. Esta restricción permite razonar sobre el orden de procesamiento y mantiene independiente la implementación interna.

Un array de JavaScript ya ofrece ese comportamiento con `push` y `pop`, según la sección 9.6. Definir un tipo propio no agrega capacidad: quita la que sobra. Nadie puede leer el elemento del medio ni ordenar la pila, y el algoritmo se vuelve más difícil de escribir mal.

B.3 Dos pilas para una expresión

La evaluación usa dos pilas con responsabilidades diferentes:

Pila	Contenido
valores	números y resultados parciales
operadores	operadores pendientes y paréntesis de apertura

Cuando se resuelve una operación se retiran tres elementos:

1. el operador;
2. el operando derecho `b`;
3. el operando izquierdo `a`.

Después se calcula `a operador b` y se apila el resultado. El orden es fundamental: `8 - 3` no es igual que `3 - 8`, y `8 / 2` no es igual que `2 / 8`.

La tabla de prioridad expresa una regla del dominio, no una propiedad de la pila:

```
const prioridad = {
  "+": 1,
  "-": 1,
  "*": 2,
  "/": 2
};
```

B.4 Recorrer y resolver

Antes de evaluar, la expresión se separa en tokens. Esta versión espera que los paréntesis estén separados de los números y operadores:

```
3 + 4 * ( 2 - 1 )
```

La expresión regular `/\s+/u` permite aceptar uno o varios espacios y también saltos de línea. Los operadores se conservan como strings y los números se convierten a `number`.

La función `resolver` implementa una única operación pendiente:

```
function resolver(valores, operadores, operacion) {  
  const operador = operadores.pop();  
  const b = valores.pop();  
  const a = valores.pop();  
  valores.push(operacion[operador](a, b));  
}
```

El algoritmo completo sigue estas reglas:

- número: se apila en `valores`;
- `(`: se apila en `operadores`;
- `)`: se resuelve hasta encontrar `(` y luego se descarta ese paréntesis;
- operador: se resuelven operadores pendientes de prioridad mayor o igual, siempre dentro del grupo actual, y después se apila el operador nuevo;
- fin: se resuelven todos los operadores restantes.

La condición “dentro del grupo actual” es importante. Un paréntesis de apertura no es un operador y no debe compararse con la tabla de prioridades.

```

function evaluar(expresion) {
    const texto = expresion.trim();

    if (texto === "") {
        throw new Error("La expresión no puede estar vacía");
    }

    const valores = new Stack();
    const operadores = new Stack();

    const prioridad = {
        "+": 1,
        "-": 1,
        "*": 2,
        "/": 2
    };

    const operacion = {
        "+": (a, b) => a + b,
        "-": (a, b) => a - b,
        "*": (a, b) => a * b,
        "/": (a, b) => a / b
    };

    const tokens = texto.split(/\s+/u).map((token) => {
        return Number.isNaN(Number(token)) ? token : Number(token);
    });

    for (const token of tokens) {
        if (typeof token === "number") {
            valores.push(token);
            continue;
        }

        if (token === "(") {
            operadores.push(token);
            continue;
        }

        if (token === ")") {
            while (!operadores.empty && operadores.top !== "(") {
                resolver(valores, operadores, operacion);
            }

            if (operadores.empty) {
                throw new Error("Hay un paréntesis de cierre sin apertura");
            }

            operadores.pop();
            continue;
        }

        if (!(token in prioridad)) {
            throw new Error(`Token no reconocido: ${token}`);
        }

        while (
            !operadores.empty &&

```

```

    operadores.top !== "(" &&
    prioridad[operadores.top] >= prioridad[token]
  ) {
    resolver(valores, operadores, operacion);
  }

  operadores.push(token);
}

while (!operadores.empty) {
  if (operadores.top === "(") {
    throw new Error("Hay un paréntesis de apertura sin cierre");
  }

  resolver(valores, operadores, operacion);
}

if (valores.contador !== 1) {
  throw new Error("La expresión no tiene una estructura válida");
}

return valores.pop();
}

console.log(evaluar("3 + 4 * ( 2 - 1 )"));
// 7

```

El analizador sigue suponiendo que la entrada tiene la forma esperada: un número, un operador, otro número y así sucesivamente. Las comprobaciones finales mejoran los mensajes de error, pero no convierten este código en un parser general.

B.5 Evaluación paso a paso

Consideremos la expresión:

```
1 + 2 * ( 3 * 4 + 5 )
```

Sus tokens son:

```
[1, "+", 2, "*", "(", 3, "*", 4, "+", 5, ")"]
```

En la tabla, la base de cada pila está a la izquierda y el tope está a la derecha.

Token	Acción	valores	operadores
Inicio	Ambas pilas están vacías	[]	[]
1	Apilar el número	[1]	[]
+	Apilar el operador	[1]	[+]
2	Apilar el número	[1, 2]	[+]
*	Tiene mayor prioridad; se apila	[1, 2]	[+, *]
(	Abrir un grupo	[1, 2]	[+, *, (]
3	Apilar el número	[1, 2, 3]	[+, *, (]
*	Apilar el operador del grupo	[1, 2, 3]	[+, *, (, *]
4	Apilar el número	[1, 2, 3, 4]	[+, *, (, *]
+	Resolver $3 * 4 = 12$; apilar +	[1, 2, 12]	[+, *, (, +]
5	Apilar el número	[1, 2, 12, 5]	[+, *, (, +]
)	Resolver $12 + 5 = 17$; retirar (	[1, 2, 17]	[+, *]
Fin	Resolver $2 * 17 = 34$	[1, 34]	[+]
Fin	Resolver $1 + 34 = 35$	[35]	[]

El único valor restante es el resultado:

```
evaluar("1 + 2 * ( 3 * 4 + 5 )"); // 35
```

B.6 Qué queda fuera de esta versión

La implementación sirve para estudiar el mecanismo, pero tiene límites explícitos:

- los tokens deben estar separados por espacios;
- el signo menos solo funciona pegado a un número, como parte de su literal;
- no admite funciones como `sin(2)`;
- no incorpora la potencia ni otros operadores;
- no informa qué operando falta, solo que la estructura no cierra.

La segunda limitación merece detenerse. Un literal negativo funciona, porque el tokenizador lo convierte en un número entero:

```
evaluar("-3 + 2"); // -1
evaluar("3 * ( -2 )"); // -6
```

Lo que no funciona es el menos como operador de un solo operando, separado de su número o aplicado a un grupo:

```
evaluar("- 5 + 1"); // Error: la expresión no tiene una estructura válida
evaluar("- ( 3 + 2 )"); // Error: la expresión no tiene una estructura válida
```

En los dos casos el `-` entra como operador binario, y cuando `resolver` va a buscar sus dos operandos encuentra uno solo. La comprobación final detecta el desbalance, aunque no puede explicar la causa. Para admitir el signo unario haría falta una etapa de tokenización que decida, según el token anterior, si `-` inicia un literal o es una operación.

Separar las etapas ayuda a extender el programa sin mezclar responsabilidades:

```
texto → tokens → validación → evaluación → resultado
```

La evaluación también puede reemplazarse por el algoritmo de Shunting Yard, que transforma la expresión infija en una expresión posfija. El enfoque de este apéndice evita construir esa segunda representación para concentrarse en el uso de las pilas.

Existe además una alternativa que no usa ninguna pila explícita: un analizador descendente recursivo define una función por nivel de precedencia y deja que cada una llame a la del nivel siguiente. La pila no desaparece, es la pila de llamadas del capítulo 16. Elegir entre las dos técnicas es elegir quién lleva la cuenta, no si alguien la lleva.

B.7 Complejidad

Cada token se apila y se retira una cantidad acotada de veces. Por lo tanto, para n tokens:

- el tiempo es $O(n)$;
- el espacio adicional es $O(n)$ en el peor caso.

La profundidad de los paréntesis y la cantidad de operadores pendientes son los principales factores que determinan cuánto ocupan las pilas.

B.8 Para recordar

- Una pila implementa el comportamiento “último en entrar, primero en salir”.
- `valores` conserva números y resultados parciales; `operadores` conserva operaciones pendientes.
- La prioridad determina cuándo resolver un operador.
- Los paréntesis detienen la resolución hasta cerrar el grupo correspondiente.

- En una operación, el segundo valor retirado es el operando izquierdo.
- La tokenización, la validación y la evaluación son responsabilidades diferentes.
- La versión presentada es didáctica y debe extenderse antes de aceptar entradas generales.

Apéndice C. Fundamentos de computación digital

Idea central

Una computadora representa información con estados discretos y construye cálculos complejos al componer operaciones lógicas simples. Los bits adquieren significado mediante una codificación; las funciones booleanas transforman entradas en salidas; las puertas y circuitos implementan físicamente esas funciones.

Este modelo conecta el nivel físico con las operaciones bit a bit de un lenguaje y muestra una idea que atraviesa toda la programación:

```
componentes simples + composición + capas de abstracción  
→ sistemas complejos
```

C.1 Dos estados distinguibles

Una computadora física trabaja con señales continuas, pero diseña rangos que interpreta como estados discretos. Podemos nombrarlos:

```
0 / 1  
falso / verdadero  
apagado / encendido  
bajo / alto
```

Lo importante no es que exista exactamente cero o cinco voltios, sino que el circuito pueda distinguir con margen dos regiones. Esa separación aporta tolerancia frente a pequeñas variaciones y ruido.

Un **bit** es una unidad capaz de representar una de dos alternativas.

C.2 Un bit no tiene significado por sí solo

El patrón **1** podría significar:

- una respuesta afirmativa;
- un píxel encendido;
- que una puerta está abierta;
- el permiso de lectura;
- la cifra uno en un número binario.

La interpretación depende del convenio. Programar siempre incluye diseñar representaciones.

C.3 Cantidad de combinaciones

Con un bit existen dos combinaciones. Con dos:

```
00
01
10
11
```

Con n bits existen 2^n patrones. Tres bits ofrecen ocho estados:

```
000 001 010 011 100 101 110 111
```

Un dado necesita seis estados, por lo que tres bits alcanzan:

Cara	Código posible
1	001
2	010
3	011
4	100
5	101
6	110

000 y 111 pueden quedar reservados. Esto ilustra dos decisiones:

1. la cantidad de bits determina la capacidad;
2. el formato decide qué significa cada patrón y qué estados son inválidos.

C.4 Cuántos bits hacen falta

Para representar k estados necesitamos el menor n que cumpla:

$$2^n \geq k$$

Ejemplos:

- 2 estados $\rightarrow$ 1 bit;
- 6 estados $\rightarrow$ 3 bits;
- 256 estados $\rightarrow$ 8 bits;
- 1000 estados $\rightarrow$ 10 bits, porque $2^{10} = 1024$.

No todos los patrones deben utilizarse. Los sobrantes pueden reservarse para errores, extensiones o control.

C.5 Del dato al cálculo

Supongamos dos dados codificados con tres bits cada uno. Queremos responder si suman siete. El sistema recibe seis bits y produce uno:

$$f(A, B) \rightarrow \{0, 1\}$$

$$1 + 6 \rightarrow 1$$

$$2 + 5 \rightarrow 1$$

$$3 + 4 \rightarrow 1$$

$$1 + 1 \rightarrow 0$$

$$2 + 2 \rightarrow 0$$

Un cálculo digital puede entenderse como una función que transforma un patrón de entrada en otro de salida. El número de entradas posibles puede ser enorme, pero la idea es la misma.

C.6 Funciones booleanas

Cuando cada variable vale 0 o 1, usamos álgebra de Boole.

NOT

Invierte un bit:

A	NOT A
0	1
1	0

Se escribe $\neg A$.

AND

Solo vale uno si ambas entradas valen uno:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Se escribe $A \wedge B$.

OR

Vale uno si al menos una entrada vale uno:

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Se escribe $A \vee B$.

XOR

Vale uno cuando las entradas son diferentes:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

XOR es útil en suma binaria, paridad, alternancia y cifrados elementales.

C.7 Componer operaciones

$(A \text{ AND } B) \text{ OR } (\text{NOT } C)$

equivale a:

$(A \wedge B) \vee \neg C$

La salida de una operación se convierte en entrada de otra. Este principio se repite:

puertas → bloques aritméticos → procesadores → computadoras
funciones → módulos → aplicaciones → sistemas

La abstracción permite usar un bloque por lo que hace sin reconstruir todos sus componentes cada vez.

C.8 Tabla de verdad

Una tabla enumera todas las entradas y la salida. Para dos variables tiene cuatro filas:

A	B	f(A, B)
0	0	0
0	1	1
1	0	1
1	1	0

Esta tabla describe XOR completamente. Para n entradas habrá 2^n filas.

Una expresión permite calcular sin almacenar toda la tabla; la tabla permite especificar y verificar la expresión.

C.9 Construir una expresión desde una tabla

Supongamos que una función vale uno en estas filas:

A=0, B=1
A=1, B=0

Cada fila verdadera produce un término AND que fija todos los valores:

(NOT A AND B)
(A AND NOT B)

Unimos los términos con OR:

$(\neg A \wedge B) \vee (A \wedge \neg B)$

Esta es una **forma normal disyuntiva**: un OR de términos AND. Permite construir una expresión para cualquier tabla booleana finita, aunque no siempre sea la forma mínima.

C.10 Simplificar expresiones

Algunas identidades:

```
A AND 1 = A
A AND 0 = 0
A OR 0 = A
A OR 1 = 1
A AND A = A
A OR A = A
A AND NOT A = 0
A OR NOT A = 1
```

Distributividad:

```
A AND (B OR C) = (A AND B) OR (A AND C)
A OR (B AND C) = (A OR B) AND (A OR C)
```

Leyes de De Morgan:

```
NOT (A AND B) = (NOT A) OR (NOT B)
NOT (A OR B) = (NOT A) AND (NOT B)
```

Estas leyes sirven en circuitos, condiciones de programas y consultas: las secciones 3.13 y 12.7 las aplican a las guardas de una función.

C.11 NAND y NOR: completitud funcional

NAND es NOT de AND. NOR es NOT de OR.

Con solo puertas NAND puede construirse NOT, AND y OR; por lo tanto, cualquier función booleana. Lo mismo ocurre con NOR.

Por ejemplo, con NAND:

```
NOT A = A NAND A
AND(A, B) = NOT(A NAND B)
```

La **completitud funcional** muestra que una colección mínima de operaciones puede construir todas las demás. En ingeniería, reducir tipos de componentes puede simplificar fabricación, aunque los diseños reales equilibran velocidad, consumo y cantidad de transistores.

C.12 Implementación física

Una puerta lógica puede construirse con transistores usados como interruptores controlados. Los detalles eléctricos incluyen tiempos de propagación, consumo, capacidad, ruido y niveles de tensión.

Desde el nivel lógico, abstraemos esos detalles:

```
entrada 0/1 → puerta → salida 0/1
```

La salida no cambia instantáneamente. Todo circuito tiene retardo. En sistemas sincronizados, un reloj coordina cuándo se considera estable el nuevo estado.

C.13 Circuitos combinacionales y secuenciales

Un circuito **combinacional** produce salida según la entrada actual:

```
salida = f(entrada actual)
```

Ejemplos: sumadores, comparadores, multiplexores.

Un circuito **secuencial** también depende del estado anterior:

```
salida y próximo estado = f(entrada, estado actual)
```

Registros, contadores y memoria requieren almacenar estado. Esta diferencia es la misma que separa una función pura de una clausura con estado, según las secciones 15.2 y 14.18.

C.14 Números binarios posicionales

En decimal, cada posición pesa una potencia de diez. En binario, una potencia de dos:

```
101102
= 1×24 + 0×23 + 1×22 + 1×21 + 0×20
= 16 + 4 + 2
= 2210
```

En JavaScript:

```
0b10110; // 22
(22).toString(2); // "10110"
```

C.15 Suma binaria

Reglas de un bit:

```
0 + 0 = 0, acarreo 0
0 + 1 = 1, acarreo 0
1 + 0 = 1, acarreo 0
1 + 1 = 0, acarreo 1
```

La suma del bit es XOR y el acarreo es AND.

C.16 Medio sumador

Entradas `A` y `B`; salidas `S` y `C`:

```
S = A XOR B
C = A AND B
```

A	B	Suma	Acarreo
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Se llama “medio” porque no recibe un acarreo anterior.

C.17 Sumador completo

Para sumar varios bits, cada posición —salvo la primera— recibe `Cin`, el acarreo previo:

```
S = A XOR B XOR Cin
Cout = (A AND B) OR (Cin AND (A XOR B))
```

Al encadenar sumadores completos se construye un sumador de varios bits. El acarreo se propaga de una posición a la siguiente; diseños más avanzados aceleran esa propagación.

C.18 Representar enteros con signo

Una representación común es complemento a dos. Con un ancho fijo de `n` bits:

- el bit más significativo participa del signo;
- el rango es -2^{n-1} a $2^{n-1} - 1$;
- negar consiste en invertir y sumar uno.

Con 8 bits:

```
5   = 00000101
-5  = 11111011
```

El ancho fijo es esencial. El patrón no tiene un valor con signo independiente del número de bits.

Los operadores bitwise de `number` en JavaScript trabajan con enteros de 32 bits con signo, salvo `>>>`, que interpreta el desplazamiento sin signo. La sección 4.11 los presenta desde el lenguaje.

C.19 Overflow

Con un ancho fijo, una suma puede necesitar un bit adicional. Si se descarta, el resultado “da la vuelta”. Para enteros sin signo de n bits, la aritmética se comporta módulo 2^n .

JavaScript `number` no es un entero fijo de 32 bits para la aritmética normal, pero las operaciones bitwise sí convierten a 32 bits. Typed arrays como `Uint8Array` también aplican rangos de ancho fijo al almacenar.

C.20 Bytes, palabras y memoria

Ocho bits forman un byte, capaz de 256 patrones. La memoria se organiza en bytes direccionables y los procesadores operan también sobre grupos mayores llamados palabras.

Un mismo conjunto de bytes puede interpretarse como:

- entero;
- número de punto flotante;
- instrucción;
- parte de un texto UTF-8;
- canales de un color;
- muestra de audio.

Otra vez, la representación y el contexto dan significado.

C.21 Máscaras de bits en JavaScript

Cada posición representa una opción:

```
const LEER = 1 << 0;    // 0001
const CREAR = 1 << 1;    // 0010
const EDITAR = 1 << 2;   // 0100
const BORRAR = 1 << 3;   // 1000
```

Activar con OR:

```
let permisos = LEER | CREAR;
```

Consultar con AND:

```
function tiene(permisos, permiso) {
  return (permisos & permiso) !== 0;
}
```

Apagar:


```
permisos &= ~CREAR;
```

Alternar con XOR:

```
permisos ^= EDITAR;
```

Presentar:

```
permisos.toString(2).padStart(4, "0");
```

C.22 Operadores lógicos y bitwise no son equivalentes

```
5 && 2; // 2, selección por truthiness
5 & 2;  // 0, AND bit a bit

5 || 2; // 5
5 | 2;  // 7
```

`&&` y `||` preservan operandos y cortocircuitan, según describe el capítulo 3. `&` y `|` convierten a enteros de 32 bits y siempre evalúan ambos lados.

C.23 Del circuito al software productivo

Comprender bits ayuda a:

- interpretar codificaciones y formatos binarios;
- diseñar flags y protocolos;
- razonar sobre rangos y overflow;
- entender por qué texto, imagen y números son interpretaciones de bytes;
- reconocer que una interfaz simple puede ocultar muchas capas.

No significa reemplazar automáticamente estructuras legibles por máscaras. La representación de bajo nivel se justifica cuando hay interoperabilidad, almacenamiento compacto o una operación bitwise real.

C.24 Para recordar

- n bits ofrecen 2^n patrones; la codificación asigna significado.
- Una función booleana puede especificarse con tabla de verdad y construirse mediante composición.
- NOT, AND y OR son una base; NAND o NOR por sí solas también son completas.
- La suma binaria surge de XOR, AND y propagación de acarreo.

- Bits, bytes y señales son la base; las capas de abstracción permiten trabajar productivamente por encima de ella.

Apéndice D. Una agenda de terminal con React e Ink

Idea central

Una interfaz declarativa se describe a partir del estado actual: en lugar de indicarle a cada parte de la pantalla cómo corregirse después de una acción, se declara cómo debe verse y el renderizador calcula la diferencia. React aporta ese modelo y Ink lo lleva a la terminal, de modo que los mismos componentes, props y estado sirven para una página web y para una TUI.

Este apéndice integra buena parte del libro en un programa que funciona:

- objetos, arrays y desestructuración;
- funciones como valores y funciones de orden superior;
- actualización inmutable con `map`, `filter` y expansión;
- lectura y escritura de JSON;
- normalización de texto Unicode para buscar sin acentos;
- separación entre cálculo y efecto.

Un tutorial para construir un **CRUD de contactos** componiendo componentes de biblioteca. La aplicación permite buscar en caliente, seleccionar personas en una lista maestra y ver sus datos en un panel de detalle. Al crear o editar, ese mismo panel muestra un formulario. Los contactos se conservan en JSON y **el legajo es su identificador único**.

Para seguirlo alcanza con conocer funciones, objetos, arrays y módulos de JavaScript. Todo el código, la configuración y los datos necesarios están incluidos aquí.

D.1 React: describir una interfaz a partir de datos

React es una biblioteca para construir interfaces mediante **componentes**: piezas reutilizables que reciben información y describen qué mostrar. Una aplicación se forma combinando esas piezas, unas dentro de otras. [Conceptos de React](#).

La idea central es que la interfaz depende de los datos actuales. Si cambia la persona seleccionada, cambia la ficha; si cambia la búsqueda, cambia la lista.

En lugar de indicarle a cada parte de la pantalla cómo corregirse después de una acción, describimos cómo debe verse con el estado actual. React vuelve a calcular esa descripción cuando corresponde. A este enfoque se lo llama **declarativo**. El capítulo 21 construye la misma idea con Arrow.js, una biblioteca mucho más pequeña, y la sección 22.9 usa Ink para una aplicación que consulta el clima.

Componentes, props y estado

Concepto	Qué representa	Ejemplo en la agenda
Componente	Una pieza de interfaz	Lista, buscador, ficha o formulario
Props	Información que recibe una pieza	El contacto que debe mostrar una ficha
Estado	Información que se conserva entre renderizados	Texto buscado o legajo seleccionado
Evento	Una acción a la que responde el programa	Cambió el texto o se envió un formulario
Renderizado	Calcular la descripción de la interfaz	Obtener la lista y la ficha correspondientes

Las props pueden ser textos, objetos, arrays o funciones. Un padre entrega datos a sus hijos y les pasa funciones para que comuniquen acciones. Por ejemplo, la lista recibe contactos y una función a la que llama cuando cambia la selección.

El estado no es cualquier variable. React debe conservarlo entre ejecuciones del componente y saber cuándo cambia. `useState` ofrece ese mecanismo: devuelve un valor y una función para actualizarlo.

JSX: una forma de escribir componentes

Este fragmento muestra una ficha mínima:

```
function Ficha({contacto}){  
  return <Text>{contacto.apellido}, {contacto.nombre}</Text>;  
}
```

Sin JSX, el mismo componente puede escribirse usando `React.createElement`:

```
function Ficha({contacto}) {  
  return React.createElement(  
    Text,  
    null,  
    contacto.apellido, ', ', contacto.nombre  
  );  
}
```

`Ficha` es una función que recibe props. `{contacto}` usa desestructuración. La expresión entre etiquetas es **JSX**; las llaves permiten incluir JavaScript dentro de esa descripción. `<Text>` será un componente de Ink.

JSX no es una cadena de texto ni obliga a usar HTML. Una herramienta lo transforma en llamadas de JavaScript que crean elementos de React. Por convención, los nombres de componentes comienzan con mayúscula.

Qué ocurre después de una acción

Supongamos que el buscador contiene `an` y agregamos una `a`:

1. El campo informa el nuevo texto, `ana`, mediante una función recibida por props.
2. Esa función actualiza el estado de búsqueda.
3. React vuelve a ejecutar el componente que calcula los resultados.
4. Sus hijos reciben las nuevas coincidencias y el contacto seleccionado.
5. El renderizador actualiza la presentación.

Los componentes deben describir la interfaz sin modificar datos externos durante el render. Las operaciones como guardar un archivo se ejecutan al responder a una acción.

React conserva el estado según la identidad y posición de los componentes. Si un componente se desmonta, pierde su estado local. Usaremos ese comportamiento para descartar el borrador al cancelar un formulario.

D.2 Ink: llevar React a una terminal

React necesita un **renderizador** que transforme sus elementos en una interfaz concreta. En el navegador se usa React DOM; **Ink es un renderizador de React para la terminal**. Los mismos conceptos de componentes, props y estado se aplican en ambos casos. [Documentación de Ink](#).

Nuestra aplicación es un proceso de Node.js que escucha el teclado y presenta texto, colores, bordes y paneles. Es una **TUI**: una interfaz de usuario de terminal. Sus dimensiones se miden en columnas y filas, no en píxeles.

Las piezas básicas

Elemento	Responsabilidad
Text	Mostrar texto y aplicar estilos como color o negrita
Box	Distribuir componentes en filas o columnas
render	Montar la aplicación en la terminal
useInput	Responder a teclas dentro de un componente

Ink distribuye las cajas mediante un modelo basado en Flexbox. Para esta agenda basta con unas pocas props:

```
<Box flexDirection="row">
  <Box width="30%"><Text>Maestro</Text></Box>
  <Box width="70%"><Text>Detalle</Text></Box>
</Box>
```

`flexDirection="row"` coloca elementos uno al lado del otro; `"column"` los apila. `borderStyle="round"` dibuja un borde y `paddingX={1}` agrega espacio interior horizontal. El texto visible se coloca dentro de `Text`.

`render` se llama al iniciar. Después, las actualizaciones de estado provocan nuevos renderizados; no llamamos a `render` por cada tecla ni imprimimos toda la pantalla manualmente.

Componer con bibliotecas

Ink proporciona la base. Otros paquetes ofrecen controles listos para integrar:

Paquete	Componente	Qué aporta
ink-text-input	TextInput	Edición de texto y cursor
ink-select-input	SelectInput	Lista navegable, indicador y desplazamiento
ink-form	Form	Campos, borrador, navegación y envío de formulario

Nuestro trabajo será **adaptar datos y conectar eventos**. Las bibliotecas administran la interacción de sus controles; la agenda decide qué contacto está seleccionado, qué operaciones son válidas y cuándo persistirlas.

Usar un componente externo requiere entender su contrato: qué props acepta, qué estado conserva y qué eventos produce. `TextInput`, `SelectInput`, `Form`.

D.3 Crear el proyecto

Necesitas Node.js 22 o posterior, npm y una terminal interactiva. Para mostrar el formulario cómodamente, usá una ventana de aproximadamente **120 columnas y 40 filas**. En VS Code ejecutá el programa en **Terminal**, no en el panel Salida.

```
node --version
npm --version
mkdir agenda-ink
cd agenda-ink
```

Creá `package.json` con este contenido:

```
{
  "name": "agenda-ink",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {"start": "tsx app.jsx"},
  "dependencies": {
    "react": "19.2.0",
    "ink": "7.1.1",
    "ink-text-input": "6.0.0",
    "ink-select-input": "6.2.0",
    "ink-form": "2.0.2"
  },
  "devDependencies": {"tsx": "4.23.13"},
  "overrides": {"ink-form": {"ink-select-input": "6.2.0"}}
}
```

Instalá:

```
npm install
```

Estas versiones fueron comprobadas juntas. El `override` hace que `ink-form` use el mismo `ink-select-input@6.2.0` que la agenda: su dependencia original apunta a una versión con requisitos anteriores de React e Ink.

`tsx` ejecuta nuestro JavaScript con JSX; no hace falta escribir TypeScript. `type: "module"` habilita módulos ES en los archivos `.js`.

Conservá `package-lock.json`. Para instalar el mismo proyecto en otra computadora, usá `npm ci`, que reproduce exactamente las versiones registradas en ese archivo.

Un asistente de programación puede encargarse de esta preparación. No hace falta dictarle cada archivo ni repetir los pasos: alcanza con darle este apéndice como especificación, indicar el resultado esperado y pedirle que verifique su propio trabajo.

Configurá este proyecto como una aplicación de Node.js para React e Ink. Instalá `ink-text-input`, `ink-select-input` e `ink-form` en las versiones indicadas, agregá el `override` de `ink-select-input` y dejá listo el script `start` con `tsx`. Después ejecutalo y mostrame la salida.

Primer componente ejecutable

Creá `app.jsx`:

```
import React from 'react';
import {render, Text} from 'ink';

function App() {
  return <Text color="cyan" bold>Agenda de contactos</Text>;
}

render(<App />);
```

Ejecutá desde la carpeta del proyecto:

```
npm start
```

Este ejemplo puede mostrar el título y terminar: todavía no escucha el teclado. `<App />` usa nuestro componente; `Text` viene de una biblioteca. Ambos se componen con la misma sintaxis.

Agregar estado y eventos

Reemplazá `app.jsx` por este ejemplo breve:

```
import React, {useState} from 'react';
import {render, Box, Text, useInput} from 'ink';

function App() {
  const [numero, setNumero] = useState(0);

  useInput((input, key) => {
    if (key.upArrow) setNumero(actual => actual + 1);
    if (key.downArrow) setNumero(actual => actual - 1);
  });

  return (
    <Box flexDirection="column">
      <Text>Número: {numero}</Text>
      <Text dimColor>↑ / ↓: cambiar · Ctrl+C: salir</Text>
    </Box>
  );
}

render(<App />);
```

`useState(0)` establece el valor inicial. `setNumero` solicita un cambio y React vuelve a calcular la interfaz. `actual => actual + 1` calcula el próximo valor a partir del anterior.

Una variable local como `let numero = 0` se crearía otra vez al ejecutar el componente; cambiarla tampoco notificaría a React. El setter conserva el dato y solicita la actualización. No cambia de inmediato la variable del manejador que ya se está ejecutando. [Estado en React](#).

`useState` y `useInput` son **hooks**. Se llaman en el nivel superior de un componente, fuera de condiciones, bucles y manejadores. Para desactivar un `useInput` se cambia su opción `isActive`, conservando la llamada al hook.

D.4 Los datos: el legajo es la identidad

Cada contacto tiene exactamente cinco campos:

Campo	Tipo	Regla
nombre	String	Obligatorio
apellido	String	Obligatorio
legajo	String	Obligatorio, único e identificador del contacto
telefono	String	Opcional
github	String	Opcional; nombre de usuario

El legajo es texto para conservar ceros iniciales: "00123". No hay una propiedad `id` adicional. Para seleccionar, modificar y eliminar buscaremos por `legajo`.

Se permite corregir el legajo al editar. Para hacerlo hay que recordar el **legajo anterior**, localizar ese registro y reemplazarlo por el contacto nuevo. Si el legajo nuevo ya pertenece a otro contacto, rechazamos el cambio.

Creá `contactos.json` con estos datos ficticios:

```
[
  {
    "nombre": "Ana",
    "apellido": "Torres",
    "legajo": "00123",
    "telefono": "381 555-0101",
    "github": "ana-torres-demo"
  },
  {
    "nombre": "Bruno",
    "apellido": "Díaz",
    "legajo": "00124",
    "telefono": "381 555-0102",
    "github": "bruno-diaz-demo"
  },
  {
    "nombre": "Carla",
    "apellido": "Núñez",
    "legajo": "00125",
    "telefono": "381 555-0103",
    "github": "carla-nunez-demo"
  }
]
```

Para empezar con una agenda vacía, el contenido del archivo debe ser `[]`.

Archivos de la aplicación

Archivo	Responsabilidad
contactos.json	Guardar los datos
archivo.js	Leer y escribir JSON
componentes.jsx	Integrar controles externos y mostrar el detalle
app.jsx	Coordinar estado, búsqueda y CRUD

D.5 Leer y escribir JSON con lo mínimo

Creá `archivo.js` completo:

```
import {readFileSync, writeFileSync} from 'node:fs';

export function cargarContactos() {
  return JSON.parse(readFileSync('contactos.json', 'utf8'));
}

export function guardarContactos(contactos) {
  writeFileSync('contactos.json', JSON.stringify(contactos, null, 2));
}
```

La lectura hace dos cosas: `readFileSync` obtiene texto y `JSON.parse` lo convierte en un array de objetos.

La escritura hace el recorrido inverso: `JSON.stringify` convierte los objetos en texto y `writeFileSync` reemplaza el contenido del archivo. `null, 2` aplica una sangría de dos espacios para que el JSON sea legible. [Archivos en Node.js](#).

El nombre relativo `contactos.json` se resuelve desde la carpeta donde corre el proceso. Por eso ejecutamos `npm start` dentro de `agenda-ink`.

Supuesto del ejemplo: el archivo existe y contiene un array válido, con los cinco campos como strings y legajos únicos. El JSON incluido cumple ese formato. Las funciones leen y escriben directamente; un error de lectura o de sintaxis se comunica como una excepción.

Usamos operaciones sincrónicas porque trabajamos con un archivo local pequeño. Cargamos al iniciar y guardamos después de cada alta, modificación o baja completada. Escribir una letra en el buscador o en un borrador no escribe el archivo.

D.6 Cómo se hace un CRUD inmutable

Immutable significa que una operación construye el nuevo estado sin modificar el estado anterior. No prohíbe que la aplicación cambie: cambia reemplazando valores, en lugar de alterar objetos que ya estaban en uso.

En React tratamos los arrays y objetos del estado como valores de solo lectura. Esto hace explícito qué cambió y evita que dos partes del programa alteren accidentalmente el mismo objeto. Actualizar arrays en estado.

Un ejemplo completo, paso a paso

Partimos de este array:

```
const contactos = [{
  nombre: 'Ana',
  apellido: 'Torres',
  legajo: '00123',
  telefono: '381 555-0101',
  github: 'ana-torres-demo'
}];
```

Create — agregar. Construimos un array que contiene los elementos existentes y el nuevo:

```
const nuevo = {
  nombre: 'Bruno',
  apellido: 'Díaz',
  legajo: '00124',
  telefono: '',
  github: ''
};

const conAlta = [...contactos, nuevo];
```

`conAlta` tiene dos contactos; `contactos` sigue teniendo uno. `...contactos` incorpora sus elementos al array nuevo. La comprobación de legajo repetido se realiza antes de aceptar el alta.

Read — consultar. Leer no requiere cambiar el array:

```
const encontrado = conAlta.find(c => c.legajo === '00123');
const coincidencias = conAlta.filter(c => c.apellido.includes('Torres'));
```

`find` devuelve el objeto encontrado o `undefined`. `filter` construye un array con coincidencias. Los objetos encontrados siguen siendo referencias a los originales: para editarlos, crearemos copias.

Update — reemplazar. Creamos el objeto actualizado y luego un array que lo sustituya en la posición correspondiente:

```
const actualizado = {...encontrado, telefono: '381 555-9999'};
const conCambio = conAlta.map(c =>
  c.legajo === encontrado.legajo ? actualizado : c
);
```

Hay dos niveles: `{...encontrado, telefono: ...}` crea un objeto nuevo; `map` crea un array nuevo. Los contactos que no cambian pueden conservar sus objetos. No necesitamos clonar todo el árbol de datos.

Si corregimos también el legajo, comparamos con el anterior:

```
const legajoAnterior = '00123';
const corregido = {...actualizado, legajo: '00999'};
const conLegajoCorregido = conCambio.map(c =>
  c.legajo === legajoAnterior ? corregido : c
);
```

La regla es encontrar por la clave anterior y reemplazar por el objeto completo con la clave nueva, después de verificar que no esté ocupada.

Delete — excluir. Creamos un array que conserve a los demás:

```
const sinAna = conLegajoCorregido.filter(c => c.legajo !== '00999');
```

`sinAna` contiene solamente a Bruno. Los arrays anteriores no fueron alterados.

Por qué no basta con copiar el array

Este fragmento **sí modifica un objeto compartido**:

```
const copia = [...contactos];
copia[0].telefono = 'otro'; // También afecta al objeto dentro de contactos.
```

El array es nuevo, pero sus elementos siguen apuntando a los mismos objetos. Para cambiar un contacto debemos crear también su objeto nuevo, como hicimos con `actualizado`.

Además, `const` impide reasignar una variable; no vuelve inmutable el objeto al que apunta. Las copias con `...` son superficiales. En nuestra agenda todos los campos son strings, por lo que copiar el contacto es suficiente. Actualizar objetos en estado.

Conectar el resultado con React y JSON

En la aplicación, cada operación calcula un array `proximos`. Luego hacemos:

```
guardarContactos(proximos);
setContactos(proximos);
```

Primero intentamos escribir; después actualizamos el estado visible. La función `persistir` del código final captura errores de escritura y los muestra. Si la escritura falla, no anuncia éxito ni actualiza el estado aceptado.

El cálculo del array es la parte inmutable; la escritura del archivo es un efecto externo deliberado. La ejecutamos en el manejador de la acción, no durante el render ni dentro de una función actualizadora de estado.

D.7 Integrar los componentes externos

Creá `componentes.jsx` con este contenido completo:

```

import React from 'react';
import {Box, Text, useInput} from 'ink';
import TextInput from 'ink-text-input';
import SelectInput from 'ink-select-input';
import {Form} from 'ink-form';

export function Buscador({valor, onCambiar, activo}) {
  return (
    <Box>
      <Text color={activo ? 'cyan' : undefined}>Buscar: </Text>
      <TextInput
        value={valor}
        onChange={onCambiar}
        focus={activo}
        placeholder="Nombre, apellido, legajo, teléfono o GitHub"
      />
    </Box>
  );
}

export function Lista({items, legajoSeleccionado, onElegir, activa}) {
  if (items.length === 0) return <Text dimColor>Sin coincidencias</Text>;

  return (
    <SelectInput
      items={items.map(contacto => ({
        label: `${contacto.apellido}, ${contacto.nombre}`,
        value: contacto.legajo
      }))}
      initialIndex={Math.max(0, items.findIndex(c => c.legajo === legajoSeleccionado))}
      limit={8}
      isFocused={activa}
      onHighlight={item => onElegir(item.value)}
    />
  );
}

export function Detalle({contacto}) {
  if (!contacto) return <Text dimColor>No hay un contacto seleccionado.</Text>;

  return (
    <Box flexDirection="column">
      <Text bold color="cyan">Detalle</Text>
      <Text>Nombre: {contacto.nombre}</Text>
      <Text>Apellido: {contacto.apellido}</Text>
      <Text>Legajo: {contacto.legajo}</Text>
      <Text>Teléfono: {contacto.telefono || '-'}</Text>
      <Text>GitHub: {contacto.github || '-'}</Text>
    </Box>
  );
}

export const campos = [
  {name: 'nombre', label: 'Nombre', required: true},
  {name: 'apellido', label: 'Apellido', required: true},
  {name: 'legajo', label: 'Legajo', required: true},
  {name: 'telefono', label: 'Teléfono'},
  {name: 'github', label: 'GitHub'}
]

```

```

];

export function Formulario({contacto, onGuardar, onCancel}) {
  useInput((input, key) => {
    if (key.ctrl && input === 'x') onCancel();
  });

  return (
    <Form
      form={{
        title: contacto.legajo ? 'Editar contacto' : 'Nuevo contacto',
        sections: [{
          title: 'Datos del contacto',
          fields: campos.map(campo => ({
            ...campo,
            type: 'string',
            initialValue: contacto[campo.name] ?? ''
          }))
        }]
      }}
      onSubmit={onGuardar}
    />
  );
}

```

El buscador: una entrada controlada

`TextInput` recibe `value` y llama a `onChange` cuando se edita. Nuestro componente traduce esas props a `valor` y `onCambiar`.

El texto de búsqueda vive en `App` porque cada cambio debe recalcular los resultados. `focus` establece si el campo escucha el teclado. La biblioteca administra cursor, letras y retroceso; nuestra aplicación interpreta el texto.

Pasamos la función que debe ejecutarse, como `onChange={onCambiar}`. Una expresión como `onChange={setBusqueda('Ana')}` la ejecutaría durante el render, en lugar de esperar un evento.

El maestro: adaptar los contactos a opciones

El selector espera opciones con `label` y `value`. Las construimos con `map`:

```

{
  label: `${contacto.apellido}, ${contacto.nombre}`,
  value: contacto.legajo
}

```

`label` es presentación; `value` identifica al contacto. Al omitir una `key` específica en las opciones, el selector usa `value` para identificar sus filas. Así el legajo también distingue los elementos de la lista.

Prop de SelectInput	Uso
items	Opciones que debe mostrar
initialIndex	Posición elegida al montarse
limit={8}	Hasta ocho filas visibles, con desplazamiento
isFocused	Habilitar o deshabilitar el teclado
onHighlight	Informar el contacto destacado al navegar

`onHighlight` actualiza el detalle mientras recorremos la lista. `onSelect` corresponde a confirmar una opción; este ejemplo usa `e` para abrir la edición y no necesita ese evento. La navegación circular y el desplazamiento los resuelve la biblioteca. Props del selector.

El detalle: una pieza específica de la agenda

`Detalle` recibe un contacto y muestra sus campos. Si no hay seleccionado, devuelve un mensaje. Un componente propio puede ser pequeño: aquí la presentación de cinco datos no requiere otra dependencia.

El formulario: describir campos

`campos` define nombres internos, etiquetas y obligatoriedad. El `map` agrega `type: 'string'` y el valor inicial de cada campo.

`Form` conserva un borrador y administra los editores y el foco. Usamos su modo **no controlado**: damos valores iniciales y recibimos el resultado por `onSubmit`. El buscador, en cambio, es controlado porque necesitamos su valor en cada edición. Contrato de Form.

La biblioteca impide enviar si faltan campos requeridos. La agenda además recorta espacios y verifica que el legajo no esté repetido: esas reglas pertenecen a nuestros datos.

`onSubmit={onGuardar}` conecta directamente el evento de la biblioteca con la función que recibe un objeto de cinco campos. El legajo viaja dentro de ese objeto; no se genera otro identificador.

Interacción del formulario

`ink-form` edita un campo por vez. Al abrirlo, pulsá Tab para enfocar el primero. Enter abre un campo y otro Enter acepta su valor en el borrador. El botón **Submit form** envía el contacto completo.

Escape descarta la edición del campo abierto. Nuestro adaptador agrega **Ctrl+X** para cancelar todo el formulario. Al volver al detalle, el formulario se desmonta y pierde su borrador. Hasta que se envíe y se guarde correctamente, los contactos aceptados permanecen intactos.

Las ayudas internas de esta biblioteca están en inglés. Su funcionamiento y sus teclas forman parte del componente que estamos incorporando.

D.8 Decidir qué estado necesita la agenda

Antes de conectar la pantalla, identifiquemos los datos que cambian:

Estado de App	Para qué se conserva
contactos	Colección aceptada
busqueda	Texto del filtro
legajoSeleccionado	Identidad de la persona elegida
modo	Consulta, nuevo o editar
foco	Lista o buscador
mensaje	Resultado o error de una operación

Los contactos filtrados, el objeto seleccionado y si estamos editando **se calculan** a partir de esos valores. No necesitan otro estado que haya que sincronizar. Diseñar el estado en React.

El formulario conserva su borrador internamente. El selector conserva su posición y ventana visible. Aunque ese estado no aparezca en `App`, debemos entenderlo para coordinar las piezas.

Filtro, identidad y selección

`initialIndex` solo determina la selección al montar el selector. Para reiniciarlo cuando cambia la búsqueda o el conjunto de legajos, calcularemos una clave:

```
const claveLista = JSON.stringify([busqueda, filtrados.map(c => c.legajo)]);
```

`key={claveLista}` hace que React monte otra instancia de la lista cuando esa clave cambia. La instancia nueva toma el `initialIndex` que corresponde al legajo seleccionado, o al primer resultado.

No incluimos el legajo seleccionado en la clave: navegar no debe reiniciar la lista a cada pulsación. Usamos `key` para expresar cuándo queremos conservar o reiniciar el estado de un componente. Identidad y estado en React.

Una parte activa del teclado por tarea

Mientras se escribe en el buscador, el selector queda inactivo. Mientras se edita un formulario, tanto el buscador como el selector y los comandos de consulta quedan inactivos.

`App` mantiene su llamada a `useInput`, pero le pasa `{isActive: !editando}`. Cada biblioteca recibe su prop de foco. Así una letra del apellido no se interpreta como un comando de eliminación.

D.9 El CRUD completo

Con `contactos.json`, `archivo.js` y `componentes.jsx` creados, reemplazá `app.jsx` por este archivo completo:

```

import React, {useState} from 'react';
import {render, Box, Text, useInput} from 'ink';
import {cargarContactos, guardarContactos} from './archivo.js';
import {Buscador, Lista, Detalle, Formulario} from './componentes.jsx';

const vacio = {nombre: '', apellido: '', legajo: '', telefono: '', github: ''};

function normalizar(texto) {
  return texto.normalize('NFD').replace(/[\u0300-\u036f]/g, '').toLowerCase();
}

function App({iniciales}) {
  const [contactos, setContactos] = useState(iniciales);
  const [busqueda, setBusqueda] = useState('');
  const [legajoSeleccionado, setLegajoSeleccionado] = useState(null);
  const [modo, setModo] = useState('consulta');
  const [foco, setFoco] = useState('lista');
  const [mensaje, setMensaje] = useState('');

  const editando = modo !== 'consulta';
  const texto = normalizar(busqueda.trim());
  const filtrados = contactos.filter(contacto =>
    ['nombre', 'apellido', 'legajo', 'telefono', 'github'].some(campo =>
      normalizar(contacto[campo]).includes(texto)
    )
  );
  const seleccionado = filtrados.find(c => c.legajo === legajoSeleccionado) ??
  filtrados[0];
  const clavelista = JSON.stringify([busqueda, filtrados.map(c => c.legajo)]);

  function cambiarBusqueda(valor) {
    setBusqueda(valor);
    setLegajoSeleccionado(null);
    setMensaje('');
  }

  function abrirFormulario(nuevoModo) {
    setModo(nuevoModo);
    setMensaje('');
  }

  function cancelar() {
    setModo('consulta');
    setMensaje('Operación cancelada.');
  }

  function persistir(proximos) {
    try {
      guardarContactos(proximos);
      setContactos(proximos);
      return true;
    } catch (error) {
      setMensaje(`No se pudo guardar: ${error.message}`);
      return false;
    }
  }

  function guardar(borrador) {

```

```

const contacto = {...borrador};
for (const campo of Object.keys(vacio)) contacto[campo] = contacto[campo].trim();

if (!contacto.nombre || !contacto.apellido || !contacto.legajo) {
  setMensaje('Completá nombre, apellido y legajo.');
```

return;

const esNuevo = modo === 'nuevo';

const legajoAnterior = esNuevo ? null : seleccionado.legajo;

if (contactos.some(c => c.legajo === contacto.legajo && c.legajo !== legajoAnterior))

{

setMensaje('Ya existe un contacto con ese legajo.');

return;

}

const proximos = esNuevo

? [...contactos, contacto]

: contactos.map(c => c.legajo === legajoAnterior ? contacto : c);

if (!persistir(proximos)) return;

setBusqueda('');

setLegajoSeleccionado(contacto.legajo);

setModo('consulta');

setMensaje(esNuevo ? 'Contacto creado y guardado.' : 'Contacto actualizado y guardado.');

}

function eliminar() {

if (!seleccionado) return;

const proximos = contactos.filter(c => c.legajo !== seleccionado.legajo);

if (!persistir(proximos)) return;

setLegajoSeleccionado(null);

setMensaje('Contacto eliminado y JSON actualizado.');

}

useInput((input, key) => {

if (key.escape) {

cambiarBusqueda('');

setFoco('lista');

return;

}

if (key.tab) {

setFoco(actual => actual === 'lista' ? 'buscar' : 'lista');

return;

}

if (foco === 'buscar') {

if (key.return) setFoco('lista');

return;

}

if (input === '/') setFoco('buscar');

if (input === 'n') abrirFormulario('nuevo');

if (input === 'e' && seleccionado) abrirFormulario('editar');

if (input === 'd') eliminar();

}, {isActive: !editando});

return (

<Box flexDirection="column">

<Text bold color="cyan">AGENDA · {editando ? 'formulario' : foco}</Text>

<Buscador

```

    valor={busqueda}
    onChange={cambiarBusqueda}
    activo={!editando && foco === 'buscar'}
  />
  <Box flexDirection="row" height={editando ? 28 : undefined}>
    <Box width="30%" flexDirection="column" borderStyle="round" paddingX={1}>
      <Text bold>Contactos</Text>
      <Lista
        key={claveLista}
        items={filtrados}
        legajoSeleccionado={seleccionado?.legajo}
        onElegir={setLegajoSeleccionado}
        activa={!editando && foco === 'lista'}
      />
    </Box>
    <Box width="70%" flexDirection="column" borderStyle="round" paddingX={1}>
      {editando ? (
        <Formulario
          contacto={modo === 'nuevo' ? vacio : seleccionado}
          onGuardar={guardar}
          onCancel={cancelar}
        />
      ) : <Detalle contacto={seleccionado} />}
    </Box>
  </Box>
  <Text dimColor>
    {filtrados.length} de {contactos.length} contactos
    {' · '}<Archivo: contactos.json
  </Text>
  <Text color="yellow" wrap="truncate">{mensaje || ' '}</Text>
  <Text dimColor>
    {editando ? 'Tab / ↑ ↓: campo · Enter: editar o aceptar campo · Submit form:
guardar' : foco === 'buscar'
  ? 'Escribí para filtrar · Enter: lista · Esc: limpiar'
  : 'n: nuevo · e: editar · d: eliminar · /: buscar · ↑ ↓: elegir'}
  </Text>
  <Text dimColor>{editando
  ? 'Esc: deshacer campo · Ctrl+X: cancelar formulario · Ctrl+C: salir'
  : 'Tab: lista / búsqueda · Esc: limpiar · Ctrl+C: salir'}</Text>
</Box>
);
}

render(<App iniciales={cargarContactos()} />);

```

Leer el flujo principal

Al iniciar, `cargarContactos()` obtiene el array y se lo pasa a `App`. La prop `iniciales` sirve para inicializar el estado; las operaciones posteriores usan `contactos`.

Cada edición del buscador cambia `busqueda`, reinicia la selección y recalcula `filtrados`. La búsqueda ignora mayúsculas y diacríticos, incluyendo la equivalencia entre ñ y n. Si el legajo elegido no aparece en los resultados, mostramos el primero. Si no hay resultados, mostramos el estado vacío.

Al pulsar `no`, cambia `modo`. El panel derecho monta `Formulario` con campos vacíos o con el contacto seleccionado. La lista permanece visible y no cambia de selección mientras se edita.

Guardar un contacto

`guardar` recibe el borrador y crea una copia. Recorta espacios exteriores y comprueba obligatorios y unicidad del legajo.

En un alta, `legajoAnterior` es `null`: ningún contacto existente puede tener el legajo propuesto. En una edición, el legajo anterior identifica al registro que estamos reemplazando. Este registro se excluye del control de duplicados.

Después se aplica el CRUD inmutable: `[...]` agrega o `map` reemplaza. `persistir` intenta escribir el array y después lo asigna al estado. Si funciona, se limpia la búsqueda, se selecciona el legajo guardado y vuelve el detalle. El mismo flujo admite corregir el legajo.

Eliminar un contacto

`eliminar` usa `filter` para excluir el legajo seleccionado. Tras guardar, conserva el filtro y reinicia la selección al primer resultado disponible. Si se elimina el último contacto, el array y el JSON quedan vacíos.

La eliminación es directa, con un mensaje de resultado en la pantalla. El foco en la lista es lo que habilita ese comando.

Renderizado condicional

Esta elección es la que transforma el detalle en formulario:

```
{editando ? (
  <Formulario contacto={contacto} onGuardar={guardar} onCancel={cancelar} />
) : (
  <Detalle contacto={seleccionado} />
)}
```

Es un fragmento explicativo; el archivo completo calcula el contacto del formulario según el modo. La estructura exterior del panel se conserva y cambia el componente que contiene.

Reservamos 28 filas para el panel durante la edición para que entren el formulario, sus bordes y sus ayudas. Los mensajes de aplicación quedan debajo.

D.10 Usar y comprobar la aplicación

Ejecutá desde `agenda-ink`:

```
npm start
```

Contexto	Tecla	Acción
Lista	↑ / ↓	Recorrer; después del último vuelve al primero
Lista	n	Crear
Lista	e	Editar seleccionado
Lista	d	Eliminar seleccionado
Lista	/	Activar búsqueda
Búsqueda	Escribir	Filtrar en caliente
Búsqueda	Enter	Volver a la lista conservando filtro
Consulta	Tab	Alternar lista y búsqueda
Consulta	Esc	Limpiar filtro y volver a la lista
Formulario, campo cerrado	Tab, Shift+Tab, ↑ o ↓	Recorrer campos y botón de envío
Campo seleccionado	Enter	Abrir el editor
Campo abierto	Enter	Aceptar el campo en el borrador
Campo abierto	Esc	Descartar la edición de ese campo
Botón Submit form	Enter	Enviar y guardar el contacto
Formulario	Ctrl+X	Cancelar el formulario completo
Aplicación	Ctrl+C	Salir

El código fue comprobado con Node.js 24.19.0 y las versiones incluidas. Estas pruebas permiten verificar el funcionamiento:

Prueba	Resultado esperado
Crear un contacto con legajo 00007	Se conserva exactamente ese texto
Crear otro con 00007	Rechaza el duplicado
Editar nombre y teléfono	Reemplaza los datos del mismo legajo
Cambiar el legajo a uno libre	Desaparece la clave anterior y se selecciona la nueva
Cambiar el legajo al de otro contacto	Rechaza el cambio y mantiene abierto el formulario
Aceptar un campo y cancelar con Ctrl+X	No cambia la colección ni se guarda el borrador
Buscar por apellido, legajo, teléfono o GitHub	Lista y detalle coinciden con el resultado
Buscar un texto inexistente	Presenta una lista y un detalle vacíos
Escribir n, e o d en el buscador	Son texto, no operaciones del CRUD
Eliminar desde un filtro	Elimina solo al seleccionado
Eliminar el último	JSON []; se puede volver a crear
Cerrar y volver a abrir	Recupera los cambios guardados

También conviene probar más de ocho contactos para observar el desplazamiento y cambiar una búsqueda sin alterar sus coincidencias para comprobar el reinicio de selección.

Para inspeccionar la persistencia, abrí `contactos.json` después de una operación. Cada guardado reemplaza el array completo. Si querés editar ese archivo a mano, cerrá primero la agenda y respetá el formato de los datos.

D.11 Para recordar

- Un componente describe la interfaz a partir de sus props y su estado; no la corrige paso a paso después de cada acción.
- Lo que puede calcularse a partir del estado no debe guardarse como estado: acá, los contactos filtrados, el seleccionado y el modo de edición se derivan.
- `key` expresa cuándo conservar y cuándo reiniciar el estado interno de un componente.
- Integrar un componente externo es entender su contrato: qué props acepta, qué estado conserva y qué eventos produce.
- El CRUD inmutable construye la colección siguiente en lugar de modificar la anterior, y por eso la copia superficial del array no alcanza: también hay que crear el objeto que cambia.
- El cálculo es puro; escribir el archivo es un efecto deliberado, y va en el manejador de la acción, nunca durante el render.

- El legajo es la identidad: seleccionar, editar y eliminar lo usan, y corregirlo obliga a recordar el valor anterior.

Apéndice E. Referencia rápida de expresiones regulares

Idea central

Una expresión regular se lee como la combinación de cuatro preguntas: qué carácter, cuántas veces, en qué posición y cómo se agrupa. Esta referencia reúne la sintaxis en tablas de consulta; la explicación de cada construcción, con sus casos límite, está en el capítulo 18.

Todos los patrones de estas tablas suponen la bandera `u`, que activa la semántica Unicode y una sintaxis más estricta.

E.1 Átomos: qué carácter

Expresión	Significado
abc	la secuencia literal abc
.	cualquier carácter, salvo los terminadores de línea
\d	un dígito ASCII, equivale a [0-9]
\D	cualquier cosa que no sea un dígito ASCII
\w	carácter de palabra: [A-Za-z0-9_]
\W	lo contrario de \w
\s	espacio en blanco: espacio, tabulación, salto de línea
\S	lo contrario de \s
[abc]	uno de estos caracteres
[a-z]	un carácter del rango
[^abc]	cualquier carácter salvo estos
\p{L}	carácter Unicode con propiedad «letra»
\p{N}	carácter Unicode con propiedad «número»
\p{Script=Greek}	carácter del alfabeto griego
\P{L}	lo contrario de \p{L}
\uXXXX	un carácter por su código de cuatro dígitos hexadecimales
\u{1F600}	un punto de código Unicode completo
\n, \t, \r	salto de línea, tabulación, retorno de carro

Los metacaracteres `. ^ $ * + ? ( ) [ ] { } | \` se escriben con barra invertida cuando se buscan literalmente. Dentro de una clase `[...]` las reglas cambian: `-`, `]`, `^` y `\` pueden necesitar atención según su posición.

E.2 Cuantificadores: cuántas veces

Se aplican al átomo inmediatamente anterior.

Expresión	Significado
<code>x?</code>	cero o una <code>x</code>
<code>x*</code>	cero o más <code>x</code>
<code>x+</code>	una o más <code>x</code>
<code>x{3}</code>	exactamente tres
<code>x{2,5}</code>	entre dos y cinco
<code>x{2,}</code>	dos o más
<code>x*?, x+?, x??, x{2,}? </code>	la variante perezosa: consume lo mínimo

Por defecto los cuantificadores son codiciosos: consumen lo máximo posible y retroceden si hace falta. Cuando existe un delimitador conocido, una clase negada suele ser más precisa y más barata que una variante perezosa: `<[>]*>` frente a `<.*?>`.

E.3 Posiciones: dónde

Las anclas no consumen caracteres; solo exigen una posición.

Expresión	Significado
<code>^</code>	comienzo del texto, o de cada línea con la bandera <code>m</code>
<code>\$</code>	final del texto, o de cada línea con la bandera <code>m</code>
<code>\b</code>	frontera de palabra, definida a partir de <code>\w</code>
<code>\B</code>	posición que no es frontera de palabra
<code>(?=...)</code>	<i>lookahead</i> positivo: lo que sigue coincide
<code>(?!...)</code>	<i>lookahead</i> negativo: lo que sigue no coincide
<code>(?<=...)</code>	<i>lookbehind</i> positivo: lo anterior coincide
<code>(?<!...)</code>	<i>lookbehind</i> negativo: lo anterior no coincide

Sin anclas, un patrón pregunta si existe una coincidencia en alguna parte. Con `^` y `$` pregunta si el texto completo tiene esa forma. Es la diferencia entre buscar y validar.

E.4 Grupos y alternativas

Expresión	Significado
(...)	grupo capturante: agrupa y guarda lo coincidente
(?:...)	grupo no capturante: agrupa sin ocupar una posición
(?<nombre>...)	grupo con nombre
a b	alternativa: a o b
\1, \2	referencia a la captura número 1, 2
\k<nombre>	referencia a la captura con ese nombre

La alternativa tiene la precedencia más baja: `rojo|verde$` significa `rojo` en cualquier posición o `verde` al final. Los paréntesis hacen visible el alcance.

E.5 Banderas

Bandera	Efecto
g	recorre todas las coincidencias y usa <code>lastIndex</code>
i	ignora diferencias de mayúsculas
m	<code>^</code> y <code>\$</code> operan por línea
s	<code>.</code> también coincide con terminadores de línea
u	semántica Unicode y sintaxis estricta
y	coincidencia adherida a <code>lastIndex</code>
d	expone los índices de cada rango coincidente

```
const patron = /hola/giu;

patron.flags; // "giu"
patron.global; // true
patron.source; // "hola"
```

E.6 Métodos: qué devuelve cada uno

Llamada	Devuelve	Necesita g
<code>patron.test(texto)</code>	booleano	no; con g conserva <code>lastIndex</code>
<code>patron.exec(texto)</code>	detalle de una coincidencia, o <code>null</code>	no; con g avanza en cada llamada
<code>texto.match(patron)</code>	detalle de la primera coincidencia	no
<code>texto.match(patron)</code> con g	array de textos completos, sin grupos	sí
<code>texto.matchAll(patron)</code>	iterador con el detalle de cada coincidencia	sí, es obligatoria
<code>texto.replace(patron, x)</code>	texto nuevo, solo la primera coincidencia	no
<code>texto.replaceAll(patron, x)</code>	texto nuevo, todas las coincidencias	sí, es obligatoria
<code>texto.search(patron)</code>	índice de la primera coincidencia, o <code>-1</code>	no
<code>texto.split(patron)</code>	array de fragmentos	no

En un reemplazo, `$1` y `$<nombre>` insertan capturas, `&` la coincidencia completa y `$$` un signo de peso literal. Una función de reemplazo recibe la coincidencia, las capturas, el índice y el texto completo.

E.7 Recetas comprobadas

```
// normalizar espacios
texto.trim().replace(/\s+/gu, " ");

// extraer números, con signo y decimales
[...texto.matchAll(/[+-]?[0-9]+(?:[.]\d+)?/gu)].map(m => m[0]);

// etiquetas Unicode
[...texto.matchAll(/#(<etiqueta>[\p{L}\p{N}_]+)/gu)].map(m => m.groups.etiqueta);

// hora de 24 horas, texto completo
/^(?:[01]\d|2[0-3]):[0-5]\d$/u;

// código con la forma ABC-1234
/^[A-Z]{3}-\d{4}$/u;

// fecha: solo la forma; el calendario se valida aparte
/^(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})$/u;

// escapar texto del usuario antes de interpolarlo
texto.replace(/[\.\*\?\^\$\{\}\|\[\]\\"/gu, "\\$&");
```

E.8 Antes de escribir una regex

Si el problema es...	La herramienta es...
buscar una secuencia fija	includes, startsWith, endsWith, indexOf
reemplazar un texto fijo	replaceAll con un string
separar por un carácter fijo	split con un string
interpretar JSON	JSON.parse
interpretar HTML o XML	un analizador estructural
interpretar CSV completo	una biblioteca de CSV
segmentar palabras de un idioma	Intl.Segmenter
comparar u ordenar texto humano	Intl.Collator
estructuras anidadas sin límite	un analizador sintáctico; el apéndice B construye uno

E.9 Para recordar

- La bandera `u` debería ser el punto de partida de todo patrón moderno; `\d` y `\w` siguen siendo ASCII, y para alfabetos generales van las propiedades Unicode.

- Sin anclas se busca; con `^` y `$` se valida.
- `g` no es un detalle: cambia lo que devuelve `match`, obliga en `matchAll` y `replaceAll`, y agrega estado a `test` y `exec`.
- Una captura que nadie usa conviene escribirla como grupo no capturante.
- Los cuantificadores anidados y ambiguos son la causa habitual del retroceso excesivo; ante entradas externas, limitá el tamaño y probá con casos adversos.
- Una expresión regular describe la forma de un texto. El significado —que una fecha exista, que un correo se entregue— lo valida el código que viene después.

Apéndice F. Glosario

Términos técnicos que usa el libro, con el capítulo o apéndice donde se desarrolla cada uno.

Acarreo. En una suma, el dígito que se traslada a la columna siguiente cuando el resultado excede la base. En el medio sumador es una de las dos salidas. *Apéndice C.*

Accesibilidad. Posibilidad de percibir, comprender y operar una interfaz con distintas capacidades y herramientas, como un lector de pantalla o la navegación por teclado. *Capítulo 20.*

Alcance (*scope*). Región del código desde la cual una vinculación resulta accesible. JavaScript tiene alcance global, de función y de bloque. *Capítulo 2.*

Alcance léxico. Regla por la cual el ámbito de una función queda determinado por el lugar donde fue escrita, no por el lugar desde donde se la invoca. Es la base de las clausuras. *Capítulos 2 y 14.*

Analizador sintáctico (*parser*). Programa que reconoce la estructura de un texto según una gramática y la convierte en una representación manipulable. Una expresión regular no alcanza cuando esa estructura admite anidamiento sin límite. *Capítulo 18 y apéndice B.*

Ancla. En una expresión regular, construcción que exige una posición sin consumir caracteres: `^`, `$`, `\b`. *Capítulo 18 y apéndice E.*

Anfitrión (*host*). Plataforma que ejecuta el motor y le aporta capacidades externas: el navegador con el DOM y los eventos, Node.js con archivos, procesos y red. *Capítulo 1.*

API (*Application Programming Interface*). Interfaz mediante la que un programa usa capacidades de otro componente. El DOM es una API del navegador; una API web ofrece sus servicios por HTTP. *Capítulos 20 y 22.*

Argumento. Valor concreto que se pasa a una función al invocarla. Se distingue del parámetro, que es el nombre declarado. *Capítulo 14.*

Array. Colección ordenada cuyos elementos se asocian con índices enteros que comienzan en cero. *Capítulo 9.*

Array disperso. Array con posiciones inexistentes, distintas de posiciones con `undefined`. Algunos métodos las saltan y otros las materializan. *Capítulos 6 y 9.*

Asociatividad. Regla que decide cómo se agrupan operadores de la misma precedencia. La mayoría asocia de izquierda a derecha; la exponenciación y la asignación, de derecha a izquierda. *Capítulo 2 y apéndice B.*

Atributo (HTML). Información que acompaña a la etiqueta de apertura de un elemento, como `class`, `id` o `href`. Los atributos booleanos se activan por su sola presencia. *Capítulo 20.*

Autómata finito. Máquina abstracta con una cantidad fija de estados que lee símbolos uno por uno. Las expresiones regulares describen, en su forma teórica, exactamente los lenguajes que un autómata finito puede reconocer. *Capítulo 18.*

Bandera (*flag*). En expresiones regulares, opción que modifica el modo de trabajo del patrón: `g`, `i`, `m`, `s`, `u`, `y`, `d`. *Capítulo 18 y apéndice E.*

BigInt. Tipo primitivo para enteros de precisión arbitraria. Su literal termina en `n` y no puede mezclarse directamente con `number` en operaciones aritméticas. *Capítulo 4.*

Bit. Unidad capaz de representar una de dos alternativas. Con `n` bits existen 2^n patrones posibles. *Apéndice C.*

BOM (*Byte Order Mark*). Marca opcional al comienzo de un archivo de texto. UTF-8 no la necesita, pero algunos programas la escriben y aparece como `U+FEFF`. *Capítulo 19.*

BPE (*Byte Pair Encoding*). Algoritmo que construye un vocabulario partiendo de bytes y fusionando, en orden, los pares más frecuentes. *Apéndice A.*

Buffer. Vista de bytes de Node.js. `readFile` sin codificación devuelve un `Buffer`; con codificación devuelve un `string`. *Capítulo 19.*

Byte. Grupo de ocho bits, capaz de 256 patrones. Es la unidad direccionable habitual de la memoria y la base del tokenizador del apéndice A. *Apéndice C y capítulo 19.*

Cabecera (*header*). Línea `Nombre: valor` de un mensaje HTTP que aporta metadatos sobre el pedido o la respuesta, como `Content-Type`. *Capítulo 22.*

Cadena de alcances. Secuencia de ámbitos que el motor consulta al resolver un nombre, desde el más interno hacia el global. *Capítulo 2.*

Callback. Función que se pasa como argumento para que otra la invoque. Puede ejecutarse enseguida, varias veces o en el futuro; el contrato debe aclararlo. *Capítulos 14 y 20.*

Camino feliz. Recorrido de un programa que supone que todo salió bien. Un ejemplo didáctico puede seguirlo deliberadamente, siempre que la decisión esté escrita. *Apéndice B.*

Cascada. Proceso por el que CSS decide qué declaración se aplica cuando varias compiten por la misma propiedad, según su origen, importancia, especificidad y orden. *Capítulo 20.*

Clausura (*closure*). Función que conserva acceso a las vinculaciones del ámbito donde fue creada, incluso después de que ese ámbito terminó. *Capítulos 2 y 14.*

Clave de lista (*key*). En una lista de componentes, identificador estable que permite saber qué elemento corresponde a qué dato cuando la lista se filtra o se reordena. *Capítulo 21 y apéndice D*.

Cliente. Programa que realiza una solicitud: un navegador, `curl` o una llamada a `fetch`. *Capítulos 20 y 22*.

Codificación de caracteres. Convenio que asigna secuencias de bytes a caracteres. UTF-8 es la codificación habitual de los archivos de texto. *Capítulos 5 y 19*.

Código de estado. Número de tres cifras con que una respuesta HTTP resume qué pasó con el pedido. El primer dígito indica la familia: `2xx` éxito, `4xx` error del cliente, `5xx` error del servidor. *Capítulo 22*.

Coerción. Conversión automática de tipo que aplica una operación porque necesita otro tipo del que recibió. *Capítulo 7*.

Collation. Orden de textos según reglas de un idioma, distinto del orden por unidades de código. `Intl.Collator` lo implementa. *Capítulo 5*.

Combinador de promesas. Función que coordina varias promesas: `Promise.all`, `allSettled`, `race` y `any`. Cada una responde una pregunta diferente. *Capítulo 17*.

Compilador. Programa que traduce código a otra representación antes de ejecutarlo. Un motor de JavaScript compila durante la ejecución las rutas que se usan con frecuencia. *Capítulo 1*.

Complejidad funcional. Propiedad de un conjunto de operaciones lógicas que permite construir todas las demás. NAND por sí sola es funcionalmente completa; NOR también. *Apéndice C*.

Componente. Pieza reutilizable de interfaz que recibe datos y describe qué mostrar. *Capítulo 21 y apéndice D*.

Contrapresión (*backpressure*). Mecanismo que impide que un productor de datos avance mucho más rápido de lo que el consumidor puede procesar. *Capítulo 19*.

Copia profunda. Duplicado que recorre todos los niveles de una estructura. `structuredClone` la realiza para muchos tipos integrados. *Capítulo 10*.

Copia superficial. Duplicado de un solo nivel: la estructura exterior es nueva y los valores internos siguen siendo las mismas referencias. Es lo que hacen el `spread`, `Object.assign` y `slice`. *Capítulos 9 y 10*.

CORS (*Cross-Origin Resource Sharing*). Mecanismo por el que un servidor indica qué accesos admite desde páginas de otros orígenes. Se configura en el servidor, no en el cliente. *Capítulo 20*.

Cortocircuito. Comportamiento de `&&` y `||` que deja de evaluar en cuanto el resultado queda determinado, y devuelve un operando en lugar de un booleano. *Capítulo 3*.

CRUD. Las cuatro operaciones básicas sobre datos: crear, leer, actualizar y borrar. En REST se traducen a `POST`, `GET`, `PUT` o `PATCH` y `DELETE`. *Capítulo 22 y apéndice D.*

Cuantificador. En una expresión regular, construcción que indica cuántas veces puede repetirse el átomo anterior: `?`, `*`, `+`, `{n,m}`. *Capítulo 18 y apéndice E.*

Desenrollado de la pila. Proceso por el cual una excepción abandona las llamadas pendientes hasta encontrar un `catch`. *Capítulo 13.*

Desestructuración. Sintaxis que extrae valores de un array o propiedades de un objeto y los vincula con nombres locales. *Capítulos 9, 10 y 14.*

Directorio de trabajo. Carpeta desde la que se interpretan las rutas relativas de un proceso, disponible en `process.cwd()`. No coincide necesariamente con la carpeta del módulo. *Capítulo 19.*

DOM (*Document Object Model*). Representación del documento como árbol de nodos, con APIs para consultarlo y modificarlo. JavaScript cambia el DOM, no el archivo HTML. *Capítulo 20.*

ECMAScript. Estándar que define el núcleo del lenguaje. JavaScript es su implementación más conocida; las APIs del navegador y de Node.js pertenecen a otras especificaciones. *Capítulo 1.*

Efecto secundario. Cambio observable fuera de una función: escribir un archivo, mutar un argumento, imprimir, consultar el reloj. *Capítulo 15.*

Elevación (*hoisting*). Procesamiento de las declaraciones antes de ejecutar el bloque. Las declaraciones de función quedan utilizables; `let` y `const` quedan en la zona muerta temporal. *Capítulo 2.*

Encadenamiento opcional (`?.`). Operador que interrumpe una cadena de acceso y produce `undefined` cuando el valor anterior es `null` o `undefined`. *Capítulo 6.*

Endpoint. Dirección de una operación o de un recurso de una API. *Capítulos 20 y 22.*

Especificidad. Peso de un selector CSS, leído en tres columnas —identificadores; clases, atributos y pseudoclases; elementos— que se comparan de izquierda a derecha, sin sumarse. *Capítulo 20.*

Estado de la aplicación. Datos que describen la situación actual de una interfaz. En un enfoque declarativo, la pantalla se calcula a partir de él. *Capítulos 20 y 21, apéndice D.*

Estado etiquetado. Modelo que representa una situación con un campo explícito —«pendiente», «lista», «error»— en lugar de deducirla de un `null` ambiguo. *Capítulo 6.*

Evento. Notificación de algo ocurrido en el entorno —un clic, un envío, una tecla— a la que el programa responde con una función registrada. *Capítulo 20.*

Excepción. Señal de que una operación no pudo cumplir su contrato. Se propaga por la pila de llamadas hasta encontrar quien la maneje. *Capítulo 13.*

Expresión. Porción de código que produce un valor. Se distingue de una sentencia, que ejecuta una acción estructural. *Capítulo 2.*

Expresión regular. Descripción de un conjunto de secuencias de texto, construida con átomos, cantidades, alternativas y posiciones. *Capítulo 18 y apéndice E.*

Fall-through. En un `switch`, continuación de la ejecución en el caso siguiente por ausencia de `break`, `return` o `throw`. Puede ser intencional o un olvido. *Capítulo 12.*

Falsy. Valor cuya conversión booleana produce `false`. Son ocho: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined` y `NaN`. *Capítulo 3.*

FileHandle. Manejador devuelto por `open` que permite lecturas parciales, posiciones explícitas y varias operaciones sobre la misma apertura. Siempre debe cerrarse. *Capítulo 19.*

Flexbox. Modelo de distribución de CSS que organiza los hijos de un contenedor a lo largo de un eje principal. *Capítulo 20.*

Forma normal disyuntiva. Expresión booleana escrita como OR de términos AND. Permite construir una expresión para cualquier tabla de verdad finita. *Apéndice C.*

Frontend y backend. El frontend es la parte de una aplicación con la que interactúa la persona, y corre en su dispositivo; el backend procesa operaciones en servidores, cerca de los datos y de las reglas. *Capítulos 1 y 20.*

Función de orden superior. Función que recibe funciones como argumento o devuelve una función. *Capítulos 14 y 15.*

Función pura. Función que devuelve el mismo resultado para las mismas entradas y no cambia estado observable fuera de ella. *Capítulo 15.*

Generador. Función declarada con `function*` que produce valores bajo demanda mediante `yield`. Implementa el protocolo iterable sin escribirlo a mano. *Capítulo 14.*

Geocodificación. Conversión de una referencia textual, como el nombre de una ciudad, en ubicaciones con coordenadas. *Capítulos 20 y 22.*

Grafema (*grapheme cluster*). Unidad que una persona percibe como un solo carácter, aunque esté formada por varios puntos de código. `Intl.Segmenter` los recorre. *Capítulo 5.*

Grid. Modelo de distribución de CSS que organiza los hijos de un contenedor en filas y columnas. *Capítulo 20.*

Guarda. Condición al comienzo de una función que descarta un caso imposible y devuelve o lanza de inmediato, dejando visible el camino principal. *Capítulo 12.*

Hook. En React, función que da acceso a capacidades del renderizador, como `useState` o `useInput`. Se llama siempre en el nivel superior del componente. *Apéndice D.*

HTTP (*HyperText Transfer Protocol*). Protocolo de pedido y respuesta entre un cliente y un servidor. No guarda memoria entre un pedido y el siguiente. *Capítulo 22.*

Idempotente. Dicho de un método HTTP, que repetirlo deja el servidor igual que ejecutarlo una sola vez. `GET`, `PUT` y `DELETE` lo son; `POST`, no. *Capítulo 22.*

Identidad. Propiedad de un objeto de ser él mismo y no otro con el mismo contenido. `===` compara identidad, no igualdad de contenido. *Capítulo 10.*

Identificador. Nombre válido para una vinculación. Puede contener letras Unicode, dígitos, `_` y `$`, y no puede comenzar con un dígito. *Capítulo 2.*

Inmutabilidad. Práctica de producir una versión nueva en lugar de modificar un valor compartido. No exige clonar todo: se copia el camino que cambia y se comparte el resto. *Capítulos 15 y 16.*

Interpolación. Inserción del valor de una expresión dentro de una plantilla literal mediante `${...}`. *Capítulo 5.*

Intérprete. Programa que ejecuta código sin producir antes un binario separado. Los motores modernos combinan interpretación y compilación durante la misma ejecución. *Capítulo 1.*

Invariante. Condición que una estructura debe cumplir siempre. En un árbol binario de búsqueda, la relación de orden entre cada nodo y sus dos subárboles. *Capítulo 16.*

Iterable. Objeto que implementa `Symbol.iterator` y por eso puede recorrerse con `for...of`, expandirse con `spread` y desestructurarse. *Capítulos 8 y 12.*

JIT (Just-In-Time). Compilación durante la ejecución, con información recogida sobre los valores reales. Si las suposiciones dejan de cumplirse, el motor descarta la optimización. *Capítulo 1.*

JSON (JavaScript Object Notation). Formato de texto para intercambiar datos estructurados: objetos, arrays, cadenas, números, booleanos y `null`. No admite comentarios, fechas ni `undefined`. *Capítulos 10, 19 y 22.*

JSX. Sintaxis que describe elementos de React dentro de JavaScript. Una herramienta la transforma en llamadas que crean esos elementos. *Apéndice D.*

Lenguaje regular. Conjunto de cadenas que un autómata finito puede reconocer. Es exactamente lo que describe una expresión regular en su forma matemática. *Capítulo 18.*

LIFO (last in, first out). Disciplina de acceso de una pila: el último elemento que entra es el primero que sale. *Capítulo 9 y apéndice B.*

Lookahead y lookbehind. Construcciones de una expresión regular que exigen la presencia o ausencia de un patrón antes o después de la coincidencia, sin incluirlo en ella. *Capítulo 18.*

Map. Colección de pares clave-valor que acepta claves de cualquier tipo y conserva el orden de inserción. *Capítulo 11.*

Máscara de bits. Técnica que usa cada bit de un entero como una opción independiente, activada con `|`, consultada con `&`, apagada con `& ~` y alternada con `^`. *Capítulo 4 y apéndice C.*

Media query. Condición de CSS que aplica un grupo de reglas solo cuando se cumple, por ejemplo un ancho mínimo del viewport. *Capítulo 20.*

Medio sumador. Circuito que suma dos bits y produce suma y acarreo. La suma es XOR y el acarreo es AND. *Apéndice C.*

Metacarácter. Carácter con significado especial en una expresión regular: `.`, `^`, `$`, `*`, `+`, `?`, `(`, `)`, `[`, `]`, `{`, `}`, `|`, `\`. *Capítulo 18.*

Método HTTP. Verbo de un pedido que indica la acción sobre el recurso: `GET`, `POST`, `PUT`, `PATCH` o `DELETE`. *Capítulo 22.*

Modo estricto. Semántica que convierte en errores varios comportamientos históricos permisivos. Los módulos ECMAScript lo aplican automáticamente. *Capítulo 1.*

Motor. Programa que lee y ejecuta código JavaScript, como V8, SpiderMonkey o JavaScriptCore. Compartir motor no implica compartir APIs. *Capítulo 1.*

Mutación. Modificación de un valor existente, en oposición a producir uno nuevo. `sort` y `splice` mutan; `toSorted` y `slice` no. *Capítulos 9 y 10.*

NaN (*Not a Number*). Valor del tipo `number` que representa un resultado numérico indeterminado. No es igual a sí mismo; se detecta con `Number.isNaN`. *Capítulo 4.*

Normalización Unicode. Transformación de un texto a una forma acordada —`NFC`, `NFD`, `NFKC`, `NFKD`— para que secuencias equivalentes se comparen como iguales. *Capítulo 5.*

Objeto. Colección de propiedades con nombre que representa una entidad. Las variables guardan una referencia, no el objeto. *Capítulo 10.*

Parámetro. Nombre declarado en la definición de una función. *Capítulo 14.*

Parámetro rest. Último parámetro escrito con `...`, que reúne en un array los argumentos restantes. *Capítulo 14.*

Path traversal. Vulnerabilidad que permite salir de un directorio autorizado mediante segmentos `..` en una ruta controlada por el usuario. *Capítulo 19.*

Pila (*stack*). Colección en la que se agrega y se quita por un solo extremo, llamado tope. Es la memoria mínima necesaria para tratar estructuras anidadas. *Apéndice B.*

Pila de llamadas (*call stack*). Registro de las llamadas a función pendientes de retornar. Determina el recorrido de una excepción y limita la profundidad de la recursión. *Capítulos 13 y 16.*

Pipeline. Encadenamiento de transformaciones donde cada etapa tiene una responsabilidad: seleccionar, transformar, acumular. *Capítulo 15.*

Plantilla etiquetada. Plantilla literal precedida por una función, que recibe por separado las partes estáticas y los valores interpolados. *Capítulos 5 y 21.*

Plantilla literal. Cadena delimitada por comillas invertidas, que admite interpolación y varias líneas. *Capítulo 5.*

Polyfill. Implementación en JavaScript de una funcionalidad ausente en un entorno. Una transformación de sintaxis no reemplaza a un polyfill. *Capítulo 1.*

Precedencia. Regla que decide cómo se agrupan operadores distintos cuando no hay paréntesis. *Capítulo 2 y apéndice B.*

Primitivo. Valor que no es un objeto. JavaScript tiene siete tipos primitivos: `boolean`, `number`, `bigint`, `string`, `undefined`, `null` y `symbol`. *Capítulos 3 a 8.*

Promesa. Objeto que representa un valor que todavía no llegó. Tiene tres estados — pendiente, cumplida, rechazada— y solo cambia una vez. *Capítulos 17, 20 y 22.*

Props. En React, información que un componente recibe de quien lo usa. Pueden ser datos o funciones. *Apéndice D.*

Punto de código. Número que Unicode asigna a un carácter. En UTF-16 puede necesitar una o dos unidades de código. *Capítulo 5.*

Reactividad. Propiedad de una interfaz que se actualiza sola cuando cambian los datos de los que depende. *Capítulo 21.*

Recursividad. Técnica en la que una función se llama a sí misma sobre una entrada más pequeña. Necesita un caso base, una reducción y una combinación. *Capítulo 16.*

Referencia. Valor que identifica a un objeto. Asignar una variable copia la referencia, no el objeto. *Capítulo 10.*

Relanzamiento (*rethrow*). Volver a lanzar el error capturado, en lugar de envolverlo en uno nuevo, cuando esta capa no puede manejarlo. *Capítulo 13.*

Renderizador. Biblioteca que transforma los elementos de React en una interfaz concreta: React DOM en el navegador, Ink en la terminal. *Apéndice D.*

Renderizar. Producir o actualizar una representación visible a partir de contenido o datos. *Capítulo 20 y apéndice D.*

REST (*Representational State Transfer*). Estilo de diseño de APIs web en el que cada recurso tiene una URL y los métodos HTTP son las acciones sobre él. *Capítulo 22.*

Selector. Expresión de CSS que identifica los elementos a los que se aplica una regla, como `.tarjeta` o `#estado`. `querySelector` usa la misma sintaxis desde JavaScript. *Capítulo 20.*

Semántica. Significado de los elementos de un documento y de sus relaciones, independiente de cómo se ven. *Capítulo 20.*

Servidor. Programa que recibe solicitudes y entrega respuestas. *Capítulos 20 y 22.*

Set. Colección de valores únicos que conserva el orden de la primera inserción y compara objetos por identidad. *Capítulo 11.*

Sombreado (*shadowing*). Declaración de un nombre en un ámbito interior que oculta otro del mismo nombre en un ámbito exterior. *Capítulo 2.*

Stream. Flujo de datos que llega por partes, sin cargar todo el contenido en memoria. Los fragmentos no coinciden con límites semánticos como una línea. *Capítulo 19.*

Symbol. Tipo primitivo cuyos valores son identidades únicas. Usado como clave de propiedad evita colisiones; los símbolos conocidos permiten participar en protocolos del lenguaje. *Capítulo 8.*

Tabla de verdad. Enumeración de todas las combinaciones de entrada de una función booleana y su salida. Para n entradas tiene 2^n filas. *Apéndice C.*

Tipado dinámico. Comprobación de tipos durante la ejecución, con valores reales. Cada valor tiene un tipo, aunque una variable no quede asociada a uno. *Capítulo 1.*

Tipado estático. Comprobación de tipos antes de ejecutar, realizada por una herramienta como TypeScript. Sus anotaciones desaparecen en tiempo de ejecución. *Capítulo 1.*

Token. En el apéndice A, identificador que representa un byte base o la concatenación de dos tokens anteriores. En un analizador, unidad léxica de la entrada. *Apéndices A y B.*

Transpilación. Transformación de código fuente a otro código fuente de nivel parecido, por ejemplo de sintaxis moderna a una compatible con entornos antiguos. *Capítulo 1.*

Truthy. Valor cuya conversión booleana produce `true`. Son todos los que no están en la lista de falsy, incluidos `[]`, `{}` y `"0"`. *Capítulo 3.*

TUI (*terminal user interface*). Interfaz de usuario dibujada con texto en una terminal, medida en columnas y filas. *Apéndice D.*

TypeScript. Lenguaje que agrega análisis estático de tipos sobre JavaScript. Se transforma a JavaScript para ejecutarse y no valida datos externos por sí solo. *Capítulo 1.*

Unicode. Estándar que asigna un punto de código a cada carácter. No define cómo se almacenan esos códigos: de eso se ocupan UTF-8 y UTF-16. *Capítulos 5 y 19.*

URL. Dirección de un recurso, formada por esquema, host, puerto, ruta, query y fragmento. *Capítulos 20 y 22.*

UTF-16. Representación interna de los strings de JavaScript. `length` y los índices cuentan unidades de código UTF-16, no caracteres visibles. *Capítulo 5.*

UTF-8. Codificación de ancho variable que representa cada punto de código con uno a cuatro bytes. Es la habitual en archivos y en la red. *Capítulos 5 y 19, apéndice A.*

Variable. Nombre vinculado con un valor durante una parte de la ejecución. `const` fija el vínculo, no el valor. *Capítulo 2.*

Viewport. Área de la ventana que el navegador usa como referencia para presentar el documento. *Capítulo 20.*

Vinculación (*binding*). Asociación entre un nombre y un valor dentro de un alcance. *Capítulo 2.*

WeakMap y **WeakSet.** Colecciones que asocian datos con objetos sin impedir que el recolector los libere. No son enumerables ni tienen `size`. *Capítulo 11.*

Zona muerta temporal (*Temporal Dead Zone*). Intervalo entre el comienzo de un bloque y la inicialización de un `let` o `const`, durante el cual leer el nombre lanza `ReferenceError`. *Capítulo 2.*

Fuentes

Documentos originales citados en el texto: estándares, historia de la Web y documentación de las bibliotecas y servicios que usan los proyectos.

Capítulo 20. Desarrollo web

- Historia de la Web: [CERN](#), [el nacimiento de la Web](#) y [los componentes de 1990](#).
- Historia de CSS: [W3C](#).
- HTML: [sintaxis y estructura en MDN](#).
- CSS: [cascada](#), [modelo de caja](#), [Flexbox](#) y [Grid](#).
- JavaScript: [estándar ECMAScript](#) y [modelo de ejecución](#).
- APIs del navegador: [DOM](#), [eventos](#) y [Fetch](#).
- Servicios del proyecto: [geocodificación de Open-Meteo](#), [datos meteorológicos y códigos](#), [condiciones de uso](#) y [licencia y atribución](#).

Capítulo 21. Arrow.js

- Arrow.js: [plantillas](#), [componentes](#) y [la función `html`](#), con [sus vinculaciones de propiedades y eventos](#).
- MDN: [plantillas etiquetadas](#) y [el elemento `dialog`](#).

Capítulo 22. API REST

- [API de pronóstico de Open-Meteo](#), la misma del capítulo 20.
- JSONPlaceholder, la API de práctica: <https://jsonplaceholder.typicode.com>.
- REST Client, extensión de Visual Studio Code de Huachao Mao.

Apéndice D. Agenda con React e Ink

- React: [describir la interfaz](#), [el estado como memoria de un componente](#), [actualizar arrays y objetos en el estado](#), [diseñar el estado](#) y [conservar o reiniciar el estado](#).
- Ink: [documentación](#), [TextInput](#), [SelectInput](#) y [sus props](#), [Form](#) y [su contrato](#).
- Node.js: [el módulo `fs`](#).